AlphaZero-style MCTS for Yolah

The network, the search, the self-play learning loop, and the code that runs them

2026-10-25

Table of contents

1 What this document covers 3

2 The game, as far as the search is concerned 4

3 Position and move encoding 5

4 The network 6
- 4.1 Architecture . . . 6
- 4.2 Training . . . 7
- 4.3 Export: folding BatchNorm . . . 8
- 4.4 The CPU backend . . . 9
- 4.5 The libtorch backend . . . 9
- 4.6 The seam: `nn::Evaluator` . . . 9
- 4.7 The network cache . . . 10

5 Monte-Carlo tree search with a network 10
- 5.1 One simulation . . . 10
- 5.2 PUCT, in KataGo's formulation . . . 11
  - 5.2.1 The exploration coefficient . . . 11
  - 5.2.2 The utility-stdev factor $\hat{\sigma}$ . . . 12
- 5.3 First play urgency . . . 12
- 5.4 Backup and the sign convention . . . 13
- 5.5 Virtual loss, batching and collisions . . . 13
- 5.6 Graph search . . . 14
  - 5.6.1 The node key . . . 15
  - 5.6.2 Tree reuse and garbage collection . . . 15
- 5.7 At the root . . . 16
  - 5.7.1 Dirichlet noise . . . 16
  - 5.7.2 Forced playouts . . . 16
  - 5.7.3 Policy target pruning . . . 16
  - 5.7.4 LCB move selection . . . 17
- 5.8 Playout cap randomization . . . 17
- 5.9 Choosing the move, and the policy target . . . 17

6 Learning by self-play 18
- 6.1 The idea . . . 18
- 6.2 The pieces and how they talk . . . 19

6.3 Self-play: many games, one GPU . . . 19
6.3.1 Why many games at once . . . 19
6.3.2 The self-play settings, and why . . . 21
6.3.3 What a training row contains . . . 21
6.3.4 Hot-swapping the network . . . 22
6.4 The trainer . . . 22
6.4.1 The replay window . . . 22
6.4.2 How much to train: the reuse ratio . . . 23
6.4.3 The loss . . . 24
6.4.4 Blending the result with the search value . . . 24
6.4.5 Optimiser and numerics . . . 25
6.4.6 Data augmentation: the 8 symmetries . . . 25
6.4.7 Averaging the weights before export . . . 26
6.4.8 Export and publication . . . 26
6.5 Measuring progress . . . 27
6.5.1 The match protocol . . . 27
6.5.2 From a score to Elo . . . 28
6.5.3 A book of balanced openings . . . 28
6.6 Running it on the cluster . . . 30
6.6.1 The cluster . . . 30
6.6.2 The image . . . 31
6.6.3 The job script . . . 31
6.6.4 Splitting the GPUs over several nodes . . . 31
6.6.5 Stopping cleanly . . . 33
6.6.6 What to watch . . . 34
6.7 Sizing for the cluster . . . 34
6.7.1 What was measured . . . 34
6.7.2 Why black wins so often . . . 35
6.7.3 From the measurements to the settings . . . 36
6.8 Parameters of the learning loop . . . 37
6.9 KataGo's auxiliary heads . . . 39
6.9.1 Why predicting more helps . . . 39
6.9.2 Ownership in Yolah . . . 40
6.9.3 The rest of the margin . . . 41
6.9.4 The heads . . . 41
6.9.5 The losses . . . 42
6.9.6 Recording the targets . . . 43
6.9.7 Two variants, one code . . . 43

7 Solving Yolah **44**
7.1 What there is to compute . . . 44
7.2 Retrograde analysis, the textbook way . . . 45
7.3 Regions: what makes endgames small . . . 46
7.4 An exact endgame solver . . . 46
7.5 Can Yolah be solved? . . . 47

8 Code map **48**

9 Parameter reference **49**
9.1 Budget and parallelism . . . 50
9.2 PUCT . . . 50
9.3 First play urgency . . . 50

9.4 Graph and root . . . 51
9.5 Self-play . . . 51

10 Measurements **51**
10.1 What graph search actually buys . . . 51
10.2 Node allocation . . . 52
10.3 What LCB and pruning change . . . 53

11 References **53**

12 Annotated source **53**
12.1 The search . . . 53
12.1.1 `player/alphazero_mcts.h` . . . 54
12.1.2 `player/alphazero_mcts.cpp` . . . 59
12.1.3 `player/alphazero_mcts_player.h` . . . 75
12.1.4 `player/alphazero_mcts_player.cpp` . . . 77
12.1.5 `player/nn_cache.h` . . . 80
12.2 The network interface and its backends . . . 82
12.2.1 `nnue/nn_evaluator.h` . . . 82
12.2.2 `nnue/nn_evaluator.cpp` . . . 84
12.2.3 `nnue/batched_evaluator.h` . . . 85
12.2.4 `nnue/batched_evaluator.cpp` . . . 87
12.2.5 `nnue/resnet.h` . . . 89
12.2.6 `nnue/resnet.cpp` . . . 91
12.2.7 `nnue/resnet_torch.h` . . . 96
12.2.8 `nnue/resnet_torch.cpp` . . . 97
12.3 Training and export . . . 99
12.3.1 `nnue/cnn_resnet_value_policy_chunked.py` . . . 99
12.3.2 `nnue/resnet_export.py` . . . 114
12.4 The self-checks . . . 118
12.4.1 `test/resnet_check.cpp` . . . 118
12.4.2 `test/alphazero_mcts_check.cpp` . . . 120
12.5 The learning loop . . . 123
12.5.1 `player/alphazero_selfplay.h` . . . 123
12.5.2 `player/alphazero_selfplay.cpp` . . . 126
12.5.3 `test/alphazero_learn_main.cpp` . . . 132
12.5.4 `test/opening_book_main.cpp` . . . 136
12.5.5 `nnue/alphazero_learn.py` . . . 143
12.5.6 `nnue/alphazero_aux.py` . . . 160
12.5.7 `config/alphazero_mcts_selfplay_player.cfg` . . . 163
12.5.8 `config/alphazero_mcts_eval_player.cfg` . . . 163
12.5.9 `nnue/alphazero_learn.def` . . . 164
12.5.10 `nnue/alphazero_common.sh` . . . 165
12.5.11 `nnue/alphazero_learn.sh` . . . 167
12.5.12 `nnue/alphazero_match.sh` . . . 170
12.5.13 `nnue/alphazero_selfplay.sh` . . . 172

1 What this document covers

The Yolah engine has two halves that only meet through a very thin interface. On one side a two-headed residual network, trained in PyTorch, that looks at a position and answers two

questions: *who is winning?* and *what would a strong player consider here?* On the other side a Monte-Carlo tree search that calls that network a few hundred times per move and turns its two opinions into an actual move — one much stronger than either opinion alone.

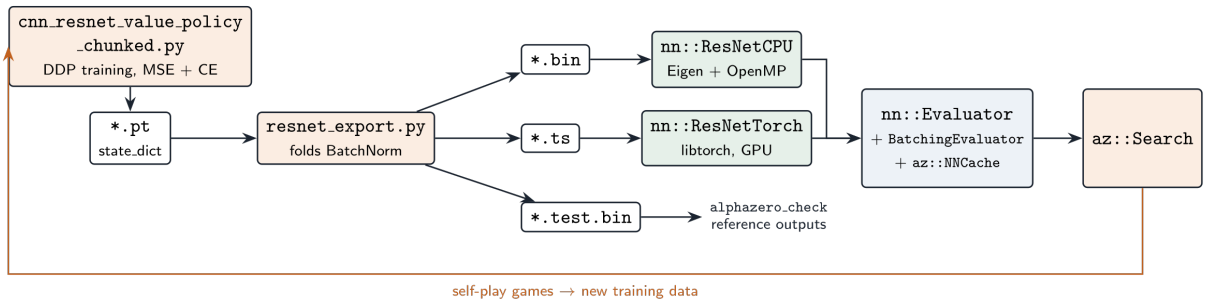


Figure 1: From the training script to the search: every artefact and who consumes it.

This document explains both halves and the seam between them: the algorithm and its formulas first, then the code that implements them, file by file. The search is the one in `player/alphazero_mcts.{h,cpp}`; it started as plain AlphaZero and has since been moved onto KataGo's formulation of PUCT, its first-play-urgency rule, LCB move selection, graph search, and the two self-play tricks from the KataGo paper (forced playouts and policy target pruning). Section 6 then closes the loop: the engine plays against itself on a GPU cluster, learns from those games and gets stronger, for weeks, with its progress measured by regular matches against its former selves, and Section 7 asks what it would take to compute exact values — endgame tables, an exact solver — and whether Yolah could be solved.

Conventions used throughout:

- s is a position, a a legal move, $s \cdot a$ the position after it.
- $v \in (-1, 1)$ is the network's value, always **for the player to move**.
- $z \in \{-1, 0, +1\}$ is the final game result, also for the player to move.
- $P(s, a)$ is the network's prior, $Q(s, a)$ the search's running estimate, N a visit count, W a sum of values.

2 The game, as far as the search is concerned

Yolah is played on 8×8 with four stones per side. A stone slides like a queen, and **the square it leaves becomes a permanent hole**. The player who moves scores one point per move; the game ends when neither side can move, and the higher score wins.

Two consequences drive the design of the search.

The state graph is acyclic. In `Yolah::play` the move does `empty ^= square_bb(m.from_sq())`, and `from_sq` always holds a stone, never a hole — so the hole count increases by exactly one on every real move and can never decrease. A position can therefore never repeat. No repetition detection, no cycle handling, no draw-by-repetition: the search space is a DAG, and a depth-first walk over it always terminates.

Move orders commute. The four stones of a side move independently, so playing a then b with two different stones reaches exactly the same position as b then a. The same position is reachable through many paths, which is what makes both the network cache and the graph search worth having.

The one wrinkle: a player with no legal move passes (`Move::none()`) and scores nothing, while the opponent keeps moving. Two positions can therefore share a board and a side to move while differing in how the points so far split between the players — and the final result depends on that split. This matters for hashing, and Section 5.6.1 comes back to it.

3 Position and move encoding

The network sees four 8×8 binary planes: black stones, white stones, holes, and a constant plane for the side to move.

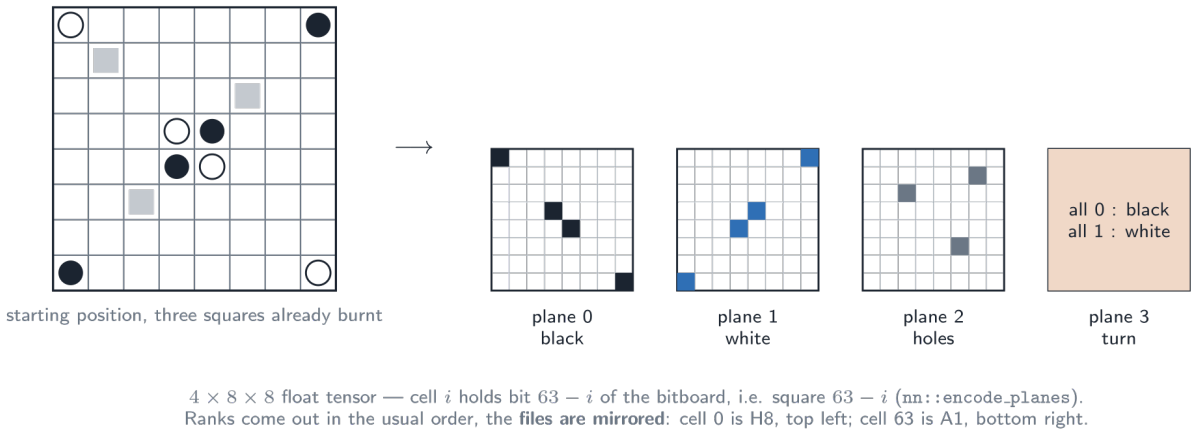


Figure 2: `nn::encode_planes` (C++) and `encode_cnn` (Python) must agree bit for bit.

Two details are easy to get wrong and are worth stating explicitly.

The planes come out mirrored, and not on purpose. Squares are numbered `SQ_A1 = 0` through `SQ_H8 = 63`, and bit s of a bitboard is square s. The training cache turns a bitboard into a plane like this (`preprocess.py`):

```
np.unpackbits(np.array([n], dtype='>u8').view(np.uint8)).reshape(8, 8)
```

`>u8` writes the integer big-endian — most significant byte first — and `np.unpackbits` takes the most significant bit of each byte first. The 64 values therefore come out in the order bit 63, bit 62, …, bit 0, so element i is bit $63 - i$, which is square $63 - i$. That is the whole origin of the $63 - i$, and `nn::encode_planes` has no choice but to repeat it:

```
plane[i] = static_cast<float>((bb >> (63 - i)) & 1);
```

Geometrically: write $s = 8\,\text{rank} + \text{file}$ and reshape row-major, and you get row $= 7 - \text{rank}$, col $= 7 - \text{file}$. The ranks land the usual way round, rank 8 on the top row — but **the files are mirrored**, H on the left and A on the right. Cell 0 is H8 and cell 63 is A1.

Nothing about the network minds. A fixed mirror of every input is just a relabelling of the board, and the convolutions learn whichever labelling they are shown. What matters is that training and inference agree *exactly*: an encoder that helpfully "fixes" the order would feed the trained weights a mirrored position, and the failure would be quiet — a network that has simply become much weaker.

There is one place it could have bitten, and happens not to. The policy head indexes actions by the **true** square numbers, 64 from + to, not the mirrored ones — so the input geometry and the output indexing disagree. It costs nothing because both heads flatten the trunk and pass through a fully-connected layer, which absorbs any fixed permutation of its input. A convolutional policy head emitting one logit per board square, as some AlphaZero implementations use, would have had to be told about the mirror explicitly.

The score is not an input. The planes carry no score at all. The sum of the two scores is recoverable — it is the number of holes — but the split is not. The network is expected to learn what it needs from the piece and hole configuration.

Moves use a flat AlphaZero-style action space of $64 \times 64 = 4096$ logits:

$$\text{action}(m) = 64 \cdot \text{from}(m) + \text{to}(m)$$

Roughly three quarters of that space is never legal. Illegal actions are not masked during training — they are simply never the target, so the softmax starves them — and the search applies the legal mask itself: the evaluator returns the logits of *exactly the legal moves*, in `Yolah::moves` order, and the search softmaxes over those. A pass maps to A1→A1, index 0, which is never a real move.

4 The network

4.1 Architecture

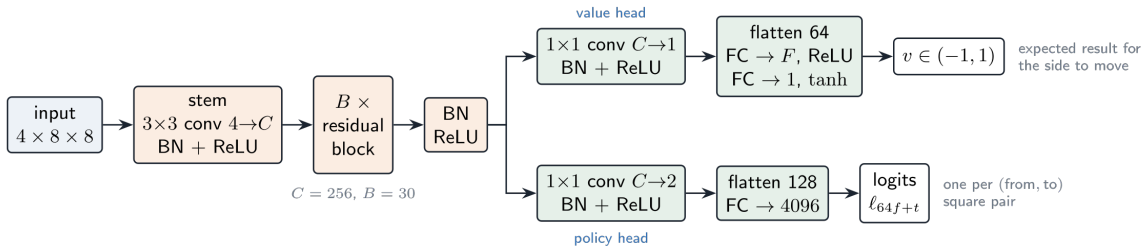
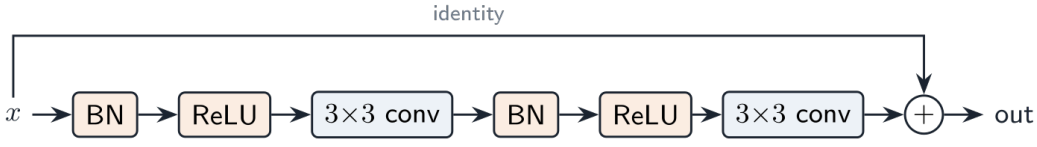


Figure 3: `Net` in `nnue/cnn_resnet_value_policy_chunked.py`. The shipped model is $C = 256$, $B = 30$, $F = 256$: about 35 M parameters and ≈ 4.5 GFLOP per position.

The trunk is a stem convolution followed by B pre-activation residual blocks and a final BatchNorm + ReLU. Both heads read the same $(C, 8, 8)$ feature map.



pre-activation (ResNet-v2): the BN sits *before* the convolution, so it cannot be folded into it at export time — the C++ backend keeps the folded affine (s, t) and applies $\max(0,\ sx + t)$ while gathering the im2col patches

Figure 4: The residual block is ResNet-v2 style: BN and ReLU come *before* each convolution.

```python
class ResBlock(nn.Module):
    def forward(self, x):
        residual = x
        x = self.conv1(torch.relu(self.bn1(x)))
        x = self.conv2(torch.relu(self.bn2(x)))
        return x + residual
```

The **value head** squeezes C channels to one with a 1×1 convolution, flattens the resulting 8×8 map, and runs two fully-connected layers into a tanh. The output is the expected result for the side to move; a winning probability is $(v + 1)/2$.

The **policy head** squeezes to two channels, flattens to 128 features, and projects to 4096 logits with a single fully-connected layer. That last layer holds $4096 \times 128 \approx 524\,\text{k}$ weights and is, per position, cheaper than one trunk block.

4.2 Training

Two scripts define the same network and differ only in how they feed it. `cnn_resnet_value_policy_chunked.py` is the one the shipped weights come from and the one `resnet_export.py` imports; `cnn_resnet_value_policy.py` is its sibling with a stock `DataLoader`, which is the right choice when the position cache fits in page cache. Everything in this section applies to both unless stated otherwise.

Both train the two heads at once on a supervised corpus of positions:

$$\mathcal{L} = \lambda_v \, (v - z)^2 \; + \; \lambda_p \, \mathrm{CE}(\ell, \; a^\star)$$

with $\lambda_v = \lambda_p = 1$, as in AlphaZero. The value target z is the game result from the point of view of the player to move at s; the policy target $a^\star$ is the single move actually played, so the policy loss is a plain cross-entropy over the 4096 logits. Terminal positions have no next move and carry `POLICY_IGNORE_INDEX = -1`, which `nn.CrossEntropyLoss(ignore_index=…)` skips.

Note the asymmetry with a true AlphaZero loop: here the policy target is a *one-hot move*, not a visit distribution. When the self-play loop of Section 6 is running, the target becomes the search's π and the loss term becomes $-\sum_a \pi(a) \log p(a)$ — the same cross-entropy, with a soft target.

The training scripts are built around one constraint: the dataset does not fit in memory. Positions live in three `np.memmap` files (`positions` as `uint8`, `values` as `int8`, `policies` as `int16`) written by `preprocess.py`, roughly 253 GB in total, and most of what is unusual in the code exists only to make that workable.

Why the chunked loader. Sampling that file uniformly at random — which is what a stock `DataLoader` does — becomes millions of scattered ~256-byte reads. On a spinning disk that is seek-bound, and the only rescue is pre-loading the whole 253 GB into page cache. `ChunkedShuffleLoader` instead reads ~1 GB **contiguous** chunks sequentially, shuffles *within* a chunk and reshuffles the chunk order every epoch. That is an approximate shuffle, but a chunk holds around 4 M positions, which is more decorrelation than a 2048-position batch can use. One background thread per rank does the reading and the batch construction while the main thread drives the GPU; the overlap is real because numpy's memcpy and CUDA's pin-memory both release the GIL, and the bounded queue between them is the ping-pong buffer.

The DDP trap. Every rank must yield *exactly* the same number of batches. Each backward pass is a collective all-reduce, so a rank that runs one step fewer leaves the others hanging forever.

Hence the floor division when the chunks are sharded across ranks — up to `world_size - 1` chunks are dropped — and the dropped trailing partial batch of each chunk.

Two more decisions that are easy to get wrong:

- The train/val split is **contiguous**, not random: positions are stored game by game, so a contiguous cut keeps whole games on one side of the split. A per-position random split would leak the same game into both halves.
- Mixed precision is selected on the compute capability, not on `torch.cuda.is_bf16_supported()` — that returns `True` on Turing, where bf16 is emulated and *slower* than fp32. Ampere and later get bf16 with the gradient scaler disabled; Turing gets fp16 with a live `GradScaler`.

The sibling script solves the same problem differently, and its two tricks are worth knowing if you ever go back to a `DataLoader`: `GameDataset.__getstate__` serialises only `(cache_dir, start, end)`, because a raw `np.memmap` pickles *by value* and a spawned worker would copy the whole backing file; and `LeanDistributedSampler` replaces the stock `DistributedSampler`, whose `torch.randperm(N).tolist()` turns a 4 GB index tensor into a ~30 GB list of Python ints **per rank**.

4.3 Export: folding BatchNorm

`resnet_export.py` turns one `.pt` checkpoint into three files. The interesting part is what it does to the BatchNorms.

In eval mode a BatchNorm is a per-channel affine map:

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta = s\,x + t, \qquad s = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}, \qquad t = \beta - \mu\, s$$

Where a BN **follows** a convolution — the stem, and the two 1×1 head convolutions — the scale folds straight into the convolution weights and the shift becomes the bias the convolution did not have:

$$W' = W \odot s, \qquad b' = t$$

Where a BN **precedes** a ReLU and a convolution — every pre-activation block, and the trunk's output BN — it cannot be merged into anything, so the export keeps (s, t) and the C++ backend applies $\max(0,\ sx + t)$ as it gathers the im2col patches, without an extra pass over the activations.

The three artefacts:

file	consumer	contents
`*.bin`	`nn::ResNetCPU`	flat little-endian float32, BN folded, convolutions laid out `(out, tap, in)` with `tap` $= 3\,dr + dc$
`*.ts`	`nn::ResNetTorch`	TorchScript module, traced in eval mode on a dummy $(1, 4, 8, 8)$ batch
`*.test.bin`	`alphazero_check`	a few positions with PyTorch's exact fp32 value and 4096 logits

The `(out, tap, in)` layout is not cosmetic — it is exactly the operand order the CPU backend's GEMM wants (Section 4.4).

The header is `"YOLAHRSN"`, a version, and then (in, C, B, F, A), so the C++ side never has to be told the architecture; it reads it.

4.4 The CPU backend

`nnue/resnet.cpp` is a from-scratch inference of the same network with Eigen and OpenMP, with no PyTorch at run time. It exists so the player always works, and as the reference the GPU backend is checked against.

Activations are kept **pixel-major** — `[position][pixel 0..63][channel]`, i.e. NHWC — which turns each 3×3 convolution into an im2col gather followed by one large GEMM:

$$\underbrace{\text{Out}}_{64n \times C} = \underbrace{\text{Col}}_{64n \times 9C} \cdot \underbrace{W^{\mathsf{T}}}_{9C \times C}$$

With $C = 256$ that is a 512×2304 by 2304×256 product per convolution — a shape Eigen's GEMM runs close to peak on with `-march=native -mfma`. The im2col copy itself is under 5 % of the layer cost, and the pre-activation BN+ReLU is applied *during* the gather, so the activations are read once.

Batching is done by splitting the batch into chunks of `MAX_CHUNK = 8` positions and giving the chunks to an OpenMP team, each thread with its own scratch buffers and Eigen running single-threaded inside. The backend is therefore happiest with **one** search thread handing it 32–128 positions at a time; several search threads must go through `nn::BatchingEvaluator` so their requests are merged rather than nested.

> ⚠ Denormals
>
> The CMake LTO link line does not pass `-ffast-math`, so FTZ/DAZ are not set and the trunk's denormal activations make inference roughly 30× slower. `nnue/resnet.cpp` sets FTZ/DAZ explicitly, per thread. Any new thread that calls into the network must do the same.

4.5 The libtorch backend

`nnue/resnet_torch.cpp` loads the TorchScript module and runs the whole batch as one `(n,4,8,8)` forward pass. It is compiled only with `-DENABLE_TORCH=ON` and a libtorch install (the one inside a pip `torch` package works: `-DCMAKE_PREFIX_PATH=<site-packages>/torch`). In half precision on an RTX-class GPU it is 20–50× the CPU backend at batch 256, which is why the search's `preferred_batch_size()` is backend-specific.

A mutex serialises the forwards: callers are batched upstream anyway, and libtorch does not love concurrent `forward` on the same module.

4.6 The seam: `nn::Evaluator`

```cpp
struct Request { const Yolah* state; const Move* moves; uint16_t nb_moves; };
struct Result  { float value; float logits[Yolah::MAX_NB_MOVES]; };

class Evaluator {
    virtual void evaluate(std::span<const Request>, std::span<Result>) = 0;
    virtual size_t preferred_batch_size() const = 0;
    virtual void reload(const std::string& weights_filename) = 0;
};
```

The search never touches a tensor. It hands over positions with their legal moves and gets back a value and the logits of *those* moves, in that order. The softmax, the temperature, and everything else are the search's business. Three implementations sit behind the interface:

- **`ResNetCPU`** and **`ResNetTorch`**, the two backends above.
- **`BatchingEvaluator`**, a wrapper. Each search thread enqueues its span and blocks; a worker thread fires as soon as the queue reaches `max_batch`, or every registered client is waiting, or `max_wait` has elapsed. With T search threads submitting B leaves each, the backend sees up to $T{\cdot}B$ positions per call. An *empty* request is a barrier join — a thread that has nothing to evaluate but wants to wait for the next batch, which is what a thread whose descents all collided does instead of spinning.

4.7 The network cache

`player/nn_cache.h` is a direct-mapped, lock-striped table of network outputs keyed by the Zobrist hash of `(board, side to move)` — exactly the network's input, so the cached value is a pure function of the key. It stores the value and the priors of the legal moves as 16-bit fixed point, and verifies the number of legal moves on lookup so a 64-bit collision cannot silently hand back someone else's priors.

It survives across moves and games (it depends only on the weights, so `reload_weights` clears it) and it is the second line of defence behind the node table of Section 5.6: it also catches positions that have fallen out of the search graph entirely.

5 Monte-Carlo tree search with a network

5.1 One simulation

A simulation walks down from the root picking children with the PUCT rule until it reaches a position the search has not expanded yet, asks the network what it is worth, and pushes that answer back up the path it came from. Classical MCTS would play a random game from the leaf instead; the network replaces both the rollout (the value head) and the move ordering heuristic (the policy head).

The four node states are `UNEXPANDED`, `EXPANDING` (a thread has claimed it and the network call is in flight), `EXPANDED`, and `TERMINAL`. The transitions are a `compare_exchange` on a single atomic byte, and `EXPANDED` is published with release semantics so that a thread observing it is guaranteed to see the fully built edge vector.

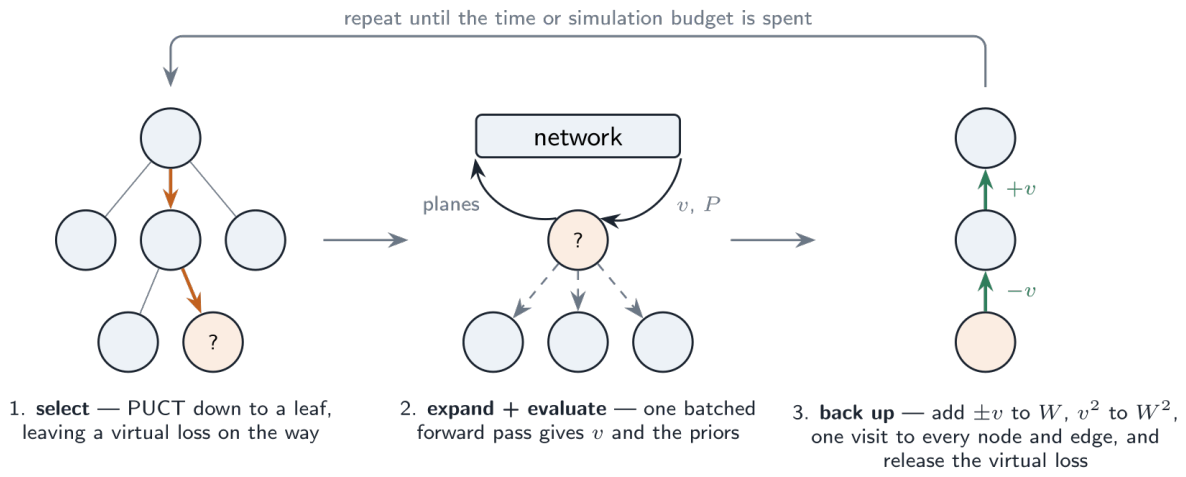


Figure 5: One simulation, in `Search::run_worker`. Steps 1 and 3 are per descent; step 2 is per batch.

5.2 PUCT, in KataGo's formulation

The child to descend into is

$$a^\star \;=\; \arg\max_a \left[\; Q(s,a) \;+\; c(s)\,\hat{\sigma}(s)\; P(s,a)\; \frac{\sqrt{W(s)+\varepsilon}}{1+W(s,a)} \;\right]$$

with $\varepsilon = 0.01$ (KataGo's `TOTALCHILDWEIGHT_PUCT_OFFSET`, which keeps the exploration term positive when nothing has been visited yet) and

$$W(s) \;=\; \sum_a W(s,a)$$

the total weight of the **edges leaving** s — note: not the node's own visit count, which is one larger. Every visit weighs 1 and every in-flight virtual loss also weighs 1.

The first term exploits, the second explores. $P(s,a)$ makes the exploration term follow the network's opinion, $\sqrt{W(s)}$ makes it grow as the parent is studied, and $1/(1+W(s,a))$ makes it collapse for a move already tried many times. In the limit, visit counts follow the policy tempered by the values.

5.2.1 The exploration coefficient

$$c(s) \;=\; c_{\text{puct}} \;+\; c_{\log}\,\log\!\left(\frac{W(s)+c_{\text{base}}}{c_{\text{base}}}\right)$$

Defaults are KataGo's `basicDecentParams`: $c_{\text{puct}} = 1.0$, $c_{\log} = 0.45$, $c_{\text{base}} = 500$. Setting $c_{\log} = 0$ recovers a constant c; AlphaZero's published variant is $c_{\log} = 1$ with $c_{\text{base}} = 19652$, which with any realistic budget is numerically a constant — at $W = 10^4$ the log term contributes about 0.4 over the whole search. With base 500 it is live from the first few hundred visits, which is the point: a node with a lot of visits should widen, because the top move's lead is by then well established and the remaining doubt is elsewhere.

5.2.2 The utility-stdev factor $\hat{\sigma}$

This is the piece that has no AlphaZero analogue. The exploration coefficient is scaled by how much the values coming back through this node *disagree*:

$$\hat{\sigma}(s) \;=\; 1 \;+\; \lambda \left(\frac{\sigma(s)}{\sigma_0} - 1 \right)$$

with $\lambda = 0.85$ (`cpuct_utility_stdev_scale`) and $\sigma_0 = 0.40$ (`cpuct_utility_stdev_prior`). $\sigma(s)$ is the standard deviation of the values backed up through s, regularised towards σ_0 with a weight of $w_p = 2$ visits:

$$\sigma(s)^2 \;=\; \frac{(\bar{u}^2 + \sigma_0^2)\, w_p \;+\; \overline{u^2}\, N(s)}{w_p + N(s) - 1} \;-\; \bar{u}^2, \qquad \bar{u} = \frac{W(s)}{N(s)}, \quad \overline{u^2} = \frac{W^2(s)}{N(s)}$$

Every node therefore carries a second accumulator $W^2 = \sum v^2$ alongside $W = \sum v$. The negamax sign flip disappears in the square, so the same v^2 is added at every ply of a backup — one extra `fetch_add`, no extra bookkeeping.

The effect is sharp and worth internalising. A node whose children all evaluate around $+0.99$ has $\sigma \approx 0.02$, so $\hat{\sigma} \approx 0.2$ and exploration is cut fivefold: the position is decided, stop spending on it. A node with one child at $+0.6$ and another at -0.4 has a large σ and gets *more* exploration than plain PUCT would give it.

> **i** σ_0 is not calibrated for this game
>
> KataGo's $\sigma_0 = 0.40$ is tuned for its utility scale, which is win/loss (± 1) **plus** score utility terms, for a radius of about 1.4. Yolah's values are pure win/loss in $[-1, 1]$, so observed stdevs run smaller and $\hat{\sigma}$ sits below 1 more often than KataGo intends. Sweeping σ_0 downwards (0.25–0.30) before drawing conclusions from A/B games is the obvious first experiment.

5.3 First play urgency

$Q(s, a)$ is undefined for a move that has never been tried. Assuming 0 is a disaster in a game where values are mostly far from 0: in a won position every unvisited move looks like a draw, i.e. terrible, and the search never widens; in a lost position every unvisited move looks like an improvement, and the search sprays.

The FPU value answers with the parent's own value, minus a penalty that grows as more of the policy mass has already been tried:

$$Q_{\text{fpu}}(s) \;=\; U(s) \;-\; r\sqrt{m(s)}, \qquad m(s) = \sum_{a:\, W(s,a) > 0} P(s, a)$$

$m(s)$ is the **visited policy mass**. When nothing has been tried, the penalty is zero and an unvisited move is worth exactly what the parent is worth. As the search works through the plausible moves, the bar for trying yet another one rises.

The baseline $U(s)$ is a blend of what the search has learned about s and what the network thought of it:

$$U(s) \;=\; \alpha\,\bar{q}(s) \;+\; (1-\alpha)\,v_{\text{net}}(s), \qquad \alpha = \min(1,\; m(s)^{p})$$

so a barely-explored node trusts the network and a well-explored one trusts its own statistics. Finally `fpu_loss_prop` can drag the estimate a proportion of the way towards a loss (-1); it defaults to 0.

The root uses `root_fpu_reduction = 0.1` rather than `0.2`: at the root, unlike everywhere else, being wrong about a move is expensive and there are visits to spare.

5.4 Backup and the sign convention

The one thing to keep straight in a negamax MCTS. A node's W accumulates values **from the point of view of the player who moved into it** — which is precisely the quantity the *parent* needs, because $Q(s,a) = W(s\cdot a)/N(s\cdot a)$ is read by the player to move at s. The network's v is for the player to move at the leaf, so the leaf itself receives $-v$, and the sign flips once per ply going up:

```
float v = -leaf_value;
const float v2 = leaf_value * leaf_value;   // same at every ply
for (size_t i = length; i-- > 0;) {
    path[i].node->w.fetch_add(v, relaxed);
    path[i].node->w2.fetch_add(v2, relaxed);
    path[i].node->n.fetch_add(1, relaxed);
    path[i].node->pending.fetch_sub(1, relaxed);
    if (Edge* e = path[i].edge) { e->n.fetch_add(1, relaxed); e->pending.fetch_sub(1, relaxed); }
    v = -v;
}
```

Terminal positions skip the network entirely and back up the exact result $\{-1, 0, +1\}$ from `terminal_value`, which compares the two scores from the point of view of the player to move.

5.5 Virtual loss, batching and collisions

A single leaf per network call would leave a GPU idle 99 % of the time. Each thread therefore gathers a whole batch of leaves before evaluating, which requires the descents to end up at *different* leaves — otherwise they would all follow the same argmax.

Every descent increments a `pending` counter on each node and edge it passes, and PUCT treats a pending visit as a **loss**:

$$Q(s,a) \;=\; \frac{W(s\cdot a) - \text{pending}(s\cdot a)}{N(s\cdot a) + \text{pending}(s\cdot a)}$$

so the next descent sees the branch as temporarily worse and goes elsewhere. When the batch comes back, `backup` converts each pending visit into a real one; a descent that is abandoned calls `undo_pending` instead.

Three things can happen when a descent reaches a node in the `EXPANDING` state:

- **it is one of our own batch's leaves** — the descent piggybacks on the same network call. It records *its own path*, because two descents can reach the same leaf along different routes (and under graph search, through different parents entirely); the same value is then backed up along each path separately;

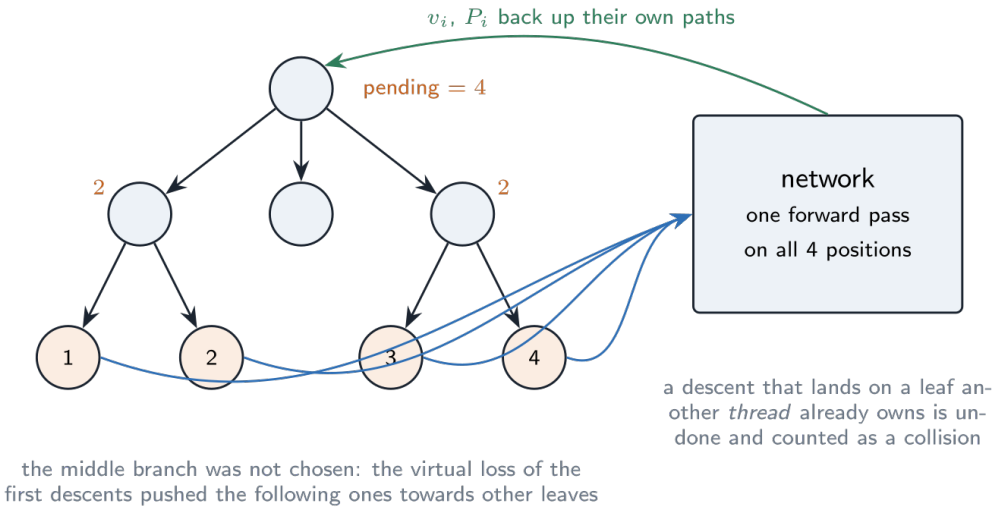


Figure 6: Gathering a batch. The `pending` counters are the virtual loss.

- **it belongs to another thread** — the descent is undone and counted as a collision. After `max_collisions` of those the batch is closed early;
- the state changed under us, in which case the loop simply re-reads it.

5.6 Graph search

Since move orders commute (Section 3), a tree stores the same position many times over and splits its statistics across those copies. The search instead keeps **one node per position**, in a sharded hash table, and lets several parents point at it.

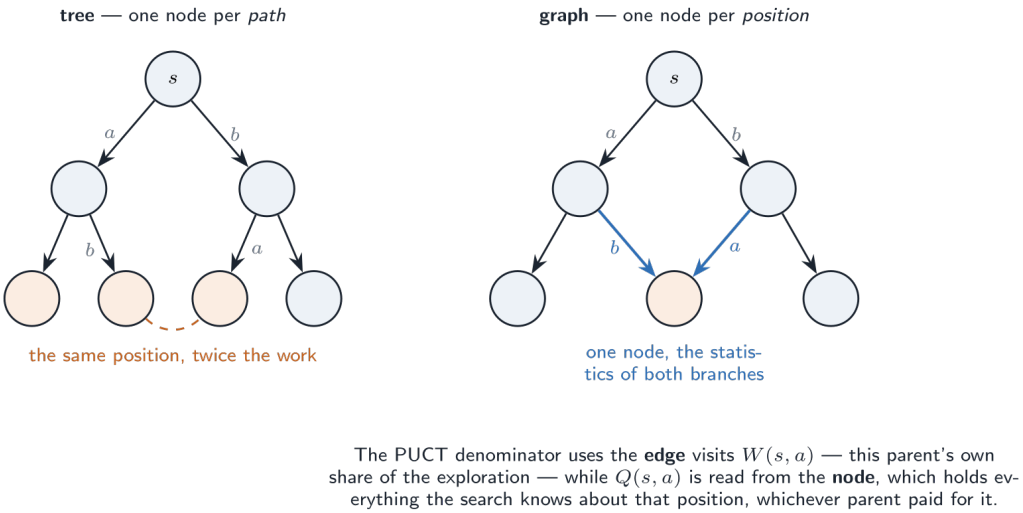


Figure 7: A tree duplicates transposed positions; a graph shares them.

The data structures split in two:

```
struct Node {                  // one per position
    atomic<uint32_t> n, pending;
    atomic<float>    w, w2;
```

```
    atomic<uint8_t>  state;
    float            net_v;
    pmr::vector<Edge> children;
};
struct Edge {                 // one per (position, move)
    Move action; float prior;
    atomic<Node*>    child;   // created on first descent
    atomic<uint32_t> n, pending;   // THIS parent's share
};
```

and PUCT reads from both:

- $W(s,a)$, the denominator, is the **edge** count — this parent's own share of the exploration, so a node that other parents have hammered does not look exhausted from here;
- $Q(s,a)$ is the **node**'s average — everything the search knows about that position, whoever paid for it.

That split is exactly KataGo's `childWeight = rawChildWeight · edgeVisits / childVisits`, which for a search weighting every visit by 1 collapses to "the edge visits".

Child nodes are created **lazily**, on the first descent through their edge, not when the parent is expanded. Expansion allocates $|\text{moves}|$ edges (16 bytes each) instead of $|\text{moves}|$ full nodes, which is the difference between ~3 200 nodes and ~20 000 nodes for a 3 200-simulation search.

5.6.1 The node key

The Zobrist hash covers the board and the side to move, and nothing else — which is right for the network cache, because that is exactly the network's input. It is *not* enough for a node, which holds statistics and whose terminal value depends on the score split that passes can skew (Section 3). The node key therefore mixes the black score in:

```
uint64_t node_key(uint64_t zobrist_hash, const Yolah& state) {
    return zobrist_hash ^ mix(uint64_t(state.score().first) + 1);
}
```

The white score is implied: the two sum to the number of holes, which the board already determines.

The hash is maintained incrementally during a descent with `zobrist::update`, so linking an edge costs a multiply and a table probe, not a 64-square scan.

5.6.2 Tree reuse and garbage collection

With a node table, re-rooting stops being a search. `Search::reuse` looks the new position up by key; if it is there, it becomes the root with every statistic it has accumulated — at whatever depth it sat in the old graph, reached through whatever move order. The two-ply scan the tree version needed (root, children, grandchildren) is gone.

What re-rooting does need is a sweep, or memory grows without bound over a game. `collect()` marks everything reachable from the new root — a plain recursion, safe because the graph is acyclic — and destroys the rest. This runs once per move, and the mark is a generation counter rather than a per-node flag reset.

Setting `graph search: false` keeps the tree behaviour (every edge gets a fresh node under a unique key, and `reuse` falls back to the two-ply scan), which is how the two are A/B-compared.

5.7 At the root

Three things happen at the root and nowhere else.

5.7.1 Dirichlet noise

For self-play only, the root priors are mixed with Dirichlet noise so that repeated self-play games do not all follow the same line:

$$P'(s_0, a) \;=\; (1-\epsilon)\, P(s_0, a) \;+\; \epsilon\, \eta_a, \qquad \eta \sim \mathrm{Dir}(\alpha)$$

with $\epsilon = 0.25$ and $\alpha = 0.3$ typical. The noisy priors live in `root_priors` and are used by PUCT *and* reported as the priors — the tree's stored priors are untouched, so a re-rooted subtree is not contaminated.

5.7.2 Forced playouts

Noise only helps if the move it promotes actually gets tried more than once. A single bad evaluation is enough for PUCT to drop a move for the rest of the search, which wastes the noise. So, at the root, any child that has been visited **at least once** is forced up to

$$n_{\text{forced}}(a) \;=\; \sqrt{k\, P(s_0, a)\, W(s_0)}$$

visits ($k = 2$ in the KataGo paper) by giving it an overwhelming selection score while it is below that bar. Off by default (`forced_playouts_k = 0`); it belongs to self-play, and it is also automatically disabled on the fast searches of Section 5.8.

5.7.3 Policy target pruning

Forced playouts are deliberate distortions of the visit distribution — training on them would teach the network the noise. They are taken back out afterwards: for every child except the most-explored one, ask what visit count PUCT would *retrospectively* have wanted, by inverting the selection formula at the best child's selection value

$$V^\star = Q(s_0, a^\star) + c\,\hat{\sigma}\,\sqrt{W + \varepsilon}\;\frac{P(s_0, a^\star)}{1 + W(s_0, a^\star)}$$

$$n'(a) \;=\; \min\!\left(n(a),\; \left\lceil \frac{c\,\hat{\sigma}\,\sqrt{W+\varepsilon}\;P(s_0, a)}{V^\star - Q(s_0, a)} - 1 \right\rceil\right)$$

A child whose value alone already beats $V^\star$ keeps all its visits; one that only got there because it was forced collapses towards zero. These *play values* — not the raw visit counts — are what becomes π and what the move is chosen from. KataGo applies this at the root unconditionally, not only in self-play, and so does this implementation.

5.7.4 LCB move selection

The most-visited move is not always the best one to play: a move can accumulate visits early and then stop improving, while a less-visited sibling has a genuinely better and tighter value. The move played is instead the one with the best **lower confidence bound**

$$\mathrm{lcb}(a) \;=\; Q(s_0,a) \;-\; \lambda_{\mathrm{lcb}}\,\hat{\sigma}_{\bar{Q}}(a), \qquad \hat{\sigma}_{\bar{Q}}(a) = \sqrt{\frac{\mathrm{Var}(a)}{\mathrm{ess}(a)}}$$

with $\lambda_{\mathrm{lcb}} = 5$, restricted to children holding at least `min_visit_prop_for_lcb` = 20 % of the leader's play value — below that the bound is meaningless. The variance comes from the child node's W and W^2, regularised by a prior saying the variance could be as large as the whole utility range, with a weight that vanishes as $1/n^2$:

$$\overline{u^2} \leftarrow \frac{\overline{u^2}\,n + (\overline{u^2} + R^2)\,w_p}{n + w_p}, \qquad w_p = \frac{1}{n^2}, \quad R = 1$$

The winner does not simply override the choice: following KataGo, it is given just enough play value to beat every other child, which keeps π, the sort order and the move choice consistent with one another.

5.8 Playout cap randomization

A self-play move needs a large budget only if its policy target is going to be recorded. Most of the value of a self-play game is in the *game* — the sequence of positions and its result — and that only needs decent play.

So, with probability `playout_cap_fast_prob`, a search runs `nb_simulations_fast` simulations with **no root noise, no forced playouts**, and reports `full_search = false`; the driver records the position and the result but no policy target. The remaining moves get the full budget, the noise, the forced playouts, and a recorded π. The paper's numbers are roughly 75 % fast at a quarter of the budget, which buys most of a full-budget game for a fraction of the network calls.

Off by default: it needs both `nb_simulations` and `nb_simulations_fast` set.

5.9 Choosing the move, and the policy target

```
play values ──► normalise ──► π   (training target)
            └► argmax          (competitive play)
            └► sample ∝ π^(1/τ) (self-play opening, ply < temperature_cutoff)
```

τ is the sampling temperature and `temperature_cutoff` the ply after which the search stops sampling and starts playing the best move. If the search could not complete a single simulation — a time budget too small for one network call — the priors stand in for π and the highest-prior move is played.

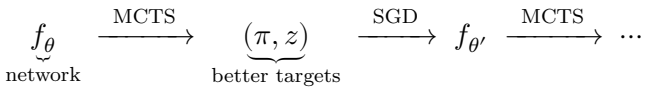
6 Learning by self-play

The network of the previous chapters was trained by *supervised* learning: it imitated the moves and the results of an existing corpus of games. It can therefore never become much stronger than whoever played those games. This chapter describes the loop that removes that ceiling: the engine plays against itself, learns from those games, plays better games with what it learned, and so on — AlphaZero's recipe, with the refinements of KataGo.

The chapter follows the data. First the idea (Section 6.1) and the processes (Section 6.2); then self-play, which produces the data (Section 6.3); the trainer, which consumes it (Section 6.4); the evaluation matches, which measure the result (Section 6.5); how it all runs on the cluster and how the resources were sized (Section 6.6, Section 6.7); and finally KataGo's auxiliary heads, the most promising next step (Section 6.9).

6.1 The idea

Look at what one search does to the network's opinion. The network gives a prior $p(\cdot \mid s)$ and a value $v(s)$; 800 simulations later the search returns a distribution $\pi(\cdot \mid s)$ over the moves and a value. Both are **better** than what went in — that is the whole point of searching. So the search is a *policy improvement operator*: it maps a network to a stronger player. If we now train the network to predict π (instead of p) and the final result z of the game (instead of v), the next network starts where the last search ended. Iterate:



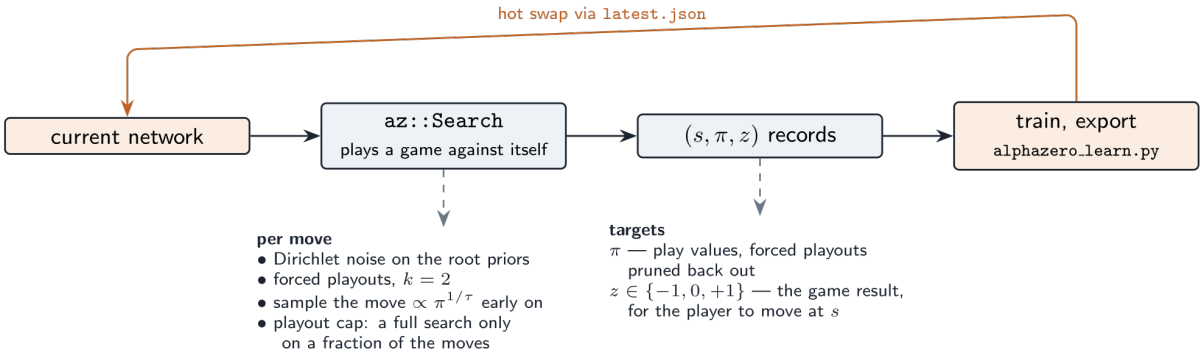


Figure 8: One turn of the loop: the network plays, the games become targets, the targets train the network.

This is *generalised policy iteration*: the search does the policy improvement step and the gradient descent does the policy evaluation step. No human games, no hand-made evaluation: the only knowledge injected is the rules, through the legal moves and the final score.

Why does it not just go round in circles? Because the two targets pull the network towards something strictly better than itself. π is the result of looking ahead with the current network — a network that predicts π directly has "compiled" 800 simulations of look-ahead into one evaluation. z is the *actual* result of the game, which no amount of self-deception can change: a network that believes a position is won while it keeps losing from there gets corrected. Each iteration the searches become stronger because the network guiding them is better, and the targets become better because the searches are stronger.

Our starting point is not a random network but the supervised `cnn_resnet_256x30_value_policy.pt`. That changes the tuning (small learning rate, no need to explore wildly at first) but not the algorithm.

6.2 The pieces and how they talk

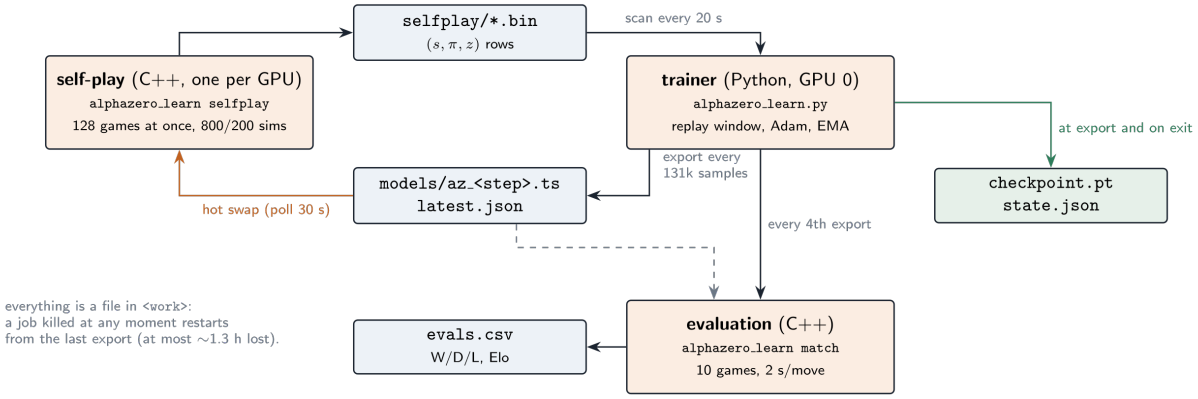


Figure 9: The learning loop. Three kinds of process, and every exchange between them is a file in the work directory.

Three programs run at the same time on one node (Figure 9):

1. **Self-play** — `alphazero_learn selfplay`, C++ (`player/alphazero_selfplay.cpp`), one process per GPU. It plays games with the newest network and writes training rows to `selfplay/*.bin`.
2. **The trainer** — `nnue/alphazero_learn.py`, Python. It is also the orchestrator: it launches and supervises the other two. It keeps a *replay window* of recent rows, takes gradient steps, and every `--export-every` training samples exports a new network (`models/az_<step>.ts`) and announces it in `latest.json`. Self-play picks the new network up at its next poll.
3. **Evaluation** — `alphazero_learn match`, C++ (`test/alphazero_learn_main.cpp`), started by the trainer every 4 hours. It plays 20 games of 2 s per move between the newest network and the previously evaluated one and appends the result to `evals.csv`.

Why files rather than sockets or shared memory? Because the run must survive being killed. A cluster job ends after two weeks and the loop must continue in the next one. With every exchange being a file that is renamed into place once it is complete (a POSIX rename is atomic), a process that dies at any moment leaves only complete files behind. The next job re-reads them and continues (Section 6.6).

There is no *gating*. The original AlphaGo Zero only promoted a new network after it beat the current one by 55 %; AlphaZero and KataGo dropped this and always self-play with the latest network, which is simpler and was found to be at least as good. The evaluation matches here are a **thermometer**, not a gate: they tell you whether learning progresses, they do not steer it.

6.3 Self-play: many games, one GPU

6.3.1 Why many games at once

A 30-block network is only efficient on the GPU with large batches: about 7 600 positions/s at batch 256 in fp16 on an RTX 3080, a small fraction of that at batch 8. A single MCTS, on the

other hand, degrades when it evaluates many leaves at once: with a batch of B leaves it must choose B descents *before* knowing any of their results (virtual loss), and beyond a few dozen leaves the descents collide and wander.

The solution is to decouple the two: run **many independent games**, each with its own single-threaded `az::Search` asking for only 8 leaves at a time, and let an `nn::BatchingEvaluator` merge the requests of all the games into one GPU call (Figure 10). With 128 games the GPU sees batches of up to 1024 positions, while each search behaves almost like a sequential one.

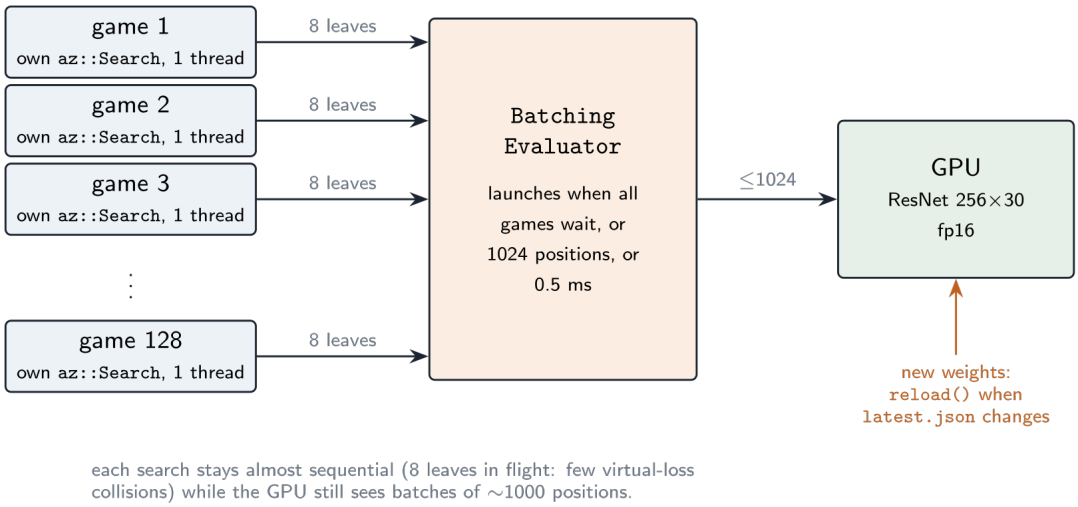


Figure 10: Many games share one network. Each search is almost sequential; the batcher gives the GPU large batches.

The batcher launches a GPU call as soon as *every* game is waiting, or the maximum batch (8 × number of games) is queued, or 0.5 ms have passed. When a game ends, its thread immediately starts the next one, so the number of games in flight stays constant; `set_nb_clients` is lowered only when a thread really exits (at the end of the run), otherwise the batcher would wait 0.5 ms for a client that will never come.

How many games? Enough to saturate the GPU, no more. On the RTX 3080, 128 games already keep it at 95 % and reach ∼8 500 evaluations/s; 256 games do not do better (the batches get larger but the GPU was already busy). Two processes of 64 games on the same GPU do *worse*, ∼6 000/s in total: each batch is half as large and the two processes contend for the device. Hence one process per GPU, with as many games as the GPU can absorb: 256 on the 48 GB cluster GPUs, which (A40 and RTX 8000) should have 2–2.5 times the fp16 throughput of a 3080 (Section 6.7).

The code of one game is short enough to read in full:

```
Yolah y;
std::vector<TrainingSample> samples;
search.reset();                                  // new game: forget the graph
while (!y.game_over() && !stop.load()) {
    const uint64_t g = generation.load(std::memory_order_acquire);
    if (g != seen) {                             // the network changed:
        search.reset();                          //   the graph's priors and values
        search.clear_cache();                    //   and the cached evaluations are stale
        seen = g;
    }
    const SearchResult r = search.search(y);     // 800 or 200 simulations
    if (r.full_search && !r.children.empty())    // only full searches are recorded
        samples.push_back(make_sample(y, r, model_step.load()));
```

```
    y.play(r.best_move);                          // sampled ∝ π^(1/τ) early, best later
}
// ... then z is filled in for every sample, and the game goes to the writer
```

6.3.2 The self-play settings, and why

`config/alphazero_mcts_selfplay_player.cfg`:

key	value	why
`nb simulations`	800	the budget of a *full* search (AlphaZero)
`playout cap fast prob` / `nb simulations fast`	0.75 / 200	playout cap randomization, Section 5.8
`dirichlet epsilon` / `alpha`	0.25 / 0.3	root noise: tries moves the prior would never try
`temperature` / `temperature cutoff`	1.0 / 20	the first 20 plies are *sampled* $\propto \pi$: diverse openings
`forced playouts k`	2	every root move tried at least $\sqrt{k\,PN}$ times
`policy target pruning`	true	… and those forced visits are removed from π again
`batch size`	8	leaves per search per network call (see above)
`nn cache`	16 MB	per game; the cluster job raises it to 32 MB (768 games × 32 MB = 25 GB)

Playout cap randomization deserves a word, since it shapes the data. Value targets are cheap: every position of a game gets the game result z. Policy targets are expensive: a good π needs a big search. KataGo's observation is that one can play *most* moves with a small search (200 simulations, no noise) — the game advances four times faster — and do a full 800-simulation search, and record a training row, only on a random 25 % of the moves. More games per hour means more value targets and more diverse positions; the policy targets that are recorded all come from full searches. The fast moves are **not recorded at all**: their π would be a poor target, and recording only their z would over-weight the value head.

Per recorded row this costs, on average, one full search plus three fast ones: $800 + 3 \times 200 = 1\,400$ simulations, a little less in network calls since tree reuse and the network cache save some. Measured: ~1 200 network evaluations per training row. That single number converts any GPU's evaluation rate into a data rate (Section 6.7).

6.3.3 What a training row contains

Each recorded position becomes a fixed-size, 336-byte record (`az::TrainingSample` in `player/alphazero_selfplay.h`, mirrored field by field by `SAMPLE_DTYPE` in `alphazero_learn.py`):

field	type	meaning
`black`, `white`, `empty`	$3 \times$ `uint64`	the bitboards — exactly the network's input
`turn`	`uint8`	0 = black to move, 1 = white
`z`	`int8`	final result for the side to move: $-1, 0, +1$
`nb_moves`	`uint8`	number of legal moves stored
`root_q`	`float`	the search's value of the position (monitoring)
`model_step`	`uint32`	which network played this move
`action[75]`	`uint16`	policy index `from*64+to` of each legal move
`prob[75]`	`uint16`	$\pi(a) \cdot 65535$, rounded

π is stored sparsely, over the legal moves only (at most 75 in Yolah), quantised to 16 bits — a resolution of 1.5×10^{-5}, far below the noise of an 800-visit estimate. A dense 4096-float target would take 16 KB per row instead of 336 bytes.

The value target is filled in when the game ends. In Yolah every move scores one point for the player who makes it, so the final scores are the numbers of moves each side managed to make. A position where colour c is to move gets

$$z = \operatorname{sign}\left(\text{score}_c - \text{score}_{\bar{c}}\right) \in \{-1, 0, +1\},$$

the same convention as the network's value head and the search's `terminal_value`.

Rows are grouped in files of about 2 000 (or every 10 minutes), named `<milliseconds since 1970>_<host>_gpu<k>_<seq>.bin`. Sorting the names sorts the files by age, across GPUs, hosts and jobs, which is all the trainer needs.

6.3.4 Hot-swapping the network

The supervisor thread of `run_selfplay` reads `latest.json` every 30 s. When the step changes it calls `evaluator.reload(path)`. That is safe while hundreds of games are searching: the libtorch backend takes its own mutex both in `evaluate` and in `load`, so a reload simply waits for the batch in flight and the next batch runs on the new weights. The supervisor then bumps an atomic `generation`; each game notices it before its next move and throws away its graph and its network cache, whose values came from the old weights. The rest of the game is played by the new network — a game may thus mix two consecutive networks, which KataGo also accepts: consecutive networks differ by a few hundred gradient steps.

6.4 The trainer

6.4.1 The replay window

Which rows should the network learn from? Only the most recent ones are made by a network close to the current one; old rows describe the play of a weaker player and their π is a worse target. But too small a window over-fits the last few thousand games. KataGo's answer (`python/shuffle.py`) is a window that grows **sub-linearly** with the total number of rows N generated so far:

$$W(N) = N_0 \left(1 + \beta \frac{(N/N_0)^\alpha - 1}{\alpha}\right), \qquad \alpha = 0.75,\ \beta = 0.4.$$

For $N \le N_0$ everything is used. After that the window keeps growing — a stronger network deserves more data to fit — but slower and slower, so the fraction of the history in use shrinks: with KataGo's $N_0 = 2.5 \cdot 10^5$ (the cluster setting), W is 1.1 M rows (31 %) at $N = 3.5$ M and 12 M (12 %) at $N = 100$ M (Figure 11). The derivative tells the same story: $W'(N) = \beta\,(N/N_0)^{\alpha-1}$ is the fraction of each new batch of rows that *enlarges* the window rather than replacing its oldest rows — 40 % at the start, 9 % after 100 M rows. W is also capped by `--max-window` (16 M rows, 5.4 GB of RAM on the cluster).

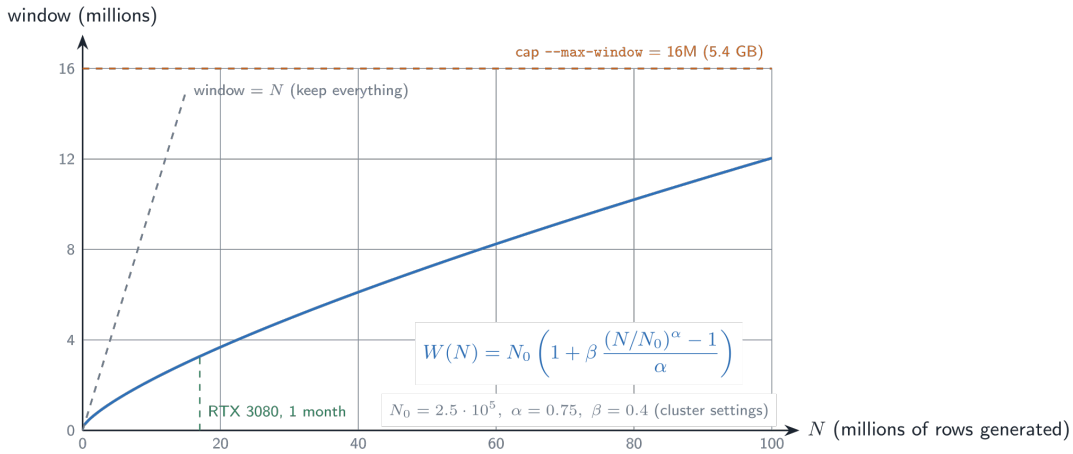


Figure 11: Replay window size as a function of the amount of data generated, with the cluster settings.

The implementation is a **ring buffer** of `--max-window` rows. New files are appended at the head; a batch draws uniform indices among the last $W(N)$ rows:

```
def sample(self, batch_size, rng):
    w = self.window_size()
    back = rng.integers(1, w + 1, size=batch_size)      # 1 = newest row
    return self.buf[(self.head - back) % self.capacity]
```

On a restart, only the files that can still be in the buffer are read (the tail of the history, newest first until the capacity is reached); the headers of the others are enough to know N.

6.4.2 How much to train: the reuse ratio

The trainer and self-play run concurrently, so something must keep them in step. If the trainer ran freely it would take thousands of steps on the first rows and over-fit them; if it were too slow, self-play would keep playing with an outdated network. The rule is a **fixed number of training samples per generated row** (`--reuse`, default 4):

$$\text{samples trained} \ \le\ r \cdot N.$$

When the trainer reaches the bound it sleeps until more rows arrive. With $r = 4$ each row is drawn about 4 times while it is in the window (in expectation; the draws are random). Training starts when $N \ge N_0$.

The trainer is never the bottleneck: one GPU trains the 256×30 network at ~3 400 samples/s (RTX 3080, batch 1024, 4.9 GB), while even three cluster GPUs of self-play generate a few tens of rows per second, i.e. a hundred or two training samples per second at $r = 4$.

6.4.3 The loss

For a row (s, π, z) and the network output (v, ℓ) — value and the 4096 logits — the loss is the AlphaZero one with a **soft** policy target:

$$\mathcal{L}(\theta) \;=\; \lambda_v \,(v_\theta(s) - z)^2 \;-\; \lambda_p \sum_{a \,\in\, \text{legal}(s)} \pi(a)\, \log p_\theta(a \mid s), \qquad p_\theta(a \mid s) = \frac{e^{\ell_a}}{\sum_{b=1}^{4096} e^{\ell_b}}.$$

With $\lambda_v = \lambda_p = 1$. Three remarks:

- The softmax runs over all 4096 logits, like in supervised training; the illegal moves are simply never targets, so their logits are pushed down. The search itself only ever reads the legal ones.
- The policy term is a cross-entropy, which decomposes as $H(\pi, p) = H(\pi) + \mathrm{KL}(\pi \,\|\, p)$. The entropy $H(\pi)$ of the targets does not depend on θ, so what the gradient reduces is the KL divergence. The trainer logs both `p` (the cross-entropy) and `kl` — the latter is the interpretable one: 0 means the network already predicts what the search finds.
- The gradient of the policy term with respect to the logits is simply $\partial\mathcal{L}/\partial\ell_a = \lambda_p\,(p_\theta(a \mid s) - \pi(a))$: every logit is pushed up or down by how much the network under- or over-estimates what the search found.

In code, with the sparse targets gathered from the log-softmax:

```
v, logits = net(x)
logp   = F.log_softmax(logits.float(), dim=1).gather(1, actions)   # (B, 75)
p_loss = -(probs * logp).sum(1).mean()                              # padding has prob 0
v_loss = F.mse_loss(v.float(), z)
loss   = args.value_weight * v_loss + args.policy_weight * p_loss
```

6.4.4 Blending the result with the search value

The value head learns to predict z, the result of the game. For a given position that target is **exact but noisy**: everything played afterwards counts in it, and in Yolah — a third of draws, most wins by one or two points (Section 6.7.2) — a position ends up labelled 0 or +1 by the accidents of the rest of the game as much as by its own merit.

Every row also carries a second estimate of the same quantity: q, the value the full search (800 simulations) found for the position when it was played (`root_q`, for the side to move, the convention of z and of the value head). q summarises hundreds of continuations, so it is **much less noisy** — **but biased**: it comes from the current network, and inherits its errors. The option `--q-weight` w trains the value on a blend of the two:

$$\text{target} = (1 - w)\, z + w\, q, \qquad \mathcal{L}_v = (v_\theta(s) - \text{target})^2.$$

It is the classic bias–variance trade-off of reinforcement learning: z is the Monte-Carlo return, q a bootstrapped estimate — the temporal-difference side, which KataGo's short-term value targets (Section 6.9) develop further. Some AlphaZero re-implementations report a gain from averaging

the two. w must stay well below 1: with q alone, the network would learn only from itself and could lock into its own mistakes; the share of z keeps it anchored to real results.

On local test games, q and z are correlated at 0.82, the blend ($w = 0.5$) lowers the variance of the target from 0.69 to 0.49, and the positions of the drawn games are far from being all "draws" in the eyes of the search (q has a standard deviation of 0.27 on them). The option costs nothing: the C++ side does not change — `root_q` has been recorded in every row since the first version of the format — and it applies at once to the rows already generated.

```
z = torch.from_numpy(rows["z"].astype(np.float32)).to(dev)
if self.q_weight > 0:
    q = torch.from_numpy(rows["root_q"].astype(np.float32)).to(dev).clamp(-1.0, 1.0)
    z = (1.0 - self.q_weight) * z + self.q_weight * q
```

On the cluster: `sbatch --export=ALL,Q_WEIGHT=0.5 alphazero_learn.sh`; the job's header and log print the target in use.

6.4.5 Optimiser and numerics

- **Adam**, learning rate 10^{-4} (the supervised run ended its cosine schedule at 10^{-5} from 10^{-3}), L2 weight decay 10^{-4} as in supervised training, a linear warm-up over the first 500 steps (the optimiser's moment estimates start from zero after a fresh start), gradient norm clipped at 5.
- **Mixed precision** chosen as in the supervised trainer: bf16 on Ampere and newer (the cluster's A40), fp16 with a gradient scaler on Turing (its RTX 8000, whose bf16 is emulated and slow). Consecutive jobs of one run may land on either, so the checkpoint must survive the switch: a bf16 job saves the state of a *disabled* gradient scaler, which is empty and which an enabled scaler refuses to load. The trainer then simply starts the scaler afresh — its scale adapts within a few steps. (`--amp` forces one or the other; that is how both switches were tested.)
- **Batch 1024**, channels-last memory layout (faster convolutions on tensor cores), `cudnn.benchmark` on — without it, cuDNN's default algorithms for this layout were an order of magnitude slower in our measurements.
- BatchNorm is in training mode: its running statistics follow the self-play data distribution.

6.4.6 Data augmentation: the 8 symmetries

The rules of Yolah do not change if the board is rotated by a quarter turn or mirrored: pieces move like queens and the initial position is symmetric. The position s and its image $g(s)$ under any of the 8 symmetries g of the square (the dihedral group D_4) therefore have the same value, and the policy of $g(s)$ is the policy of s with every move transformed by g:

$$v(g\,s) = v(s), \qquad \pi(g\,a \mid g\,s) = \pi(a \mid s).$$

Each training row is shown to the network under a random one of the 8 transformations — eight times more data for free, and a network pushed towards the symmetry it should respect (Figure 12).

The only subtlety is bookkeeping. The network input stores square q in plane cell $c = 63 - q$ (Section 3), and moves are indexed by *squares*. The tables are therefore computed on cells and converted:

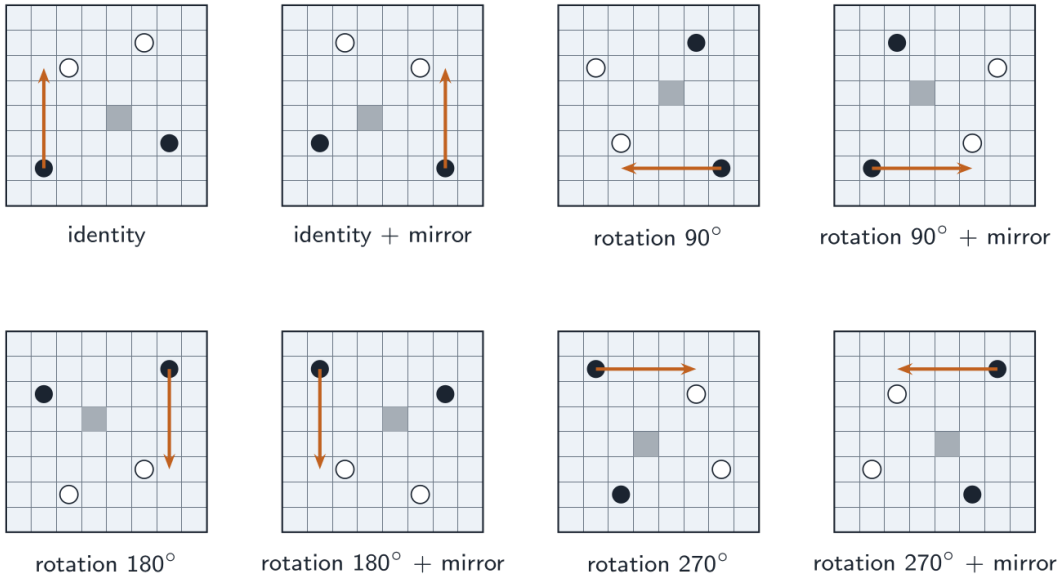


Figure 12: One position and one move under the 8 symmetries of the board. The value is the same for all eight; the move turns with the board.

```
cells = torch.arange(64).view(8, 8)
src = [f(rot90(cells, k)) for k in 0..3 for f in (identity, flip)]  # (8, 64)
# new plane cell j takes the value of old cell src[g, j]
inv = argsort(src)                          # inv[g, old cell] = new cell
sq  = 63 - inv[:, 63 - arange(64)]          # old square -> new square
act = sq[:, :, None] * 64 + sq[:, None, :]  # (8, 4096): move index -> move index
act[:, 0] = 0                               # index 0 is the pass, and stays the pass
```

The planes are permuted with one `gather`, the move indices with one table lookup, both on the GPU. The tables were checked against the rules: for 200 self-play positions and all 8 symmetries, the transformed legal moves of s are exactly the legal moves of $g(s)$ computed by `server/yolah.py`.

6.4.7 Averaging the weights before export

The weights after the last gradient step are a noisy point: SGD bounces around the minimum. KataGo exports a *stochastic weight average* of recent snapshots; here we keep an **exponential moving average** of the weights, updated after every step:

$$\bar{\theta} \leftarrow \mu\,\bar{\theta} + (1-\mu)\,\theta, \qquad \mu = 0.995.$$

Unrolled, $\bar{\theta}_t = (1-\mu)\sum_{k\geq 0} \mu^k\,\theta_{t-k}$: step $t-k$ weighs μ^k, the weights have mean age $\mu/(1-\mu) \approx 200$ steps. It is $\bar{\theta}$ that is exported and played by self-play, θ that keeps training. The BatchNorm running statistics are averaged the same way (`AveragedModel(..., use_buffers=True)`), so that the exported network is consistent.

6.4.8 Export and publication

Every `--export-every` training samples (524 288 = 512 steps of 1024 on the cluster; with 4 samples per row, one network every 131 072 new rows):

1. `az_<step>.pt` — the plain `state_dict` of $\bar{\theta}$ (it can be fed to `resnet_export.py` for the CPU `.bin` backend);
2. `az_<step>.ts` — the network traced with `torch.jit.trace` in eval mode, exactly like `resnet_export.py` does, for the libtorch backend;
3. `latest.json` rewritten — self-play switches within 30 s;
4. `checkpoint.pt` (weights, EMA, optimiser, gradient scaler) and `state.json` (counters, evaluation state) rewritten.

Every write goes to a temporary name and is `os.replace`d into place.

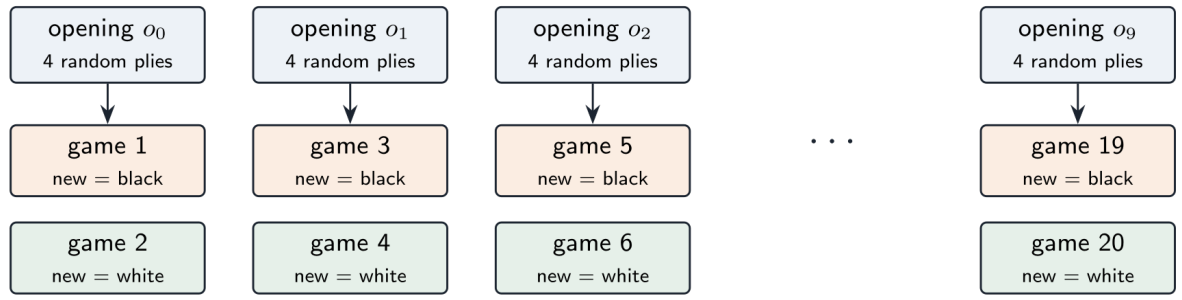
A 256×30 network is 141 MB per file and a month produces hundreds of exports, so old files are pruned (`cleanup_models`): an export that was never evaluated disappears at the next export; an evaluated one keeps its `.pt` — the history of the run, at 141 MB per evaluation — but loses its `.ts` once it can no longer be an opponent. If it is needed again (`--eval-opponents lag:K`), `Evaluator.ensure_ts` re-traces it from the `.pt`; the re-traced module gives bit-identical outputs. `--keep-all` disables pruning.

6.5 Measuring progress

6.5.1 The match protocol

Every 4 hours (`--eval-hours`; the first export of a run is evaluated at once), the newest network plays **20 games** against an older one, **2 seconds per move**, both sides using the competitive settings of `config/alphazero_mcts_eval_player.cfg` (no noise, no sampling, LCB). The opponent is by default the **previously evaluated network** (`--eval-opponents previous`; `initial` and `lag:K` are also available and can be combined). The schedule is in wall-clock time rather than in exports because the export rate depends on the GPUs, which differ between the development machine and the cluster; `--eval-every K` switches to one evaluation every K exports.

A detail matters here. Two deterministic players repeat the same game from the same position: 10 games with the new network as black would be *one* game played ten times. So each pair of games starts from its own opening, and the new network plays it once with each colour (Figure 13). The pair cancels the bias of the opening, whichever side it favours — and the first player's advantage in Yolah is large (Section 6.7.2). The openings were at first random (4 plies); they now come from a book of balanced openings (Section 6.5.3), after which the colour decides far fewer pairs.



Both players search 2 s per move, no noise, no sampling: from a given position they are (almost) deterministic. Without the random openings the 10 games with the same colours would be the same game ten times. The pair of games sharing an opening cancels its bias (one colour may be favoured by it). Score $s = (W + D/2)/20$.

Figure 13: The 20 evaluation games: 10 openings, each played once with each colour.

A match takes $20 \times 55 \times 2$ s ≈ 37 minutes. It runs on the last GPU while that GPU also plays self-play games: both players suffer the same contention, so the comparison is fair, but "2 s per move" buys fewer simulations than on an idle GPU.

6.5.2 From a score to Elo

With W wins, D draws and L losses out of $n = 20$ games, the score is $s = (W + D/2)/n$. Under the Elo model the expected score of a player Δ points stronger is $1/(1 + 10^{-\Delta/400})$; inverting,

$$\Delta = 400 \log_{10} \frac{s}{1 - s}.$$

A 20–0 result would give $\Delta = \infty$, so the estimate uses a half-game prior: $\tilde{s} = (W + D/2 + 1/2)/(n + 1)$, which caps $|\Delta|$ at 645. Each evaluation adds its Δ to the opponent's rating, giving a **chained Elo** for every evaluated network relative to the initial one (`model_elo` in `evals.csv`).

> ⚠ Twenty games is a thermometer, not a microscope
>
> The standard error of a score over n games is at most $\sqrt{s(1-s)/n}$, i.e. 0.11 for $n = 20$ near $s = 0.5$ (less if there are many draws). Through the slope of the Elo curve at $s = 0.5$, $d\Delta/ds = 1600/\ln 10 \approx 695$, that is **±78 Elo** for one evaluation. The chained Elo accumulates the errors of every link like a random walk: $\pm 78 \cdot \sqrt{k}$ after k links. Read the **trend** over many evaluations, not any single one; a run that learns shows a chained Elo that climbs steadily. For a precise comparison of two networks at the end, play a few hundred games (200 games: ±25 Elo, 400: ±17) with `nnue/alphazero_match.sh`, a separate 1-GPU job that can run alongside the learning job:
>
> ```
> sbatch alphazero_match.sh                              # latest vs initial, 200 games, 0.5
> ↪  s/move
> sbatch --export=ALL,A=7168,B=4096,GAMES=400 alphazero_match.sh
> sbatch --export=ALL,BOOK=none alphazero_match.sh       # random openings instead of the book
> ```
>
> It copies the two networks first (the learning job prunes old files at every export), re-traces a network from its `.pt` if its `.ts` was pruned, plays the games in pairs like the evaluations, and appends A's result with its 95 % confidence interval to `WORK_DIR/matches.csv`. Adding `initial` to `--eval-opponents` gives an unchained measurement against a fixed reference, at the cost of another 20 games per evaluation.

6.5.3 A book of balanced openings

A pair of games tells which network is stronger only when the two games end differently *because of the networks*. With random openings they often do not: Black wins the game in which A is black and the game in which B is black, the pair ends 1–1, and those two games — two hours of GPU at 1 s per move — have measured nothing. The long matches of October showed it plainly (most pairs 1–1, Black winning almost every game). The cure used by every engine-testing framework is a book of openings in which neither side is clearly ahead. Yolah has no opening theory to borrow one from, so the book is built by search: `test/opening_book_main.cpp`, written to `config/openings/balanced.txt`.

1. Candidates. 200 000 random openings of 6 plies (3 per side; uniformly random legal moves, none ending the game). Two openings can lead to the same game up to a symmetry: 4 of the

8 symmetries of the board (Section 6.4.6) map the initial position to itself — the identity, the half turn and the two diagonal flips; the other four send the black pieces to the white ones' initial squares. Each opening is reduced to the smallest of its 4 images (`canonical`) and only one opening per class is kept. (That the 4 transformations turn a game into a game was checked on 2 000 random games of 30 plies: the transformed moves are always legal and give the transformed positions; for an axis mirror they never are.)

2. Values. Each candidate is searched by the minimax player (the dev player with the `spacelite` network, one thread) at a **fixed depth** rather than a fixed time, so that the book is reproducible from its command line, which is written in its header. The values are turned to Black's point of view, on the search's scale where ±30 000 is tanh ±1. Most random openings are far from balanced (Figure 14, left): the initial position is worth +0.35 to Black at depth 12, the random openings +0.47 on average, and only 15 % lie within ±0.27. A search at depth 6 (≈ 0.1 s) discards the others — its values correlate at 0.95 with those of depth 10 — and the 29 598 left are searched again at depth 12 (≈ 0.25 s each; 24 minutes on 18 cores in all, two thirds of them for the depth-6 filter).

3. Calibration. A first book took the 1 000 openings closest to a value of 0, and was too good for White: from it the minimax player against itself scored only 0.41–0.47 with Black, depending on the depth. A value of 0 is not an even game. To find the value that is, 6 000 of the 29 598 openings (all values, from −0.58 to +0.54) were played out, minimax against itself at depth 10, and Black's score was regressed on the depth-12 value (Figure 14, right): it reaches 1/2 at **+0.12** (+3 500; 95 % interval of the regression's zero +2 500 to +3 900). The book is therefore the 1 000 openings whose value is closest to +3 500 — all within 3 500 ± 312 — written in a random order so that a short match does not only use the most balanced ones.

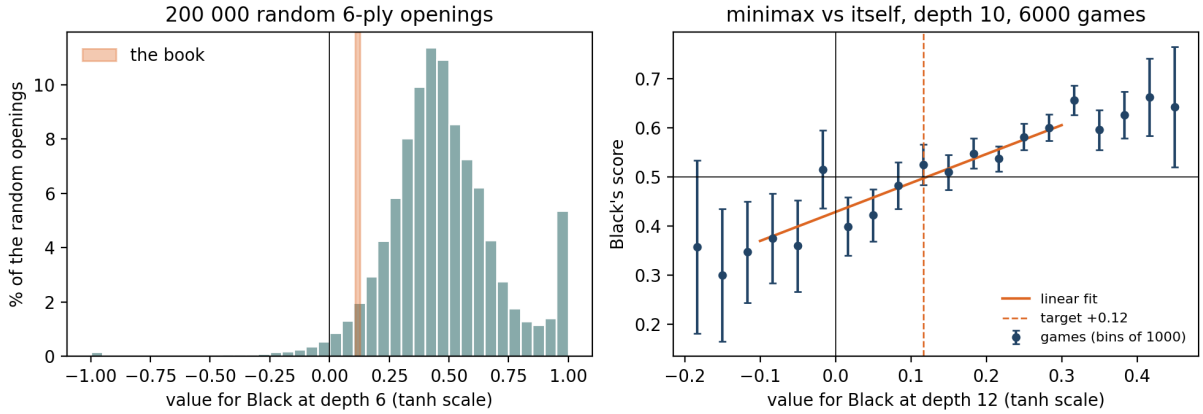


Figure 14: Left: the depth-6 value of 200 000 random 6-ply openings for Black, and the depth-12 window the book is taken from. Right: Black's score in 6 000 games of the minimax player against itself at depth 10, by the depth-12 value of the opening; 1/2 is reached at +0.12, not 0.

The regression also says how little a value says about one game: the correlation between an opening's value and the game's result is 0.17. Over this range, the opening shifts the odds by a few percent per 0.03; the rest is the play. That is the point — the openings should leave the result to the players.

4. Checks. `opening_book check` plays from the book (and, for comparison, from random 4-ply openings) with the minimax player:

games from 1 000 openings	Black's score (±0.025)
book, minimax vs itself at depth 6 / 8 / 10 / 12	0.496 / 0.498 / 0.520 / 0.499
random 4-ply openings, depth 10	0.724

pairs (each opening, both colours)	colour decides the pair	stronger wins both	stronger's score (Elo)
depth 8 vs 6, random openings	30.9 %	30.9 %	0.698 (+146)
depth 8 vs 6, **book**	**18.6 %**	**38.2 %**	0.739 (+180)
depth 10 vs 8, random openings	26.9 %	25.9 %	0.692 (+141)
depth 10 vs 8, **book**	**16.9 %**	**33.1 %**	0.713 (+158)

"Colour decides" counts the pairs whose two games were won by the same colour. With the book their share drops by 40 %, the pairs won twice by the stronger player become more frequent, and the same difference of strength shows as a larger score: random openings compress the measured gaps towards 0, by 10–20 % here. The book is calibrated with the minimax player; whether its +0.12 is also the even point for the networks, which see the game differently, is measured for free by every match (`evals/match_*.json` records the opening and the colours of each game).

Use. `nnue/alphazero_match.sh` uses the book by default (`BOOK=none`: random openings as before), and so do the evaluations of the learning job (`EVAL_BOOK`, i.e. `--eval-book`; `none` to switch back). Pair k of a match plays opening k of the book in an order shuffled by the match's seed, so 1 000 openings cover 2 000 games without repetition. Changing the openings in the middle of a run changes the scale of the chained Elo a little (gaps look larger, see above): compare evaluations made with the same kind of openings.

```
cd build
./opening_book build --plies 6 --candidates 200000 --pre-depth 6 --pre-max 8000 \
    --depth 12 --target 3500 --max-value 1500 --size 1000 --threads 18 --seed 1 \
    --out ../config/openings/balanced.txt                    # 25 min on 18 cores
./opening_book check --games 1000 --depth 10                 # Black's score from the book
./opening_book check --games 1000 --depth 10 --opponent-depth 8  # pairs, as in the matches
```

To recalibrate (another player, another network): build with `--target 0 --max-value 30000 --size 100000` to keep every value, play them with `check --random-plies 0 --csv calib.csv`, and take the value at which Black's score crosses 1/2 (`doc/figs/make_book_fig.py` draws it).

6.6 Running it on the cluster

6.6.1 The cluster

resource	what there is	what the run uses
GPU nodes	5 nodes, 13 cards of 48 GB: 7 A40, 6 RTX 8000	3 cards (the per-user maximum)
CPU nodes	18 nodes, 592 cores, 8 TB of RAM	nothing (see Section 6.7)
home (`/home/<user>`)	visible from every node	the work directory (~60 GB)

resource	what there is	what the run uses
`/calcul`	17 TB, distributed	nothing

The run lives in the home directory, `~/AlphaZeroLearn`: it must survive for a month and be reachable from whichever node the next job lands on, and its I/O is modest (about 1.2 GB of self-play rows a day, a 141 MB network per export, small reads by the trainer), so no fast scratch space is needed.

6.6.2 The image

`nnue/alphazero_learn.def` builds a *toolchain* image: CUDA 12.4 with cuDNN, g++-13 (the code is C++23), Boost, oneTBB, Google benchmark, CMake, Python 3 and torch 2.6.0 — whose wheel also contains libtorch, the C++ library the TorchScript backend links against. Unlike the other `.def` files it does **not** contain the Yolah sources: the job bind-mounts the repository and compiles `alphazero_learn` on the compute node, at the start of every job (about a minute). Two reasons: the CMake flags use `-march=native`, and a binary compiled on the machine that builds the image may use instructions the cluster CPUs lack (a crash with `SIGILL`); and changing the code then requires a `git pull`, not a new image. The build is cached in `WORK_DIR/build/<hash of the CPU model>`.

```
# once, on a machine where you are root (or with --fakeroot)
cd nnue && sudo singularity build alphazero_learn.sif alphazero_learn.def
# copy the .sif to the cluster, clone the repo there, copy the initial network
scp alphazero_learn.sif cluster:AlphaZeroLearn/
# on the cluster: git clone the repository into ~/Yolah, then
scp cnn_resnet_256x30_value_policy.pt cluster:Yolah/nnue/
```

The network is stored in the repository with Git LFS, which the cluster does not have: a clone there holds a 134-byte *pointer* in place of the 144 MB file, hence the `scp`. Both the job script and the trainer check for a pointer and stop with an explicit message rather than a cryptic `torch.load` error.

6.6.3 The job script

`nnue/alphazero_learn.sh` asks for **3 GPUs, 32 CPUs, 128 GB of RAM and 14 days** (Figure 16). Its header lists every tunable as an environment variable (`SELFPLAY_GAMES`, `LR`, `REUSE`, `EVAL_HOURS`, …); the paths (`YOLAH_DIR`, default `~/Yolah`; `SIF` and `WORK_DIR`, default under `~/AlphaZeroLearn`) and the shell functions shared with the next script live in `nnue/alphazero_common.sh`. It does two things: compile, then run `alphazero_learn.py` in the container. The first lines of its output name the GPUs it got (`nvidia-smi`).

6.6.4 Splitting the GPUs over several nodes

The GPU nodes have two or three cards each, and a job asking for three GPUs on *one* node may wait a long time for one to be completely free. Because the loop only communicates through files in the shared home, the three GPUs do not have to be on the same node (Figure 17):

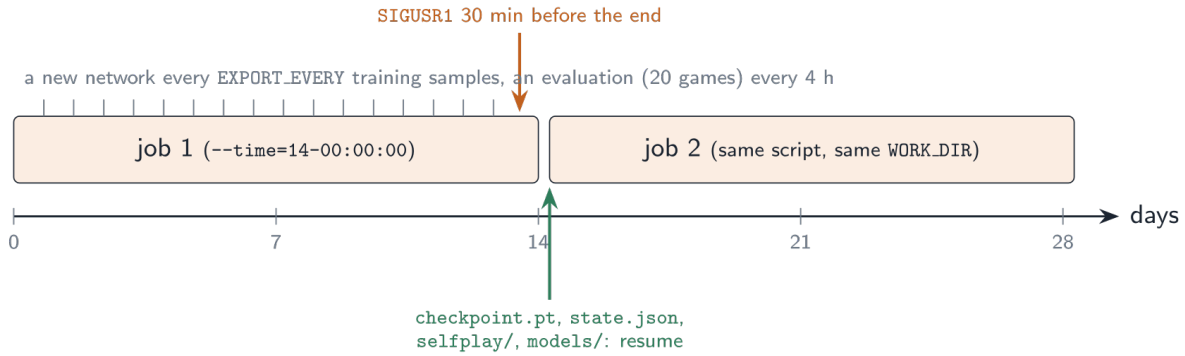


Figure 15: Two 2-week jobs make one month-long run.

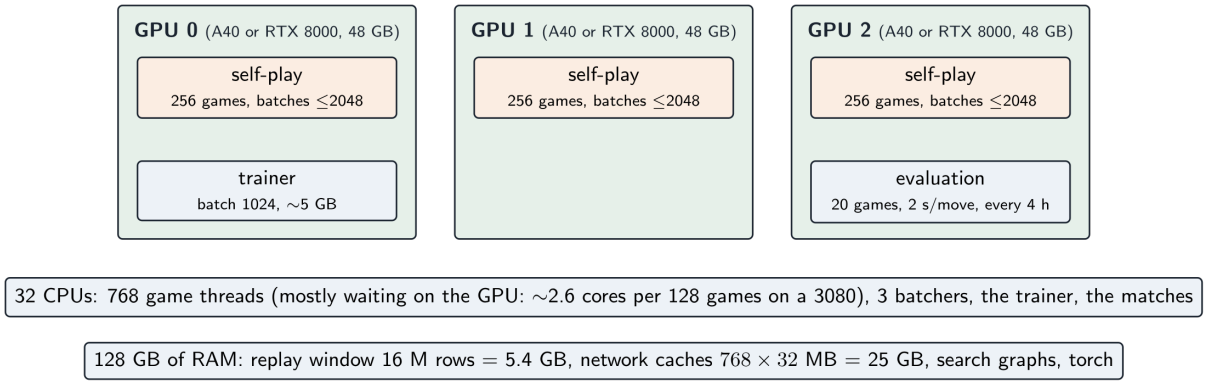


Figure 16: How the job uses the node.

```
sbatch --gres=gpu:1 alphazero_learn.sh     # trainer + self-play + evaluations
sbatch alphazero_selfplay.sh               # one more self-play GPU, any node
sbatch alphazero_selfplay.sh               # and another
```

`nnue/alphazero_selfplay.sh` (1 GPU, 12 CPUs, 48 GB) runs only `alphazero_learn selfplay` on the same work directory. It waits until the main job has published a network, compiles for its own node (a lock serialises two jobs compiling for the same CPU model), hot-swaps every new network like the main job's own self-play, and writes its games under a name that carries its host and job id — two jobs on one node both see their GPU as number 0, so the GPU number alone would not be unique. It stops by itself when no new network has appeared for 6 hours (`--max-stale-hours`): when the main job is over, it does not keep playing with an outdated network until its own time limit.

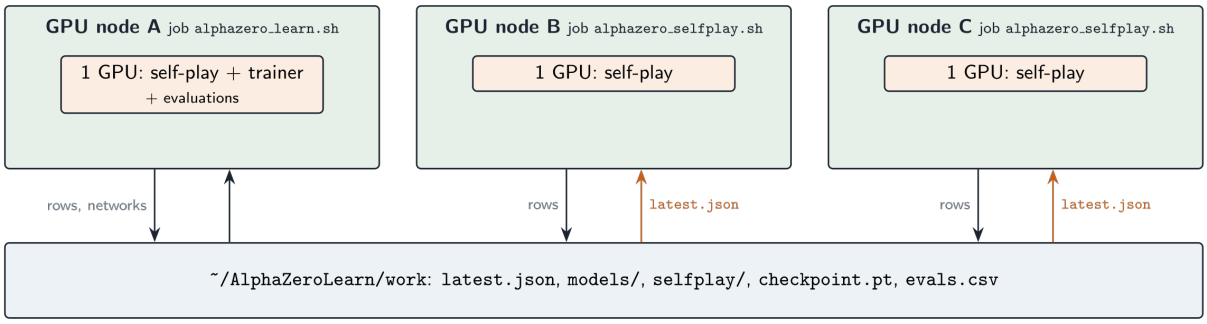


Figure 17: Three 1-GPU jobs on three nodes, one run: every exchange is a file in the shared home directory.

6.6.5 Stopping cleanly

Stopping cleanly matters, because the last minutes of a job are the ones that would otherwise be lost:

- `#SBATCH --signal=B:USR1@1800` makes SLURM send `SIGUSR1` to the batch script 30 minutes before the time limit. The script's `trap` forwards it as `SIGTERM` to the trainer, which stops at its next step, writes `checkpoint.pt`, sends `SIGTERM` to the self-play processes — they finish their current searches, write the finished games and exit — and stops the evaluation (which stays in `pending_evals` and is played first by the next job).
- In case the signal is lost, the script also reads the job's remaining time (`squeue -o %L`) and passes `--max-hours` = that minus 20 minutes.
- The job script starts the container with `exec`, so the signal reaches Singularity itself, which forwards it into the container. Sending it to a bash subshell around Singularity would kill the subshell and leave the trainer and its self-play processes running as orphans. This was caught in testing.
- The trainer refuses to start if it cannot see a GPU. The usual cause is a container whose CUDA driver is not bound, and without the check self-play would fall back to the CPU and run about 50 times slower for two weeks. The self-play log's second line names the backend and device actually used.
- Even a `kill -9` loses at most the work since the last export and the games in progress: self-play rows are on disk as soon as a file is complete.

To continue, submit **the same script with the same `WORK_DIR`**:

```
sbatch alphazero_learn.sh          # days 0-14
sbatch alphazero_learn.sh          # days 14-28: resumes from WORK_DIR
```

`RESUBMIT=1` makes the job resubmit itself after a clean stop (never after a crash, to avoid a resubmission loop).

6.6.6 What to watch

file	what it tells you
`alphazero_learn_<job>.out`	the trainer's log: losses every 2 min, exports, evaluations
`WORK_DIR/evals.csv`	one line per match: W/D/L, score, Elo difference, chained Elo
`WORK_DIR/train_log.csv`	step, rows, window, value loss, policy loss, KL, samples/s
`WORK_DIR/logs/selfplay_gpu<k>.log`	games/s, rows/s, evals/s, game length, colour balance (every 5 min)
`WORK_DIR/logs/eval.log`	every evaluation game, move by move result
`WORK_DIR/evals/*.json`	the complete record of every evaluation game

Healthy signs: the value loss slowly decreasing, the KL staying small (the network keeps up with the search, typically a few tenths), the chained Elo climbing, the colour balance of self-play evolving slowly. Warning signs: a value loss that collapses towards 0 (the window is too small: it memorises), a KL that keeps growing (the learning rate is too small or the reuse too low), games getting systematically shorter (a degenerate strategy — look at `evals/*.json`).

6.7 Sizing for the cluster

6.7.1 What was measured

On the development machine (one RTX 3080, 10 GB, WSL2), with the initial network and 800/200 simulations:

quantity	value
self-play, 128 games	~8 500 evaluations/s, GPU at 95 %
self-play, 256 games	no better (the GPU is saturated)
self-play, 2 processes × 64 games	~6 000 evaluations/s in total (worse)
games	~0.51 games/s — ~44 000 per day, 55 plies on average
training rows (full searches only)	~6.9 rows/s — ~600 000 per day
evaluations per row	~1 200
trainer, batch 1024	~3 400 samples/s, 4.9 GB (alone on the GPU)
colour balance	black wins 61 %, white 8 %, draws 31 %

The last line is a property of Yolah, not of the loop; it is explained just below.

6.7.2 Why black wins so often

On the cluster the three self-play GPUs report the same balance: black wins ~57 %, white ~8 %, the rest are draws. That is not a weakness of the network nor a bug in the counting: it follows from how Yolah is scored.

Every move scores one point for the player who makes it, and black moves first. The final margin, black minus white, is therefore the sum of two parts:

- **a structural bonus for black of 0 or +1.** As long as nobody has to pass, black has made either as many moves as white (the game ended after a white move: a draw) or one more (it ended after a black move: black wins by one). On average, black gets half a point for free.
- **the passes.** A player who is blocked while the other can still move loses one point per pass. It is the only way white can win.

And the margins are tiny (Figure 18): in 84 % of the games the final margin is between −2 and +2. With margins that small, black's free half point decides a large share of the games: it turns would-be draws into black wins, and small white wins into draws.

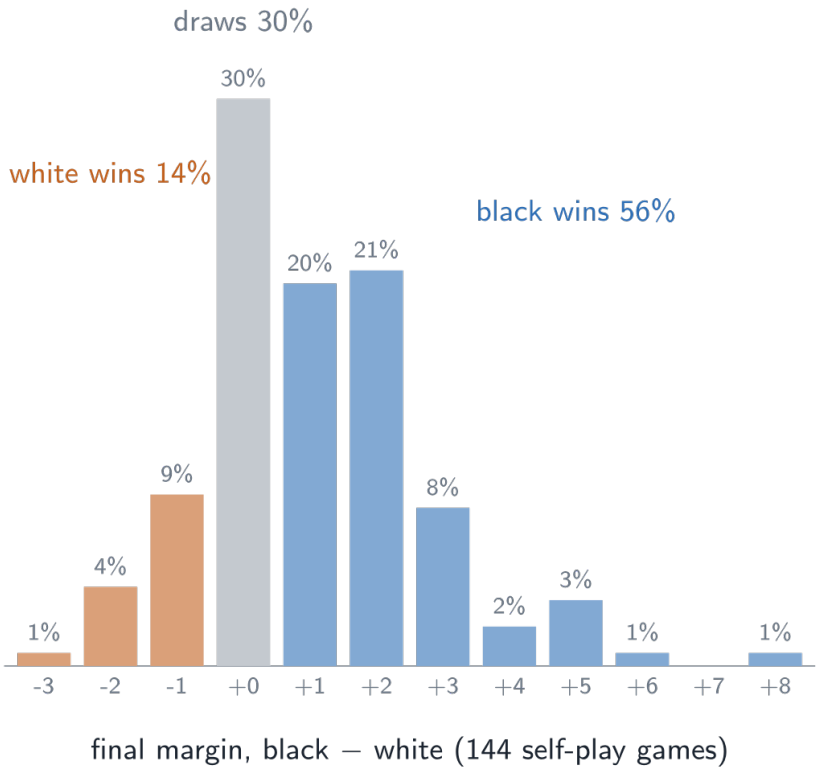


Figure 18: Distribution of the final margin over 144 self-play games observed from the initial position (the margin is read off the future ownership map described in the section on the auxiliary heads).

The same balance appears in games played by entirely different engines. The 480 000 games of the supervised training set (minimax and NNUE players) start with 0 to 9 random plies, and splitting them by that number is instructive:

random plies at the start	black wins	draws	white wins
0 (the real initial position)	**60.6 %**	**32.0 %**	**7.5 %**
2, 4, 6, 8	52–61 %	26–34 %	13–14 %
1, 3, 5, 7, 9	34–43 %	29–38 %	19–38 %

From the initial position, the minimax engines show exactly the imbalance of AlphaZero's self-play. Over the whole set white wins 20 %, but only because of the random openings: after an *odd* number of random plies, black has played one random — hence usually bad — move more than white, and the more random plies, the better white does. A comparison of colour balances is therefore only meaningful between games that start from the same kind of position. (The evaluation matches play each opening with both colours, from a book of balanced openings, Section 6.5.3.)

Consequences for the learning loop:

- none for the value head: the network sees whose turn it is (plane 3 of the input) and learns that the same board is worth more to black;
- none for the evaluation: every opening is played once with each colour;
- but z (win / draw / loss) says little in such a game: with a third of draws and most wins by one or two points, the *exact* margin carries much more information than its sign — which is what the score-margin head of Section 6.9 learns.

A game designer who wanted balanced results would give white a compensation (half a point, like komi in Go); that is a change of rules, not of the engine, and has no effect on learning.

6.7.3 From the measurements to the settings

Self-play is the bottleneck, as in every AlphaZero-like system; the rest is arithmetic.

Data rate. With ~1 200 evaluations per row, a GPU doing E evaluations per second produces $E/1200$ rows/s, i.e. $72\,E$ rows per day. How fast are the cluster's cards? The network is convolutions on 8×8 boards with 256 channels: compute-bound, in fp16 with fp32 accumulation. The A40 (Ampere, ~150 TFLOPS of fp16 tensor work) and the RTX 8000 (Turing, ~130 TFLOPS) do that at full rate, whereas a GeForce RTX 3080 does it at half its nominal rate (~60 TFLOPS). From the specifications, then, a cluster card should be 2 to 2.5 times a 3080. Taking a conservative 2×: $3 \times 17\,000$ evaluations/s $\approx$ 3.6 M rows per day, **~100 M rows in 28 days** ($\approx$ 7 M games). Each self-play log prints `evals/s`, so the real number is known after five minutes; everything below scales linearly with it.

Games per GPU: 256. Enough to saturate a GPU 2–2.5 times faster than the 3080, which saturates at 128. More games cost only RAM and threads, both plentiful. One process per GPU (measured above: two per GPU is worse).

Replay window: $N_0 = 250\,000$, cap 16 M. KataGo's own N_0. At 3.6 M rows/day the first 250 000 rows arrive within two hours, and after a month $W(N) \approx 12$ M rows (Figure 11) — 4 GB, well under the 16 M cap (5.4 GB) chosen to leave headroom if the GPUs are faster than assumed.

Export every 524 288 samples. At $r = 4$ that is a new network every 131 072 rows, about every 50 minutes at 3.6 M rows/day — frequent enough for self-play to play with a fresh network, rare enough that the export (a trace and two 141 MB files) is negligible.

Evaluation every 4 hours, 20 games. Six per day, 37 minutes each on the last GPU: about 15 % of that GPU's self-play. About 170 evaluations over the month, i.e. a 24 GB history of evaluated networks.

RAM and CPUs. 768 games × 32 MB of network cache = 25 GB, the replay buffer 5.4 GB, plus the search graphs and torch: well under the 128 GB requested. The game threads spend most of their time waiting for the GPU: on the 3080, 128 games used ~2.6 cores. The CPU work (descents, backups, encoding the positions) is proportional to the number of *evaluations*, not to the number of games, so a GPU twice as fast needs about twice that: $3 \times 2 \times 2.6 \approx 16$ cores for self-play, plus the trainer's data pipeline and the 4 search threads of each evaluation player. 32 CPUs leave a comfortable margin, and asking for less than a whole node makes the job easier to schedule.

Heterogeneous GPUs. A job may get A40s, RTX 8000s or, in principle, a mix. Self-play does not care. The trainer switches between bf16 and fp16 (see the optimiser notes above). The evaluation compares two networks on the *same* GPU, so the match is fair whatever the card; but "2 s per move" buys more simulations on an A40 than on an RTX 8000, so the absolute level of play of two evaluations made in different jobs may differ slightly — another reason to read the Elo trend rather than single values. To pin a card type, use the cluster's gres type (`--gres=gpu:a40:3`; `sinfo -o "%N %G"` lists them).

Why not the CPU nodes? Self-play could in principle run on the 592 CPU cores with the pure C++ backend (`nn::ResNetCPU`). Measured on the development machine it evaluates 130–160 positions/s on about ten cores; a 33-core node would do perhaps 450/s, and all 18 nodes ~8 000/s — about half of one A40, for a gain of roughly 15 % on three GPUs. That would not pay for the complexity (a second network format to export at every update, jobs spread over 18 nodes) nor for occupying a shared resource. The GPUs are where the network is fast.

Disk. ~34 GB of self-play rows for 100 M rows, ~24 GB of evaluated networks: plan for ~60 GB in `WORK_DIR`.

6.8 Parameters of the learning loop

All are command-line options of `alphazero_learn.py`; the `.sh` exposes the important ones as environment variables and sets the cluster values:

option	default	cluster (`.sh`)	meaning
`--selfplay-games`	128	256	concurrent games per self-play process (one per GPU)
`--selfplay-set 'k=v'`		`nn cache=32`	override a key of the self-play config
`--batch-size`	1024	1024	training batch
`--lr`	1e-4	1e-4	Adam learning rate (after warm-up)
`--warmup-steps`	500	500	linear warm-up
`--weight-decay`	1e-4	1e-4	L2
`--ema-decay`	0.995	0.995	weight averaging for export; 0 = raw weights
`--reuse`	4	4	training samples per generated row

option	default	cluster (`.sh`)	meaning
`--q-weight`	0	`Q_WEIGHT`	value target $(1-w)\,z + w\,q$ (Section 6.4.4); 0 = the result alone
`--min-rows`	100 000	250 000	N_0: rows before training starts, window base
`--max-window`	4 M	16 M	ring buffer capacity
`--export-every`	131 072	524 288	training samples between two networks
`--eval-hours`	4	4	time between two evaluations
`--eval-every`	0		evaluate every K exports instead (0 = use `--eval-hours`)
`--eval-games` / `--eval-seconds`	20 / 2	20 / 2	the match
`--eval-opening-plies`	4	4	random plies before each pair of games, without a book
`--eval-book`	none	`EVAL_BOOK` = `config/openings/balanced.txt`	balanced openings (Section 6.5.3)
`--eval-opponents`	previous	previous	`previous`, `initial`, `lag:K`, comma separated
`--train-gpu` / `--eval-gpu`	0 / −1		trainer on the first GPU, matches on the last
`--keep-all`	off		keep every exported network
`--max-hours`	0	time limit − 20 min	stop cleanly after this long
`--no-selfplay`	off		train only (self-play running elsewhere)
`--allow-cpu`	off		run without CUDA (tests only)
`--amp`	auto		trainer precision: `auto` (bf16 on Ampere+, else fp16), `bf16`, `fp16`
`--aux`	off	`AUX=1`	train the auxiliary heads (Section 6.9)
`--own-weight`	1.5		weight of the ownership loss
`--score-weight` / `--score-cdf-weight`	0.02 / 0.02		weights of the margin losses
`--aux-ramp-steps`	2 000		linear ramp of the three auxiliary weights from 0

6.9 KataGo's auxiliary heads

Everything so far trains the network on two targets per position: one scalar z and one distribution π. KataGo's most effective idea on top of AlphaZero is to ask the network for **more predictions than the search needs** — extra *heads* that exist only during training (Figure 19). This section explains what they are and why they help, how the two most useful ones — ownership and score — translate to Yolah, and how they are implemented here, as an **option** (`--aux`): without it, everything described in the previous sections runs unchanged, file formats included.

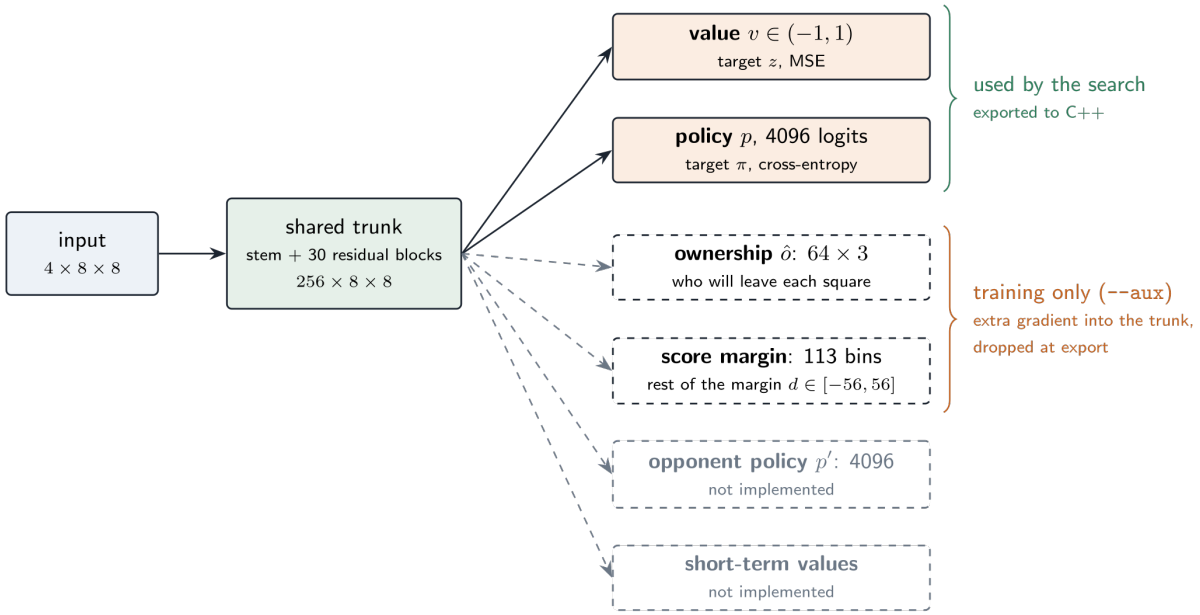


Figure 19: The two heads the search uses, and the auxiliary heads that exist only to train the trunk. Only ownership and score margin are implemented.

6.9.1 Why predicting more helps

Think of the information a game gives to the value head: the result, win, draw or loss — $\log_2 3 \approx 1.6$ bits — shared by the 55 positions of the game. A position early in the game gets the credit or the blame for 50 later moves it had nothing to do with. The value target is extremely *noisy*, and that noise is what makes value learning the slow part of AlphaZero. In Yolah it is worse than usual: with a third of draws and most wins by one or two points (Section 6.7.2), the result hides almost everything that happened.

An auxiliary target is any quantity that is (a) known for free at the end of the game, (b) much richer than z, and (c) related to what the network must understand. Its head is small — a 1×1 convolution, a couple of linear layers on top of the shared trunk — and it is thrown away at export time. What remains is its **gradient into the trunk**: the trunk must now compute features that also predict the auxiliary target, and features that explain *where* and *by how much* a game is won are exactly the features that make the value head accurate. It is regularisation by multi-task learning. In the KataGo paper's ablations, removing these targets slows learning down markedly, most of all early in a run.

Two caveats before expecting miracles: our network starts from a supervised model that already plays well, so the gain will be smaller than for KataGo, which started from scratch; and only evaluation matches can tell how much.

6.9.2 Ownership in Yolah

In Go, KataGo predicts for each of the 361 points who owns it at the end of the game — 361 targets per position instead of one. Yolah has an exact equivalent, and a particularly clean one. Every move leaves a hole on the square the piece came from and scores one point for its player (`Yolah::play`). So at the end of a game **each square is either a hole left by black, a hole left by white, or neither**, and the number of holes a player left *is* their score (Figure 20, a real game of the initial network: 29–26). The ownership map is the final score decomposed square by square.

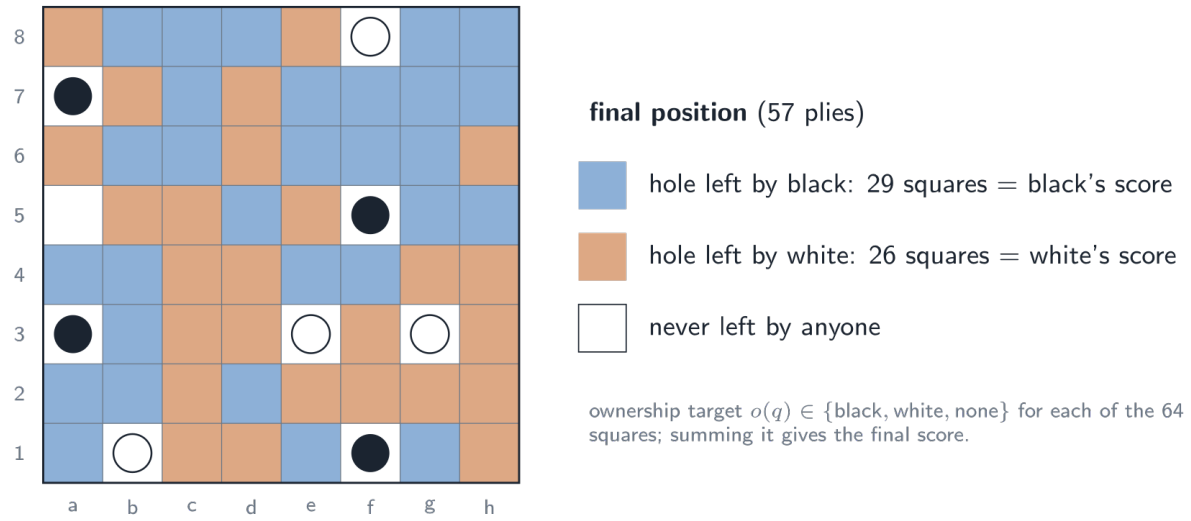


Figure 20: Final ownership in a real game: every square is labelled by the player who left it. Counting the labels gives the final score.

Yolah is a cousin of the Game of the Amazons, and ownership means the same thing in both: as the game goes on, the board splits into regions sealed off by holes, and the squares of a region where I am alone are going to be "mine". The ownership head has to see that partition coming long before it is fixed — which is precisely the strategic understanding the value head needs.

The target must be predictable from the position. The network's input shows the holes, but not *who* made them. The final label of a square that is already a hole is therefore history, not something the position determines: asking the network to predict it would be asking it to guess, i.e. noise. So the target is the **future ownership**, relative to the side to move:

$$o_t(q) = \begin{cases} \text{no target} & \text{if } q \text{ is already a hole at } t, \\ \text{mine} & \text{if the side to move at } t \text{ will leave } q, \\ \text{opponent's} & \text{if the opponent will leave } q, \\ \text{nobody's} & \text{otherwise (it stays free, or a piece ends the game there).} \end{cases}$$

Figure 21 shows it on the same game, after 24 plies.

Being relative to the side to move ("mine / the opponent's" rather than "black / white") makes the target independent of the colour, like the value.

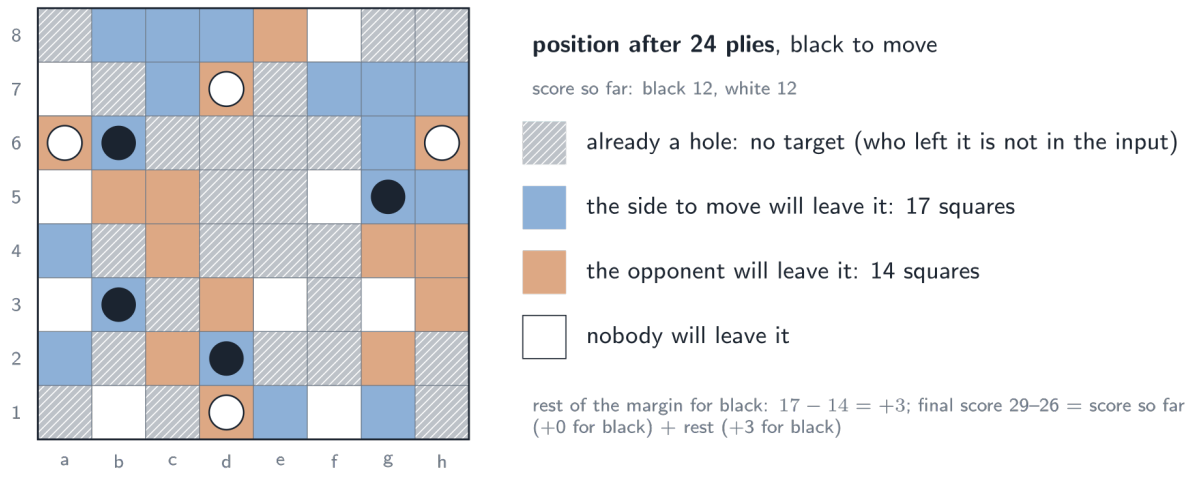


Figure 21: The future ownership target of a mid-game position of the same game. Holes already made carry no target; the rest of the board is labelled by who will leave each square. The counts give the rest of the score: 12 + 17 = 29 for black, 12 + 14 = 26 for white.

6.9.3 The rest of the margin

Summing the future ownership map gives the number of moves each side will still make, hence the **rest of the score margin** for the side to move:

$$d_t = \#\{q : o_t(q) = \text{mine}\} - \#\{q : o_t(q) = \text{opponent's}\} \ \in [-56, 56]$$

(64 squares minus 8 pieces). The final margin is the margin so far plus d_t (29–26 = (12–12) + (17–14) in the figure). Why predict the *rest* rather than the final margin, as KataGo does? Because the score so far is not in the input either: it can be guessed from the number of holes and whose turn it is — but not when there have been passes. The rest of the margin, on the other hand, is entirely determined by the position and the play to come.

The score head predicts the whole **distribution** of d_t — 113 logits, one per possible value — rather than a single number: whether a position is a sure +1 or a coin toss between −2 and +4 is exactly the kind of information the value lacks. It needs no extra data: d_t is read off the ownership map.

6.9.4 The heads

`AuxNet` (`nnue/alphazero_aux.py`) is `Net` — the same trunk, the same value and policy heads, the same parameter names — plus two heads on the trunk's output $h \in \mathbb{R}^{C\times 8\times 8}$:

- **ownership**: a 1×1 convolution $C \to 3$, i.e. three logits per square, one softmax per square over (nobody, mine, opponent's) — a per-square classification;
- **score**: global pooling of h over the 64 squares, mean and max ($2C = 512$ numbers), then a linear layer to 256, a ReLU, and a linear layer to the 113 logits. Pooling rather than flattening, because the margin is a property of the whole board.

```
def forward_all(self, x):
    h = self.trunk(x)                                        # (B, C, 8, 8)
    v, p = self.heads(h)                                     # the two heads of Net
    own = self.own_conv(h).flatten(2)                        # (B, 3, 64)
    pooled = torch.cat([h.mean(dim=(2, 3)), h.amax(dim=(2, 3))], dim=1)
    score = self.score_fc2(torch.relu(self.score_fc1(pooled)))   # (B, 113)
    return v, p, own, score
```

`forward()` still returns `(v, p)`, so an `AuxNet` can be used wherever a `Net` is.

6.9.5 The losses

With $\hat{o}_q$ the predicted distribution of square q, $\hat{P}$ the predicted distribution of the margin and $\hat{F}$ its cumulative distribution:

$$\mathcal{L}_{\text{own}} = -\frac{1}{64} \sum_{q\,:\,o_t(q) \neq \text{no target}} \log \hat{o}_q(o_t(q)), \qquad \mathcal{L}_{\text{pdf}} = -\log \hat{P}(d_t), \qquad \mathcal{L}_{\text{cdf}} = \sum_{x=-56}^{56} \left(\hat{F}(x) - \mathbb{1}[x \geq d_t]\right)^2.$$

- The ownership loss is a cross-entropy per square, averaged over the whole board (squares without target count for 0). It is KataGo's $1/b^2$ times the sum over the points.
- The pdf term is an ordinary cross-entropy against the true margin. On its own it treats every wrong value alike; the cdf term adds the notion of distance: predicting $+11$ for a true $+12$ moves the cumulative distribution by one step, predicting -30 by many (it is the squared-error form of the earth mover's distance between the two distributions).

The total loss adds them to the AlphaZero loss with small weights, because the auxiliary heads must shape the trunk without dominating the two heads that matter:

$$\mathcal{L} = \mathcal{L}_v + \mathcal{L}_p + \rho(t)\,(w_{\text{own}}\,\mathcal{L}_{\text{own}} + w_{\text{pdf}}\,\mathcal{L}_{\text{pdf}} + w_{\text{cdf}}\,\mathcal{L}_{\text{cdf}}), \qquad w_{\text{own}} = 1.5,\ w_{\text{pdf}} = w_{\text{cdf}} = 0.02,$$

KataGo's weights. $\rho(t)$ is a **ramp** from 0 to 1 over the first 2 000 steps (`--aux-ramp-steps`): the new heads start random, their first gradients are large and meaningless, and they must not shake a trunk that already plays well. Rows without auxiliary targets (see below) are masked out of the three terms.

The code, in `aux_losses`:

```
ce = F.cross_entropy(own_logits, own, ignore_index=OWN_PAST, reduction="none")  # (n, 64)
own_loss = ce.sum(1).mean() / 64.0
target = margin_from_ownership(own) + MAX_MARGIN                # index of d_t in 0..112
logp = F.log_softmax(score_logits, dim=1)
pdf_loss = F.nll_loss(logp, target)
cdf_pred = logp.exp().cumsum(1)
cdf_true = (torch.arange(SCORE_BINS)[None, :] >= target[:, None]).float()
cdf_loss = ((cdf_pred - cdf_true) ** 2).sum(1).mean()
```

The 8 symmetries apply unchanged: the ownership map is a plane like the input planes and goes through the same `gather`; the margin, a count, is invariant.

6.9.6 Recording the targets

The target of a sample depends on the rest of its game, so it is computed when the game ends — like z. During the game, `run_selfplay` notes who leaves each square (`leaver[q]`); at the end, `with_ownership` turns that into the map of every recorded sample:

```
for (int q = 0; q < 64; q++) {
    uint8_t code;
    if ((s.empty >> q) & 1)      code = OWN_PAST;      // already a hole: no target
    else if (leaver[q] == 0xFF)  code = OWN_NONE;      // nobody leaves it
    else                         code = leaver[q] == s.turn ? OWN_MINE : OWN_OPP;
    a.own[q / 4] |= static_cast<uint8_t>(code << (2 * (q % 4)));
}
```

A square that is not yet a hole at the sample and is left later can only have been left *after* the sample — once left, a square is a hole forever — so the map counts exactly the moves still to come.

Two bits per square, 16 bytes per row: the rows become `TrainingSampleAux` (352 bytes instead of 336), in files of **version 2**. On the Python side the map is unpacked on the GPU into the plane-cell order of the input (cell c holds square $63 - c$) so that it lines up with the planes and their symmetries.

The targets were checked against the rules on 8 570 rows, with and without the random symmetry: every square already a hole is exactly a square without target; no square holding one of my pieces is ever labelled "the opponent will leave it", and vice versa; and on the 4 655 rows of early game (no pass possible yet), the result z is the sign of margin so far $+ d_t$ in 100 % of the cases.

6.9.7 Two variants, one code

The option is decided by the trainer alone, and everything else follows:

	without `--aux` (default)	with `--aux` (`AUX=1`)
network trained	`Net`	`AuxNet` (`Net` + 2 heads)
network exported (`.pt`, `.ts`)	`Net`	`Net` — the heads are dropped
C++ side	unchanged	unchanged
`latest.json`	`{"step", "ts"}`	`{"step", "ts", "aux": true}`
self-play rows	version 1, 336 bytes	version 2, 352 bytes
logs	`train_log.csv`	`train_log.csv` + `train_log_aux.csv`

- **Self-play follows `latest.json`**: the key `"aux"` switches the recording on or off at the next poll, in the main job and in the extra self-play jobs alike. No flag has to be kept in sync by hand.
- **The trainer reads both formats**: version-1 rows read by an `--aux` trainer have no target (every square "no target", masked in the loss); version-2 rows read by a plain trainer simply lose their map.
- **A run can switch** from one job to the next, in either direction:

```
sbatch --export=ALL,AUX=1 alphazero_learn.sh    # the next two weeks with the heads
```

The shared parameters are kept; the heads start from scratch (or are dropped); the optimizer, whose state is per parameter, restarts with a new warm-up, and so does the weight average. The log says so (`variant switch: with auxiliary heads from step …`). This was tested in both directions on a single run.

- **The export is the plain network**: `export_model` rebuilds a `Net` from the non-auxiliary parameters, so the `.pt` and `.ts` files are byte-for-byte the same kind as without the option, and the traced module gives exactly the value and policy of `AuxNet` (checked: difference 0).

What to watch with `--aux`, in `train_log_aux.csv` (and every 2 minutes in the job's log): the ownership loss and its accuracy (the fraction of squares whose future owner is predicted right), and the two margin losses. They should fall quickly in the first thousands of steps — the heads are learning — and then slowly. The two KataGo heads not implemented here, the opponent's reply and the short-term values, would need rows paired with the next rows of the same game; the version header of the files leaves room for that.

7 Solving Yolah

Two questions come naturally once a program plays a game well. Could we compute *exact* values instead of estimating them — at least near the end of the game, as chess engines do with endgame tablebases built by retrograde analysis? And could the whole game be *solved*? This chapter works out what that would take for Yolah. Nothing in it is implemented yet; the numbers come from the measurements of the previous chapter and from the rules.

The vocabulary is Allis's. A game is **ultra-weakly solved** when the result of perfect play from the initial position is known, **weakly solved** when a strategy achieving it is known too, and **strongly solved** when the value of every position is known. Checkers was weakly solved in 2007, after almost two decades of work; Othello (8×8) was weakly solved in 2023 — a draw.

7.1 What there is to compute

The quantity to compute is the one the score head of Section 6.9 predicts: the **rest of the margin** $V(s)$, the number of moves the side to move will still make minus the number the opponent will, under perfect play by both. Three properties of Yolah make it pleasant.

It depends on the board alone. The score so far does not matter: the final margin is the margin so far plus $V(s)$, and the margin so far is a constant that neither player can change any more. A position is therefore just the two sets of four pieces, the set of holes and the side to move.

It obeys a simple recurrence. A move scores one point for its player and hands the move to the opponent, whose rest of the margin counts against us; a player who cannot move passes and scores nothing:

$$V(s) = \begin{cases} \max_m \left(1 - V(s \cdot m)\right) & \text{if the side to move has moves,} \\ -V(\bar{s}) & \text{if only the opponent has moves } (\bar{s} : s \text{ with the other side to move}), \\ 0 & \text{if nobody can move (the game is over).} \end{cases}$$

It also solves win / draw / loss. From the initial position the margin so far is 0, so black wins, draws or loses according to the sign of $V(s_0)$. That this sign is also the value of the game played for win / draw / loss is not obvious — the two objectives could call for different moves — but it holds because the sign is monotone and minimax commutes with monotone functions: $\operatorname{sign}(\max(a, b)) = \max(\operatorname{sign} a, \operatorname{sign} b)$, and likewise for min. Solving the margin game solves the result, and gives more: by how much.

The game is acyclic and layered. Every move adds a hole, and holes never disappear, so a game lasts at most 56 moves and the positions fall into layers by their number of holes: a move always goes from layer k to layer $k + 1$, and a pass stays in the layer but can happen at most once in a row. There are no cycles, no repetitions, no draws by repetition: the recurrence above is a plain backward induction.

7.2 Retrograde analysis, the textbook way

Retrograde analysis computes values from the end of the game backwards. For Yolah it would go layer by layer, from the full boards to the empty one:

1. Every position of the deepest layers is terminal (nobody can move): value 0.
2. For layer k, every value needed is either in layer $k + 1$ (after a move) or in layer k with the other side to move (after a pass, and that position is not itself a pass position). Compute them by the recurrence.
3. Store one byte per position ($V \in [-56, 56]$), and divide the work by 16 with the 8 symmetries of the board and the colour swap ($V(\text{black}, \text{white}, H, \text{black to move}) = V(\text{white}, \text{black}, H, \text{white to move})$ with the pieces' colours exchanged).

Classic retrograde analysis walks *backwards* with **unmoves** rather than enumerating every position of a layer: from a solved position it generates its predecessors and updates them. In Yolah an unmove is easy to state: take a piece of the player who has just moved, standing on b; pick a hole a on a queen line through b whose squares in between are all free; move the piece back to a and turn a back into a free square. Unmoves visit only positions that can actually lead to the solved ones, which matters when most of the formal positions are unreachable.

Why whole-board tables are out of reach. In chess, an endgame table is small because few pieces are left. In Yolah the number of pieces never decreases — there are always eight — and what shrinks is the free space. But the holes that bound it can be *anywhere*: a table of all positions with only F free squares has to choose which $8 + F$ squares are not holes, where the pieces are among them, and whose they are:

free squares F	positions (after the 16 symmetries)
4	1.4×10^{16}
8	5.5×10^{19}
12	2.2×10^{22}

Even the table of the last few moves is larger than the biggest endgame databases ever built. A tablebase of *whole positions* is not the right tool.

7.3 Regions: what makes endgames small

What saves the day is that late positions fall apart (Figure 22). A queen move only crosses free squares, and consecutive squares on a line are neighbours (8-connectivity). So a piece can only ever reach the free squares of the **regions** — 8-connected groups of free squares — it touches, and two regions never merge: squares only ever stop being free. A late position is a **sum of independent sub-games**, one per region.

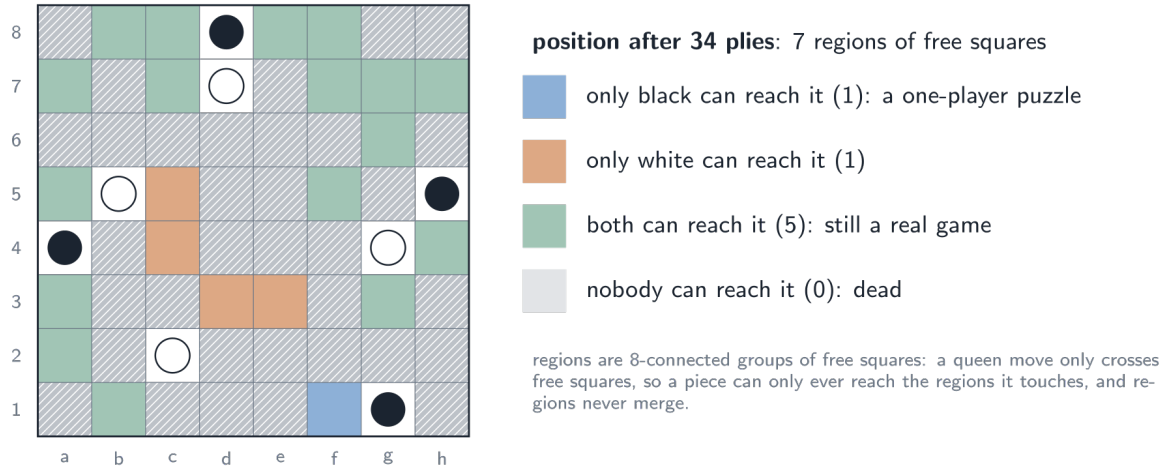


Figure 22: A mid-game position of the real game of Figure 20, split into its regions of free squares.

A region only one player can reach is a one-player puzzle: how many moves can my pieces make in it? The answer depends only on the region's shape and on where my pieces stand along it, not on the rest of the board. Regions are small — a few to a dozen squares after the middle game — and their shapes recur, so their values can be **tabulated once**: the table is keyed by the region's shape (normalised by translation and by the 8 symmetries) and the pieces' positions in it. This is where retrograde analysis fits Yolah: not over whole boards, but over regions. When every region is single-owner, the game is over in all but name: black makes M_{black} more moves, white M_{white}, in whatever order, and $V = \pm(M_{\text{black}} - M_{\text{white}})$.

A region both players can reach is still a real game, and a sum of such games is not simply the sum of their values: each turn, the player chooses *which* region to play in, and tempo matters — the theory of such sums is combinatorial game theory, which was built for exactly this kind of endgame in the Game of the Amazons, Yolah's closest cousin. In practice an exact solver searches the sum, with the regions as its unit: a transposition table per region, the single-owner regions resolved by the table above, and bounds per region (a region of n free squares cannot change the margin by more than n) for pruning.

7.4 An exact endgame solver

The practical form of all this is an exact solver for the last part of the game, and Figure 23 shows why it is within reach. The branching factor collapses as the holes accumulate: about 55 legal moves at the start, 22 on average, fewer than 10 after 35 holes. The whole game tree is around 10^{64} move sequences, but the last 20 plies weigh only about 10^{13} — and alpha-beta with good move ordering explores roughly the square root of that, $10^{6.5}$ positions.

The solver would combine standard techniques, each helped by a property of Yolah:

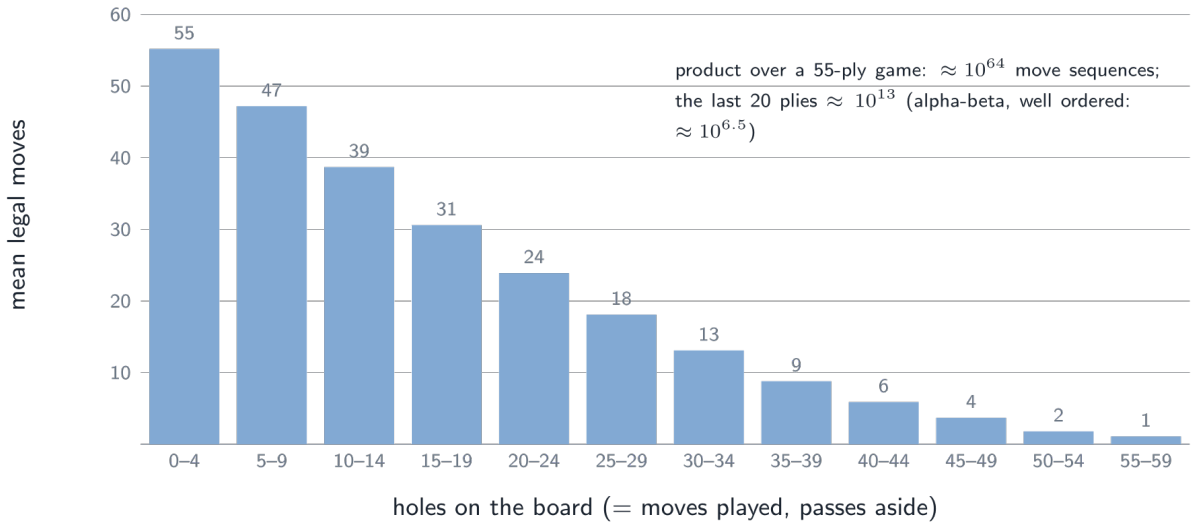


Figure 23: Mean number of legal moves as a function of the number of holes, measured on self-play positions.

- **Null-window alpha-beta (MTD(f)).** A search with window $[\beta - 1, \beta]$ only answers "is $V \geq \beta$?", and prunes far more than a full-window search. Margins in Yolah are tiny (Figure 18: between -2 and $+2$ in 84 % of the games), so two or three such tests, starting from the network's estimate, pin the exact value down.
- **Move ordering by the network.** Alpha-beta is only efficient if it tries the best move first; the policy head gives exactly that ordering.
- **A transposition table**, and the region decomposition above: tables for single-owner regions, per-region bounds, and search restricted to the contested regions.
- **Parallelism.** Independent sub-problems (the children of a node, the positions of a batch) spread over many cores — and the cluster has 592 CPU cores that self-play does not use.

Even without solving the whole game, such a solver pays off:

- **in the search**: below a number of free squares, the MCTS can replace the network's estimate by the exact value — what chess engines do with their tablebases;
- **in training**: exact values are perfect targets for late positions, better than the noisy result z;
- **as a measuring stick**: the network's value can be compared with the truth, ply by ply.

7.5 Can Yolah be solved?

Strongly, no. The positions are at most $\binom{64}{4}\binom{60}{4} \times 2^{56} \times 2 \approx 4.5 \times 10^{28}$ (the placements of the pieces, any subset of the other squares as holes, the side to move). Most are unreachable, but even a small fraction is far beyond any storage.

Weakly: maybe, with a serious effort. The orders of magnitude put Yolah in the same class as Othello:

	checkers	Othello 8×8	Yolah
state space (upper bound)	$\sim 5 \times 10^{20}$	$\sim 10^{28}$	$\sim 4.5 \times 10^{28}$
game length	~50–100 plies	60 plies	56 moves, ~55 plies

	checkers	Othello 8×8	Yolah
weakly solved	2007	2023	—

A weak solution does not need the whole state space: it needs a proof tree, in which the solving side plays one good move per position and only the opponent's replies are all explored. What decides feasibility is how quickly that tree grows with the depth, and there Yolah has arguments on both sides. Against it: a very wide opening, with 55 legal moves at the start where Othello has a handful. For it: endgames that decompose into independent regions, tiny margins that make null-window searches very effective, and a network that already orders moves well.

How to find out. The honest answer comes from measurement, not from estimates:

1. write the exact endgame solver of Section 7.4 and check it against brute force on small positions;
2. solve self-play positions at 50, 45, 40, 35, … holes, recording the time and the number of nodes;
3. fit the growth per ply and extrapolate to the initial position.

If the cost grows by a factor g per ply, going back k plies costs g^k more: the curve says within days whether a weak solution is a matter of weeks of cluster time, of years, or out of reach — and the solver is useful anyway.

What the games already suggest. The self-play games and the 480 000 supervised games agree (Section 6.7.2): from the initial position black wins about 60 % of the games, white about 8 %, and the final margin is almost always between −2 and +2. That makes it likely that the value of Yolah is small — a draw or a black win by one or two moves — but that is a statistic of imperfect play, not a proof. Deciding between "black wins" and "draw" is precisely what a weak solution would do.

8 Code map

file	what lives there
`nnue/cnn_resnet_value_policy_chunked.py`	`Net`, `ResBlock`, `encode_cnn`, `ChunkedShuffleLoader`, the DDP trainer — the production script
`nnue/cnn_resnet_value_policy.py`	the same network with a stock `DataLoader` + `LeanDistributedSampler`
`nnue/resnet_export.py`	BatchNorm folding, the `.bin` writer, the TorchScript trace, the test vectors
`nnue/resnet.{h,cpp}`	`nn::ResNetCPU` — im2col + Eigen GEMM, OpenMP chunking, FTZ/DAZ
`nnue/resnet_torch.{h,cpp}`	`nn::ResNetTorch` — TorchScript on the GPU
`nnue/nn_evaluator.{h,cpp}`	the `Request`/`Result`/`Evaluator` interface, `encode_planes`, `action_index`, `make_evaluator`
`nnue/batched_evaluator.{h,cpp}`	`nn::BatchingEvaluator` — merges several search threads into one backend batch
`player/nn_cache.h`	`az::NNCache` — direct-mapped, lock-striped cache of network outputs

file	what lives there
`player/alphazero_mcts.h`	`SearchParams`, `ChildStat`, `SearchResult`, `Search` — and the algorithm overview
`player/alphazero_mcts.cpp`	everything below: the node table, selection, backup, the worker loop, the root logic
`player/alphazero_mcts_player.{h,cpp}`	the `Player` wrapper and the JSON schema
`test/alphazero_mcts_check.cpp`	search / reuse self-checks and a head-to-head harness
`test/alphazero_check_main.cpp`	the `alphazero_check` CLI
`player/alphazero_selfplay.{h,cpp}`	`TrainingSample` (the row format), `run_selfplay` — many games on one batched network, hot swap
`test/alphazero_learn_main.cpp`	the `alphazero_learn` CLI: `selfplay` and `match`
`nnue/alphazero_learn.py`	the trainer and orchestrator: replay window, loss, symmetries, EMA, export, evaluations
`nnue/alphazero_aux.py`	optional auxiliary heads (`--aux`): future ownership and rest of the margin
`nnue/alphazero_learn.{def,sh}`	the cluster image and the SLURM job
`config/alphazero_mcts_selfplay_player.cfg`	the search settings of self-play
`config/alphazero_mcts_eval_player.cfg`	the search settings of the evaluation matches

The functions worth knowing in `alphazero_mcts.cpp`:

function	role
`select_child`	Section 5.2 — one PUCT argmax, plus forced playouts at the root
`utility_stdev_factor`	$\hat{\sigma}(s)$ from the node's W and W^2
`fpu_value`	Section 5.3 — what an unvisited child is assumed to be worth
`priors_from_logits`	softmax over the legal moves, with `policy_temperature`
`expand`	build the edge vector, store `net_v`, publish `EXPANDED`
`backup` / `undo_pending`	negamax accumulation / virtual-loss rollback
`run_worker`	gather → evaluate → expand → back up, per thread
`get_or_create`, `lookup`, `collect`, `mark`	the node table and its garbage collection
`reuse`, `new_root`, `expand_root`	re-rooting between moves
`root_play_values`	policy target pruning and the LCB adjustment
`make_result`	the statistics, π, and the move choice

9 Parameter reference

All of these are JSON keys of `AlphaZeroMCTSPlayer` (spaces, not underscores) and fields of `az::SearchParams` (underscores).

9.1 Budget and parallelism

key	default	meaning
`microseconds`	1 000 000	wall-clock budget; 0 = use the simulation budget only
`nb simulations`	0	simulation budget; 0 = time only
`nb threads`	1	search threads sharing the graph (`"hardware concurrency"` allowed)
`batch size`	backend hint	leaves gathered per thread per network call
`max collisions`	= batch size	cross-thread collisions that close a batch early
`nn cache`	64	MB of network-output cache; 0 disables it

9.2 PUCT

key	default	meaning
`cpuct exploration`	1.0	c_{puct}
`cpuct exploration log`	0.45	$c_{\log}$; 0 gives a constant coefficient
`cpuct exploration base`	500	c_{base}
`cpuct utility stdev scale`	0.85	λ; 0 disables $\hat{\sigma}$ (plain AlphaZero)
`cpuct utility stdev prior`	0.40	σ_0 — see the calibration note in Section 5.2
`cpuct utility stdev prior weight`	2.0	w_p, in visits
`policy temperature`	1.0	softmax temperature on the priors; >1 flattens

9.3 First play urgency

key	default	meaning
`fpu reduction`	0.2	r away from the root
`root fpu reduction`	0.1	r at the root
`fpu loss prop` / `root fpu loss prop`	0.0	drag the estimate towards a loss
`fpu parent weight by visited policy`	true	blend $\bar{q}$ and v_{net} by the visited mass
`fpu parent weight by visited policy pow`	1.0	the exponent p
`fpu parent weight`	0.0	constant blend, used only when the above is false

9.4 Graph and root

key	default	meaning
`graph search`	true	one node per position; false = a plain tree
`use lcb`	true	play the best lower confidence bound
`lcb stdevs`	5.0	λ_{lcb}
`min visit prop for lcb`	0.20	eligibility threshold, as a fraction of the leader
`policy target pruning`	true	invert PUCT to undo the forced playouts
`reuse tree`	true	re-root instead of starting from scratch

9.5 Self-play

key	default	meaning
`dirichlet alpha` / `dirichlet epsilon`	0.3 / 0.0	root noise; $\epsilon = 0$ disables it
`temperature` / `temperature cutoff`	0.0 / 0	sample the move $\propto \pi^{1/\tau}$ below the cutoff ply
`forced playouts k`	0.0	k; the paper uses 2.0, self-play only
`playout cap fast prob`	0.0	fraction of moves that get the small budget
`nb simulations fast`	0	that small budget

10 Measurements

All figures below: `ResNetCPU`, 256×30 weights, one search thread, WSL2 on a desktop CPU. Reproduce with

```
./alphazero_check --no-bench --sims 3200 --set 'graph search=false'
```

10.1 What graph search actually buys

Four positions, 3 200 simulations each:

position		graph	tree
0 (ply 0)	network calls	3 195	2 552 + 646 cache hits
	distinct positions	**3 195**	2 552
	sims/s	156	181
1 (ply 17)	network calls	3 141	2 632 + 515 cache hits
	distinct positions	**3 141**	2 632

position		graph	tree
	sims/s	159	175
2 (ply 19)	network calls	3 175	2 947 + 231 cache hits
	distinct positions	**3 175**	2 947
	sims/s	156	162
3 (ply 10)	network calls	3 195	2 630 + 570 cache hits
	distinct positions	**3 195**	2 630
	sims/s	155	178

Read this carefully, because the naive reading is backwards. Graph search is **not** a speedup — it is 5–15 % *slower* per simulation. What it changes is what a simulation buys.

In tree mode, 7–20 % of the leaves are transpositions. The search allocates a duplicate node for a position it has already evaluated, the network cache serves the value for free, and the simulation backs that value up along its own path. Those are cheap and they are genuine MCTS updates — the path they refresh is a real one — they are simply *redundant*: no new information entered the search. In graph mode the descent links to the existing node and *keeps going*, ending at a position nobody has evaluated, which costs a real network call.

Same simulation count, same number of nodes, but **20–25 % more distinct positions examined**, and every shared node pools the statistics of all its parents instead of splitting them across copies. Correcting for the lower simulation rate, the advantage is **4–8 % more distinct positions per wall-clock second** — consistently across all four positions — plus the pooling, which no eval count captures.

So the trade is: cheap redundant simulations against fewer, more informative ones. Which of those wins games is not something an eval counter can answer.

> **!** This is not yet a strength measurement
>
> More distinct positions per second is a proxy, not a result. The question "does graph search win games" needs a head-to-head:
>
> ```
> ./alphazero_check --games 200 --time 1000000 --opponent <tree-mode.cfg>
> ```
>
> and a 4–8 % node advantage is worth, on the experience of other engines, some single-digit number of Elo — which a few hundred games cannot separate from noise (200 games carry a standard error of roughly ±25 Elo). Resolving it means thousands of games on the libtorch backend.
> The default is `graph search: true` on the strength of the proxy and of the statistic pooling, not on a measured Elo gain. Nothing in this document should be read as one. The flag exists so the decision can be re-tested cheaply.

10.2 Node allocation

Children are now created lazily, one per descent, instead of eagerly at expansion. At 400 simulations the eager version allocated **20 335** node objects; the lazy one allocates **396** — one per position actually reached. The unreached children cost a 16-byte `Edge` instead of a 56-byte `Node` with its own vector. This one is unambiguous: fewer allocations, less memory, better locality, and it is what makes a node table cheap enough to maintain.

10.3 What LCB and pruning change

At 400 simulations, position 2 (ply 19) with the raw visit counts:

```
f2:d2  N: 194  Q: +0.639  lcb: +0.479
f2:e3  N:  87  Q: +0.759  lcb: +0.533   ← played
```

The most-visited move is not the one with the best lower bound, and LCB plays the second. That is the intended behaviour, but at 400 simulations with $\lambda_{\text{lcb}} = 5$ it is an aggressive override — the eligibility threshold ($87 \quad 0.2 \times 194$) is doing a lot of work. Worth re-checking at the budgets the engine will actually play at.

Policy target pruning is similarly consequential: at position 0 it drives several children with a dozen visits each to $\pi = 0$, because their priors are so small that PUCT would never have visited them — they were reached through FPU, not through their policy. That is exactly what the mechanism is for, but it makes π markedly peakier than the visit distribution, and it should be eyed before it is trained on.

11 References

- Silver et al., *Mastering the game of Go without human knowledge*, Nature 2017 — AlphaGo Zero: PUCT, Dirichlet noise, the two-headed network.
- Silver et al., *A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play*, 2018 — AlphaZero, and the $c_{\text{init}} + \log((N + c_{\text{base}} + 1)/c_{\text{base}})$ coefficient.
- D. J. Wu, *Accelerating Self-Play Learning in Go*, arXiv:1902.10565, 2019 — the KataGo paper: playout cap randomization, forced playouts, policy target pruning.
- KataGo source, `cpp/search/` — `searchexplorehelpers.cpp` for the selection formula, `getFpuValueForChildrenAssumeVisited` for FPU and $\hat{\sigma}$, `searchresults.cpp` for the play-selection values and LCB, `searchhelpers.cpp` for `getSelfUtilityLCBAndRadius`, `searchparams.cpp` for the defaults quoted here.
- Leela Chess Zero — the FPU-reduction formulation this and KataGo share.

12 Annotated source

Every file the document describes, in full, generated from the working tree by `doc/make_appendix.py`. Each one carries an orientation note, a table of the symbols it defines, and its complete source with line numbers — so the sections above can point at `alphazero_mcts.cpp:212` and mean it.

The running commentary lives in the sources themselves rather than being duplicated here; what the tables add is the one-line answer to "what is this for" for every symbol, which a comment sitting next to a definition cannot give you at a glance.

12.1 The search

These five files are the search proper. They depend on `game.h`, `misc.h`, `zobrist.h` and the `nn::Evaluator` interface of the next part, and on nothing else.

12.1.1 `player/alphazero_mcts.h`

317 lines.

The public surface: the parameter block, what a search reports, and the class. The header comment is the algorithm overview — the same one Section 5.2 through Section 5.8 expand on.

`SearchParams` is a plain aggregate with the KataGo defaults baked in, so `az::SearchParams{}` is a usable configuration once a budget is set. Everything private is forward-declared: `Node`, `Edge`, `PathStep` and `Worker` are defined in the `.cpp`, so nothing that includes this header sees the graph representation.

symbol	role
`SearchParams`	every knob, grouped: budget, parallelism, PUCT, FPU, graph, root, self-play
`ChildStat`	one root move as reported: move, prior, raw edge visits, `q`, `q2`, play value, LCB
`SearchResult`	the move, the root value, the counters, `full_search`, the children and π
`SearchResult::to_string`	the one-line-plus-children summary printed by `"verbose": true`
`Search`	the search object: owns the node table, the network cache and the parameters
`Search::search`	run one search from a position and pick a move
`Search::reset`	drop the graph (new game); the network cache survives
`Search::clear_cache`	drop the cached network outputs (after new weights)
`Search::params`	read/write access to the live parameters
`Search::Shard`	one stripe of the node table: a mutex and a `hash → Node*` map

```
#ifndef ALPHAZERO_MCTS_H
#define ALPHAZERO_MCTS_H
// alphazero_mcts.h — AlphaZero-style Monte-Carlo tree search for Yolah, with
// the refinements KataGo added on top of it.
//
// Inspired by MCTS_mem_player.h (same pool-allocated storage, same lock-free
// multi-threading style) but the leaf is evaluated by the two-headed ResNet
// instead of a random playout, and children are selected with PUCT, in the
// KataGo formulation (cpp/search/searchexplorehelpers.cpp):
//
//     a* = argmax_a  Q(s,a) + c(s) · P(s,a) · √(W(s) + 0.01) / (1 + W(s,a))
//     c(s) = [cpuct + cpuct_log · log((W(s) + cpuct_base) / cpuct_base)] · σ̂(s)
//     σ̂(s) = 1 + stdev_scale · (stdev(s) / stdev_prior − 1)
//
// W(s) = Σ_a W(s,a) is the total weight of the *edges* leaving s (one per
// visit, plus one per in-flight virtual loss) and stdev(s) is the running
// standard deviation of the values backed up through s, blended with a prior.
// The σ̂ factor explores more where the children disagree and less where the
// node's value has settled. AlphaZero is the special case
// cpuct_log = stdev_scale = 0.
//
// GRAPH SEARCH ──────────────────────────────────────────────────────────────
// The search space is a DAG, not a tree. Every Yolah move turns the square it
// leaves into a hole (`empty ^= from`), so the number of holes increases by
```

```
// exactly one per move and a position can never repeat: no cycles, ever. At
// the same time the four stones of a side move independently, so move orders
// commute and the same position is reached along very many paths.
//
// The search therefore stores ONE node per distinct position, in a shared
// hash table keyed by the Zobrist hash (plus the score split, which the hash
// does not cover — see node_key in the .cpp). A node reached through k
// different parents accumulates the statistics of all k of them: a line
// refuted in one branch is refuted in every branch that transposes into it.
//
//   • Node  — one per position: visit count, ΣV, ΣV², the network's value,
//             and the edges to its successors.
//   • Edge  — one per (position, move): prior, pointer to the successor node
//             (created on first descent), and this parent's own visit count.
//
// PUCT uses the EDGE visits in the denominator (this parent's share of the
// exploration) and the CHILD NODE's average for Q (everything the search
// knows about that position, whoever explored it). That is exactly KataGo's
// childWeight = rawChildWeight · edgeVisits / childVisits, which for a search
// weighting every visit by 1 collapses to "edge visits".
//
// Search loop (per thread, repeated until the time / simulation budget is spent):
//   1. GATHER up to `batch_size` leaves. Each descent applies a *virtual loss*
//      (a pending visit that counts as a loss) to every node and edge on its
//      path so that the following descents — of this thread and of the others
//      — are steered towards different leaves. A descent that reaches a leaf
//      some other thread is already expanding is a "collision"; it is undone
//      and, after `max_collisions` of them, the batch is closed early.
//      Terminal leaves are scored exactly and backed up at once.
//   2. EVALUATE the batch with one call to nn::Evaluator (a GPU wants a few
//      hundred positions per call; the CPU backend a few dozen).
//   3. EXPAND every leaf with the network's priors (softmax over the legal
//      moves only, optional temperature) and BACK UP its value along every
//      path that reached it, flipping the sign at each ply (negamax
//      convention), converting the pending visits into real ones.
//
// Values are stored per node as W = Σ v from the point of view of the player
// who moved INTO the node, i.e. Q(s,a) = W(child)/N(child) is exactly the
// quantity PUCT needs at the parent. The network's value is for the player to
// move at the evaluated position, so the leaf itself receives -v.
//
// Network cache: before asking the network for a leaf, the search looks the
// position up in an NNCache keyed by its Zobrist hash (player/nn_cache.h). It
// is a second line of defence behind the node table — it also catches
// positions that have dropped out of the graph, and it survives across moves
// and games.
//
// Tree reuse: search(state) looks `state` up in the node table and, if it is
// there, re-roots on it — keeping every statistic gathered so far, however
// deep in the old graph the position was. The nodes that are no longer
// reachable from the new root are then collected (mark and sweep).
//
// ROOT REFINEMENTS ──────────────────────────────────────────────────────────
// Three things happen at the root that do not happen anywhere else:
//   • FORCED PLAYOUTS (KataGo paper §5.1, self-play only): a child that has
//     been visited at least once is forced up to √(k·P(s,a)·W(s)) visits, so
//     that a move the noise made interesting is actually tried a few times
//     instead of being dropped after one bad evaluation.
//   • POLICY TARGET PRUNING (same section): those forced visits are taken
//     back out afterwards — each non-best child is reduced to the visit count
//     that PUCT would retrospectively have given it — so the training target
//     is not polluted by playouts the search was forced to make.
//   • LCB MOVE SELECTION: the move played is the one with the best lower
```

```
//     confidence bound Q − λ·σ(Q), not simply the most visited, among the
//     children with enough visits for the bound to mean anything.
//
// Self-play: everything the driver needs is on SearchResult — the play-value
// distribution π over the root's moves (policy target), the root value, and
// move sampling with a temperature. Dirichlet noise on the root priors and
// PLAYOUT CAP RANDOMIZATION (a large budget with noise and a recorded policy
// target on a fraction of the moves, a small one without on the rest) are
// parameters. A new network is exported with nnue/resnet_export.py and
// swapped in through nn::Evaluator::reload(). See
// player/alphazero_mcts_player.h for the JSON keys.
#include "game.h"
#include "misc.h"
#include "nn_evaluator.h"
#include "nn_cache.h"
#include <atomic>
#include <chrono>
#include <cstdint>
#include <memory_resource>
#include <mutex>
#include <string>
#include <unordered_map>
#include <vector>

namespace az {

struct SearchParams {
    // ── budget: the search stops when either bound is hit (0 = no bound) ──
    uint64_t microseconds     = 1'000'000;   // wall-clock budget
    uint64_t nb_simulations   = 0;           // e.g. 800 for AlphaZero-style self-play
    // ── parallelism ──
    size_t   nb_threads       = 1;           // search threads sharing the graph
    size_t   batch_size       = 32;          // leaves gathered per thread per network call
    size_t   max_collisions   = 0;           // cross-thread collisions closing a batch (0 =
    ↪  batch_size)
    size_t   nn_cache_mb      = 64;          // transposition cache of network outputs (0 = off)

    // ── PUCT (KataGo defaults, cpp/search/searchparams.cpp) ──
    float    cpuct_exploration      = 1.0f;    // c
    float    cpuct_exploration_log  = 0.45f;   // c_log (0 = AlphaZero: no log term)
    float    cpuct_exploration_base = 500.0f;  // c_base
    // Exploration is scaled by σˆ= 1 + scale · (stdev / prior − 1), where stdev
    // is the standard deviation of the values backed up through the node,
    // regularised towards `prior` with weight `prior_weight` visits. Scale 0
    // disables it (σˆ= 1) and gives the plain AlphaZero/LC0 behaviour.
    float    cpuct_utility_stdev_scale        = 0.85f;
    float    cpuct_utility_stdev_prior        = 0.40f;
    float    cpuct_utility_stdev_prior_weight = 2.0f;

    // ── First play urgency ──
    // An unvisited child is worth
    //     baseline − fpu_reduction · √(prior mass of the visited siblings)
    // instead of 0, so that the search does not spread over every child of a
    // clearly bad position. `baseline` is the value of the parent for the
    // player to move there: its running average Q blended with its raw network
    // value, the average weighing (visited prior mass)^pow — an unexplored
    // node trusts the network, a well explored one its own statistics.
    // fpu_loss_prop finally drags the estimate a proportion of the way towards
    // a loss (−1). The root uses its own, gentler, constants.
    float    fpu_reduction      = 0.2f;
    float    root_fpu_reduction = 0.1f;
    float    fpu_loss_prop      = 0.0f;
    float    root_fpu_loss_prop = 0.0f;
```

```
    bool     fpu_parent_weight_by_visited_policy     = true;
    float    fpu_parent_weight_by_visited_policy_pow = 1.0f;
    // Used only when the above is false: a constant weight for the network
    // value in the blend (0 = use the running average alone).
    float    fpu_parent_weight  = 0.0f;

    // Softmax temperature applied to the policy logits at expansion (>1
    // flattens the priors; LC0 uses ~1.36 for its nets).
    float    policy_temperature = 1.0f;

    // ── graph search ──
    // Share one node between every path that reaches the same position. Off
    // reverts to a plain tree (each path gets its own node), which is only
    // useful for measuring what the sharing is worth.
    bool     graph_search     = true;

    // ── root: move choice ──
    // Play the move with the best lower confidence bound Q − lcb_stdevs·σ(Q)
    // among the children holding at least min_visit_prop_for_lcb of the most
    // visited child's visits, instead of the most visited move.
    bool     use_lcb               = true;
    float    lcb_stdevs            = 5.0f;
    float    min_visit_prop_for_lcb = 0.20f;
    // Take the forced playouts back out of the root's play values, so that the
    // policy target reflects what PUCT would have wanted (KataGo always does
    // this at the root; it is what makes forced playouts safe to train on).
    bool     policy_target_pruning  = true;

    // ── self-play exploration ──
    float    dirichlet_alpha    = 0.3f;      // Dir(α) noise mixed into the root priors
    float    dirichlet_epsilon  = 0.0f;      // 0 = no noise (competitive play)
    float    temperature        = 0.0f;      // move sampling ∝ π^(1/τ); 0 = argmax
    int      temperature_cutoff = 0;         // sample only while ply < cutoff (0 = never sample)
    // Force every root child that has been visited at least once up to
    // √(k·P·W) visits. 0 = off; the paper uses k = 2 during self-play only.
    float    forced_playouts_k  = 0.0f;
    // Playout cap randomization: with probability `fast_prob` the search runs
    // `nb_simulations_fast` simulations with no root noise, no forced playouts
    // and SearchResult::full_search = false — the driver must not record a
    // policy target for those moves. Needs nb_simulations > 0. 0 = off.
    float    playout_cap_fast_prob = 0.0f;
    uint64_t nb_simulations_fast   = 0;

    // ── misc ──
    bool     reuse_tree       = true;
    uint64_t seed             = 0;           // 0 = random_device
};

struct ChildStat {
    Move  move;
    float prior;        // network prior P(s,a) (after noise if any)
    uint32_t visits;    // edge visits N(s,a), before any pruning
    float q;            // Q(s,a) for the side to move at the root, ∈ [-1, 1]
    float q2;           // mean of Q² over the child's visits (for the LCB variance)
    float play_value;   // visits after policy target pruning / LCB adjustment
    float lcb;          // Q(s,a) − lcb_stdevs · σ(Q), or -2 if not computed
};

struct SearchResult {
    Move     best_move  = Move::none();
    float    root_value = 0.0f;         // Q of the root for the side to move (from visits)
    float    net_value  = 0.0f;         // raw network value of the root position
    uint64_t nb_simulations = 0;        // simulations run in this call (reused ones excluded)
```

```
    uint32_t root_visits = 0;           // total visits of the root (including reused)
    uint64_t nb_collisions = 0;         // cross-thread collisions (undone descents)
    uint64_t nb_cache_hits = 0;         // leaves expanded from the network cache
    uint64_t nb_evaluations = 0;        // positions sent to the network
    uint64_t nb_transpositions = 0;     // edges that linked to an existing node
    bool     full_search = true;        // false = a fast playout-cap search: no policy target
    double   seconds = 0.0;
    size_t   tree_nodes = 0;            // distinct positions currently in the graph
    std::vector<ChildStat> children;    // sorted by play value, descending
    // Training target π(a): the root's play values, normalised. With
    // policy_target_pruning these are the visit counts with the forced
    // playouts taken back out.
    std::vector<float> policy;
    std::string to_string(size_t max_children = 8) const;
};

class Search {
public:
    Search(nn::Evaluator& evaluator, const SearchParams& params, std::pmr::memory_resource*
    ↪ memory);
    ~Search();
    Search(const Search&) = delete;
    Search& operator=(const Search&) = delete;

    // Run a search from `state` (must not be game over) and pick a move.
    SearchResult search(const Yolah& state);
    // Forget the graph (call at the start of a new game). The network cache is
    // kept: it only depends on the weights.
    void reset();
    // Drop the cached network outputs (after loading new weights).
    void clear_cache();

    SearchParams&       params()       { return prm; }
    const SearchParams& params() const { return prm; }
    size_t nb_nodes() const { return node_count.load(std::memory_order_relaxed); }

private:
    struct Node;
    struct Edge;
    struct Worker;   // per-thread state (buffers, PRNG)
    struct PathStep; // one ply of a descent: the node reached and the edge taken

    enum : uint8_t { UNEXPANDED = 0, EXPANDING = 1, EXPANDED = 2, TERMINAL = 3 };

    // ── node table (one entry per distinct position) ──
    static constexpr size_t NB_SHARDS = 64;
    struct Shard {
        std::mutex mutex;
        std::unordered_map<uint64_t, Node*> nodes;
    };
    Node* get_or_create(uint64_t key, Worker* w);
    Node* lookup(uint64_t key);
    Node* new_node();
    void  destroy_all();
    void  collect(Node* keep);        // mark and sweep from the new root
    void  mark(Node* node);

    // graph management
    bool  reuse(const Yolah& state, uint64_t key);
    void  new_root(const Yolah& state, uint64_t key);
    void  expand_root(const Yolah& state);
    void  apply_root_noise(Worker& w);
    // search
```

```
    void  run_worker(Worker& w, const Yolah& root_state);
    uint32_t select_child(const Node& node, bool is_root) const;
    float utility_stdev_factor(const Node& node, uint32_t visits) const;
    float fpu_value(const Node& node, uint32_t visits, float visited_policy_mass, bool is_root)
     ↪  const;
    void  priors_from_logits(const nn::Result& result, size_t nb_moves, float* priors) const;
    void  expand(Node& node, const Yolah::MoveList& moves, const float* priors, float value);
    void  backup(const PathStep* path, size_t length, float leaf_value);
    void  undo_pending(const PathStep* path, size_t length);
    static float terminal_value(const Yolah& state);
    // result
    SearchResult make_result(const Yolah& state, double seconds, uint64_t sims,
                             uint64_t collisions, Worker& w) const;
    void root_play_values(std::vector<ChildStat>& children) const;

    nn::Evaluator& evaluator;
    SearchParams prm;
    std::pmr::polymorphic_allocator<> alloc;
    Shard shards[NB_SHARDS];
    Node* root = nullptr;
    Yolah root_state;                       // position at the root (valid when root != nullptr)
    uint64_t root_key = 0;                  // its node-table key
    std::vector<float> root_priors;         // root priors after Dirichlet noise (empty = none)
    uint32_t mark_counter = 0;
    std::atomic<uint64_t> node_count{0};
    std::atomic<uint64_t> simulations{0};  // simulations completed in the current search
    std::atomic<uint64_t> reserved{0};     // simulations claimed by the workers (budget
     ↪  accounting)
    std::atomic<uint64_t> collisions{0};
    std::atomic<uint64_t> cache_hits{0};
    std::atomic<uint64_t> evaluations{0};
    std::atomic<uint64_t> transpositions{0};
    std::unique_ptr<NNCache> cache;
    std::atomic<bool> stop{false};
    std::chrono::steady_clock::time_point deadline;
    uint64_t sim_budget = 0;                // simulations for the current search (playout cap)
    bool     full_search = true;            // false = fast search: no noise, no forced playouts
    PRNG     rng;                           // search-level PRNG (playout cap randomization)
    uint64_t base_seed;
    uint64_t search_counter = 0;
};

} // namespace az

#endif
```

12.1.2 `player/alphazero_mcts.cpp`

857 lines.

The implementation, in the order the search uses it: the graph representation, the node table and its garbage collection, re-rooting, the selection rule, the per-thread worker loop, and finally the root post-processing that turns visit counts into a move and a training target.

Two invariants hold everywhere and explain most of the code:

- a node's `w` is the sum of values **for the player who moved into it**, so `w/n` read from the parent is already the parent's $Q(s, a)$ and the sign flips once per ply on the way up;

- `state` is the only atomic with ordering that matters. `EXPANDED` is stored with release and read with acquire, which is what makes the edge vector safe to read without a lock; every statistic is relaxed, because a stale read only changes which leaf a descent picks.

symbol	role
`Search::Edge`	one per (position, move): prior, successor pointer, this parent's visit and pending counts
`Search::Node`	one per position: `n`, `pending`, `w`, `w2`, `state`, the GC mark, the network value, the edges
`Search::PathStep`	one ply of a descent: the node reached and the edge taken to reach it
`Search::Worker`	per-thread buffers: PRNG, the batch's leaves, its descents, the path arena, the request/result spans
`same_position`	full position equality, used by the tree-mode re-rooting scan
`mix`	splitmix64 finaliser, used to fold the score into the node key
`node_key`	Zobrist hash ⊕ mixed black score — see Section 5.6.1 for why the score is needed
`Search::terminal_value`	the exact result of a finished game, for the player to move
`Search::new_node`	allocate one node from the pool resource and count it
`Search::lookup`	find a position in the node table (used by re-rooting)
`Search::destroy_all`	destroy every node; the table is the ownership list
`Search::mark` / `collect`	mark and sweep from the new root; safe as a plain recursion because the graph is acyclic
`Search::get_or_create`	table lookup or insert, under the shard lock
`Search::new_root`	start a fresh graph at a position
`Search::reuse`	re-root by table lookup (graph mode) or by the two-ply scan (tree mode), then collect
`Search::expand_root`	the one unbatched network call, for the root itself
`Search::apply_root_noise`	mix Dirichlet noise into a *copy* of the root priors, leaving the graph's own priors alone
`Search::utility_stdev_factor`	$\hat{\sigma}(s)$ from the node's `w` and `w2` — Section 5.2
`Search::fpu_value`	what an unvisited child is assumed to be worth — Section 5.3
`Search::select_child`	one PUCT argmax, plus the forced-playout override at the root
`Search::priors_from_logits`	softmax over the legal moves only, with `policy_temperature`
`Search::expand`	build the edge vector, store the network value, publish `EXPANDED`
`Search::backup`	negamax accumulation into every node and edge of one path
`Search::undo_pending`	roll the virtual loss back on an abandoned descent

symbol	role
`Search::run_worker`	gather → evaluate → expand → back up, until the budget is spent
`Search::root_play_values`	policy target pruning and the LCB adjustment — Section 5.7
`Search::make_result`	assemble the statistics, π and the move choice

```cpp
// alphazero_mcts.cpp — see alphazero_mcts.h for the algorithm overview.
#include "alphazero_mcts.h"
#include "misc.h"
#include "zobrist.h"
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <limits>
#include <random>
#include <sstream>
#include <thread>

namespace az {

// ── Graph node and edge ──────────────────────────────────────────────
// One Node per distinct position, one Edge per (position, move). All
// statistics are atomics updated with relaxed ordering: they are heuristics, a
// slightly stale read only changes which leaf a descent picks, never the
// graph's integrity. The only synchronisation that matters is `state`
// (acquire/release) — a thread that observes EXPANDED is guaranteed to see the
// fully built edge vector — and Edge::child, published the same way.
struct Search::Edge {
    Move  action = Move::none();        // move leading to the successor
    float prior  = 0.0f;                // P(s,a) from the policy head
    std::atomic<Search::Node*> child{nullptr};   // successor, created on first descent
    std::atomic<uint32_t> n{0};         // edge visits: this parent's share of the child
    std::atomic<uint32_t> pending{0};   // in-flight visits through this edge

    Edge() = default;
    Edge(Move m, float p) : action(m), prior(p) {}
    // Atomics are not movable; the vector needs a move constructor anyway.
    Edge(const Edge& o)
        : action(o.action), prior(o.prior), child(o.child.load()),
          n(o.n.load()), pending(o.pending.load()) {}
    Edge& operator=(const Edge& o) {
        action = o.action; prior = o.prior;
        child = o.child.load(); n = o.n.load(); pending = o.pending.load();
        return *this;
    }
};

struct Search::Node {
    std::atomic<uint32_t> n{0};         // completed visits N(s), from every parent
    std::atomic<uint32_t> pending{0};   // in-flight visits (virtual loss), each counted as a loss
    std::atomic<float>    w{0.0f};      // Σ backed-up values for the player who moved INTO this node
    std::atomic<float>    w2{0.0f};     // Σ of their squares (sign-free: stdev of the subgraph's values)
    std::atomic<uint8_t>  state{UNEXPANDED};
    uint32_t mark = 0;                  // garbage collection: last mark_counter that reached it
    float    net_v = 0.0f;              // network value at this node (player to move here)
    std::pmr::vector<Edge> children;

    using allocator_type = std::pmr::polymorphic_allocator<>;
```

```
    explicit Node(allocator_type a = {}) : children(a) {}
};

// One ply of a descent: the node reached and the edge that was taken to reach
// it (null for the root, which no edge leads to).
struct Search::PathStep {
    Node* node;
    Edge* edge;
};

// ── Per-thread state ──────────────────────────────────────────────────────
struct Search::Worker {
    static constexpr size_t MAX_PATH = Yolah::MAX_NB_PLIES + 8;   // root → deepest leaf
    struct Leaf {
        Node*    node;
        uint64_t hash;           // Zobrist key, for the network cache
        Yolah    state;
        Yolah::MoveList moves;
    };
    // A descent that reached `leaf`: the value evaluated there is backed up
    // along this path. Two descents of the same batch can reach the same leaf
    // through different paths — under graph search even through different
    // parents — so every descent keeps its own path.
    struct Descent {
        uint32_t leaf;
        uint32_t path_len;
    };
    PRNG prng;
    std::vector<Leaf> leaves;            // distinct leaves to evaluate
    std::vector<Descent> descents;       // one per gathered visit
    std::vector<PathStep> paths;         // (batch_size + 1) × MAX_PATH steps
    std::vector<nn::Request> requests;
    std::vector<nn::Result> results;

    Worker(uint64_t seed, size_t batch_size)
        : prng(seed ? seed : 0x9E3779B97F4A7C15ull),
          paths((batch_size + 1) * MAX_PATH, PathStep{nullptr, nullptr}),
          results(batch_size) {
        leaves.reserve(batch_size);
        descents.reserve(batch_size);
        requests.reserve(batch_size);
    }
    PathStep* path(size_t k) { return paths.data() + k * MAX_PATH; }
};

// ── helpers ───────────────────────────────────────────────────────────────
namespace {
    bool same_position(const Yolah& a, const Yolah& b) {
        return a.nb_plies() == b.nb_plies() &&
               a.bitboard(Yolah::BLACK) == b.bitboard(Yolah::BLACK) &&
               a.bitboard(Yolah::WHITE) == b.bitboard(Yolah::WHITE) &&
               a.empty_bitboard() == b.empty_bitboard() &&
               a.score() == b.score();
    }

    uint64_t mix(uint64_t x) {
        x ^= x >> 30; x *= 0xBF58476D1CE4E5B9ull;
        x ^= x >> 27; x *= 0x94D049BB133111EBull;
        return x ^ (x >> 31);
    }

    // Key of a position in the node table. zobrist::hash covers the board and
    // the side to move — exactly what the network sees, which is why the
```

```cpp
    // network cache can key on it alone. It does NOT cover how the points
    // scored so far split between the two players: two positions with the same
    // board can differ there (a pass scores nothing), and the terminal value
    // depends on the split. Nodes carry statistics, not just network outputs,
    // so their key has to separate them.
    uint64_t node_key(uint64_t zobrist_hash, const Yolah& state) {
        return zobrist_hash ^ mix(uint64_t(state.score().first) + 1);
    }
}

float Search::terminal_value(const Yolah& state) {
    const auto [black, white] = state.score();
    const int diff = (state.current_player() == Yolah::BLACK ? 1 : -1) * (int(black) -
    ↪  int(white));
    return diff > 0 ? 1.0f : (diff < 0 ? -1.0f : 0.0f);
}

// ── construction / node table ──────────────────────────────────────
Search::Search(nn::Evaluator& evaluator, const SearchParams& params, std::pmr::memory_resource*
↪  memory)
    : evaluator(evaluator), prm(params), alloc(memory),
      rng(params.seed ? params.seed : 0x2545F4914F6CDD1Dull),
      base_seed(params.seed ? params.seed : std::random_device{}()) {
    if (prm.microseconds == 0 && prm.nb_simulations == 0)
        throw std::invalid_argument("az::Search: either microseconds or nb_simulations must be >
        ↪  0");
    prm.nb_threads = std::max<size_t>(1, prm.nb_threads);
    prm.batch_size = std::max<size_t>(1, prm.batch_size);
    cache = std::make_unique<NNCache>(prm.nn_cache_mb << 20);
}

Search::~Search() {
    destroy_all();
}

void Search::clear_cache() {
    cache->clear();
}

Search::Node* Search::new_node() {
    node_count.fetch_add(1, std::memory_order_relaxed);
    return alloc.new_object<Node>();
}

Search::Node* Search::lookup(uint64_t key) {
    Shard& sh = shards[key % NB_SHARDS];
    std::lock_guard lock(sh.mutex);
    auto it = sh.nodes.find(key);
    return it == sh.nodes.end() ? nullptr : it->second;
}

void Search::destroy_all() {
    for (Shard& sh : shards) {
        for (auto& [key, node] : sh.nodes) alloc.delete_object(node);
        sh.nodes.clear();
    }
    root = nullptr;
    node_count = 0;
}

void Search::reset() {
    destroy_all();
    root_priors.clear();
```

```
}

// Depth-first marking of everything still reachable from the new root. The
// position graph is acyclic (every move fills a square, so the number of holes
// strictly increases), hence a plain recursion with a "already marked" test
// terminates and visits each node once.
void Search::mark(Node* node) {
    if (!node || node->mark == mark_counter) return;
    node->mark = mark_counter;
    if (node->state.load(std::memory_order_relaxed) != EXPANDED) return;
    for (Edge& e : node->children) mark(e.child.load(std::memory_order_relaxed));
}

void Search::collect(Node* keep) {
    ++mark_counter;
    mark(keep);
    uint64_t freed = 0;
    for (Shard& sh : shards) {
        for (auto it = sh.nodes.begin(); it != sh.nodes.end();) {
            if (it->second->mark != mark_counter) {
                alloc.delete_object(it->second);
                it = sh.nodes.erase(it);
                ++freed;
            } else {
                ++it;
            }
        }
    }
    node_count.fetch_sub(freed, std::memory_order_relaxed);
}

// Find (or create) the node for `key`. Under graph search the key identifies
// the position, so every path reaching it gets the same node; in tree mode the
// caller passes a fresh unique key and every edge gets its own node.
Search::Node* Search::get_or_create(uint64_t key, Worker* w) {
    Shard& sh = shards[key % NB_SHARDS];
    std::lock_guard lock(sh.mutex);
    auto [it, inserted] = sh.nodes.try_emplace(key, nullptr);
    if (!inserted) {
        transpositions.fetch_add(1, std::memory_order_relaxed);
        return it->second;
    }
    (void)w;
    it->second = new_node();
    return it->second;
}

void Search::new_root(const Yolah& state, uint64_t key) {
    destroy_all();
    root = get_or_create(key, nullptr);
    root_state = state;
    root_key = key;
}

bool Search::reuse(const Yolah& state, uint64_t key) {
    if (!root) return false;
    if (prm.graph_search) {
        // The node table is the whole tree-reuse mechanism: whatever depth the
        // position sat at in the old graph, its statistics are right there.
        Node* node = lookup(key);
        if (!node) return false;
        root = node;
        root_state = state;
```

```
        root_key = key;
        collect(root);                // drop everything the new root cannot reach
        return true;
    }
    // Tree mode: the position can only be the root, one of its children or one
    // of its grandchildren.
    if (same_position(state, root_state)) return true;
    if (root->state.load(std::memory_order_acquire) != EXPANDED) return false;
    for (Edge& e : root->children) {
        Node* child = e.child.load(std::memory_order_acquire);
        if (!child) continue;
        Yolah s1 = root_state;
        s1.play(e.action);
        if (same_position(s1, state)) {
            root = child; root_state = state; root_key = key; collect(root);
            return true;
        }
        if (child->state.load(std::memory_order_acquire) != EXPANDED) continue;
        for (Edge& g : child->children) {
            Node* grandchild = g.child.load(std::memory_order_acquire);
            if (!grandchild) continue;
            Yolah s2 = s1;
            s2.play(g.action);
            if (same_position(s2, state)) {
                root = grandchild; root_state = state; root_key = key; collect(root);
                return true;
            }
        }
    }
    return false;
}

void Search::expand_root(const Yolah& state) {
    if (state.game_over()) {
        root->state.store(TERMINAL, std::memory_order_release);
        return;
    }
    Yolah::MoveList moves;
    state.moves(moves);
    nn::Request request{&state, moves.begin(), static_cast<uint16_t>(moves.size())};
    nn::Result result;
    evaluator.evaluate(std::span<const nn::Request>(&request, 1), std::span<nn::Result>(&result,
↪ 1));
    root->state.store(EXPANDING, std::memory_order_relaxed);
    float priors[Yolah::MAX_NB_MOVES];
    priors_from_logits(result, moves.size(), priors);
    expand(*root, moves, priors, result.value);
}

void Search::apply_root_noise(Worker& w) {
    root_priors.clear();
    if (!full_search || prm.dirichlet_epsilon <= 0.0f || root->children.empty()) return;
    std::mt19937_64 gen(w.prng.rand<uint64_t>());
    std::gamma_distribution<float> gamma(prm.dirichlet_alpha, 1.0f);
    const size_t nb = root->children.size();
    root_priors.resize(nb);
    float sum = 0.0f;
    for (size_t i = 0; i < nb; i++) { root_priors[i] = gamma(gen); sum += root_priors[i]; }
    for (size_t i = 0; i < nb; i++) {
        const float noise = sum > 0.0f ? root_priors[i] / sum : 1.0f / nb;
        root_priors[i] = (1.0f - prm.dirichlet_epsilon) * root->children[i].prior +
↪ prm.dirichlet_epsilon * noise;
    }
```

```cpp
}

// ── search primitives ──────────────────────────────────────────────────
// KataGo's σ^factor: the exploration constant is scaled by how much the values
// backed up through this node disagree, measured against a prior stdev.
float Search::utility_stdev_factor(const Node& node, uint32_t visits) const {
    if (prm.cpuct_utility_stdev_scale <= 0.0f) return 1.0f;
    const float prior = prm.cpuct_utility_stdev_prior;
    float stdev = prior;
    if (visits > 1) {
        const float n = static_cast<float>(visits);
        // The sign of the mean is irrelevant here, only its square is used.
        const float mean    = node.w.load(std::memory_order_relaxed) / n;
        const float mean_sq = mean * mean;
        // Relaxed reads of w and w2 can be slightly out of step; clamp so that
        // the variance never comes out negative.
        const float sq_avg  = std::max(node.w2.load(std::memory_order_relaxed) / n, mean_sq);
        const float pw = prm.cpuct_utility_stdev_prior_weight;
        stdev = std::sqrt(std::max(0.0f,
            ((mean_sq + prior * prior) * pw + sq_avg * n) / (pw + n - 1.0f) - mean_sq));
    }
    return 1.0f + prm.cpuct_utility_stdev_scale * (stdev / prior - 1.0f);
}

// Value assumed for a child that has not been visited yet (first play urgency).
float Search::fpu_value(const Node& node, uint32_t visits, float visited_policy_mass, bool
↪  is_root) const {
    // Everything here is from the point of view of the player to move at
    // `node`: node.w is for the player who moved into it, hence the sign flip,
    // and node.net_v is the network's value for the player to move.
    const float parent_q = visits > 0 ? -node.w.load(std::memory_order_relaxed) / visits :
    ↪  node.net_v;
    float baseline = parent_q;
    if (prm.fpu_parent_weight_by_visited_policy) {
        const float p = prm.fpu_parent_weight_by_visited_policy_pow;
        const float mass = p == 1.0f ? visited_policy_mass : std::pow(visited_policy_mass, p);
        const float avg_weight = std::min(1.0f, mass);
        baseline = avg_weight * parent_q + (1.0f - avg_weight) * node.net_v;
    } else if (prm.fpu_parent_weight > 0.0f) {
        baseline = prm.fpu_parent_weight * node.net_v + (1.0f - prm.fpu_parent_weight) * parent_q;
    }
    const float reduction = (is_root ? prm.root_fpu_reduction : prm.fpu_reduction) *
    ↪  std::sqrt(visited_policy_mass);
    const float loss_prop = is_root ? prm.root_fpu_loss_prop : prm.fpu_loss_prop;
    const float fpu = baseline - reduction;
    return fpu + (-1.0f - fpu) * loss_prop;    // values live in [-1, 1]: a loss is -1
}

uint32_t Search::select_child(const Node& node, bool is_root) const {
    const auto& children = node.children;
    const uint32_t nb = static_cast<uint32_t>(children.size());
    const uint32_t n_parent = node.n.load(std::memory_order_relaxed);
    const float* priors = (is_root && !root_priors.empty()) ? root_priors.data() : nullptr;

    // Total weight of the edges leaving this node (KataGo's totalChildWeight —
    // the node's own visit is not part of it) and prior mass of the moves that
    // have been tried at least once.
    float total_child_weight = 0.0f, visited_mass = 0.0f;
    for (uint32_t i = 0; i < nb; i++) {
        const Edge& e = children[i];
        const uint32_t w = e.n.load(std::memory_order_relaxed) +
        ↪  e.pending.load(std::memory_order_relaxed);
        total_child_weight += static_cast<float>(w);
```

```
        if (w > 0) visited_mass += priors ? priors[i] : e.prior;
    }

    const float cpuct = prm.cpuct_exploration + prm.cpuct_exploration_log *
        std::log((total_child_weight + prm.cpuct_exploration_base) / prm.cpuct_exploration_base);
    // The √W of the PUCT formula, with KataGo's offset keeping it positive at W = 0.
    const float explore_scaling = cpuct * utility_stdev_factor(node, n_parent)
                                * std::sqrt(total_child_weight + 0.01f);
    const float fpu = fpu_value(node, n_parent, visited_mass, is_root);
    // Forced playouts (self-play only): a move that has been tried once is
    // pushed up to √(k·P·W) visits before PUCT is allowed to drop it.
    const bool forced = is_root && full_search && prm.forced_playouts_k > 0.0f;
    const float forced_scale = forced ? prm.forced_playouts_k * total_child_weight : 0.0f;

    uint32_t best = 0;
    float best_score = -std::numeric_limits<float>::infinity();
    for (uint32_t i = 0; i < nb; i++) {
        const Edge& e = children[i];
        const uint32_t n = e.n.load(std::memory_order_relaxed);
        const uint32_t pending = e.pending.load(std::memory_order_relaxed);
        const uint32_t n_eff = n + pending;
        const float p = priors ? priors[i] : e.prior;
        // Q comes from the child NODE — everything the search knows about that
        // position, whichever parent paid for it — while the PUCT denominator
        // is this EDGE's own visit count.
        float q = fpu;
        const Node* child = e.child.load(std::memory_order_acquire);
        if (child) {
            const uint32_t cn = child->n.load(std::memory_order_relaxed);
            const uint32_t cp = child->pending.load(std::memory_order_relaxed);
            // Every pending visit is counted as a loss (-1): the virtual loss.
            if (cn + cp > 0)
                q = (child->w.load(std::memory_order_relaxed) - static_cast<float>(cp)) / (cn +
↪   cp);
        }
        float score = q + explore_scaling * p / (1.0f + n_eff);
        if (forced && n_eff > 0 && static_cast<float>(n_eff) < std::sqrt(forced_scale * p))
            score += 1e6f;
        if (score > best_score) { best_score = score; best = i; }
    }
    return best;
}

void Search::priors_from_logits(const nn::Result& result, size_t nb_moves, float* priors) const {
    // Softmax over the legal moves only, with the policy temperature.
    float max_logit = -std::numeric_limits<float>::infinity();
    for (size_t m = 0; m < nb_moves; m++) max_logit = std::max(max_logit, result.logits[m]);
    float sum = 0.0f;
    const float inv_t = 1.0f / prm.policy_temperature;
    for (size_t m = 0; m < nb_moves; m++) {
        priors[m] = std::exp((result.logits[m] - max_logit) * inv_t);
        sum += priors[m];
    }
    for (size_t m = 0; m < nb_moves; m++) priors[m] /= sum;
}

void Search::expand(Node& node, const Yolah::MoveList& moves, const float* priors, float value) {
    const size_t nb = moves.size();
    node.children.reserve(nb);
    for (size_t m = 0; m < nb; m++) node.children.emplace_back(moves[m], priors[m]);
    node.net_v = value;
    node.state.store(EXPANDED, std::memory_order_release);   // publishes the edges
}
```

```

void Search::backup(const PathStep* path, size_t length, float leaf_value) {
    // The leaf's value is for the player to move at the leaf; a node's W is
    // for the player who moved into it, hence the initial sign flip, and one
    // more flip per ply on the way up (negamax).
    float v = -leaf_value;
    // Σ v² is the same at every ply: the negamax flip disappears in the square.
    const float v2 = leaf_value * leaf_value;
    for (size_t i = length; i-- > 0;) {
        Node* node = path[i].node;
        node->w.fetch_add(v, std::memory_order_relaxed);
        node->w2.fetch_add(v2, std::memory_order_relaxed);
        node->n.fetch_add(1, std::memory_order_relaxed);
        node->pending.fetch_sub(1, std::memory_order_relaxed);
        if (Edge* e = path[i].edge) {
            e->n.fetch_add(1, std::memory_order_relaxed);
            e->pending.fetch_sub(1, std::memory_order_relaxed);
        }
        v = -v;
    }
}

void Search::undo_pending(const PathStep* path, size_t length) {
    for (size_t i = 0; i < length; i++) {
        path[i].node->pending.fetch_sub(1, std::memory_order_relaxed);
        if (Edge* e = path[i].edge) e->pending.fetch_sub(1, std::memory_order_relaxed);
    }
}

// ── the per-thread search loop ──────────────────────────────────────
void Search::run_worker(Worker& w, const Yolah& root_position) {
    Node* const root_node = root;
    const uint64_t root_hash = zobrist::hash(root_position);
    const size_t max_collisions = prm.max_collisions ? prm.max_collisions : prm.batch_size;
    uint64_t local_collisions = 0, local_hits = 0, local_evals = 0;
    float priors[Yolah::MAX_NB_MOVES];
    // Tree mode: every edge gets its own node, under a key nothing else uses.
    static std::atomic<uint64_t> unique_key{0x8000000000000000ull};

    while (!stop.load(std::memory_order_relaxed)) {
        // ── 1. gather ──
        w.leaves.clear();
        w.descents.clear();
        w.requests.clear();
        size_t budget = prm.batch_size;
        if (sim_budget) {
            // Claim visits from the shared budget so that the threads together
            // never exceed it (unused claims are returned below).
            uint64_t claimed = reserved.load(std::memory_order_relaxed);
            for (;;) {
                if (claimed >= sim_budget) { budget = 0; break; }
                const uint64_t want = std::min<uint64_t>(budget, sim_budget - claimed);
                if (reserved.compare_exchange_weak(claimed, claimed + want,
                 ↪  std::memory_order_relaxed)) {
                    budget = want;
                    break;
                }
            }
            if (budget == 0) {
                if (simulations.load(std::memory_order_relaxed) >= sim_budget) break;
                // Other threads still hold claims: wait for their batch instead of spinning.
                evaluator.evaluate({}, {});
                std::this_thread::yield();
```

```
                continue;
            }
        }
        size_t batch_collisions = 0, gathered_visits = 0;
        while (gathered_visits < budget && batch_collisions <= max_collisions) {
            PathStep* path = w.path(w.descents.size());
            size_t len = 0;
            Node* node = root_node;
            Yolah state = root_position;
            uint64_t hash = root_hash;
            node->pending.fetch_add(1, std::memory_order_relaxed);
            path[len++] = {node, nullptr};
            for (;;) {
                uint8_t st = node->state.load(std::memory_order_acquire);
                if (st == EXPANDED) {
                    Edge& e = node->children[select_child(*node, node == root_node)];
                    const uint8_t mover = state.current_player();
                    state.play(e.action);
                    hash = zobrist::update(hash, mover, e.action);
                    e.pending.fetch_add(1, std::memory_order_relaxed);
                    Node* child = e.child.load(std::memory_order_acquire);
                    if (!child) {
                        // Link the edge to its successor, sharing the node with
                        // every other path that reaches the same position. The
                        // shard lock serialises the linking of this edge: in
                        // graph mode it is picked by the position key, in tree
                        // mode by the edge itself.
                        const uint64_t key = prm.graph_search ? node_key(hash, state)
                                                              : unique_key.fetch_add(1,
                                                                std::memory_order_relaxed);
                        Shard& sh = shards[(prm.graph_search ? key : std::hash<const void*>{}(&e))
% NB_SHARDS];
                        std::unique_lock lock(sh.mutex);
                        child = e.child.load(std::memory_order_acquire);
                        if (!child) {
                            auto [it, inserted] = sh.nodes.try_emplace(key, nullptr);
                            if (inserted) it->second = new_node();
                            else          transpositions.fetch_add(1, std::memory_order_relaxed);
                            child = it->second;
                            e.child.store(child, std::memory_order_release);
                        }
                    }
                    child->pending.fetch_add(1, std::memory_order_relaxed);
                    path[len++] = {child, &e};
                    node = child;
                    continue;
                }
                if (st == TERMINAL) {
                    backup(path, len, terminal_value(state));
                    simulations.fetch_add(1, std::memory_order_relaxed);
                    ++gathered_visits;
                    break;
                }
                if (st == UNEXPANDED) {
                    uint8_t expected = UNEXPANDED;
                    if (node->state.compare_exchange_strong(expected, EXPANDING,
                        std::memory_order_acq_rel,
                                                            std::memory_order_acquire)) {
                        ++gathered_visits;
                        if (state.game_over()) {
                            node->state.store(TERMINAL, std::memory_order_release);
                            backup(path, len, terminal_value(state));
                            simulations.fetch_add(1, std::memory_order_relaxed);
```

```
                        break;
                    }
                    Worker::Leaf& leaf = w.leaves.emplace_back();
                    leaf.node = node;
                    leaf.state = state;
                    state.moves(leaf.moves);
                    leaf.hash = hash;
                    // Transposition the node table missed (a position that
                    // dropped out of the graph, or tree mode): expand from
                    // the network cache, no network call.
                    float value;
                    if (cache->lookup(leaf.hash, leaf.moves.size(), value, priors)) {
                        expand(*node, leaf.moves, priors, value);
                        backup(path, len, value);
                        simulations.fetch_add(1, std::memory_order_relaxed);
                        ++local_hits;
                        w.leaves.pop_back();
                    } else {
                        w.descents.push_back({static_cast<uint32_t>(w.leaves.size() - 1),
                                              static_cast<uint32_t>(len)});
                    }
                    break;
                }
                if (expected != EXPANDING) continue;   // became EXPANDED/TERMINAL meanwhile:
                 ↪ go on
            }
            // st == EXPANDING: the leaf is being evaluated. If it is ours
            // (same batch), this descent rides on the same evaluation —
            // its path is kept separately, it may well differ. Otherwise
            // the descent is undone.
            uint32_t mine = UINT32_MAX;
            for (uint32_t i = 0; i < w.leaves.size(); i++)
                if (w.leaves[i].node == node) { mine = i; break; }
            if (mine != UINT32_MAX) {
                w.descents.push_back({mine, static_cast<uint32_t>(len)});
                ++gathered_visits;
            } else {
                undo_pending(path, len);
                ++batch_collisions;
            }
            break;
        }
    }
    local_collisions += batch_collisions;
    if (sim_budget && gathered_visits < budget)
        reserved.fetch_sub(budget - gathered_visits, std::memory_order_relaxed);

    // ── 2. evaluate ──
    const size_t nb = w.leaves.size();
    if (nb == 0 && gathered_visits == 0) {
        // Every descent collided with leaves another thread is evaluating:
        // join the evaluator's barrier (an empty request) so we resume
        // right after that batch has been expanded, instead of spinning.
        evaluator.evaluate({}, {});
        std::this_thread::yield();
    }
    if (nb > 0) {
        for (Worker::Leaf& leaf : w.leaves)
            w.requests.push_back({&leaf.state, leaf.moves.begin(),
↪ static_cast<uint16_t>(leaf.moves.size())});
        evaluator.evaluate(w.requests, std::span<nn::Result>(w.results.data(), nb));
        local_evals += nb;
        // ── 3. expand ... ──
```

```
            for (size_t i = 0; i < nb; i++) {
                Worker::Leaf& leaf = w.leaves[i];
                const nn::Result& r = w.results[i];
                priors_from_logits(r, leaf.moves.size(), priors);
                cache->store(leaf.hash, leaf.moves.size(), r.value, priors);
                expand(*leaf.node, leaf.moves, priors, r.value);
            }
            // ── ... and back up, once per descent, along its own path ──
            for (size_t d = 0; d < w.descents.size(); d++) {
                const Worker::Descent& desc = w.descents[d];
                backup(w.path(d), desc.path_len, w.results[desc.leaf].value);
            }
            simulations.fetch_add(w.descents.size(), std::memory_order_relaxed);
        }

        if (prm.microseconds && std::chrono::steady_clock::now() >= deadline) stop.store(true,
        ↪  std::memory_order_relaxed);
        if (sim_budget && simulations.load(std::memory_order_relaxed) >= sim_budget)
            stop.store(true, std::memory_order_relaxed);
    }
    collisions.fetch_add(local_collisions, std::memory_order_relaxed);
    cache_hits.fetch_add(local_hits, std::memory_order_relaxed);
    evaluations.fetch_add(local_evals, std::memory_order_relaxed);
}

// ── top level ──────────────────────────────────────────────────────────
SearchResult Search::search(const Yolah& state) {
    using namespace std::chrono;
    const auto start = steady_clock::now();

    const uint64_t key = node_key(zobrist::hash(state), state);
    if (!(prm.reuse_tree && reuse(state, key))) new_root(state, key);
    if (root->state.load(std::memory_order_acquire) == UNEXPANDED) expand_root(state);

    // Playout cap randomization: most self-play moves get a small budget, no
    // root noise and no policy target; a minority get the full treatment.
    full_search = true;
    sim_budget = prm.nb_simulations;
    if (prm.nb_simulations && prm.nb_simulations_fast && prm.playout_cap_fast_prob > 0.0f) {
        const float u = (rng.rand<uint64_t>() >> 11) * 0x1.0p-53f;
        if (u < prm.playout_cap_fast_prob) {
            full_search = false;
            sim_budget = prm.nb_simulations_fast;
        }
    }

    std::vector<std::unique_ptr<Worker>> workers;
    for (size_t t = 0; t < prm.nb_threads; t++) {
        // Distinct, non-zero seeds per thread and per search.
        const uint64_t seed = (base_seed + 0x9E3779B97F4A7C15ull * (search_counter *
        ↪  prm.nb_threads + t + 1)) | 1;
        workers.push_back(std::make_unique<Worker>(seed, prm.batch_size));
    }
    ++search_counter;

    simulations = 0;
    reserved = 0;
    collisions = 0;
    cache_hits = 0;
    evaluations = 0;
    transpositions = 0;
    stop = false;
    if (root->state.load(std::memory_order_acquire) == EXPANDED) {
```

```
        apply_root_noise(*workers[0]);
        deadline = prm.microseconds ? start + microseconds(prm.microseconds) :
↪  steady_clock::time_point::max();
        if (prm.nb_threads == 1) {
            run_worker(*workers[0], state);
        } else {
            std::vector<std::jthread> threads;
            for (size_t t = 0; t < prm.nb_threads; t++)
                threads.emplace_back([this, &workers, t, &state] { run_worker(*workers[t], state);
↪  });
        }
    }
    const double seconds = duration<double>(steady_clock::now() - start).count();
    SearchResult res = make_result(state, seconds, simulations.load(), collisions.load(),
↪  *workers[0]);
    res.root_visits = root->n.load();
    return res;
}

// Play values of the root's moves: the edge visits, with the forced playouts
// taken back out (KataGo's getReducedPlaySelectionWeight) and the LCB bonus
// applied. They serve both as the training target π and as the move choice.
void Search::root_play_values(std::vector<ChildStat>& children) const {
    const size_t nb = children.size();
    if (nb == 0) return;

    float total = 0.0f;
    for (const ChildStat& c : children) total += static_cast<float>(c.visits);
    for (ChildStat& c : children) c.play_value = static_cast<float>(c.visits);

    // ── 1. the most stably explored child (KataGo's "non LCB best") ──
    size_t best = 0;
    {
        float best_goodness = -1e30f;
        for (size_t i = 0; i < nb; i++) {
            const float v = static_cast<float>(children[i].visits);
            // Discount one visit's worth: the most recent one may be an outlier.
            const float g = v * std::max(0.0f, v - 1.0f) / std::max(1.0f, v) + 2.0f *
↪  children[i].prior;
            if (g > best_goodness) { best_goodness = g; best = i; }
        }
    }

    // ── 2. policy target pruning ──
    // How many visits would PUCT have given each child, in retrospect, if it
    // had never been forced? Invert the selection formula at the best child's
    // selection value and keep the smaller of the two counts.
    if (prm.policy_target_pruning && total > 0.0f) {
        const float cpuct = prm.cpuct_exploration + prm.cpuct_exploration_log *
            std::log((total + prm.cpuct_exploration_base) / prm.cpuct_exploration_base);
        const float explore_scaling = cpuct * utility_stdev_factor(*root,
↪  root->n.load(std::memory_order_relaxed))
                                    * std::sqrt(total + 0.01f);
        const ChildStat& b = children[best];
        const float best_value = b.q + explore_scaling * b.prior / (1.0f +
↪  static_cast<float>(b.visits));
        for (size_t i = 0; i < nb; i++) {
            if (i == best) continue;
            ChildStat& c = children[i];
            if (c.visits == 0) { c.play_value = 0.0f; continue; }
            const float excess = best_value - c.q;     // the exploration term it would need
            if (excess <= 0.0f) continue;              // still the best move on value alone: keep
↪  it
```

```cpp
            const float wanted = explore_scaling * c.prior / excess - 1.0f;
            c.play_value = std::min(c.play_value, std::ceil(std::max(0.0f, wanted)));
        }
    }

    // ── 3. LCB: give the best lower bound enough play value to be chosen ──
    if (!prm.use_lcb || total <= 0.0f) return;
    std::vector<float> radius(nb, 0.0f);
    const float best_play = children[best].play_value;
    float best_lcb = -1e10f;
    size_t best_lcb_index = SIZE_MAX;
    for (size_t i = 0; i < nb; i++) {
        ChildStat& c = children[i];
        if (c.visits == 0) { c.lcb = -2.0f; radius[i] = 2.0f * prm.lcb_stdevs; continue; }
        // Weighted-sample machinery from KataGo, specialised to unit weights:
        // every visit weighs 1, so weightSum = weightSqSum = edge visits and
        // the effective sample size is the edge visit count itself.
        const float n = static_cast<float>(c.visits);
        const float mean_sq = c.q * c.q;
        float sq_avg = std::max(c.q2, mean_sq + 1e-8f);
        // A prior saying the variance could be as large as the whole utility
        // range (radius 1), with a weight that vanishes as the visits grow.
        const float prior_w = 1.0f / (n * n);
        sq_avg = (sq_avg * n + (sq_avg + 1.0f) * prior_w) / (n + prior_w);
        const float weight_sum = n + prior_w;
        const float ess = weight_sum * weight_sum / (n + prior_w * prior_w);
        const float var = std::max(0.0f, sq_avg - mean_sq);
        radius[i] = std::sqrt(var / ess) * prm.lcb_stdevs;
        c.lcb = c.q - radius[i];
        if (c.play_value > 0.0f && c.play_value >= prm.min_visit_prop_for_lcb * best_play) {
            if (c.lcb > best_lcb) { best_lcb = c.lcb; best_lcb_index = i; }
        }
    }
    if (best_lcb_index == SIZE_MAX) return;
    // The winner is given just enough play value to beat every other child:
    // a child whose bound is `excess` worse would need its radius to grow by
    // `factor` to catch up, and the radius shrinks like 1/√weight.
    float adjusted = children[best_lcb_index].play_value;
    for (size_t i = 0; i < nb; i++) {
        if (i == best_lcb_index) continue;
        const float excess = best_lcb - children[i].lcb;
        if (excess < 0.0f) continue;   // that child only lost on the visit threshold
        const float factor = (radius[i] + excess) / (radius[i] + 0.20f * excess);
        adjusted = std::max(adjusted, factor * factor * children[i].play_value);
    }
    children[best_lcb_index].play_value = adjusted;
}

SearchResult Search::make_result(const Yolah& state, double seconds, uint64_t sims,
                                 uint64_t nb_collisions, Worker& w) const {
    SearchResult res;
    res.seconds = seconds;
    res.nb_simulations = sims;
    res.nb_collisions = nb_collisions;
    res.nb_cache_hits = cache_hits.load();
    res.nb_evaluations = evaluations.load();
    res.nb_transpositions = transpositions.load();
    res.full_search = full_search;
    res.tree_nodes = node_count.load();
    res.net_value = root->net_v;
    const uint32_t root_n = root->n.load();
    res.root_value = root_n > 0 ? -root->w.load() / root_n : root->net_v;

```

```
    const auto& children = root->children;
    res.children.reserve(children.size());
    for (size_t i = 0; i < children.size(); i++) {
        const Edge& e = children[i];
        const Node* c = e.child.load(std::memory_order_acquire);
        const uint32_t cn = c ? c->n.load() : 0;
        ChildStat st{};
        st.move   = e.action;
        st.prior  = root_priors.empty() ? e.prior : root_priors[i];
        st.visits = e.n.load();
        st.q      = cn > 0 ? c->w.load() / cn : 0.0f;
        st.q2     = cn > 0 ? c->w2.load() / cn : 0.0f;
        st.lcb    = -2.0f;
        res.children.push_back(st);
    }
    root_play_values(res.children);
    std::stable_sort(res.children.begin(), res.children.end(), [](const ChildStat& a, const
    ↪  ChildStat& b) {
        return a.play_value != b.play_value ? a.play_value > b.play_value : a.q > b.q;
    });
    if (res.children.empty()) return res;   // terminal root

    // Training target π: the play values, normalised. If the search could not
    // complete a single simulation, fall back to the priors.
    float total = 0.0f;
    for (const ChildStat& c : res.children) total += c.play_value;
    res.policy.resize(res.children.size());
    for (size_t i = 0; i < res.children.size(); i++)
        res.policy[i] = total > 0 ? res.children[i].play_value / total : res.children[i].prior;

    // Move choice: the best play value or, in the self-play opening phase, a
    // sample ∝ π^(1/τ).
    size_t pick = 0;
    if (total > 0.0f && prm.temperature > 0.0f && state.nb_plies() < prm.temperature_cutoff) {
        std::vector<double> weights(res.children.size());
        double sum = 0.0;
        for (size_t i = 0; i < weights.size(); i++) {
            weights[i] = std::pow(static_cast<double>(res.children[i].play_value), 1.0 /
    ↪  prm.temperature);
            sum += weights[i];
        }
        double r = (w.prng.rand<uint64_t>() >> 11) * 0x1.0p-53 * sum;
        for (pick = 0; pick + 1 < weights.size(); pick++) {
            r -= weights[pick];
            if (r <= 0.0) break;
        }
    } else if (total == 0.0f) {
        for (size_t i = 1; i < res.children.size(); i++)
            if (res.children[i].prior > res.children[pick].prior) pick = i;
    }
    res.best_move = res.children[pick].move;
    return res;
}

std::string SearchResult::to_string(size_t max_children) const {
    std::ostringstream os;
    os << std::fixed << std::setprecision(3);
    os << "[ move ]: " << best_move << "  Q: " << std::showpos << root_value << std::noshowpos
       << "  net: " << std::showpos << net_value << std::noshowpos
       << "  sims: " << nb_simulations << (full_search ? "" : " (fast)")
       << " (" << root_visits << " root visits, " << nb_evaluations
       << " net evals, " << nb_cache_hits << " cache hits, " << nb_transpositions
       << " transpositions, " << nb_collisions << " collisions)  "
```

```
       << std::setprecision(1) << seconds << "s, "
       << std::setprecision(0) << (seconds > 0 ? nb_simulations / seconds : 0) << " sims/s, nodes:
        ↪  " << tree_nodes << "\n";
    os << std::setprecision(3);
    for (size_t i = 0; i < std::min(max_children, children.size()); i++) {
        const ChildStat& c = children[i];
        os << "  " << c.move << "  N: " << std::setw(6) << c.visits << "  Q: " << std::showpos <<
↪  c.q
           << "  lcb: " << c.lcb << std::noshowpos << "  P: " << c.prior
           << "  pi: " << (policy.empty() ? 0.0f : policy[i]) << "\n";
    }
    return os.str();
}

} // namespace az
```

12.1.3 `player/alphazero_mcts_player.h`

92 lines.

The `Player` wrapper. The header comment is the authoritative JSON schema — the parameter reference in Section 9 is the same list with the defaults spelled out.

symbol	role
`AlphaZeroMCTSPlayer`	`Player` implementation: owns the pool resource, the evaluator and the `az::Search`
`AlphaZeroMCTSPlayer::play`	one search, one move; the result is kept for `last_result()`
`AlphaZeroMCTSPlayer::game_over`	reset the graph at the end of a game
`AlphaZeroMCTSPlayer::last_result`	the full root statistics of the last search — the self-play hook
`AlphaZeroMCTSPlayer::reload_weights`	hot-swap the network: resets the graph and the network cache
`AlphaZeroMCTSPlayer::config`	round-trip the live configuration back to JSON

```
#ifndef ALPHAZERO_MCTS_PLAYER_H
#define ALPHAZERO_MCTS_PLAYER_H
// alphazero_mcts_player.h — Player wrapper around az::Search + a network backend.
//
// JSON configuration (keys follow the other players' conventions):
// {
//   "name": "AlphaZeroMCTSPlayer",
//   "microseconds": 1000000,          // thinking time per move (0 = use only "nb simulations")
//   "nb simulations": 0,              // simulation budget (0 = time only); 800 for self-play
//   "nb threads": 1,                  // search threads ("hardware concurrency" allowed)
//   "batch size": 32,                 // leaves gathered per search thread per network call
//   "weights": "../nnue/cnn_resnet_256x30_value_policy.bin",   // .bin (cpu) or .ts (torch)
//   "backend": "cpu",                 // "cpu" (Eigen/OpenMP) | "torch" (libtorch,
↪  -DENABLE_TORCH=ON)
//   "device": "cuda",                 // torch only
//   "fp16": true,                     // torch only
//   "nb eval threads": 0,             // cpu only: OpenMP threads inside the network (0 = all)
//   // PUCT, KataGo style (see alphazero_mcts.h); the defaults are KataGo's
//   "cpuct exploration": 1.0, "cpuct exploration log": 0.45, "cpuct exploration base": 500,
//   "cpuct utility stdev scale": 0.85,   // 0 = plain AlphaZero exploration
//   "cpuct utility stdev prior": 0.40, "cpuct utility stdev prior weight": 2.0,
```

```
//   "fpu reduction": 0.2, "root fpu reduction": 0.1,
//   "fpu loss prop": 0.0, "root fpu loss prop": 0.0,
//   "fpu parent weight by visited policy": true,
//   "fpu parent weight by visited policy pow": 1.0, "fpu parent weight": 0.0,
//   "policy temperature": 1.0,
//   "graph search": true,             // one node per position (DAG) instead of a tree
//   // root move choice
//   "use lcb": true, "lcb stdevs": 5.0, "min visit prop for lcb": 0.2,
//   "policy target pruning": true,    // take the forced playouts back out of pi
//   "dirichlet alpha": 0.3, "dirichlet epsilon": 0.0,          // root noise (self-play)
//   "temperature": 0.0, "temperature cutoff": 0,               // move sampling (self-play)
//   "forced playouts k": 0.0,         // 2.0 during self-play, 0 for competitive play
//   "playout cap fast prob": 0.0,     // self-play: fraction of moves with a small budget
//   "nb simulations fast": 0,         //   ... and how small (needs "nb simulations")
//   "reuse tree": true,
//   "nn cache": 64,                   // MB of transposition cache for network outputs (0 = off)
//   "seed": 0,
//   "verbose": false                  // print the root statistics after every move
// }
// Only "microseconds" (or "nb simulations") and "weights" are mandatory.
//
// Threading model: the search threads share one tree and one evaluator. With
// more than one search thread the evaluator is wrapped in nn::BatchingEvaluator
// so that the backend sees merged batches (essential for a GPU, and it keeps
// the OpenMP CPU backend from being entered concurrently).
//
// Self-play: after play() the full root statistics of the last search are
// available through last_result() — its play values are the policy target π
// and the game outcome gives z. The learning loop itself (player/
// alphazero_selfplay.h, nnue/alphazero_learn.py) does not use this class: it
// runs az::Search directly, many games on one shared evaluator, with the
// parameters built by search_params(). Self-play settings: "dirichlet
// epsilon" 0.25, "temperature" 1.0, "temperature cutoff" 20, "nb simulations"
// 800 — see config/alphazero_mcts_selfplay_player.cfg.
#include "player.h"
#include "alphazero_mcts.h"
#include "nn_evaluator.h"
#include <memory>
#include <memory_resource>
#include <string>

class AlphaZeroMCTSPlayer : public Player {
public:
    explicit AlphaZeroMCTSPlayer(const json& config);
    ~AlphaZeroMCTSPlayer() override;

    Move play(Yolah) override;
    void game_over(Yolah) override;
    std::string info() override;
    json config() override;

    const az::SearchResult& last_result() const { return last; }
    az::Search& search() { return *searcher; }
    nn::Evaluator& evaluator() { return *net; }
    // Load new weights (same architecture) into the running backend.
    void reload_weights(const std::string& weights_filename);
    // The search parameters described by a JSON config (the keys above);
    // missing keys keep az::SearchParams' defaults. Used by the self-play
    // driver, which runs az::Search directly on a shared evaluator.
    static az::SearchParams search_params(const json& config);

private:
    json cfg;
```

```
    std::string weights_filename;
    bool verbose;
    std::pmr::synchronized_pool_resource memory;
    std::unique_ptr<nn::Evaluator> net;
    std::unique_ptr<az::Search> searcher;
    az::SearchResult last;
};

#endif
```

12.1.4 `player/alphazero_mcts_player.cpp`

149 lines.

JSON in, `SearchParams` out, plus the decision of whether to wrap the backend in a `BatchingEvaluator`. The only subtlety is the ordering in the constructor: the backend has to exist before `batch size` can default to `preferred_batch_size()`, and the search has to be destroyed before the pool resource it allocated the graph from — hence the explicit destructor.

symbol	role
`read_nb_threads`	accepts a number or the string `"hardware concurrency"`
`AlphaZeroMCTSPlayer::AlphaZeroMCTSPlayer`	parse the config, build the backend, wrap it if multi-threaded, build the search
`AlphaZeroMCTSPlayer::~AlphaZeroMCTSPlayer`	destroy the search first: the graph lives in `memory`

```
#include "alphazero_mcts_player.h"
#include "batched_evaluator.h"
#include <iostream>
#include <stdexcept>
#include <thread>

using std::string, std::invalid_argument;

namespace {
    size_t read_nb_threads(const json& j, const char* key, size_t default_value) {
        if (!j.contains(key)) return default_value;
        if (j[key].is_number()) return j[key].get<size_t>();
        if (j[key].is_string() && j[key].get<string>() == "hardware concurrency")
            return std::thread::hardware_concurrency();
        throw invalid_argument(string("number or \"hardware concurrency\" expected for ") + key);
    }
}

az::SearchParams AlphaZeroMCTSPlayer::search_params(const json& j) {
    az::SearchParams p;
    p.microseconds       = j.value("microseconds", uint64_t(0));
    p.nb_simulations     = j.value("nb simulations", uint64_t(0));
    p.nb_threads         = read_nb_threads(j, "nb threads", 1);
    p.batch_size         = j.value("batch size", size_t(0));
    p.max_collisions     = j.value("max collisions", size_t(0));
    p.nn_cache_mb        = j.value("nn cache", size_t(64));
    p.cpuct_exploration      = j.value("cpuct exploration", p.cpuct_exploration);
    p.cpuct_exploration_log  = j.value("cpuct exploration log", p.cpuct_exploration_log);
    p.cpuct_exploration_base = j.value("cpuct exploration base", p.cpuct_exploration_base);
    p.cpuct_utility_stdev_scale        = j.value("cpuct utility stdev scale",
↪   p.cpuct_utility_stdev_scale);
```

```
    p.cpuct_utility_stdev_prior        = j.value("cpuct utility stdev prior", p.cpuct_utility_stdev_prior);
    p.cpuct_utility_stdev_prior_weight = j.value("cpuct utility stdev prior weight", p.cpuct_utility_stdev_prior_weight);
    p.fpu_reduction      = j.value("fpu reduction", p.fpu_reduction);
    p.root_fpu_reduction = j.value("root fpu reduction", p.root_fpu_reduction);
    p.fpu_loss_prop      = j.value("fpu loss prop", p.fpu_loss_prop);
    p.root_fpu_loss_prop = j.value("root fpu loss prop", p.root_fpu_loss_prop);
    p.fpu_parent_weight_by_visited_policy =
        j.value("fpu parent weight by visited policy", p.fpu_parent_weight_by_visited_policy);
    p.fpu_parent_weight_by_visited_policy_pow =
        j.value("fpu parent weight by visited policy pow", p.fpu_parent_weight_by_visited_policy_pow);
    p.fpu_parent_weight  = j.value("fpu parent weight", p.fpu_parent_weight);
    p.policy_temperature = j.value("policy temperature", p.policy_temperature);
    p.graph_search       = j.value("graph search", p.graph_search);
    p.use_lcb            = j.value("use lcb", p.use_lcb);
    p.lcb_stdevs         = j.value("lcb stdevs", p.lcb_stdevs);
    p.min_visit_prop_for_lcb = j.value("min visit prop for lcb", p.min_visit_prop_for_lcb);
    p.policy_target_pruning  = j.value("policy target pruning", p.policy_target_pruning);
    p.forced_playouts_k      = j.value("forced playouts k", p.forced_playouts_k);
    p.playout_cap_fast_prob  = j.value("playout cap fast prob", p.playout_cap_fast_prob);
    p.nb_simulations_fast    = j.value("nb simulations fast", p.nb_simulations_fast);
    p.dirichlet_alpha    = j.value("dirichlet alpha", p.dirichlet_alpha);
    p.dirichlet_epsilon  = j.value("dirichlet epsilon", p.dirichlet_epsilon);
    p.temperature        = j.value("temperature", p.temperature);
    p.temperature_cutoff = j.value("temperature cutoff", p.temperature_cutoff);
    p.reuse_tree         = j.value("reuse tree", true);
    p.seed               = j.value("seed", uint64_t(0));
    return p;
}

AlphaZeroMCTSPlayer::AlphaZeroMCTSPlayer(const json& j) : cfg(j) {
    if (!j.contains("weights")) throw invalid_argument("weights key expected");
    if (!j.contains("microseconds") && !j.contains("nb simulations"))
        throw invalid_argument("microseconds or nb simulations key expected");
    weights_filename = j["weights"].get<string>();
    verbose = j.value("verbose", false);

    az::SearchParams p = search_params(j);

    // Backend, then the batching wrapper when several search threads share it.
    std::unique_ptr<nn::Evaluator> backend = nn::make_evaluator(j);
    if (p.batch_size == 0) p.batch_size = backend->preferred_batch_size();
    if (p.nb_threads > 1) {
        const size_t merged = j.value("merged batch size", p.batch_size * p.nb_threads);
        net = std::make_unique<nn::BatchingEvaluator>(std::move(backend), p.nb_threads, merged);
    } else {
        net = std::move(backend);
    }
    searcher = std::make_unique<az::Search>(*net, p, &memory);
}

AlphaZeroMCTSPlayer::~AlphaZeroMCTSPlayer() {
    // The tree lives in `memory`; destroy it before the pool goes away.
    searcher.reset();
}

Move AlphaZeroMCTSPlayer::play(Yolah yolah) {
    last = searcher->search(yolah);
    if (verbose) std::cout << last.to_string() << std::flush;
    return last.best_move;
}
```

```
91
92  void AlphaZeroMCTSPlayer::game_over(Yolah) {
93      searcher->reset();
94  }
95
96  void AlphaZeroMCTSPlayer::reload_weights(const std::string& filename) {
97      net->reload(filename);
98      weights_filename = filename;
99      cfg["weights"] = filename;
100     searcher->reset();         // the old tree's priors/values came from the old network
101     searcher->clear_cache();   // so did the cached evaluations
102 }
103
104 std::string AlphaZeroMCTSPlayer::info() {
105     return "alphazero mcts player (PUCT + value/policy resnet, " + net->info() + ")";
106 }
107
108 json AlphaZeroMCTSPlayer::config() {
109     json j = cfg;
110     j["name"] = "AlphaZeroMCTSPlayer";
111     const az::SearchParams& p = searcher->params();
112     j["microseconds"] = p.microseconds;
113     j["nb simulations"] = p.nb_simulations;
114     if (p.nb_threads == std::thread::hardware_concurrency()) j["nb threads"] = "hardware
        ↪  concurrency";
115     else                                                     j["nb threads"] = p.nb_threads;
116     j["batch size"] = p.batch_size;
117     j["weights"] = weights_filename;
118     j["backend"] = cfg.value("backend", string("cpu"));
119     j["cpuct exploration"] = p.cpuct_exploration;
120     j["cpuct exploration log"] = p.cpuct_exploration_log;
121     j["cpuct exploration base"] = p.cpuct_exploration_base;
122     j["cpuct utility stdev scale"] = p.cpuct_utility_stdev_scale;
123     j["cpuct utility stdev prior"] = p.cpuct_utility_stdev_prior;
124     j["cpuct utility stdev prior weight"] = p.cpuct_utility_stdev_prior_weight;
125     j["fpu reduction"] = p.fpu_reduction;
126     j["root fpu reduction"] = p.root_fpu_reduction;
127     j["fpu loss prop"] = p.fpu_loss_prop;
128     j["root fpu loss prop"] = p.root_fpu_loss_prop;
129     j["fpu parent weight by visited policy"] = p.fpu_parent_weight_by_visited_policy;
130     j["fpu parent weight by visited policy pow"] = p.fpu_parent_weight_by_visited_policy_pow;
131     j["fpu parent weight"] = p.fpu_parent_weight;
132     j["policy temperature"] = p.policy_temperature;
133     j["graph search"] = p.graph_search;
134     j["use lcb"] = p.use_lcb;
135     j["lcb stdevs"] = p.lcb_stdevs;
136     j["min visit prop for lcb"] = p.min_visit_prop_for_lcb;
137     j["policy target pruning"] = p.policy_target_pruning;
138     j["forced playouts k"] = p.forced_playouts_k;
139     j["playout cap fast prob"] = p.playout_cap_fast_prob;
140     j["nb simulations fast"] = p.nb_simulations_fast;
141     j["dirichlet alpha"] = p.dirichlet_alpha;
142     j["dirichlet epsilon"] = p.dirichlet_epsilon;
143     j["temperature"] = p.temperature;
144     j["temperature cutoff"] = p.temperature_cutoff;
145     j["reuse tree"] = p.reuse_tree;
146     j["nn cache"] = p.nn_cache_mb;
147     j["verbose"] = verbose;
148     return j;
149 }
```

12.1.5 `player/nn_cache.h`

105 lines.

A direct-mapped cache of network outputs, keyed by the Zobrist hash — which is exactly the network's input, so a hit is always valid. It is the second line of defence behind the node table: it also catches positions swept out of the graph, and it survives across moves and games.

Two details worth noting: the stored move count is verified on lookup, so a 64-bit collision cannot hand back another position's priors; and priors are 16-bit fixed point, which costs 1/65535 of resolution and halves the entry size.

symbol	role
`NNCache::Entry`	key, value, move count, and the priors as `uint16` fixed point
`NNCache::NNCache`	size in bytes, rounded down to a power-of-two entry count; 0 disables it
`NNCache::lookup`	hit only if both the key and the move count match
`NNCache::store`	always overwrite — the table is direct-mapped
`NNCache::clear`	take every stripe lock, then wipe (used by `reload_weights`)

```
#ifndef NN_CACHE_H
#define NN_CACHE_H
// nn_cache.h — transposition cache of network evaluations for the AlphaZero
// MCTS (az::Search).
//
// In Yolah the four stones of a side move independently, so the same position
// is reached through many move orders: a large share of the leaves the search
// wants to expand are transpositions of positions already evaluated. The
// network only sees (board, side to move), so its output is a pure function of
// the Zobrist hash of that pair — we cache it: value + the priors of the legal
// moves (in Yolah::moves order, which is deterministic for a position). A hit
// expands the node without a network call, which is where >99 % of the time
// goes.
//
// Direct-mapped table (index = hash & mask, always overwrite), lock-striped so
// several search threads can use it concurrently; the cost of a lookup is
// negligible next to a network evaluation. The hash also verifies the number of
// legal moves, so a 64-bit collision cannot silently produce garbage priors.
// Priors are stored as 16-bit fixed point (1/65535 resolution).
//
// Statistics are shared by everything that evaluates the same network, so the
// cache survives across moves and games; it must be cleared when the weights
// change (az::Search::clear_cache / AlphaZeroMCTSPlayer::reload_weights).
#include "game.h"
#include <array>
#include <atomic>
#include <cstdint>
#include <cstring>
#include <mutex>
#include <vector>

namespace az {

class NNCache {
public:
```

```
    struct Entry {
        uint64_t key = 0;
        float    value = 0.0f;
        uint16_t nb_moves = 0;
        uint16_t priors[Yolah::MAX_NB_MOVES] = {};
    };

    // size_bytes = 0 disables the cache. The entry count is rounded down to a
    // power of two.
    explicit NNCache(size_t size_bytes) {
        size_t n = size_bytes / sizeof(Entry);
        size_t pow2 = 1;
        while (pow2 * 2 <= n) pow2 *= 2;
        if (n >= 2) {
            table.resize(pow2);
            mask = pow2 - 1;
        }
    }

    bool enabled() const { return !table.empty(); }
    size_t capacity() const { return table.size(); }
    size_t size_bytes() const { return table.size() * sizeof(Entry); }

    // Returns true and fills value / priors[0..nb_moves) on a hit.
    bool lookup(uint64_t key, size_t nb_moves, float& value, float* priors) {
        if (table.empty()) return false;
        const Entry& e = table[key & mask];
        std::lock_guard lock(locks[key & (locks.size() - 1)]);
        if (e.key != key || e.nb_moves != nb_moves) {
            misses.fetch_add(1, std::memory_order_relaxed);
            return false;
        }
        value = e.value;
        for (size_t m = 0; m < nb_moves; m++) priors[m] = e.priors[m] * (1.0f / 65535.0f);
        hits.fetch_add(1, std::memory_order_relaxed);
        return true;
    }

    void store(uint64_t key, size_t nb_moves, float value, const float* priors) {
        if (table.empty()) return;
        Entry& e = table[key & mask];
        std::lock_guard lock(locks[key & (locks.size() - 1)]);
        e.key = key;
        e.value = value;
        e.nb_moves = static_cast<uint16_t>(nb_moves);
        for (size_t m = 0; m < nb_moves; m++)
            e.priors[m] = static_cast<uint16_t>(priors[m] * 65535.0f + 0.5f);
    }

    void clear() {
        for (size_t i = 0; i < locks.size(); i++) locks[i].lock();
        std::fill(table.begin(), table.end(), Entry{});
        hits = 0;
        misses = 0;
        for (size_t i = locks.size(); i-- > 0;) locks[i].unlock();
    }

    uint64_t nb_hits() const { return hits.load(std::memory_order_relaxed); }
    uint64_t nb_misses() const { return misses.load(std::memory_order_relaxed); }

private:
    std::vector<Entry> table;
    size_t mask = 0;
```

```
    std::array<std::mutex, 256> locks;
    std::atomic<uint64_t> hits{0}, misses{0};
};

} // namespace az

#endif
```

12.2 The network interface and its backends

Everything below `nn::Evaluator` is interchangeable: the search never learns which backend it is talking to, and `make_evaluator` picks one from the same JSON the player was given.

12.2.1 `nnue/nn_evaluator.h`

121 lines.

The interface, plus the two encodings that *must* agree with the training code: `encode_planes` (the four input planes, in the reversed cell order of Section 3) and `action_index` (`from*64 + to`). If either drifts from the Python side, the network silently reads a permuted board.

symbol	role
`PLANES`, `PLANE_CELLS`, `INPUT_FLOATS`, `NUM_ACTIONS`	the shapes, shared with the training script
`encode_planes`	position → 256 floats, NCHW, cell i = bit $63 - i$
`action_index`	move → policy index; a pass maps to 0 (A1→A1)
`Request` / `Result`	a position with its legal moves / a value and the logits of exactly those moves
`Evaluator`	the abstract backend: `evaluate`, `preferred_batch_size`, `info`, `reload`
`make_evaluator`	factory over the `"backend"` key

```
#ifndef NN_EVALUATOR_H
#define NN_EVALUATOR_H
// nn_evaluator.h — backend-agnostic interface between the AlphaZero-style MCTS
// (player/alphazero_mcts.h) and the two-headed value/policy network.
//
// The search never touches tensors: it hands the evaluator a batch of
// positions together with their legal moves and receives, per position,
//   • a scalar value v ∈ (-1, +1) from the point of view of the player to move
//     (the network's tanh output, trained on z ∈ {-1, 0, +1});
//   • the raw policy logits of *exactly the legal moves*, in the order they
//     were given. Softmax (with an optional temperature) is done by the search.
//
// Two backends implement this interface:
//   nnue/resnet.h        — pure C++ (Eigen + OpenMP), no external dependency,
//                          loads the .bin written by nnue/resnet_export.py;
//   nnue/resnet_torch.h  — libtorch/TorchScript, GPU capable, built only with
//                          -DENABLE_TORCH=ON, loads the .ts from the same script.
// plus a wrapper:
//   nnue/batched_evaluator.h — merges the requests of several search threads
//                          into one large batch before calling a backend.
```

```cpp
//
// Learning loop (nnue/alphazero_learn.py): a freshly trained network is
// exported as TorchScript and swapped in with reload() while the self-play
// games keep running — no search object needs to be rebuilt.
#include "game.h"
#include <cstdint>
#include <memory>
#include <span>
#include <string>

namespace nn {

// ── Board encoding (MUST match preprocess.py / encode_cnn in
//    cnn_resnet_value_policy_chunked.py) ──────────────────────────────
//
// The network sees 4 planes of 8x8:
//   plane 0 black stones, plane 1 white stones, plane 2 "empty" (vacated)
//   squares, plane 3 the side to move (all 0 = black, all 1 = white).
// Plane cell i (row-major, i = 8*row + col with row/col in 0..7) holds bit
// (63 - i) of the bitboard, i.e. square (63 - i), since bit s IS square s
// (SQ_A1 = 0 ... SQ_H8 = 63).
//
// Where the (63 - i) comes from: preprocess.py builds a plane with
//     np.unpackbits(np.array([bb], dtype='>u8').view(np.uint8)).reshape(8, 8)
// '>u8' dumps the integer big-endian (most significant byte first) and
// unpackbits takes the most significant bit of each byte first, so the values
// come out in the order bit 63, 62, ..., 0 — element i is bit 63 - i.
//
// Geometrically, with s = 8*rank + file, that is row = 7 - rank and
// col = 7 - file: the ranks are in the usual order (rank 8 on the top row)
// but the FILES ARE MIRRORED, H on the left. Cell 0 is H8, cell 63 is A1.
// The network does not care — a fixed mirror of every input is just a
// relabelling of the board — but training and inference must agree exactly,
// so do not "fix" this: it is what the weights were trained on.
constexpr int PLANES        = 4;
constexpr int PLANE_CELLS   = 64;
constexpr int INPUT_FLOATS  = PLANES * PLANE_CELLS;
constexpr int NUM_ACTIONS   = 64 * 64;

// Fill out[0..255] with the 4 planes of `y` in NCHW order (plane-major).
inline void encode_planes(const Yolah& y, float* out) {
    const uint64_t bbs[3] = { y.bitboard(Yolah::BLACK), y.bitboard(Yolah::WHITE),
     ↪  y.empty_bitboard() };
    for (int c = 0; c < 3; c++) {
        const uint64_t bb = bbs[c];
        float* plane = out + c * PLANE_CELLS;
        for (int i = 0; i < PLANE_CELLS; i++) {
            plane[i] = static_cast<float>((bb >> (63 - i)) & 1);
        }
    }
    const float turn = (y.current_player() == Yolah::WHITE) ? 1.0f : 0.0f;
    float* plane = out + 3 * PLANE_CELLS;
    for (int i = 0; i < PLANE_CELLS; i++) {
        plane[i] = turn;
    }
}

// Policy head index of a move: from*64 + to (preprocess.py: pol = s1*64 + s2).
// A pass (Move::none() == a1:a1) maps to index 0 exactly like in training.
constexpr int action_index(Move m) {
    return static_cast<int>(m.from_sq()) * 64 + static_cast<int>(m.to_sq());
}

```

```cpp
// ── Batch interface ──────────────────────────────────────────────────────
struct Request {
    const Yolah* state;     // position to evaluate
    const Move*  moves;     // its legal moves (Yolah::moves), nb_moves >= 1
    uint16_t     nb_moves;
};

struct Result {
    float value;                        // for the player to move at Request::state
    float logits[Yolah::MAX_NB_MOVES];  // policy logits of moves[0..nb_moves)
};

class Evaluator {
public:
    virtual ~Evaluator() = default;
    // Evaluate in.size() positions; out.size() must be >= in.size().
    // Thread-safe: may be called concurrently from several search threads.
    virtual void evaluate(std::span<const Request> in, std::span<Result> out) = 0;
    // The batch size the backend is most efficient at (a hint for the search).
    virtual size_t preferred_batch_size() const = 0;
    // Human readable description (backend, device, weights file).
    virtual std::string info() const = 0;
    // Swap in new weights (same architecture). Must not be called while
    // evaluate() is running.
    virtual void reload(const std::string& weights_filename) = 0;
};

// Build an evaluator from the player's JSON config:
//   "backend"    : "cpu" (default) | "torch"
//   "weights"    : path to the .bin (cpu) or .ts (torch) exported file
//   "device"     : torch only — "cuda" (default if available) | "cpu"
//   "fp16"       : torch only — run the network in half precision (default true on cuda)
//   "nb eval threads" : cpu only — OpenMP threads used inside one evaluate() (0 = all)
//   "batch size" : preferred batch size reported to the search (default backend specific)
std::unique_ptr<Evaluator> make_evaluator(const json& config);

} // namespace nn

#endif
```

12.2.2 `nnue/nn_evaluator.cpp`

34 lines.

The factory. Nothing more: it reads `"backend"`, `"weights"`, `"device"`, `"fp16"` and `"nb eval threads"`, and fails loudly when the libtorch backend is asked for in a build that does not have it.

symbol	role
`make_evaluator`	`"cpu"` → `ResNetCPU`, `"torch"` → `ResNetTorch` (only with `ENABLE_TORCH`)

```cpp
#include "nn_evaluator.h"
#include "resnet.h"
#include "batched_evaluator.h"
#ifdef ENABLE_TORCH
#include "resnet_torch.h"
#endif
```

```cpp
#include <stdexcept>
#include <thread>

namespace nn {

std::unique_ptr<Evaluator> make_evaluator(const json& j) {
    if (!j.contains("weights")) throw std::invalid_argument("weights key expected");
    const std::string weights = j["weights"].get<std::string>();
    const std::string backend = j.value("backend", std::string("cpu"));
    if (backend == "cpu") {
        const int eval_threads = j.value("nb eval threads", 0);
        const size_t batch = j.value("batch size", 32);
        return std::make_unique<ResNetCPU>(weights, eval_threads, batch);
    }
    if (backend == "torch") {
#ifdef ENABLE_TORCH
        const std::string device = j.value("device", std::string("cuda"));
        const bool fp16 = j.value("fp16", true);
        const size_t batch = j.value("batch size", 256);
        return std::make_unique<ResNetTorch>(weights, device, fp16, batch);
#else
        throw std::invalid_argument("backend \"torch\" requires a build with -DENABLE_TORCH=ON");
#endif
    }
    throw std::invalid_argument("unknown backend \"" + backend + "\" (expected \"cpu\" or
    ↪ \"torch\")");
}

} // namespace nn
```

12.2.3 `nnue/batched_evaluator.h`

85 lines.

The wrapper that merges several search threads into one backend call. The header comment explains the three conditions that fire a batch; the empty-request barrier is the part worth remembering, because it is what a search thread whose descents all collided uses instead of spinning.

symbol	role
`BatchingEvaluator`	`Evaluator` that owns a backend and a worker thread
`BatchingEvaluator::Job`	one caller's request/result spans plus its completion flag
`BatchingEvaluator::evaluate`	enqueue and block; an empty span is a barrier join
`BatchingEvaluator::set_nb_clients`	how many threads must be waiting before a batch fires early
`BatchingEvaluator::worker_loop`	wait, concatenate, call the backend once, scatter, wake the callers

```cpp
#ifndef BATCHED_EVALUATOR_H
#define BATCHED_EVALUATOR_H
// batched_evaluator.h — merges the evaluate() calls of several search threads
// into larger batches before handing them to a backend.
//
```

```cpp
// Why: a GPU (or the OpenMP CPU backend) is efficient only on large batches,
// whereas each MCTS thread can only gather a limited number of leaves per
// iteration without degrading the search (virtual-loss collisions). With T
// search threads each submitting ~B leaves, the backend sees batches of up to
// T·B positions instead of B.
//
// How: callers enqueue their (requests, results) spans and block. One worker
// thread wakes up as soon as
//   • the queued positions reach `max_batch`, or
//   • every registered client is waiting (`nb_clients`), or
//   • `max_wait` has elapsed with something queued,
// concatenates the queued requests, runs the backend once, scatters the
// results back and releases the callers. The queue is FIFO so no client can
// starve.
//
// An EMPTY request is a barrier join: the caller has nothing to evaluate but
// wants to block until the next batch has been processed (a search thread
// whose descents all collided with leaves owned by another thread). It counts
// as a waiting client, so it does not delay the batch.
#include "nn_evaluator.h"
#include <chrono>
#include <condition_variable>
#include <deque>
#include <memory>
#include <mutex>
#include <thread>
#include <vector>

namespace nn {

class BatchingEvaluator final : public Evaluator {
public:
    // backend    : the evaluator that does the actual work (owned)
    // nb_clients : number of threads expected to call evaluate() concurrently;
    //              a batch is launched as soon as all of them are waiting
    // max_batch  : upper bound on the merged batch size
    // max_wait   : how long the worker waits for stragglers before launching
    //              a partial batch
    BatchingEvaluator(std::unique_ptr<Evaluator> backend, size_t nb_clients, size_t max_batch,
                      std::chrono::microseconds max_wait = std::chrono::microseconds(500));
    ~BatchingEvaluator() override;

    void evaluate(std::span<const Request> in, std::span<Result> out) override;
    size_t preferred_batch_size() const override { return max_batch; }
    std::string info() const override;
    void reload(const std::string& weights_filename) override;

    Evaluator& backend() { return *impl; }
    void set_nb_clients(size_t n);

private:
    struct Job {
        std::span<const Request> in;
        std::span<Result> out;
        bool done = false;
    };
    void worker_loop();

    std::unique_ptr<Evaluator> impl;
    size_t nb_clients;
    const size_t max_batch;
    const std::chrono::microseconds max_wait;

```

```
    std::mutex mutex;
    std::condition_variable cv_worker;     // signals the worker: new job / stop
    std::condition_variable cv_clients;    // signals clients: results ready
    std::deque<Job*> queue;
    size_t queued_positions = 0;
    size_t waiting_clients = 0;
    bool stop = false;
    std::thread worker;

    // Scratch owned by the worker.
    std::vector<Request> batch_in;
    std::vector<Result> batch_out;
};

} // namespace nn

#endif
```

12.2.4 `nnue/batched_evaluator.cpp`

98 lines.

The queue. FIFO, so no client can starve; the worker fires on whichever of the three conditions comes first, and `reload` takes the same lock as the worker so weights cannot change mid-batch.

```
#include "batched_evaluator.h"
#include <algorithm>

namespace nn {

BatchingEvaluator::BatchingEvaluator(std::unique_ptr<Evaluator> backend, size_t nb_clients,
                                     size_t max_batch, std::chrono::microseconds max_wait)
    : impl(std::move(backend)), nb_clients(std::max<size_t>(1, nb_clients)),
      max_batch(std::max<size_t>(1, max_batch)), max_wait(max_wait) {
    batch_in.reserve(this->max_batch);
    batch_out.resize(this->max_batch);
    worker = std::thread([this] { worker_loop(); });
}

BatchingEvaluator::~BatchingEvaluator() {
    {
        std::lock_guard lock(mutex);
        stop = true;
    }
    cv_worker.notify_all();
    worker.join();
}

std::string BatchingEvaluator::info() const {
    return "batching(" + std::to_string(nb_clients) + " clients, max batch " +
           std::to_string(max_batch) + ") over " + impl->info();
}

void BatchingEvaluator::reload(const std::string& weights_filename) {
    // Serialise with the worker: no batch may be running while weights change.
    std::lock_guard lock(mutex);
    impl->reload(weights_filename);
}

void BatchingEvaluator::set_nb_clients(size_t n) {
```

```cpp
    std::lock_guard lock(mutex);
    nb_clients = std::max<size_t>(1, n);
}

void BatchingEvaluator::evaluate(std::span<const Request> in, std::span<Result> out) {
    Job job{in, out};   // in.empty() = barrier join, see header
    std::unique_lock lock(mutex);
    queue.push_back(&job);
    queued_positions += in.size();
    ++waiting_clients;
    cv_worker.notify_one();
    cv_clients.wait(lock, [&] { return job.done; });
    --waiting_clients;
}

void BatchingEvaluator::worker_loop() {
    std::unique_lock lock(mutex);
    for (;;) {
        // Wake-up conditions (see header). wait_for handles the straggler timeout.
        auto ready = [&] {
            return stop || queued_positions >= max_batch ||
                   (!queue.empty() && waiting_clients >= nb_clients);
        };
        if (!ready()) {
            if (!queue.empty()) cv_worker.wait_for(lock, max_wait, ready);
            else                cv_worker.wait(lock, [&] { return stop || !queue.empty(); });
        }
        if (stop && queue.empty()) return;
        if (queue.empty()) continue;

        // Take whole jobs (FIFO) until the next one would overflow max_batch.
        // The first job is always taken, even if it alone exceeds max_batch:
        // backends accept any batch size, max_batch is only the merge target.
        std::vector<Job*> taken;
        batch_in.clear();
        while (!queue.empty()) {
            Job* j = queue.front();
            if (!taken.empty() && batch_in.size() + j->in.size() > max_batch) break;
            taken.push_back(j);
            batch_in.insert(batch_in.end(), j->in.begin(), j->in.end());
            queued_positions -= j->in.size();
            queue.pop_front();
        }
        if (batch_out.size() < batch_in.size()) batch_out.resize(batch_in.size());

        // Run the backend without holding the lock so clients can keep queueing.
        if (!batch_in.empty()) {
            lock.unlock();
            impl->evaluate(batch_in, std::span<Result>(batch_out.data(), batch_in.size()));
            lock.lock();
        }

        size_t offset = 0;
        for (Job* j : taken) {
            std::copy_n(batch_out.begin() + offset, j->in.size(), j->out.begin());
            offset += j->in.size();
            j->done = true;
        }
        cv_clients.notify_all();
    }
}

} // namespace nn
```

12.2.5 `nnue/resnet.h`

98 lines.

The pure-C++ backend's declaration. The header comment is the performance rationale: NHWC activations, im2col + one GEMM per convolution, BatchNorm folded at export time, OpenMP over chunks of the batch.

symbol	role
`ResNetCPU`	`Evaluator` over the `.bin` written by `resnet_export.py`
`ResNetCPU::Block`	one residual block's folded weights: two (scale, shift) pairs and two kernels
`ResNetCPU::Scratch`	per-thread activation and im2col buffers, grown lazily
`ResNetCPU::forward`	the reference path used by the self-check: planes in, values and all 4096 logits out
`ResNetCPU::forward_chunk`	trunk plus heads on at most `MAX_CHUNK` positions
`ResNetCPU::policy_logit`	one logit from the 128-d policy feature vector — only the legal moves are computed

```
#ifndef RESNET_H
#define RESNET_H
// resnet.h — pure C++ (Eigen + OpenMP) inference of the two-headed ResNet
// trained by nnue/cnn_resnet_value_policy_chunked.py.
//
// Architecture (see the Python file for the training-side definition):
//   input (4 planes, 8x8) — 3x3 conv → BN → ReLU        (stem, BN folded)
//   — B × [ BN → ReLU → 3x3 conv → BN → ReLU → 3x3 conv, + skip ]
//   — BN → ReLU                                          (trunk output)
//   ├─ value : 1x1 conv → BN → ReLU → flatten(64) → FC → ReLU → FC → tanh
//   └─ policy: 1x1 conv → BN → ReLU → flatten(128) → FC → 4096 logits
//
// Implementation notes
// ————————————————————
// • Activations are kept pixel-major ("NHWC": [position][pixel 0..63][channel])
//   so that the 3x3 convolution becomes im2col + one big GEMM per layer:
//       Out (64n × C)  =  Col (64n × 9C)  ·  Wᵀ (9C × C)
//   Eigen's GEMM (with -march=native -mfma) runs close to peak on this shape.
//   The im2col copy itself is <5 % of the layer's cost.
// • BatchNorm is folded at export time (nnue/resnet_export.py): the stem and
//   the 1x1 head convs absorbed it into their weights; the pre-activation BNs
//   of the residual blocks become a per-channel affine map applied together
//   with the ReLU in a single pass over the activations.
// • Parallelism: one evaluate() call splits the batch into chunks of a few
//   positions and processes the chunks with an OpenMP team (each thread owns
//   its scratch buffers, Eigen runs single-threaded inside). The backend is
//   therefore most efficient with ONE search thread feeding it batches of
//   ~32-128 positions; several search threads should go through
//   nn::BatchingEvaluator so their requests are merged rather than nested.
// • Cost: ≈ 4.5 GFLOP per position for the 256×30 network — expect on the
//   order of 100 positions/s on a desktop CPU. The libtorch backend
//   (resnet_torch.h) is 20-50× faster on a GPU; this backend exists so the
//   player always works, and as the reference for correctness.
#include "nn_evaluator.h"
#include <Eigen/Core>
#include <string>
```

```cpp
#include <vector>

namespace nn {

class ResNetCPU final : public Evaluator {
public:
    using AlignedVec = std::vector<float, Eigen::aligned_allocator<float>>;

    // weights_filename : .bin produced by resnet_export.py
    // nb_threads       : OpenMP threads per evaluate() call (0 = OpenMP default)
    // preferred_batch  : batch size hint reported to the search
    explicit ResNetCPU(const std::string& weights_filename, int nb_threads = 0,
                       size_t preferred_batch = 32);
    ~ResNetCPU() override;

    void evaluate(std::span<const Request> in, std::span<Result> out) override;
    size_t preferred_batch_size() const override { return preferred_batch; }
    std::string info() const override;
    void reload(const std::string& weights_filename) override;

    // Reference forward used by the self-test: `planes` holds n × 256 floats
    // (NCHW, see encode_planes); writes n values and n × NUM_ACTIONS logits.
    void forward(const float* planes, size_t n, float* values, float* logits) const;

    int channels()  const { return C; }
    int nb_blocks() const { return B; }
    int nb_threads() const { return threads; }

private:
    struct Block {
        AlignedVec bn1_scale, bn1_shift, conv1_w;   // conv: (C, 9, C) row-major
        AlignedVec bn2_scale, bn2_shift, conv2_w;
    };
    struct Scratch;   // per-thread work buffers, defined in resnet.cpp

    void load(const std::string& weights_filename);
    // Runs the trunk + heads on n ≤ max_chunk positions. Writes n values and
    // the n × 128 policy feature vectors (input of the last policy FC).
    void forward_chunk(Scratch& s, const float* planes, size_t n,
                       float* values, float* policy_vec) const;
    // Policy logit of action index a given the 128-d policy feature vector.
    float policy_logit(const float* policy_vec, int a) const;

    std::string filename;
    int threads;
    size_t preferred_batch;
    static constexpr size_t MAX_CHUNK = 8;   // positions per GEMM (64 rows each)

    int C = 0, B = 0, F = 0, A = 0;          // channels, blocks, value fc size, actions
    AlignedVec stem_w, stem_b;               // (C, 9, 4), (C)
    std::vector<Block> blocks;
    AlignedVec out_scale, out_shift;         // trunk output BN
    AlignedVec vconv_w, vconv_b;             // (C), (1)
    AlignedVec vfc1_w, vfc1_b;               // (F, 64), (F)
    AlignedVec vfc2_w, vfc2_b;               // (F), (1)
    AlignedVec pconv_w, pconv_b;             // (2, C), (2)
    AlignedVec pfc_w, pfc_b;                 // (A, 128), (A)
};

} // namespace nn

#endif
```

12.2.6 `nnue/resnet.cpp`

299 lines.

The inference itself. The shape to keep in mind is $\mathrm{Out}_{64n\times C} = \mathrm{Col}_{64n\times 9C} \cdot W^{\mathsf{T}}_{9C\times C}$: every 3×3 convolution is one GEMM, and the pre-activation BatchNorm and ReLU are applied while the im2col patches are gathered, so the activations are read once per layer.

`flush_denormals` is not an optimisation, it is a correctness-of-performance fix: the trained network produces activations around 10^{-41}, and denormal arithmetic is ~100× slower on x86. FTZ/DAZ are per-thread MXCSR flags, so every OpenMP worker sets them itself rather than relying on link flags the LTO link line does not carry.

symbol	role
`TapTable`	precomputed source pixel for each (output pixel, kernel tap), -1 outside the board
`Reader`	little-endian reader for the `.bin` format documented in `resnet_export.py`
`bn_relu`	$y = \max(0, s_c x + t_c)$ over a row-major block — the folded pre-activation BN
`im2col`	gather the nine taps of every output pixel into one GEMM operand row
`flush_denormals`	set FTZ/DAZ on the calling thread
`ResNetCPU::load`	read the header, then every folded tensor in export order
`ResNetCPU::forward_chunk`	stem, B residual blocks, output BN, then both heads
`ResNetCPU::evaluate`	split the batch into chunks, run them under OpenMP, fill in only the legal logits

```
// resnet.cpp — see resnet.h for the design overview.
#define EIGEN_DONT_PARALLELIZE   // we parallelise over positions ourselves (OpenMP)
#include "resnet.h"
#include <Eigen/Dense>
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
#include <sstream>
#include <stdexcept>
#include <omp.h>
#include <pmmintrin.h>
#include <xmmintrin.h>

namespace nn {

namespace {

using RowMat = Eigen::Matrix<float, Eigen::Dynamic, Eigen::Dynamic, Eigen::RowMajor>;
using MapRow      = Eigen::Map<RowMat, Eigen::Aligned32>;
using MapConstRow = Eigen::Map<const RowMat, Eigen::Aligned32>;

// SRC[p][tap] = source pixel of output pixel p for kernel tap (kh, kw),
// or -1 when the tap falls outside the board (zero padding).
// p = 8*row + col, tap = 3*kh + kw, kh/kw in 0..2.
struct TapTable {
```

```
    int src[64][9];
    TapTable() {
        for (int p = 0; p < 64; p++) {
            const int r = p / 8, c = p % 8;
            for (int tap = 0; tap < 9; tap++) {
                const int rr = r + tap / 3 - 1, cc = c + tap % 3 - 1;
                src[p][tap] = (rr >= 0 && rr < 8 && cc >= 0 && cc < 8) ? rr * 8 + cc : -1;
            }
        }
    }
};
const TapTable TAPS;

// Little-endian binary reader for the .bin format documented in resnet_export.py.
struct Reader {
    FILE* f;
    std::string name;
    explicit Reader(const std::string& filename) : f(std::fopen(filename.c_str(), "rb")),
    ↪  name(filename) {
        if (!f) throw std::runtime_error("ResNetCPU: cannot open " + filename);
    }
    ~Reader() { if (f) std::fclose(f); }
    uint32_t u32() {
        uint32_t v;
        if (std::fread(&v, 4, 1, f) != 1) throw std::runtime_error("ResNetCPU: truncated file " +
        ↪  name);
        return v;
    }
    void floats(ResNetCPU::AlignedVec& dst, size_t n) {
        dst.resize(n);
        if (std::fread(dst.data(), sizeof(float), n, f) != n)
            throw std::runtime_error("ResNetCPU: truncated file " + name);
    }
};

// y = max(0, scale[c] * x + shift[c]) over a (rows × C) row-major block.
inline void bn_relu(const float* __restrict x, float* __restrict y, size_t rows, int C,
                    const float* __restrict scale, const float* __restrict shift) {
    for (size_t r = 0; r < rows; r++) {
        const float* xr = x + r * C;
        float* yr = y + r * C;
        for (int c = 0; c < C; c++) {
            yr[c] = std::max(0.0f, scale[c] * xr[c] + shift[c]);
        }
    }
}

// Col[(b*64 + p) × (tap*C + ch)] = H[b][src(p, tap)][ch]  (0 when padded).
inline void im2col(const float* __restrict h, float* __restrict col, size_t n, int C) {
    const size_t row_len = 9 * static_cast<size_t>(C);
    for (size_t b = 0; b < n; b++) {
        const float* hb = h + b * 64 * C;
        float* cb = col + b * 64 * row_len;
        for (int p = 0; p < 64; p++) {
            float* crow = cb + p * row_len;
            for (int tap = 0; tap < 9; tap++) {
                float* dst = crow + tap * C;
                const int src = TAPS.src[p][tap];
                if (src < 0) std::memset(dst, 0, C * sizeof(float));
                else         std::memcpy(dst, hb + src * C, C * sizeof(float));
            }
        }
    }
```

```cpp
}

// The trained network produces activations in the denormal range (1e-41 …),
// and denormal arithmetic is ~100× slower on x86. Flush-to-zero / denormals-
// are-zero are per-thread MXCSR flags, so set them on every worker thread of
// the parallel region instead of relying on the link flags (-ffast-math adds
// crtfastmath.o, but only for the main thread and only if it is on the link
// line — the CMake LTO link line does not have it).
inline void flush_denormals() {
    _MM_SET_FLUSH_ZERO_MODE(_MM_FLUSH_ZERO_ON);
    _MM_SET_DENORMALS_ZERO_MODE(_MM_DENORMALS_ZERO_ON);
}

} // namespace

// Per-thread scratch: activations X, H, Y (rows × C), im2col buffer (rows × 9C),
// the stem im2col (rows × 36) and the head maps. Grown lazily; thread_local.
struct ResNetCPU::Scratch {
    int C = 0;
    size_t rows = 0;
    AlignedVec X, H, Y, Col, Col0, VMap, PMap, V64, HV;
    void ensure(int channels, size_t n_positions, int F) {
        const size_t need = n_positions * 64;
        if (channels == C && need <= rows) return;
        C = channels; rows = std::max(rows, need);
        X.assign(rows * C, 0.f); H.assign(rows * C, 0.f); Y.assign(rows * C, 0.f);
        Col.assign(rows * 9 * C, 0.f);
        Col0.assign(rows * 36, 0.f);
        VMap.assign(rows, 0.f); PMap.assign(rows * 2, 0.f);
        V64.assign(64, 0.f); HV.assign(F, 0.f);
    }
};

ResNetCPU::ResNetCPU(const std::string& weights_filename, int nb_threads, size_t preferred_batch)
    : filename(weights_filename), threads(nb_threads), preferred_batch(preferred_batch) {
    load(weights_filename);
}

ResNetCPU::~ResNetCPU() = default;

void ResNetCPU::load(const std::string& weights_filename) {
    Reader r(weights_filename);
    char magic[8];
    if (std::fread(magic, 1, 8, r.f) != 8 || std::memcmp(magic, "YOLAHRSN", 8) != 0)
        throw std::runtime_error("ResNetCPU: bad magic in " + weights_filename +
                                 " (expected a file written by resnet_export.py)");
    const uint32_t version = r.u32();
    if (version != 1) throw std::runtime_error("ResNetCPU: unsupported version " +
     ↪  std::to_string(version));
    const int in_channels = static_cast<int>(r.u32());
    if (in_channels != PLANES) throw std::runtime_error("ResNetCPU: expected 4 input planes");
    const int c = static_cast<int>(r.u32()), b = static_cast<int>(r.u32());
    const int f = static_cast<int>(r.u32()), a = static_cast<int>(r.u32());
    if (C != 0 && (c != C || b != B || f != F || a != A))
        throw std::runtime_error("ResNetCPU::reload: architecture mismatch");
    if (a != NUM_ACTIONS) throw std::runtime_error("ResNetCPU: expected 4096 actions");
    C = c; B = b; F = f; A = a;

    r.floats(stem_w, static_cast<size_t>(C) * 9 * PLANES);
    r.floats(stem_b, C);
    blocks.assign(B, Block{});
    for (Block& blk : blocks) {
        r.floats(blk.bn1_scale, C); r.floats(blk.bn1_shift, C);
```

```
        r.floats(blk.conv1_w, static_cast<size_t>(C) * 9 * C);
        r.floats(blk.bn2_scale, C); r.floats(blk.bn2_shift, C);
        r.floats(blk.conv2_w, static_cast<size_t>(C) * 9 * C);
    }
    r.floats(out_scale, C); r.floats(out_shift, C);
    r.floats(vconv_w, C); r.floats(vconv_b, 1);
    r.floats(vfc1_w, static_cast<size_t>(F) * 64); r.floats(vfc1_b, F);
    r.floats(vfc2_w, F); r.floats(vfc2_b, 1);
    r.floats(pconv_w, 2 * static_cast<size_t>(C)); r.floats(pconv_b, 2);
    r.floats(pfc_w, static_cast<size_t>(A) * 128); r.floats(pfc_b, A);
    // Anything left over means the exporter and this reader disagree.
    char probe;
    if (std::fread(&probe, 1, 1, r.f) == 1)
        throw std::runtime_error("ResNetCPU: trailing bytes in " + weights_filename);
    filename = weights_filename;
}

void ResNetCPU::reload(const std::string& weights_filename) {
    load(weights_filename);
}

std::string ResNetCPU::info() const {
    std::ostringstream os;
    os << "ResNetCPU " << C << "x" << B << " (" << filename << ", "
       << (threads ? std::to_string(threads) : std::string("omp default")) << " threads)";
    return os.str();
}

float ResNetCPU::policy_logit(const float* policy_vec, int a) const {
    const float* w = pfc_w.data() + static_cast<size_t>(a) * 128;
    float s = pfc_b[a];
    for (int k = 0; k < 128; k++) s += w[k] * policy_vec[k];
    return s;
}

void ResNetCPU::forward_chunk(Scratch& s, const float* planes, size_t n,
                              float* values, float* policy_vec) const {
    s.ensure(C, std::max(n, MAX_CHUNK), F);
    const size_t rows = n * 64;

    // ── stem: im2col over the 4 input planes (NCHW → pixel-major 36-wide rows) ──
    for (size_t b = 0; b < n; b++) {
        const float* pl = planes + b * INPUT_FLOATS;
        for (int p = 0; p < 64; p++) {
            float* crow = s.Col0.data() + (b * 64 + p) * 36;
            for (int tap = 0; tap < 9; tap++) {
                const int src = TAPS.src[p][tap];
                for (int c = 0; c < PLANES; c++) {
                    crow[tap * PLANES + c] = src < 0 ? 0.f : pl[c * 64 + src];
                }
            }
        }
    }
    MapRow X(s.X.data(), rows, C);
    {
        MapConstRow col0(s.Col0.data(), rows, 36);
        MapConstRow w(stem_w.data(), C, 36);
        X.noalias() = col0 * w.transpose();
        X.rowwise() += Eigen::Map<const Eigen::RowVectorXf>(stem_b.data(), C);
        X = X.cwiseMax(0.0f);                    // the stem ends with a ReLU
    }

    // ── residual tower ──
```

```
    MapRow Y(s.Y.data(), rows, C);
    MapConstRow col(s.Col.data(), rows, 9 * C);
    for (const Block& blk : blocks) {
        bn_relu(s.X.data(), s.H.data(), rows, C, blk.bn1_scale.data(), blk.bn1_shift.data());
        im2col(s.H.data(), s.Col.data(), n, C);
        MapConstRow w1(blk.conv1_w.data(), C, 9 * C);
        Y.noalias() = col * w1.transpose();
        bn_relu(s.Y.data(), s.H.data(), rows, C, blk.bn2_scale.data(), blk.bn2_shift.data());
        im2col(s.H.data(), s.Col.data(), n, C);
        MapConstRow w2(blk.conv2_w.data(), C, 9 * C);
        X.noalias() += col * w2.transpose();      // skip connection
    }
    bn_relu(s.X.data(), s.H.data(), rows, C, out_scale.data(), out_shift.data());
    MapConstRow H(s.H.data(), rows, C);

    // ── heads ──
    // 1x1 convs are plain matrix products over the pixel rows.
    Eigen::Map<Eigen::VectorXf> vmap(s.VMap.data(), rows);
    vmap.noalias() = H * Eigen::Map<const Eigen::VectorXf>(vconv_w.data(), C);
    MapRow pmap(s.PMap.data(), rows, 2);
    pmap.noalias() = H * MapConstRow(pconv_w.data(), 2, C).transpose();

    Eigen::Map<const RowMat> Wfc1(vfc1_w.data(), F, 64);
    Eigen::Map<const Eigen::VectorXf> bfc1(vfc1_b.data(), F);
    Eigen::Map<const Eigen::VectorXf> wfc2(vfc2_w.data(), F);
    Eigen::Map<Eigen::VectorXf> v64(s.V64.data(), 64);
    Eigen::Map<Eigen::VectorXf> hv(s.HV.data(), F);
    for (size_t b = 0; b < n; b++) {
        // value: relu(conv+bias) over the 64 pixels → FC → ReLU → FC → tanh
        for (int p = 0; p < 64; p++) v64[p] = std::max(0.f, s.VMap[b * 64 + p] + vconv_b[0]);
        hv.noalias() = Wfc1 * v64 + bfc1;
        hv = hv.cwiseMax(0.f);
        values[b] = std::tanh(wfc2.dot(hv) + vfc2_b[0]);
        // policy: relu(conv+bias), flattened channel-major (c*64 + p) as in torch's flatten
        float* pv = policy_vec + b * 128;
        for (int p = 0; p < 64; p++) {
            pv[p]      = std::max(0.f, s.PMap[(b * 64 + p) * 2 + 0] + pconv_b[0]);
            pv[64 + p] = std::max(0.f, s.PMap[(b * 64 + p) * 2 + 1] + pconv_b[1]);
        }
    }
}

void ResNetCPU::forward(const float* planes, size_t n, float* values, float* logits) const {
    const size_t nb_chunks = (n + MAX_CHUNK - 1) / MAX_CHUNK;
#pragma omp parallel for schedule(dynamic, 1) num_threads(threads > 0 ? threads :
 ↪  omp_get_max_threads())
    for (size_t k = 0; k < nb_chunks; k++) {
        thread_local Scratch scratch;
        flush_denormals();
        const size_t start = k * MAX_CHUNK, cnt = std::min(MAX_CHUNK, n - start);
        float pvec[MAX_CHUNK * 128];
        forward_chunk(scratch, planes + start * INPUT_FLOATS, cnt, values + start, pvec);
        for (size_t b = 0; b < cnt; b++) {
            float* out = logits + (start + b) * NUM_ACTIONS;
            for (int a = 0; a < NUM_ACTIONS; a++) out[a] = policy_logit(pvec + b * 128, a);
        }
    }
}

void ResNetCPU::evaluate(std::span<const Request> in, std::span<Result> out) {
    const size_t n = in.size();
    if (out.size() < n) throw std::invalid_argument("ResNetCPU::evaluate: output span too small");
    // Chunk so that every OpenMP thread gets work even for small batches.
```

```
    const int nthreads = threads > 0 ? threads : omp_get_max_threads();
    const size_t chunk = std::clamp<size_t>(n / static_cast<size_t>(nthreads), 1, MAX_CHUNK);
    const size_t nb_chunks = (n + chunk - 1) / chunk;
#pragma omp parallel for schedule(dynamic, 1) num_threads(nthreads)
    for (size_t k = 0; k < nb_chunks; k++) {
        thread_local Scratch scratch;
        flush_denormals();
        const size_t start = k * chunk, cnt = std::min(chunk, n - start);
        float planes[MAX_CHUNK * INPUT_FLOATS];
        float values[MAX_CHUNK];
        float pvec[MAX_CHUNK * 128];
        for (size_t b = 0; b < cnt; b++) encode_planes(*in[start + b].state, planes + b *
        ↪  INPUT_FLOATS);
        forward_chunk(scratch, planes, cnt, values, pvec);
        for (size_t b = 0; b < cnt; b++) {
            const Request& rq = in[start + b];
            Result& rs = out[start + b];
            rs.value = values[b];
            for (uint16_t m = 0; m < rq.nb_moves; m++) {
                rs.logits[m] = policy_logit(pvec + b * 128, action_index(rq.moves[m]));
            }
        }
    }
}

} // namespace nn
```

12.2.7 `nnue/resnet_torch.h`

48 lines.

The libtorch backend's declaration, compiled only under `-DENABLE_TORCH=ON`.

symbol	role
`ResNetTorch`	`Evaluator` over the TorchScript module, on CUDA or CPU, optionally in fp16

```
#ifndef RESNET_TORCH_H
#define RESNET_TORCH_H
// resnet_torch.h — libtorch backend: runs the TorchScript module exported by
// nnue/resnet_export.py (<name>.ts) on the GPU (or CPU) through libtorch.
//
// Only compiled when the project is configured with -DENABLE_TORCH=ON; the
// build then needs a libtorch install (or the one that ships inside a pip
// torch package: -DCMAKE_PREFIX_PATH=<site-packages>/torch).
//
// The whole batch is one forward pass: (n, 4, 8, 8) → (n,), (n, 4096).
// In half precision on an RTX-class GPU the 256x30 network evaluates several
// thousand positions per second at batch 256, i.e. 20-50× the CPU backend.
#ifdef ENABLE_TORCH
#include "nn_evaluator.h"
#include <torch/script.h>
#include <torch/cuda.h>
#include <mutex>
#include <string>

namespace nn {

```

```
class ResNetTorch final : public Evaluator {
public:
    // device: "cuda", "cuda:1", "cpu" ... ; fp16: run the network in half precision
    ResNetTorch(const std::string& module_filename, const std::string& device = "cuda",
                bool fp16 = true, size_t preferred_batch = 256);

    void evaluate(std::span<const Request> in, std::span<Result> out) override;
    size_t preferred_batch_size() const override { return preferred_batch; }
    std::string info() const override;
    void reload(const std::string& module_filename) override;

private:
    void load(const std::string& module_filename);

    std::string filename;
    torch::Device device;
    bool fp16;
    size_t preferred_batch;
    torch::jit::script::Module module;
    std::mutex mutex;          // one forward at a time (callers are batched anyway)
    torch::Tensor input_cpu;   // pinned staging buffer, grown on demand
};

} // namespace nn

#endif // ENABLE_TORCH
#endif
```

12.2.8 `nnue/resnet_torch.cpp`

87 lines.

Load the traced module, stage the batch into a pinned CPU tensor, one forward, read the two outputs back. The mutex serialises forwards — callers are batched upstream anyway.

symbol	role
`ResNetTorch::load`	`torch::jit::load`, move to the device, `eval()`, optionally `to(half)`
`ResNetTorch::evaluate`	encode into the staging tensor, forward, scatter the legal logits back

```
#ifdef ENABLE_TORCH
#include "resnet_torch.h"
#include <sstream>
#include <stdexcept>

namespace nn {

ResNetTorch::ResNetTorch(const std::string& module_filename, const std::string& device_name,
                         bool fp16, size_t preferred_batch)
    : filename(module_filename),
      device(device_name == "cuda" && !torch::cuda::is_available() ? torch::Device(torch::kCPU)
                                                                   : torch::Device(device_name)),
      fp16(fp16 && device.is_cuda()),   // half precision only makes sense on the GPU
      preferred_batch(preferred_batch) {
    load(module_filename);
}

```

```
void ResNetTorch::load(const std::string& module_filename) {
    std::lock_guard lock(mutex);
    // The default profiling executor re-specialises the graph for every new
    // batch shape (several seconds each, and the search uses many shapes):
    // use the plain executor. These are process-wide flags and must be set
    // before the module is loaded (a module frozen/optimised under the other
    // executor cannot be run by this one).
    torch::jit::getProfilingMode() = false;
    torch::jit::setGraphExecutorOptimize(false);
    try {
        module = torch::jit::load(module_filename, device);
    } catch (const c10::Error& e) {
        throw std::runtime_error("ResNetTorch: cannot load " + module_filename + ": " +
            e.what_without_backtrace());
    }
    module.eval();
    if (fp16) module.to(torch::kHalf);
    filename = module_filename;
    // Warm up cuDNN / the executor on the shapes the search is likely to use.
    torch::NoGradGuard no_grad;
    for (int64_t n : {int64_t(1), int64_t(8), int64_t(32), int64_t(128),
        int64_t(preferred_batch)}) {
        torch::Tensor x = torch::zeros({n, PLANES, 8, 8}, torch::TensorOptions().dtype(fp16 ?
    torch::kHalf : torch::kFloat32).device(device));
        module.forward({x});
    }
    if (device.is_cuda()) torch::cuda::synchronize();
}

void ResNetTorch::reload(const std::string& module_filename) {
    load(module_filename);
}

std::string ResNetTorch::info() const {
    std::ostringstream os;
    os << "ResNetTorch (" << filename << ", " << device << (fp16 ? ", fp16" : ", fp32") << ")";
    return os.str();
}

void ResNetTorch::evaluate(std::span<const Request> in, std::span<Result> out) {
    const int64_t n = static_cast<int64_t>(in.size());
    if (n == 0) return;
    if (out.size() < in.size()) throw std::invalid_argument("ResNetTorch::evaluate: output span
        too small");
    std::lock_guard lock(mutex);
    torch::NoGradGuard no_grad;
    c10::InferenceMode inference;

    if (!input_cpu.defined() || input_cpu.size(0) < n) {
        auto options =
            torch::TensorOptions().dtype(torch::kFloat32).pinned_memory(device.is_cuda());
        input_cpu = torch::empty({n, PLANES, 8, 8}, options);
    }
    float* dst = input_cpu.data_ptr<float>();
    for (int64_t i = 0; i < n; i++) encode_planes(*in[i].state, dst + i * INPUT_FLOATS);

    torch::Tensor x = input_cpu.narrow(0, 0, n).to(device, fp16 ? torch::kHalf : torch::kFloat32,
                                                   /*non_blocking=*/true);
    auto outputs = module.forward({x}).toTuple();
    // Both heads back to the host in fp32, one copy each.
    torch::Tensor values = outputs->elements()[0].toTensor().to(torch::kCPU,
    torch::kFloat32).contiguous();
    torch::Tensor logits = outputs->elements()[1].toTensor().to(torch::kCPU,
    torch::kFloat32).contiguous();
```

```
    const float* v = values.data_ptr<float>();
    const float* p = logits.data_ptr<float>();
    for (int64_t i = 0; i < n; i++) {
        const Request& rq = in[i];
        Result& rs = out[i];
        rs.value = v[i];
        const float* row = p + i * NUM_ACTIONS;
        for (uint16_t m = 0; m < rq.nb_moves; m++) rs.logits[m] = row[action_index(rq.moves[m])];
    }
}

} // namespace nn
#endif // ENABLE_TORCH
```

12.3 Training and export

The Python half. `cnn_resnet_value_policy_chunked.py` is the script the shipped weights were trained with and the one `resnet_export.py` imports; its sibling `cnn_resnet_value_policy.py` defines the *same* network and differs only in the data pipeline (a stock `DataLoader` with worker processes, which is the right choice when the cache fits in page cache).

12.3.1 `nnue/cnn_resnet_value_policy_chunked.py`

875 lines.

The network definition, the encoding, and the training loop.

The loader is the part that is specific to this project rather than to AlphaZero. The position cache is a ~253 GB file; sampling it uniformly at random turns training into millions of scattered 256-byte reads, which on a spinning disk is seek-bound. `ChunkedShuffleLoader` reads ~1 GB **contiguous** chunks sequentially, shuffles within a chunk and reshuffles the chunk order each epoch — an approximate shuffle, but a chunk holds ~4 M positions, and sequential reads are fast without pre-warming 253 GB of RAM. One background thread per rank does the reading and batch construction while the main thread drives the GPU; the overlap is real because numpy's memcpy and CUDA's pin-memory both release the GIL.

The DDP detail that bites: every rank must yield *exactly* the same number of batches, or the first rank to finish hangs the others on the gradient all-reduce. Hence the floor division in the chunk sharding and the dropped trailing batches.

symbol	role
`_bitboard_to_plane`, `encode_cnn`	the Python side of Section 3 — must match `nn::encode_planes`
`ChunkedShuffleLoader`	sequential chunked reader over the memmap cache, double-buffered, DDP-sharded
`ChunkedShuffleLoader._producer`	the background thread: read a chunk, shuffle it, emit pinned batches
`ResBlock`	pre-activation residual block (Figure 4)
`Net`	stem, B blocks, output BN, value head and policy head (Figure 3)
`Net.forward`	returns `(value, policy_logits)`
`TrainerDDP`	one process per GPU: channels-last, AMP, `torch.compile`, cosine LR

symbol	role
`TrainerDDP._compute_loss`	$\lambda_v \,\mathrm{MSE}(v, z) + \lambda_p \,\mathrm{CE}(\ell, a^\star)$
`TrainerDDP._run_epoch` / `_validate`	the loop, with value sign accuracy and policy top-1 as the readable metrics
`main`	contiguous train/val split, build the loaders inside the spawned process, train

```python
"""
cnn_resnet_value_policy_chunked.py
==================================
AlphaZero-style two-headed ResNet for the game of Yolah, trained with a
custom double-buffered "chunked-shuffle" data loader.

It is functionally the same network as cnn_resnet_value_policy.py — the ONLY
difference is the data pipeline:

  cnn_resnet_value_policy.py        torch DataLoader + 8 worker processes,
                                    random-access reads into the memmap cache.
  cnn_resnet_value_policy_chunked.py (this file)  one background thread per
                                    rank, large sequential chunk reads.

Why the chunked loader exists
─────────────────────────────
The cache (produced by preprocess.py) is a ~253 GB file of encoded positions.
A standard DataLoader samples positions uniformly at random, which becomes
millions of random ~256-byte reads scattered across that file. On an
HDD-backed filesystem random reads are seek-bound and collapse throughput;
the only rescue is pre-loading the whole 253 GB into RAM page cache.

This loader instead:
  • reads large CONTIGUOUS chunks (~1 GB) *sequentially* — sequential reads
    are fast even on spinning disks, so no RAM pre-warm is needed;
  • shuffles positions WITHIN a chunk and reshuffles the chunk ORDER each
    epoch. This is an approximate shuffle, but a chunk holds ~4M positions
    (~80k games), so any one batch still mixes thousands of games — gradients
    stay decorrelated;
  • does the chunk read + batch construction in ONE background thread while
    the main thread runs the GPU. A bounded queue between them is the
    rotating "ping-pong" buffer. This overlap works because numpy's memcpy /
    dtype-convert and CUDA's pin-memory all release the Python GIL, so the
    helper thread genuinely runs concurrently with GPU compute;
  • hands the main loop batch tensors that are already pinned, so the
    host→GPU copy can be asynchronous.

Everything below the loader — the network, the loss, mixed precision,
channels-last memory format, DistributedDataParallel — is standard and
identical to the sibling script.
"""

# ── Imports ──────────────────────────────────────────────────────────────
from tqdm import tqdm                  # progress bar around the training loop
import torch
from torch import nn
import torch.multiprocessing as mp     # mp.spawn launches one process per GPU
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.distributed import init_process_group, destroy_process_group
import os
import sys
import gc                              # disabled inside main() to avoid GC pauses
import json                            # the cache's meta.json sidecar
import random                          # epoch-level chunk-order shuffle
```

```
import threading                        # the background producer thread
import queue as queue_mod               # bounded hand-off queue (the buffer)
import numpy as np

# The position encoder needs the Yolah game module, which lives in ../server.
sys.path.append("../server")
from yolah import Yolah, Move, Square

# Allow TF32 matmuls on Ampere+ (a speed/precision trade that is safe here).
torch.set_float32_matmul_precision('high')


# ── Board encoding ──────────────────────────────────────────────────
#
# Every position is a (4, 8, 8) float tensor — four 8×8 "planes":
#   plane 0 — black pieces  : 1 where a black stone sits, else 0
#   plane 1 — white pieces  : 1 where a white stone sits, else 0
#   plane 2 — empty squares : 1 where a square has been vacated (scored), else 0
#   plane 3 — turn          : a constant plane, all-0 if black is to move,
#                             all-1 if white is to move
# (preprocess.py writes this same layout to disk as uint8.)

# Policy action space: a Yolah move is (from_square, to_square), each in 0..63.
# We flatten it to a single index  action = from*64 + to  → 64*64 = 4096 logits.
NUM_ACTIONS = 64 * 64

# A terminal position has no "next move" to predict. Its policy target is set
# to this sentinel, and CrossEntropyLoss(ignore_index=...) skips those rows.
POLICY_IGNORE_INDEX = -1

# Value target z ∈ {-1, 0, +1}, from the CURRENT player's point of view:
#   +1  current player went on to win
#    0  draw
#   -1  current player lost
# The network's value head ends in tanh, so its output v ∈ (-1, +1); the
# value loss is MSE(v, z).

# Number of input planes — must match what preprocess.py wrote (a guard in the
# loader checks this against meta.json).
IN_CHANNELS = 4


def _bitboard_to_plane(n: int) -> np.ndarray:
    """
    Expand a 64-bit Yolah bitboard into an 8×8 float32 plane.

    The flattened plane is filled MSB-first: array cell i receives bit
    (63 - i) of `n`, so cell 0 holds bit 63 and cell 63 holds bit 0.

    Note this is the reverse of Yolah's own square numbering — in yolah.py
    square s is bit s, i.e. square 0 is the LEAST-significant bit — so the
    plane ends up in reverse-square order. That is harmless: a CNN does not
    care about the absolute orientation of the 8×8 plane. What matters is
    that this matches preprocess.py's bb_to_plane (np.unpackbits on a
    big-endian word, which yields the same MSB-first order), so the inference
    encoder and the cached training tensors agree. Do not "fix" the order
    here without regenerating the cache.

    Used only by encode_cnn() for inference; training reads pre-encoded
    planes straight from the cache.
    """
    b = np.zeros(64, dtype=np.float32)
    for i in range(64):
```

```
        if n & (1 << (63 - i)):
            b[i] = 1.0
    return b.reshape(8, 8)


def encode_cnn(yolah) -> torch.Tensor:
    """
    Encode a live Yolah position into the (4, 8, 8) float32 tensor the network
    expects. This is the INFERENCE-time encoder (e.g. for MCTS); training does
    not call it — preprocess.py already baked these tensors into the cache.
    """
    black = _bitboard_to_plane(yolah.black)
    white = _bitboard_to_plane(yolah.white)
    empty = _bitboard_to_plane(yolah.empty)
    # Black moves on even plies (0, 2, 4, ...), white on odd plies.
    black_to_move = (yolah.nb_plies() & 1) == 0
    turn  = np.full((8, 8), 0.0 if black_to_move else 1.0, dtype=np.float32)
    return torch.from_numpy(np.stack([black, white, empty, turn]))


# ── Chunked-shuffle double-buffered loader ──────────────────────────────
#
# CHUNK_SIZE — number of positions read per chunk.
#   4194304 = 2048 * 2048. Two deliberate properties:
#     • it is an exact multiple of the batch size (2048) → every chunk yields
#       exactly 2048 whole batches, no ragged tail;
#     • at 256 bytes/position it is ~1 GB of uint8 — small enough that a few
#       chunks fit comfortably in RAM, large enough to span ~80k games so the
#       within-chunk shuffle decorrelates batches well.
CHUNK_SIZE = 2048 * 2048

# BATCH_QUEUE_DEPTH — how many fully-built, pinned batches the producer thread
# may run ahead of the GPU. This is the depth of the ping-pong buffer: bigger
# = more slack to absorb a slow chunk read, but more pinned RAM held at once
# (each batch ≈ 2 MB → 32 batches ≈ 64 MB per rank).
BATCH_QUEUE_DEPTH = 32


class ChunkedShuffleLoader:
    """
    Streaming data loader over the memmap cache produced by preprocess.py.

    It is a drop-in iterable for the trainer:
        • iterating it yields (state, value_target, policy_target) tensors,
          one batch at a time, already pinned for async host→GPU copy;
        • len(loader) is the number of batches in one epoch;
        • set_epoch(e) reseeds the shuffling so each epoch differs.

    Distributed training (DDP):
        With `world_size` GPUs, rank r reads a DISJOINT, strided subset of the
        chunks (chunks r, r+world_size, r+2*world_size, ...). Remainder chunks
        and any partial trailing batch are dropped so EVERY rank yields exactly
        the same number of batches — this is essential: each backward pass does
        a collective all-reduce, and if one rank ran fewer steps the others
        would hang waiting on it.
    """

    def __init__(self, cache_dir, lo, hi, batch_size, rank, world_size,
                 chunk_size=CHUNK_SIZE, shuffle=True, pin_memory=True,
                 queue_depth=BATCH_QUEUE_DEPTH):
        # cache_dir   — directory holding positions.u8 / values.i8 /
        #               policies.i16 / meta.json
        # [lo, hi)    — the slice of the global dataset this loader covers
```

```
        #               (the caller passes the train slice or the val slice)
        # batch_size  — positions per batch, PER GPU
        # rank/world_size — this process's DDP rank and the total #GPUs
        # shuffle     — True for training, False for validation
        # pin_memory  — pin produced batches (page-locked RAM → async H2D copy)
        self.cache_dir   = cache_dir
        self.batch_size  = batch_size
        self.rank        = rank
        self.world_size  = world_size
        self.chunk_size  = chunk_size
        self.shuffle     = shuffle
        self.pin_memory  = pin_memory
        self.queue_depth = queue_depth
        self.epoch       = 0
        self._open()     # open the memmaps

        # Tile the half-open range [lo, hi) into whole, non-overlapping chunks.
        # `all_starts` is the list of every chunk's starting position. The
        # `hi - chunk_size + 1` upper bound discards a final partial chunk.
        all_starts = list(range(lo, hi - chunk_size + 1, chunk_size))

        # Shard the chunks across ranks. `per_rank` is floor-divided so every
        # rank gets the SAME count; up to (world_size - 1) leftover chunks are
        # dropped. Rank r then takes a strided slice starting at index r.
        per_rank = len(all_starts) // world_size
        self.my_chunks = all_starts[rank : per_rank * world_size : world_size]

        # With CHUNK_SIZE an exact multiple of batch_size this division is
        # clean; otherwise the trailing partial batch of each chunk is dropped.
        self.batches_per_chunk = chunk_size // batch_size

        # Total batches this rank yields per epoch — identical across ranks.
        self.n_batches = len(self.my_chunks) * self.batches_per_chunk

    def _open(self):
        """
        Open the three cache files as read-only memory maps.

        np.memmap does not read any data here — it only maps the file into the
        process's virtual address space. Bytes are paged in lazily by the OS as
        the producer thread actually touches them, and the page cache is shared
        by every process on the node (so 3 ranks do not triple RAM use).
        """
        with open(os.path.join(self.cache_dir, "meta.json")) as f:
            meta = json.load(f)

        # Fail loudly on a stale cache (e.g. a 6-plane cache from an older
        # preprocess.py) rather than silently mis-reading bytes.
        n_planes = meta["positions"]["shape"][1]
        if n_planes != IN_CHANNELS:
            raise ValueError(
                f"Cache has {n_planes} planes but training expects {IN_CHANNELS}. "
                "Re-run preprocess.py.")

        n = int(meta["n_positions"])
        # positions.u8  : (N, 4, 8, 8) uint8  — the board planes
        self.positions = np.memmap(
            os.path.join(self.cache_dir, meta["positions"]["path"]),
            dtype=np.uint8, mode='r', shape=tuple(meta["positions"]["shape"]))
        # values.i8     : (N,)         int8   — value target z ∈ {-1, 0, +1}
        self.values = np.memmap(
            os.path.join(self.cache_dir, meta["values"]["path"]),
            dtype=np.int8, mode='r', shape=(n,))
```

```
        # policies.i16  : (N,)         int16  — action index, or -1 if terminal
        self.policies = np.memmap(
            os.path.join(self.cache_dir, meta["policies"]["path"]),
            dtype=np.int16, mode='r', shape=(n,))

    def set_epoch(self, epoch):
        """Reseed shuffling for a new epoch (chunk order + within-chunk perm)."""
        self.epoch = epoch

    def __len__(self):
        """Number of batches per epoch — lets tqdm show a correct total."""
        return self.n_batches

    def _producer(self, chunk_order, q):
        """
        The background worker thread (one per loader, started by __iter__).

        For each chunk, in the epoch's shuffled order:
          1. read the whole chunk sequentially from the memmap into RAM;
          2. build a permutation that shuffles positions within the chunk;
          3. slice it into batches, convert dtypes, optionally pin, and push
             each batch onto the bounded queue `q`.

        The queue's `maxsize` is the back-pressure mechanism: once it holds
        `queue_depth` batches, `q.put(...)` blocks, so this thread can run at
        most that far ahead of the consumer and never grows memory unbounded.

        Why a thread (not a process) overlaps the GPU: numpy's bulk memcpy /
        astype and torch's pin_memory all drop the GIL for their heavy work,
        and so does every CUDA call in the main thread — so the two threads
        truly run at the same time.
        """
        bs = self.batch_size
        try:
            for chunk_id in chunk_order:
                start = self.my_chunks[chunk_id]
                end   = start + self.chunk_size

                # --- step 1: sequential read of the chunk into RAM ----------
                # np.array(memmap_slice) forces a contiguous copy, i.e. an
                # actual sequential read of this region off disk.
                pos = np.array(self.positions[start:end])   # (C, 4, 8, 8) uint8
                val = np.array(self.values[start:end])      # (C,)         int8
                pol = np.array(self.policies[start:end])    # (C,)         int16

                # --- step 2: within-chunk shuffle ---------------------------
                # Seed depends on both epoch and chunk start, so a given chunk
                # is permuted differently every epoch. `& 0xFFFFFFFF` keeps the
                # seed inside the 32-bit range numpy's Generator expects.
                if self.shuffle:
                    rng  = np.random.default_rng(
                        (self.epoch * 1_000_003 + start) & 0xFFFFFFFF)
                    perm = rng.permutation(self.chunk_size)
                else:
                    # Validation: keep natural order — no need to shuffle.
                    perm = np.arange(self.chunk_size)

                # --- step 3: cut the chunk into batches ---------------------
                for b in range(self.batches_per_chunk):
                    # `idx` are the rows of this chunk that form batch b.
                    idx = perm[b * bs : (b + 1) * bs]
                    # Gather rows and convert to the dtypes the model wants:
                    #   X : float32 board planes
```

```
                    #   v : float32 value target
                    #   p : int64   policy target (CrossEntropyLoss needs int64)
                    X = torch.from_numpy(pos[idx].astype(np.float32))
                    v = torch.from_numpy(val[idx].astype(np.float32))
                    p = torch.from_numpy(pol[idx].astype(np.int64))
                    # Pin into page-locked RAM so the later .to(gpu,
                    # non_blocking=True) can be a true asynchronous DMA copy.
                    if self.pin_memory:
                        X, v, p = X.pin_memory(), v.pin_memory(), p.pin_memory()
                    # Blocks here if the consumer is behind (queue full).
                    q.put((X, v, p))
            except Exception as e:                              # pragma: no cover
                # A crash in a thread is otherwise silent; surface it.
                print(f"[ChunkedShuffleLoader] producer error: {e}", flush=True)
            finally:
                # Always post the end-of-epoch sentinel, even on error, so the
                # consumer's loop terminates instead of blocking forever.
                q.put(None)

    def __iter__(self):
        """
        Start one epoch: spawn the producer thread and yield its batches.

        Called once per epoch (by the trainer's `for ... in loader`). Each call
        creates a fresh queue + thread, so epochs are independent.
        """
        # Decide the order in which this rank visits its chunks this epoch.
        order = list(range(len(self.my_chunks)))
        if self.shuffle:
            random.Random(self.epoch).shuffle(order)

        # The bounded queue IS the rotating buffer between producer & consumer.
        q = queue_mod.Queue(maxsize=self.queue_depth)
        # daemon=True → the thread will not keep the process alive on exit.
        t = threading.Thread(target=self._producer, args=(order, q), daemon=True)
        t.start()

        # Consume batches until the producer posts the None sentinel.
        while True:
            item = q.get()
            if item is None:
                break
            yield item
        t.join()   # tidy up the (already finished) producer thread


# ── Network ──────────────────────────────────────────────────────────────
class ResBlock(nn.Module):
    """
    One pre-activation residual block (the "ResNet v2" ordering:
    BatchNorm → ReLU → Conv, twice, then add the input back).

    The skip connection (x + residual) lets gradients flow straight through,
    which is what makes a deep (30-block) stack trainable.
    """
    def __init__(self, channels: int):
        super().__init__()
        # bias=False on the convs because the following BatchNorm has its own
        # learnable shift, making a conv bias redundant.
        self.bn1   = nn.BatchNorm2d(channels)
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
```

```
370
371     def forward(self, x):
372         residual = x                                    # save the input
373         x = self.conv1(torch.relu(self.bn1(x)))         # BN → ReLU → Conv
374         x = self.conv2(torch.relu(self.bn2(x)))         # BN → ReLU → Conv
375         return x + residual                             # add the skip connection
376
377
378 class Net(nn.Module):
379     """
380     AlphaZero-style two-headed network.
381
382       Trunk : (B, 4, 8, 8) − input conv − N residual blocks − BN − ReLU
383               → a shared (B, C, 8, 8) feature map.
384
385       Value head  : 1×1 conv to 1 channel → flatten(64) → FC → ReLU → FC
386                     → tanh → a scalar v ∈ (-1, +1) per position.
387                     Interpretation: expected game outcome for the player to
388                     move; P(win) = (v + 1) / 2.
389
390       Policy head : 1×1 conv to 2 channels → flatten(128) → FC
391                     → 4096 logits, one per (from, to) action.
392
393       global_pool (KataGo): both heads also see the mean and the max of the
394       trunk over the 64 squares (2C numbers), through value_gp_fc (added to
395       value_fc1 before its ReLU) and policy_gp_fc (added to the logits). The
396       1- and 2-channel maps above are narrow bottlenecks; the pooled vector
397       gives the heads a summary of the whole board. Both layers have no
398       bias, so zero weights make the network exactly the one without
399       pooling (alphazero_grow.py relies on it).
400
401     forward() returns the pair (value, policy_logits).
402     """
403     def __init__(self, channels: int = 256, nb_blocks: int = 30,
404                  value_fc_size: int = 256, num_actions: int = NUM_ACTIONS,
405                  global_pool: bool = False):
406         super().__init__()
407         self.global_pool = global_pool
408
409         # Stem: lift the 4 input planes to `channels` feature maps.
410         self.input_conv = nn.Sequential(
411             nn.Conv2d(4, channels, kernel_size=3, padding=1, bias=False),
412             nn.BatchNorm2d(channels),
413             nn.ReLU(inplace=True),
414         )
415         # Shared trunk: nb_blocks residual blocks, then a final BN.
416         self.res_blocks = nn.Sequential(*[ResBlock(channels) for _ in range(nb_blocks)])
417         self.output_bn  = nn.BatchNorm2d(channels)
418
419         # Value head. The 1×1 conv squeezes C channels down to 1, giving a
420         # (B, 1, 8, 8) map = 64 numbers, which the FC layers turn into a scalar.
421         self.value_conv = nn.Conv2d(channels, 1, kernel_size=1, bias=False)
422         self.value_bn   = nn.BatchNorm2d(1)
423         self.value_fc1  = nn.Linear(64, value_fc_size)
424         self.value_fc2  = nn.Linear(value_fc_size, 1)
425
426         # Policy head. 1×1 conv to 2 channels → (B, 2, 8, 8) = 128 numbers →
427         # one FC straight to the 4096-way action logits.
428         self.policy_conv = nn.Conv2d(channels, 2, kernel_size=1, bias=False)
429         self.policy_bn   = nn.BatchNorm2d(2)
430         self.policy_fc   = nn.Linear(2 * 64, num_actions)
431
432         if global_pool:
```

```
            self.value_gp_fc  = nn.Linear(2 * channels, value_fc_size, bias=False)
            self.policy_gp_fc = nn.Linear(2 * channels, num_actions, bias=False)

    def trunk(self, x):
        x = self.input_conv(x)
        x = self.res_blocks(x)
        return torch.relu(self.output_bn(x))          # shared (B, C, 8, 8)

    def heads(self, x):
        # Value head: (B,C,8,8) → (B,1,8,8) → (B,64) → (B,fc) → (B,1) → (B,)
        v = torch.relu(self.value_bn(self.value_conv(x)))
        v = v.flatten(1)                              # keep batch dim, flatten rest
        v = self.value_fc1(v)

        # Policy head: (B,C,8,8) → (B,2,8,8) → (B,128) → (B, NUM_ACTIONS)
        p = torch.relu(self.policy_bn(self.policy_conv(x)))
        p = p.flatten(1)
        p = self.policy_fc(p)                         # raw logits (no softmax)

        if self.global_pool:
            pooled = torch.cat([x.mean(dim=(2, 3)), x.amax(dim=(2, 3))], dim=1)   # (B, 2C)
            v = v + self.value_gp_fc(pooled)
            p = p + self.policy_gp_fc(pooled)
        v = torch.relu(v)
        v = torch.tanh(self.value_fc2(v)).squeeze(-1) # tanh → (-1,1); drop last dim
        return v, p

    def forward(self, x):
        return self.heads(self.trunk(x))


def net_from_state_dict(sd, cls=None, strict=True):
    """
    A Net (or a subclass `cls`) sized from a state_dict: channels, blocks,
    head sizes and global pooling are read off the tensors. Returns the
    network and load_state_dict's (missing, unexpected) keys.
    """
    C = sd["input_conv.0.weight"].shape[0]
    B = sum(1 for k in sd if k.endswith(".conv1.weight"))
    net = (cls or Net)(channels=C, nb_blocks=B, value_fc_size=sd["value_fc1.weight"].shape[0],
                       num_actions=sd["policy_fc.weight"].shape[0],
                       global_pool="value_gp_fc.weight" in sd)
    return net, net.load_state_dict(sd, strict=strict)

    def clip(self):
        # No-op: kept so the training loop can call model.clip() uniformly
        # across networks (the NNUE variants clamp weights here for int8
        # quantization; this CNN does not need it).
        pass


# ── Training configuration ──────────────────────────────────────────────
# Override at runtime with YOLAH_NB_EPOCHS — lets you change epoch count
# without rebuilding the .sif (the .sh's `singularity exec --env ...` passes
# it through). Default keeps the original value.
NB_EPOCHS  = int(os.environ.get("YOLAH_NB_EPOCHS", "20"))
MODEL_PATH = "/mnt/"                          # checkpoint dir (bind-mounted)
MODEL_NAME = "cnn_resnet_256x30_value_policy"
LAST_MODEL = f"{MODEL_PATH}{MODEL_NAME}.pt" # if present, resume from it
# Cache directory inside the container (the .sh bind-mounts the host cache
# onto /cache); overridable via env var for local runs.
CACHE_DIR  = os.environ.get("YOLAH_CACHE_DIR", "/cache")

```

```
# Total loss = VALUE_LOSS_WEIGHT * MSE(value) + POLICY_LOSS_WEIGHT * CE(policy).
# AlphaZero weights both at 1.0; raise the value weight if value learning lags.
VALUE_LOSS_WEIGHT  = 1.0
POLICY_LOSS_WEIGHT = 1.0


def ddp_setup(rank, world_size):
    """
    Initialise the process group for DistributedDataParallel.

    All ranks live on one node here, so they rendezvous over localhost. NCCL is
    the GPU-to-GPU communication backend used for the gradient all-reduce.
    """
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "65437"
    init_process_group(backend="nccl", rank=rank, world_size=world_size)


class TrainerDDP:
    """Owns one GPU's model replica, optimizer, and the train/validate loops."""

    def __init__(self, gpu_id, model, train_loader, val_loader, save_every=1):
        self.gpu_id = gpu_id

        # Move the model to this rank's GPU and convert it to channels-last
        # (NHWC) memory format: cuDNN's NHWC convolution kernels are faster on
        # tensor-core GPUs, and it makes the gradient layout match what DDP's
        # buckets expect (silences the "grad strides" warning).
        self.model = model.to(gpu_id).to(memory_format=torch.channels_last)

        self.train_loader = train_loader
        self.val_loader   = val_loader
        self.save_every   = save_every

        # Adam with a little weight decay (L2 regularisation).
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=0.001,
                                          weight_decay=1e-4)
        # Value head: regression → mean-squared error against z ∈ {-1, 0, +1}.
        self.value_loss_fn = nn.MSELoss()
        # Policy head: 4096-way classification; ignore_index skips terminal
        # positions whose policy target is POLICY_IGNORE_INDEX.
        self.policy_loss_fn = nn.CrossEntropyLoss(ignore_index=POLICY_IGNORE_INDEX)
        # Cosine schedule: lr eases from 1e-3 down to 1e-5 over the run.
        self.scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
            self.optimizer, T_max=NB_EPOCHS, eta_min=1e-5)

        # Mixed precision. bf16 has wide dynamic range and needs no loss
        # scaling, but only Ampere+ (compute capability major >= 8) has bf16
        # tensor cores. Turing GPUs (e.g. Quadro RTX 8000, cc 7.5) would run
        # bf16 emulated and SLOW — they do, however, have fp16 tensor cores.
        # So: gate strictly on the capability major version.
        major, _ = torch.cuda.get_device_capability(gpu_id)
        self.amp_dtype = torch.bfloat16 if major >= 8 else torch.float16
        # fp16 gradients can underflow → need a GradScaler. bf16 does not, so
        # the scaler is disabled there and its scale/step/update become no-ops.
        self.scaler = torch.amp.GradScaler(
            'cuda', enabled=(self.amp_dtype == torch.float16))

        torch.cuda.set_device(gpu_id)
        torch.cuda.empty_cache()
        # Wrap in DDP: every backward pass all-reduces gradients across ranks.
        # gradient_as_bucket_view: gradients live directly in the all-reduce
        # bucket memory → one less memcpy + one less stream sync per bucket.
```

```
        # static_graph: tells DDP the autograd graph is identical every step
        # (true for us — no conditionals, no unused params, no model surgery)
        # so the reducer is built once and DDP can fuse/overlap aggressively.
        # Both are aimed at the periodic ~50 s all-reduce stall observed on
        # multi-GPU runs that vanishes on single-GPU.
        self.model = DDP(self.model, device_ids=[gpu_id],
                         gradient_as_bucket_view=True, static_graph=True)
        # torch.compile JIT-fuses the graph for extra speed (first iterations
        # are slow while it compiles, then much faster).
        self.model = torch.compile(self.model)
        # A side CUDA stream used to prefetch the next batch's H2D copy.
        self.stream = torch.cuda.Stream(device=gpu_id)

    def _save_checkpoint(self, epoch):
        """Save the raw model weights (unwrap DDP via .module)."""
        torch.save(self.model.module.state_dict(),
                   f"{MODEL_PATH}{MODEL_NAME}.{epoch}.pt")

    def _compute_loss(self, value_pred, policy_logits, value_target, policy_target):
        """Combined value + policy loss; also returns the two parts for logging."""
        v_loss = self.value_loss_fn(value_pred, value_target)
        p_loss = self.policy_loss_fn(policy_logits, policy_target)
        return (VALUE_LOSS_WEIGHT  * v_loss
              + POLICY_LOSS_WEIGHT * p_loss, v_loss, p_loss)

    @staticmethod
    def _policy_correct(policy_logits, policy_target):
        """
        Count correct top-1 policy predictions, ignoring terminal positions.
        Returns (n_correct, n_scored) so the caller can accumulate an accuracy.
        """
        mask = policy_target != POLICY_IGNORE_INDEX     # valid (non-terminal) rows
        if mask.sum().item() == 0:
            return 0, 0
        preds = torch.argmax(policy_logits, dim=1)
        correct = ((preds == policy_target) & mask).sum().item()
        return correct, int(mask.sum().item())

    def _h2d_async(self, batch_cpu):
        """
        Issue an asynchronous host->GPU transfer for one (X, v, p) batch on
        self.stream, return the GPU tensors.

        PREFETCH: callers `wait_stream` on the default stream *before* using
        these tensors. Because the H2D runs on the side stream, it can proceed
        concurrently with the previous iteration's compute on the default
        stream — that is the whole point of the prefetch.
        """
        X_cpu, v_cpu, p_cpu = batch_cpu
        with torch.cuda.stream(self.stream):
            X = X_cpu.to(self.gpu_id, non_blocking=True)
            v = v_cpu.to(self.gpu_id, non_blocking=True)
            p = p_cpu.to(self.gpu_id, non_blocking=True)
        return (X, v, p)

    def _run_epoch(self, epoch):
        """One training pass over this rank's shard of the data.

        PREFETCH: while iteration N is computing on the default stream, the
        H2D transfer for iteration N+1 is in flight on self.stream. The body's
        wall-time therefore measures only compute, not transfer, and the
        transfer is hidden behind compute instead of adding to the iteration.
        """
```

```
        n = 0                                   # positions seen
        # Three value-accuracy variants tracked side-by-side:
        #   v_correct        — sign(v) == sign(y); draws always count as wrong
        #                      (ceiling = 1 - P(draw)).
        #   v_bucket_correct — bucket(v) ∈ {-1, 0, +1} via |v| < 0.33; fair for draws.
        #   v_signed_correct / v_signed_n — sign-acc over non-draw rows only.
        v_correct        = 0
        v_bucket_correct = 0
        v_signed_correct = 0
        v_signed_n       = 0
        p_correct = 0                           # policy top-1 hits
        p_total   = 0                           # non-terminal positions
        v_running, p_running = 0.0, 0.0         # running loss sums
        it = 0                                  # iteration counter (for periodic gc)

        # Pre-load the first batch onto the GPU (side stream). We keep the
        # CPU batch reference (cpu_cur) alive across the iteration so its
        # pinned tensors are not freed before the async H2D completes.
        loader_iter = iter(self.train_loader)
        cpu_cur = next(loader_iter, None)
        if cpu_cur is None:
            return
        gpu_cur = self._h2d_async(cpu_cur)

        pbar = tqdm(total=len(self.train_loader))
        while True:
            # Make the default stream wait until the current batch's H2D is
            # complete on self.stream. This is just a stream dependency — the
            # host does not block here.
            torch.cuda.current_stream(self.gpu_id).wait_stream(self.stream)
            X, v_target, p_target = gpu_cur
            bs = len(X)
            n += bs

            # Issue the NEXT iteration's H2D RIGHT NOW on self.stream. It will
            # run concurrently with this iteration's forward/backward/step on
            # the default stream — that overlap is the whole point.
            cpu_next = next(loader_iter, None)
            if cpu_next is not None:
                gpu_next = self._h2d_async(cpu_next)

            # Match the model's channels-last memory format (sub-ms reformat).
            X = X.contiguous(memory_format=torch.channels_last)

            self.optimizer.zero_grad()
            with torch.autocast('cuda', dtype=self.amp_dtype):
                v_pred, p_logits = self.model(X)
                loss, v_loss, p_loss = self._compute_loss(v_pred, p_logits,
                                                          v_target, p_target)
            self.scaler.scale(loss).backward()
            self.scaler.step(self.optimizer)
            self.scaler.update()

            # Accumulate metrics (.item() pulls scalars back to the CPU).
            v_running        += v_loss.item() * bs
            p_running        += p_loss.item() * bs
            v_correct        += (torch.sign(v_pred) == torch.sign(v_target)).sum().item()
            # 3-way bucketed accuracy: bucket v_pred to {-1, 0, +1} via |v|<0.33.
            v_bucket = torch.where(v_pred >  0.33,  1.0,
                       torch.where(v_pred < -0.33, -1.0, 0.0))
            v_bucket_correct += (v_bucket == v_target).sum().item()
            # Sign-acc on non-draws only (mask out v_target == 0).
            v_mask = v_target != 0
```

```
            if v_mask.any():
                v_signed_correct += (torch.sign(v_pred[v_mask]) == v_target[v_mask]).sum().item()
                v_signed_n       += int(v_mask.sum().item())
            pc, pt = self._policy_correct(p_logits, p_target)
            p_correct += pc
            p_total   += pt

            self.model.module.clip()            # no-op for this CNN

            # Manual GC every 1000 iters — auto-GC is disabled in main(), so
            # we mop up reference cycles in one controlled pause instead of
            # whatever the auto-collector would decide.
            it += 1
            if it % 1000 == 0:
                gc.collect()

            pbar.update(1)

            # Advance to the next batch, or stop if the loader is exhausted.
            # Re-binding here drops the previous iter's CPU tensors — by now
            # their H2D has long completed (we waited for it at the top of
            # this iter), so it is safe to free them.
            if cpu_next is None:
                break
            cpu_cur = cpu_next
            gpu_cur = gpu_next

        pbar.close()
        self.scheduler.step()                   # advance the lr schedule

        # Only rank 0 logs, so the output is not printed three times.
        if self.gpu_id == 0:
            lr = self.optimizer.param_groups[0]['lr']
            p_acc        = (p_correct / p_total)       if p_total    > 0 else 0.0
            v_signed_acc = (v_signed_correct / v_signed_n) if v_signed_n > 0 else 0.0
            print(f'epoch {epoch+1} '
                  f'train value_mse: {v_running/n:.4f} '
                  f'value_sign_acc: {v_correct/n:.4f} '
                  f'value_bucket_acc: {v_bucket_correct/n:.4f} '
                  f'value_signed_acc: {v_signed_acc:.4f} '
                  f'policy_ce: {p_running/n:.4f} '
                  f'policy_acc: {p_acc:.4f} '
                  f'lr: {lr:.6f}', flush=True)

    def _validate(self, epoch):
        """One pass over the validation shard — no gradients, no optimizer.
        Uses the same prefetch pattern as _run_epoch.
        """
        self.model.train(False)                 # eval mode (freezes BatchNorm)
        n = 0
        v_correct        = 0
        v_bucket_correct = 0
        v_signed_correct = 0
        v_signed_n       = 0
        p_correct = 0
        p_total   = 0
        v_running, p_running = 0.0, 0.0
        with torch.no_grad():                   # no autograd graph → less memory
            loader_iter = iter(self.val_loader)
            cpu_cur = next(loader_iter, None)
            if cpu_cur is None:
                self.model.train()
                return
```

```
            gpu_cur = self._h2d_async(cpu_cur)
            pbar = tqdm(total=len(self.val_loader))
            while True:
                torch.cuda.current_stream(self.gpu_id).wait_stream(self.stream)
                X, v_target, p_target = gpu_cur
                bs = len(X)
                n += bs

                cpu_next = next(loader_iter, None)
                if cpu_next is not None:
                    gpu_next = self._h2d_async(cpu_next)

                X = X.contiguous(memory_format=torch.channels_last)
                with torch.autocast('cuda', dtype=self.amp_dtype):
                    v_pred, p_logits = self.model(X)
                    _, v_loss, p_loss = self._compute_loss(v_pred, p_logits,
                                                           v_target, p_target)
                v_running        += v_loss.item() * bs
                p_running        += p_loss.item() * bs
                v_correct        += (torch.sign(v_pred) == torch.sign(v_target)).sum().item()
                v_bucket = torch.where(v_pred >  0.33,  1.0,
                           torch.where(v_pred < -0.33, -1.0, 0.0))
                v_bucket_correct += (v_bucket == v_target).sum().item()
                v_mask = v_target != 0
                if v_mask.any():
                    v_signed_correct += (torch.sign(v_pred[v_mask]) ==
↪ v_target[v_mask]).sum().item()
                    v_signed_n       += int(v_mask.sum().item())
                pc, pt = self._policy_correct(p_logits, p_target)
                p_correct += pc
                p_total   += pt

                pbar.update(1)

                if cpu_next is None:
                    break
                cpu_cur = cpu_next
                gpu_cur = gpu_next
            pbar.close()
        if self.gpu_id == 0:
            p_acc        = (p_correct / p_total)       if p_total    > 0 else 0.0
            v_signed_acc = (v_signed_correct / v_signed_n) if v_signed_n > 0 else 0.0
            print(f'epoch {epoch+1} '
                  f'val value_mse: {v_running/n:.4f} '
                  f'value_sign_acc: {v_correct/n:.4f} '
                  f'value_bucket_acc: {v_bucket_correct/n:.4f} '
                  f'value_signed_acc: {v_signed_acc:.4f} '
                  f'policy_ce: {p_running/n:.4f} '
                  f'policy_acc: {p_acc:.4f}', flush=True)
        self.model.train()                      # back to train mode

    def train(self, nb_epochs):
        """Top-level loop: for each epoch, train, validate, checkpoint."""
        self.model.train()
        for epoch in range(nb_epochs):
            # Reseed the loader's shuffling so this epoch differs from the last.
            self.train_loader.set_epoch(epoch)
            self._run_epoch(epoch)
            self._validate(epoch)
            if self.gpu_id == 0 and epoch % self.save_every == 0:
                self._save_checkpoint(epoch)
        self._save_checkpoint(nb_epochs - 1)    # always save the final model

```

```python

def main(rank, world_size, batch_size, cache_dir):
    """
    Per-process entry point — mp.spawn runs one copy of this on each GPU,
    passing the GPU's index as `rank`.
    """
    ddp_setup(rank, world_size)

    # Read the dataset size from meta.json and take a contiguous 95/5 split.
    # The split is contiguous (not random) because positions are stored
    # game-by-game, so a contiguous cut keeps whole games on one side — less
    # train/val leakage than a per-position random split.
    #
    # The loaders are built HERE, inside the spawned process — never passed in
    # through mp.spawn. np.memmap is a lazy mmap (virtual address space only),
    # so constructing a loader is cheap and the OS page cache is shared.
    with open(os.path.join(cache_dir, "meta.json")) as f:
        n_total = int(json.load(f)["n_positions"])
    n_train = int(0.95 * n_total)

    train_loader = ChunkedShuffleLoader(cache_dir, 0, n_train, batch_size,
                                        rank, world_size, shuffle=True)
    val_loader   = ChunkedShuffleLoader(cache_dir, n_train, n_total, batch_size,
                                        rank, world_size, shuffle=False)

    if rank == 0:
        print(f'Dataset: {n_total:,} positions -> '
              f'train {n_train:,}  val {n_total - n_train:,}', flush=True)
        print(f'Batches/rank/epoch: train {len(train_loader):,}  '
              f'val {len(val_loader):,}  (chunk_size={CHUNK_SIZE:,})', flush=True)

    net = Net()
    # Resume from a previous checkpoint if one exists.
    if os.path.isfile(LAST_MODEL):
        net.load_state_dict(torch.load(LAST_MODEL, map_location='cpu'))
    if rank == 0:
        nb_params = sum(p.numel() for p in net.parameters())
        print(net, flush=True)
        print(f'Parameters: {nb_params:,}', flush=True)

    # Disable Python's cyclic GC for the duration of training; reference
    # counting still frees everything that is not in a cycle, and we mop up
    # cycles ourselves with gc.collect() every 1000 iters in _run_epoch.
    # This trades unpredictable Gen-2 pauses for one controlled pause.
    gc.disable()

    trainer = TrainerDDP(rank, net, train_loader, val_loader)
    trainer.train(NB_EPOCHS)
    destroy_process_group()                    # clean up the NCCL group


if __name__ == "__main__":
    print(torch.cuda.is_available())
    # One training process per ALLOCATED GPU. world_size honours
    # CUDA_VISIBLE_DEVICES (the SLURM convention) when set, falling back to
    # device_count() only for local dev. This avoids the cluster pitfall
    # where Singularity --nv exposes all physical GPUs and mp.spawn over
    # device_count() then races for one the job does not own → NCCL hang.
    cvd = os.environ.get("CUDA_VISIBLE_DEVICES", "").strip()
    world_size = (len([x for x in cvd.split(",") if x.strip()])
                  if cvd else torch.cuda.device_count())
    print(world_size, flush=True)
    # mp.spawn calls main(rank, world_size, 1024, CACHE_DIR) on each GPU; it
```

```
    # prepends `rank` itself. batch_size is PER GPU → effective batch is
    # 1024 * world_size.
    mp.spawn(main, args=(world_size, 1024, CACHE_DIR), nprocs=world_size)
```

12.3.2 `nnue/resnet_export.py`

238 lines.

One checkpoint in, three artefacts out. The module docstring is the authoritative `.bin` layout; Section 4.3 is the same thing with the algebra spelled out.

The asymmetry to remember: a BatchNorm that *follows* a convolution folds into its weights and bias; a BatchNorm that *precedes* one cannot, so its (s, t) pair is written out for the C++ side to apply.

symbol	role
`bn_affine`	eval-mode BatchNorm $\to (s, t)$ with $s = \gamma/\sqrt{\sigma^2 + \epsilon}$, $t = \beta - \mu s$
`conv_to_out_tap_in`	$(out, in, 3, 3) \to (out, 9, in)$, the layout the CPU GEMM wants
`Writer`	little-endian float32 sink, counting what it wrote
`export_bin`	header, stem, blocks, output BN, both heads — the order `ResNetCPU::load` reads
`export_torchscript`	trace in eval mode on a dummy batch and save
`export_test_vectors`	random positions with PyTorch's exact fp32 outputs, for the C++ self-check
`main`	tolerates a DDP-wrapped checkpoint, infers C, B, F, A from the tensors

```
"""
resnet_export.py — export the trained two-headed ResNet (cnn_resnet_256x30_
value_policy.pt) for C++ inference by the AlphaZero-style MCTS player.

Three artefacts are produced next to the checkpoint:

  1. <name>.bin  — a flat little-endian float32 dump with every BatchNorm
                   folded into the adjacent conv / affine op. This is what the
                   pure-C++ CPU backend (nnue/resnet.h) loads. No PyTorch needed
                   at run time.
  2. <name>.ts   — a TorchScript module (traced in eval mode) for the optional
                   libtorch backend (nnue/resnet_torch.h, built with
                   -DENABLE_TORCH=ON). This one runs on the GPU.
  3. <name>.test.bin — a handful of random positions with the exact PyTorch
                   outputs (value + 4096 policy logits) so the C++ backends can
                   self-check bit-for-bit-ish (see test/alphazero_mcts_check.cpp).

Usage (from the nnue/ directory, the module imports ../server/yolah.py):

    python3 resnet_export.py [checkpoint.pt] [--no-ts] [--nb-test 32]

Binary layout of <name>.bin (all little-endian; float32 unless stated)
-----------------------------------------------------------------------
    char[8]  magic        = "YOLAHRSN"
    uint32   version      = 1
```

```
    uint32   in_channels  = 4
    uint32   channels     = C   (256)
    uint32   nb_blocks    = B   (30)
    uint32   value_fc     = F   (256)
    uint32   num_actions  = A   (4096)
    ── trunk ──
    stem_w   [C][9][4]      conv weight, BN folded, layout (out, tap, in) with
                            tap = dr*3 + dc, dr/dc ∈ {0,1,2} (kernel row/col)
    stem_b   [C]            folded bias
    for each block b in 0..B-1:
        bn1_scale [C], bn1_shift [C]      pre-activation affine (y = s*x + t)
        conv1_w   [C][9][C]               (out, tap, in), no bias
        bn2_scale [C], bn2_shift [C]
        conv2_w   [C][9][C]
    out_bn_scale [C], out_bn_shift [C]
    ── value head ──
    value_conv_w [C]        1x1 conv (C→1) with value_bn folded
    value_conv_b [1]
    value_fc1_w  [F][64], value_fc1_b [F]
    value_fc2_w  [1][F],  value_fc2_b [1]
    ── policy head ──
    policy_conv_w [2][C]    1x1 conv (C→2) with policy_bn folded
    policy_conv_b [2]
    policy_fc_w   [A][128], policy_fc_b [A]

Why fold BatchNorm: in eval mode BN is a per-channel affine map
y = γ(x-μ)/√(σ²+ε) + β = s·x + t. Where BN directly follows a conv (stem,
value_conv, policy_conv) it folds into the conv weights (W·s) and a bias (t).
Where BN precedes a ReLU+conv (the pre-activation residual blocks and the
trunk's output_bn) it cannot be merged into the conv, so we keep (s, t) and the
C++ code applies max(0, s·x+t) while it gathers the im2col patches — no extra
pass over the activations.
"""
import os
import sys
import struct
import argparse
import numpy as np
import torch

sys.path.append("../server")
from yolah import Yolah, Move, Square                    # noqa: E402
from cnn_resnet_value_policy_chunked import Net, encode_cnn, net_from_state_dict  # noqa: E402

MAGIC   = b"YOLAHRSN"
VERSION = 1
BN_EPS  = 1e-5   # nn.BatchNorm2d default


def bn_affine(sd, prefix):
    """Return (scale, shift) float32 arrays for the eval-mode BatchNorm `prefix`."""
    gamma = sd[f"{prefix}.weight"].double()
    beta  = sd[f"{prefix}.bias"].double()
    mean  = sd[f"{prefix}.running_mean"].double()
    var   = sd[f"{prefix}.running_var"].double()
    scale = gamma / torch.sqrt(var + BN_EPS)
    shift = beta - mean * scale
    return scale.float().numpy(), shift.float().numpy()


def conv_to_out_tap_in(w):
    """(out, in, 3, 3) torch conv weight → (out, 9, in) float32 numpy, tap = dr*3+dc."""
    w = w.detach().double().numpy()
```

```
    out, inn, kh, kw = w.shape
    assert kh == 3 and kw == 3
    return np.ascontiguousarray(w.transpose(0, 2, 3, 1).reshape(out, 9, inn)).astype(np.float32)


class Writer:
    def __init__(self, path):
        self.f = open(path, "wb")
        self.nb_floats = 0

    def u32(self, v):
        self.f.write(struct.pack("<I", v))

    def floats(self, arr):
        arr = np.ascontiguousarray(arr, dtype="<f4")
        self.f.write(arr.tobytes())
        self.nb_floats += arr.size

    def close(self):
        self.f.close()


def export_bin(net, sd, path):
    C = net.input_conv[0].weight.shape[0]
    B = len(net.res_blocks)
    F = net.value_fc1.weight.shape[0]
    A = net.policy_fc.weight.shape[0]

    w = Writer(path)
    w.f.write(MAGIC)
    for v in (VERSION, 4, C, B, F, A):
        w.u32(v)

    # ── stem: conv(4→C, no bias) followed by BN → fold BN into the conv ──
    s, t = bn_affine(sd, "input_conv.1")
    stem = conv_to_out_tap_in(sd["input_conv.0.weight"]) * s[:, None, None]
    w.floats(stem)
    w.floats(t)

    # ── residual blocks (pre-activation: BN→ReLU→conv, twice) ──
    for b in range(B):
        p = f"res_blocks.{b}"
        s1, t1 = bn_affine(sd, f"{p}.bn1")
        w.floats(s1); w.floats(t1)
        w.floats(conv_to_out_tap_in(sd[f"{p}.conv1.weight"]))
        s2, t2 = bn_affine(sd, f"{p}.bn2")
        w.floats(s2); w.floats(t2)
        w.floats(conv_to_out_tap_in(sd[f"{p}.conv2.weight"]))

    # ── trunk output BN (followed by ReLU in forward) ──
    so, to = bn_affine(sd, "output_bn")
    w.floats(so); w.floats(to)

    # ── value head: 1x1 conv(C→1, no bias) + BN(1) folded ──
    sv, tv = bn_affine(sd, "value_bn")
    vconv = sd["value_conv.weight"].double().numpy().reshape(1, C) * sv[:, None]
    w.floats(vconv.astype(np.float32))
    w.floats(tv)
    w.floats(sd["value_fc1.weight"].numpy()); w.floats(sd["value_fc1.bias"].numpy())
    w.floats(sd["value_fc2.weight"].numpy()); w.floats(sd["value_fc2.bias"].numpy())

    # ── policy head: 1x1 conv(C→2, no bias) + BN(2) folded ──
    sp, tp = bn_affine(sd, "policy_bn")
```

```
    pconv = sd["policy_conv.weight"].double().numpy().reshape(2, C) * sp[:, None]
    w.floats(pconv.astype(np.float32))
    w.floats(tp)
    w.floats(sd["policy_fc.weight"].numpy()); w.floats(sd["policy_fc.bias"].numpy())

    w.close()
    print(f"wrote {path}: C={C} blocks={B} value_fc={F} actions={A}, "
          f"{w.nb_floats:,} floats ({os.path.getsize(path) / 1e6:.1f} MB)")


def export_torchscript(net, path):
    """Trace the eval-mode network on a dummy batch and save it for libtorch."""
    example = torch.zeros(1, 4, 8, 8)
    with torch.no_grad():
        traced = torch.jit.trace(net, example)
    traced.save(path)
    print(f"wrote {path} ({os.path.getsize(path) / 1e6:.1f} MB)")


def random_position(rng, max_plies=60):
    """Play a random game for a random number of plies; return the Yolah state."""
    y = Yolah()
    n = int(rng.integers(0, max_plies))
    for _ in range(n):
        if y.game_over():
            break
        moves = y.moves()
        y.play(moves[int(rng.integers(0, len(moves)))])
    return y


def export_test_vectors(net, path, nb, seed=1234):
    """
    Record `nb` random (non-terminal preferred) positions with PyTorch's
    fp32 outputs. Record layout, repeated nb times after a uint32 count:
        uint64 black, uint64 white, uint64 empty, uint32 turn(0/1),
        float32 value, float32[4096] policy logits
    """
    rng = np.random.default_rng(seed)
    positions = [Yolah()]                          # always include the initial position
    while len(positions) < nb:
        y = random_position(rng)
        if not y.game_over():
            positions.append(y)
    X = torch.stack([encode_cnn(y) for y in positions])
    with torch.no_grad():
        v, p = net(X)
    v = v.numpy().astype("<f4"); p = p.numpy().astype("<f4")
    with open(path, "wb") as f:
        f.write(struct.pack("<I", nb))
        for i, y in enumerate(positions):
            f.write(struct.pack("<QQQI", y.black, y.white, y.empty, y.nb_plies() & 1))
            f.write(struct.pack("<f", float(v[i])))
            f.write(p[i].tobytes())
    print(f"wrote {path}: {nb} positions "
          f"(value range [{v.min():+.3f}, {v.max():+.3f}])")


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("checkpoint", nargs="?", default="cnn_resnet_256x30_value_policy.pt")
    ap.add_argument("--no-ts", action="store_true", help="skip the TorchScript export")
```

```python
    ap.add_argument("--nb-test", type=int, default=32, help="number of test vectors")
    args = ap.parse_args()

    sd = torch.load(args.checkpoint, map_location="cpu")
    # Tolerate a DDP-wrapped checkpoint ("module." prefix).
    sd = {(k[7:] if k.startswith("module.") else k): v for k, v in sd.items()}
    net, _ = net_from_state_dict(sd)
    net.eval()

    base = os.path.splitext(args.checkpoint)[0]
    if net.global_pool:
        # The CPU backend (nn::ResNetCPU) knows only the heads without
        # global pooling: such a network runs on the GPU backend (.ts) only.
        print("global pooling: no .bin (CPU backend), TorchScript only")
    else:
        export_bin(net, sd, base + ".bin")
    if not args.no_ts:
        export_torchscript(net, base + ".ts")
    if not net.global_pool:
        export_test_vectors(net, base + ".test.bin", args.nb_test)


if __name__ == "__main__":
    main()
```

12.4 The self-checks

`alphazero_check` is the regression suite: it verifies the C++ backends against PyTorch's own outputs, benchmarks them, and asserts the search's invariants.

12.4.1 `test/resnet_check.cpp`

102 lines.

Replays `*.test.bin` through a backend and compares. The tolerance is 2e-3 for the fp32 paths and 2e-2 for GPU fp16; the policy argmax is compared separately, because a logit difference that never changes the argmax is harmless to the search.

symbol	role
`resnet_check`	max
`resnet_bench`	positions per second at a given batch size

```cpp
#include "resnet_check.h"
#include "misc.h"
#include <chrono>
#include <cmath>
#include <cstdio>
#include <fstream>
#include <iostream>
#include <vector>

namespace test {

namespace {
    struct Vector {
        Yolah state;
```

```
        float value;
        std::vector<float> logits;   // 4096
    };

    std::vector<Vector> read_vectors(const std::string& filename) {
        std::ifstream ifs(filename, std::ios::binary);
        if (!ifs) throw std::runtime_error("cannot open " + filename);
        uint32_t n;
        ifs.read(reinterpret_cast<char*>(&n), 4);
        std::vector<Vector> res(n);
        for (Vector& v : res) {
            uint64_t black, white, empty;
            uint32_t turn;
            ifs.read(reinterpret_cast<char*>(&black), 8);
            ifs.read(reinterpret_cast<char*>(&white), 8);
            ifs.read(reinterpret_cast<char*>(&empty), 8);
            ifs.read(reinterpret_cast<char*>(&turn), 4);
            ifs.read(reinterpret_cast<char*>(&v.value), 4);
            v.logits.resize(nn::NUM_ACTIONS);
            ifs.read(reinterpret_cast<char*>(v.logits.data()), nn::NUM_ACTIONS * 4);
            // Scores are irrelevant to the network; ply parity encodes the turn.
            v.state.set_state(black, white, empty, 0, 0, static_cast<uint16_t>(turn));
        }
        if (!ifs) throw std::runtime_error("truncated " + filename);
        return res;
    }
}

bool resnet_check(nn::Evaluator& evaluator, const std::string& test_vectors_filename, float
↪  tolerance) {
    const auto vectors = read_vectors(test_vectors_filename);
    std::vector<Yolah::MoveList> moves(vectors.size());
    std::vector<nn::Request> requests(vectors.size());
    std::vector<nn::Result> results(vectors.size());
    for (size_t i = 0; i < vectors.size(); i++) {
        vectors[i].state.moves(moves[i]);
        requests[i] = { &vectors[i].state, moves[i].begin(),
↪  static_cast<uint16_t>(moves[i].size()) };
    }
    evaluator.evaluate(requests, results);
    float max_value_err = 0, max_logit_err = 0;
    size_t argmax_mismatches = 0;
    for (size_t i = 0; i < vectors.size(); i++) {
        max_value_err = std::max(max_value_err, std::fabs(results[i].value - vectors[i].value));
        size_t best_ref = 0, best_out = 0;
        for (size_t m = 0; m < moves[i].size(); m++) {
            const float ref = vectors[i].logits[nn::action_index(moves[i][m])];
            max_logit_err = std::max(max_logit_err, std::fabs(results[i].logits[m] - ref));
            if (ref > vectors[i].logits[nn::action_index(moves[i][best_ref])]) best_ref = m;
            if (results[i].logits[m] > results[i].logits[best_out]) best_out = m;
        }
        argmax_mismatches += best_ref != best_out;
    }
    std::cout << "[resnet check] " << evaluator.info() << "\n"
              << "  positions           : " << vectors.size() << "\n"
              << "  max |value error|   : " << max_value_err << "\n"
              << "  max |logit error|   : " << max_logit_err << "\n"
              << "  policy argmax diffs : " << argmax_mismatches << "\n";
    const bool ok = max_value_err <= tolerance && max_logit_err <= tolerance * 10;
    std::cout << "  " << (ok ? "OK" : "MISMATCH") << std::endl;
    return ok;
}

```

```cpp
double resnet_bench(nn::Evaluator& evaluator, size_t batch_size, size_t nb_batches) {
    PRNG prng(42);
    std::vector<Yolah> states(batch_size);
    std::vector<Yolah::MoveList> moves(batch_size);
    std::vector<nn::Request> requests(batch_size);
    std::vector<nn::Result> results(batch_size);
    for (size_t i = 0; i < batch_size; i++) {
        // random non-terminal position
        const size_t plies = reduce(prng.rand<uint32_t>(), 50);
        for (size_t k = 0; k < plies && !states[i].game_over(); k++) {
            states[i].moves(moves[i]);
            states[i].play(moves[i][reduce(prng.rand<uint32_t>(), moves[i].size())]);
        }
        states[i].moves(moves[i]);
        requests[i] = { &states[i], moves[i].begin(), static_cast<uint16_t>(moves[i].size()) };
    }
    evaluator.evaluate(requests, results);   // warm-up
    const auto start = std::chrono::steady_clock::now();
    for (size_t k = 0; k < nb_batches; k++) evaluator.evaluate(requests, results);
    const double secs = std::chrono::duration<double>(std::chrono::steady_clock::now() -
    ↪  start).count();
    const double pps = static_cast<double>(batch_size * nb_batches) / secs;
    std::cout << "[resnet bench] batch " << batch_size << ": " << pps << " positions/s ("
              << secs / nb_batches * 1e3 << " ms per batch)" << std::endl;
    return pps;
}

} // namespace test
```

12.4.2 `test/alphazero_mcts_check.cpp`

163 lines.

The search's invariants. `check_result` is the interesting part: it asserts that the child visits sum to the root visits, that π sums to 1, that Q and the priors are in range, that the budget was honoured exactly — unless this was a playout-cap fast search — and that the move played is the first child in the reported (play-value) order.

`alphazero_reuse_check` deserves its shape. Asserting that a *random* opponent reply lands on a subtree the search explored is a coin flip — with ~100 legal replies and a few hundred simulations it usually does not — so that walk only reports, and the two cases that must always reuse are checked explicitly: searching the same position twice must re-root on the root itself, and playing our own move must re-root on a child.

symbol	role
`random_position`	a random legal position, for the search check
`check_result`	the invariants above
`alphazero_search_check`	four positions, fixed simulation budget, no tree reuse
`alphazero_reuse_check`	the random walk (reported) plus the root and child reuse cases (asserted)
`alphazero_vs`	a head-to-head match against another player config, colours alternating

```cpp
#include "alphazero_mcts_check.h"
#include "alphazero_mcts_player.h"
#include "misc.h"
#include <cmath>
#include <iostream>
#include <memory>

using std::cout;

namespace test {

namespace {
    Yolah random_position(PRNG& prng, size_t max_plies) {
        Yolah y;
        Yolah::MoveList moves;
        const size_t n = reduce(prng.rand<uint32_t>(), static_cast<uint32_t>(max_plies));
        for (size_t i = 0; i < n && !y.game_over(); i++) {
            y.moves(moves);
            y.play(moves[reduce(prng.rand<uint32_t>(), moves.size())]);
        }
        return y;
    }

    bool check_result(const az::SearchResult& r, uint64_t nb_simulations, bool reused) {
        bool ok = true;
        auto fail = [&](const std::string& what) { cout << "  FAIL: " << what << "\n"; ok = false;
        ↪  };
        if (r.best_move == Move::none() && r.children.size() > 1) fail("no move chosen");
        uint64_t sum = 0;
        float pi = 0.0f;
        for (const az::ChildStat& c : r.children) {
            sum += c.visits;
            if (c.q < -1.0001f || c.q > 1.0001f) fail("Q out of range");
            if (c.prior < 0.0f || c.prior > 1.0001f) fail("prior out of range");
        }
        for (float p : r.policy) pi += p;
        if (!r.policy.empty() && std::fabs(pi - 1.0f) > 1e-3f) fail("policy does not sum to 1");
        if (!reused && sum != r.root_visits) fail("children visits != root visits");
        // A playout-cap "fast" search deliberately runs a smaller budget.
        if (r.full_search) {
            if (!reused && r.nb_simulations != nb_simulations) fail("simulation budget not
            ↪  honoured exactly");
            if (r.nb_simulations < nb_simulations) fail("fewer simulations than requested");
        } else if (r.nb_simulations == 0) {
            fail("fast search ran no simulation");
        }
        if (!r.children.empty() && r.children[0].move != r.best_move &&
            r.children[0].visits != r.children[1].visits)
            fail("best move is not the most visited child");
        return ok;
    }
}

bool alphazero_search_check(const json& player_config, uint64_t nb_simulations) {
    json cfg = player_config;
    cfg["microseconds"] = 0;
    cfg["nb simulations"] = nb_simulations;
    cfg["verbose"] = false;
    cfg["reuse tree"] = false;
    cfg["temperature"] = 0.0;
    AlphaZeroMCTSPlayer player(cfg);
    cout << "[search check] " << player.info() << "\n";
    bool ok = true;
```

```
    PRNG prng(7);
    for (int k = 0; k < 4; k++) {
        Yolah y = k == 0 ? Yolah() : random_position(prng, 40);
        if (y.game_over()) continue;
        player.play(y);
        const az::SearchResult& r = player.last_result();
        cout << "position " << k << " (ply " << y.nb_plies() << "):\n" << r.to_string(5);
        ok &= check_result(r, nb_simulations, false);
    }
    cout << "  " << (ok ? "OK" : "FAILED") << std::endl;
    return ok;
}

bool alphazero_reuse_check(const json& player_config, uint64_t nb_simulations) {
    json cfg = player_config;
    cfg["microseconds"] = 0;
    cfg["nb simulations"] = nb_simulations;
    cfg["verbose"] = false;
    cfg["reuse tree"] = true;
    AlphaZeroMCTSPlayer player(cfg);
    cout << "[reuse check] " << player.info() << "\n";
    PRNG prng(11);
    Yolah y;
    Yolah::MoveList moves;
    bool ok = true;
    size_t reused_plies = 0;
    for (int ply = 0; ply < 6 && !y.game_over(); ply++) {
        Move m = player.play(y);
        const az::SearchResult& r = player.last_result();
        ok &= check_result(r, nb_simulations, ply > 0);
        const uint64_t new_nodes = r.tree_nodes;
        cout << "  ply " << y.nb_plies() << ": " << m << "  root visits " << r.root_visits
             << " (this search: " << r.nb_simulations << "), nodes " << new_nodes << "\n";
        if (ply > 0 && r.root_visits > r.nb_simulations) ++reused_plies;
        y.play(m);
        if (y.game_over()) break;
        // random opponent reply
        y.moves(moves);
        y.play(moves[reduce(prng.rand<uint32_t>(), moves.size())]);
    }
    // The opponent replies at random, so whether its move is one the search
    // actually explored — and the subtree therefore reusable — is luck; the
    // count above is informational. The two cases below must always reuse.
    cout << "  reused subtrees on " << reused_plies << " of 5 plies (random replies)\n";

    Yolah z = random_position(prng, 12);
    if (!z.game_over()) {
        player.game_over(z);              // drop the tree
        player.play(z);
        const uint64_t first = player.last_result().root_visits;
        player.play(z);                   // same position again → the root itself
        const az::SearchResult& again = player.last_result();
        ok &= check_result(again, nb_simulations, true);
        if (again.root_visits <= first) { cout << "  FAIL: the root was not reused\n"; ok = false;
        ↪  }
        cout << "  same position twice: " << first << " → " << again.root_visits << " root
↪  visits\n";
        // Same search driving both sides: the next position is a child of the root.
        z.play(again.best_move);
        if (!z.game_over()) {
            player.play(z);
            const az::SearchResult& child = player.last_result();
            ok &= check_result(child, nb_simulations, true);
```

```
            // More visits at the root than this search alone produced: the
            // rest came from the previous one (the budget may differ from
            // nb_simulations when playout cap randomization is on).
            if (child.root_visits <= child.nb_simulations) { cout << "  FAIL: the child was not reused\n"; ok = false; }
            cout << "  after our own move: " << child.root_visits << " root visits\n";
        }
    }
    cout << "  " << (ok ? "OK" : "FAILED") << std::endl;
    return ok;
}

double alphazero_vs(const json& player_config, const json& opponent_config, size_t nb_games, bool verbose) {
    double score = 0.0;
    for (size_t g = 0; g < nb_games; g++) {
        const bool az_black = (g % 2) == 0;
        std::unique_ptr<Player> az = Player::create(player_config);
        std::unique_ptr<Player> opp = Player::create(opponent_config);
        Player* black = az_black ? az.get() : opp.get();
        Player* white = az_black ? opp.get() : az.get();
        Yolah y;
        while (!y.game_over()) {
            Player* p = y.current_player() == Yolah::BLACK ? black : white;
            Move m = p->play(y);
            if (verbose) cout << (y.current_player() == Yolah::BLACK ? "black " : "white ") << m << "\n";
            y.play(m);
        }
        az->game_over(y);
        opp->game_over(y);
        const auto [bs, ws] = y.score();
        const int az_diff = az_black ? int(bs) - int(ws) : int(ws) - int(bs);
        score += az_diff > 0 ? 1.0 : (az_diff == 0 ? 0.5 : 0.0);
        cout << "game " << g + 1 << ": alphazero as " << (az_black ? "black" : "white")
             << ", score " << bs << "/" << ws << " → " << (az_diff > 0 ? "win" : az_diff == 0 ? "draw" : "loss")
             << std::endl;
    }
    cout << "[alphazero vs " << opponent_config.value("name", std::string("?")) << "] "
         << score / nb_games * 100 << " %" << std::endl;
    return score / nb_games;
}

} // namespace test
```

12.5 The learning loop

The self-play learning loop of Section 6: the C++ self-play driver and its command-line front end, the book of balanced openings of the matches, the Python trainer that orchestrates everything, the two search configurations, and the cluster image and job scripts.

12.5.1 `player/alphazero_selfplay.h`

115 lines.

The row format and the self-play entry point. `TrainingSample` is written to disk as is (336 bytes, no padding — the `static_assert`s guard that) and read by numpy through `SAMPLE_DTYPE` in `alphazero_learn.py`: the two must change together.

symbol	role
`TrainingSample`	one training row: the bitboards, the side to move, z, the sparse π, the search value, the network step
`OwnershipCode`	future owner of a square: nobody, mine, the opponent's, or no target (already a hole)
`TrainingSampleAux`	a version-2 row: `TrainingSample` + the 2-bit future ownership map (`--aux`)
`SampleFileHeader`	24-byte file header: magic, version, row size, row and game counts
`ModelRef` / `read_latest_model`	parse `latest.json`, resolving a relative path against the work directory
`SelfPlayOptions`	the driver's knobs: concurrency, flush policy, poll period
`run_selfplay`	play until stopped — Section 6.3

```cpp
#ifndef ALPHAZERO_SELFPLAY_H
#define ALPHAZERO_SELFPLAY_H
// alphazero_selfplay.h — the self-play half of the AlphaZero learning loop.
//
// The loop (driven by nnue/alphazero_learn.py, see player/README_ALPHAZERO_LEARN.md):
//
//   self-play (this file, C++) ──samples──▶ <work>/selfplay/*.bin
//        ▲                                          │
//        │ latest.json                              ▼
//   <work>/models/az_<step>.ts ◀──export── trainer (alphazero_learn.py)
//
// run_selfplay() plays many games at once, one az::Search per game, all the
// searches sharing ONE network behind an nn::BatchingEvaluator: with ~100
// games each asking for a few leaves at a time, the GPU sees large batches
// while every individual search stays close to sequential (few virtual-loss
// collisions). It polls <work>/latest.json and hot-swaps the weights as soon
// as the trainer exports a new network.
//
// Only the "full" searches of playout cap randomization are recorded (KataGo:
// the fast searches exist to make games longer/cheaper, their visit counts are
// too noisy to be a policy target). A sample is the position, the policy
// target π over its legal moves and — once the game is over — its outcome z.
//
// Optional auxiliary targets (KataGo's ownership / score heads, see the doc,
// "KataGo's auxiliary heads"): when the trainer runs with --aux it writes
// "aux": true in latest.json, and the games then record, for every sample,
// its FUTURE OWNERSHIP map — for each square, who will leave it between this
// position and the end of the game (every move leaves a hole on the square
// it comes from and scores one point, so the map sums to the rest of the
// score). Those rows are TrainingSampleAux in files of version 2; without
// the option nothing changes (TrainingSample, version 1).
#include "alphazero_mcts.h"
#include "nn_evaluator.h"
#include <atomic>
#include <cstdint>
#include <string>

namespace az {

// ── On-disk format (little-endian). Mirrored by SAMPLE_DTYPE in
//    nnue/alphazero_learn.py — change both together. ──────────────────────
struct TrainingSample {
    uint64_t black, white, empty;        // bitboards (the network input, see nn::encode_planes)
```

```
    uint8_t  turn;                      // 0 = black to move, 1 = white
    int8_t   z;                         // outcome for the side to move: -1, 0, +1
    uint8_t  nb_moves;                  // entries used in action[] / prob[]
    uint8_t  flags;                     // reserved (0)
    float    root_q;                    // search value of the root, side to move
    uint32_t model_step;                // trainer step of the network that searched
    uint16_t action[Yolah::MAX_NB_MOVES];  // policy index from*64+to (nn::action_index)
    uint16_t prob[Yolah::MAX_NB_MOVES];    // π(action) · 65535, rounded
};
static_assert(sizeof(TrainingSample) == 336, "keep in sync with alphazero_learn.py");

// Future ownership of a square, relative to the side to move at the sample.
enum OwnershipCode : uint8_t {
    OWN_NONE = 0,   // nobody will leave it (stays free, or a piece ends the game there)
    OWN_MINE = 1,   // the side to move will leave it: +1 for the side to move
    OWN_OPP  = 2,   // the opponent will leave it: +1 for the opponent
    OWN_PAST = 3,   // already a hole: who made it is not in the position → no target
};

// A version-2 row: the version-1 row followed by the future ownership map,
// 2 bits per square, square q in byte q/4 at bits 2·(q%4). Mirrored by
// SAMPLE_DTYPE_AUX in nnue/alphazero_learn.py.
struct TrainingSampleAux {
    TrainingSample base;
    uint8_t        own[16];
};
static_assert(sizeof(TrainingSampleAux) == 352, "keep in sync with alphazero_learn.py");

struct SampleFileHeader {
    char     magic[8];                  // "YOLAHSP1"
    uint32_t version;                   // 1 = TrainingSample, 2 = TrainingSampleAux
    uint32_t sample_size;               // sizeof(TrainingSample)
    uint32_t nb_samples;
    uint32_t nb_games;
};
static_assert(sizeof(SampleFileHeader) == 24);

// The network self-play must use, published by the trainer in
// <work>/latest.json as {"step": 12000, "ts": "models/az_00012000.ts"}, plus
// "aux": true when the trainer wants the auxiliary targets. A relative path
// is relative to the work directory.
struct ModelRef {
    uint64_t    step = 0;
    std::string path;
    bool        aux = false;                // record TrainingSampleAux rows
};
bool read_latest_model(const std::string& work_dir, ModelRef& out);

struct SelfPlayOptions {
    json        player;                 // AlphaZeroMCTSPlayer keys: search + backend (weights
    ↪ ignored)
    std::string work_dir;               // reads latest.json, writes selfplay/
    std::string tag;                    // part of the file names (default host_pid)
    size_t      nb_games       = 128;   // games played concurrently
    uint64_t    max_games      = 0;     // stop after this many games (0 = until stopped)
    size_t      flush_samples  = 2000;  // write a file every N samples...
    double      flush_seconds  = 600;   // ...or every N seconds, whichever comes first
    double      poll_seconds   = 30;    // how often latest.json is checked
    double      report_seconds = 300;   // how often the throughput line is printed
    // Stop when latest.json has not changed for this long (0 = never). For
    // self-play jobs running apart from the trainer (alphazero_selfplay.sh):
    // when the trainer's job is over, no new network comes and they stop
    // instead of burning GPU hours on an outdated one.
```

```
    double      max_stale_hours = 0;
};

// Plays until `stop` is set (or max_games). Unfinished games are dropped;
// finished ones are always written before returning.
void run_selfplay(const SelfPlayOptions& options, const std::atomic<bool>& stop);

} // namespace az

#endif
```

12.5.2 `player/alphazero_selfplay.cpp`

328 lines.

One thread per game, all sharing one `BatchingEvaluator`; the calling thread is the supervisor (hot swap, periodic flush, statistics). Games stopped mid-way are dropped; finished ones always reach the disk before `run_selfplay` returns.

symbol	role
`SampleWriter`	collects finished games; writes ~2000-row files under a temporary name and renames them
`make_sample`	a `SearchResult` → a `TrainingSample` (π quantised to 16 bits)
`with_ownership`	a sample + who left each square during the game → its future ownership map
`run_selfplay` / `game_loop`	one game after another; z filled in at the end of each game

```
#include "alphazero_selfplay.h"
#include "alphazero_mcts_player.h"
#include "batched_evaluator.h"
#include <unistd.h>
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstring>
#include <filesystem>
#include <format>
#include <fstream>
#include <iostream>
#include <memory_resource>
#include <mutex>
#include <thread>
#include <vector>

namespace fs = std::filesystem;
using std::string;

namespace az {

bool read_latest_model(const string& work_dir, ModelRef& out) {
    std::ifstream in(fs::path(work_dir) / "latest.json");
    if (!in) return false;
    try {
        const json j = json::parse(in);
```

```
        ModelRef r;
        r.step = j.at("step").get<uint64_t>();
        fs::path p = j.at("ts").get<string>();
        if (p.is_relative()) p = fs::path(work_dir) / p;
        r.path = p.string();
        r.aux = j.value("aux", false);
        out = r;
        return true;
    } catch (const std::exception&) {
        return false;   // being rewritten (the trainer renames atomically, but be safe)
    }
}

namespace {

double now_seconds() {
    using namespace std::chrono;
    return duration<double>(steady_clock::now().time_since_epoch()).count();
}

string default_tag() {
    char host[256] = {0};
    gethostname(host, sizeof(host) - 1);
    return std::format("{}_{}", host, getpid());
}

// Collects finished games and writes them in files of ~flush_samples samples.
// Files are written under a temporary name and renamed, so the trainer never
// sees a partial file. Names start with the wall-clock time in milliseconds:
// sorting them by name is sorting them by age, across hosts and restarts.
//
// Rows are raw bytes of one fixed size per file: TrainingSample (version 1)
// or TrainingSampleAux (version 2). A file never mixes the two: when the
// format of the incoming games changes (the trainer switched --aux), what is
// pending is written first.
class SampleWriter {
public:
    SampleWriter(fs::path dir, string tag, size_t flush_samples, double flush_seconds)
        : dir(std::move(dir)), tag(std::move(tag)), flush_samples(flush_samples),
          flush_seconds(flush_seconds), last_flush(now_seconds()) {
        fs::create_directories(this->dir);
    }

    template <class Row>
    void add_game(const std::vector<Row>& game, uint32_t version) {
        std::lock_guard lock(mutex);
        if (nb_pending && (version != pending_version || sizeof(Row) != row_size)) write_locked();
        pending_version = version;
        row_size = sizeof(Row);
        const char* bytes = reinterpret_cast<const char*>(game.data());
        pending.insert(pending.end(), bytes, bytes + game.size() * sizeof(Row));
        nb_pending += game.size();
        ++pending_games;
        if (nb_pending >= flush_samples) write_locked();
    }

    void flush_if_old() {
        std::lock_guard lock(mutex);
        if (nb_pending && now_seconds() - last_flush >= flush_seconds) write_locked();
    }

    void flush() {
        std::lock_guard lock(mutex);
```

```cpp
        if (nb_pending) write_locked();
    }

private:
    void write_locked() {
        using namespace std::chrono;
        const auto ms =
        ↪  duration_cast<milliseconds>(system_clock::now().time_since_epoch()).count();
        const string name = std::format("{:013d}_{}_{:05d}.bin", ms, tag, seq++);
        const fs::path tmp = dir / ("." + name + ".tmp");
        {
            std::ofstream out(tmp, std::ios::binary);
            SampleFileHeader h{};
            std::memcpy(h.magic, "YOLAHSP1", 8);
            h.version = pending_version;
            h.sample_size = static_cast<uint32_t>(row_size);
            h.nb_samples = static_cast<uint32_t>(nb_pending);
            h.nb_games = static_cast<uint32_t>(pending_games);
            out.write(reinterpret_cast<const char*>(&h), sizeof(h));
            out.write(pending.data(), static_cast<std::streamsize>(pending.size()));
            if (!out) {
                std::cerr << "selfplay: cannot write " << tmp << ", keeping the samples in
                ↪  memory\n";
                return;
            }
        }
        fs::rename(tmp, dir / name);
        pending.clear();
        nb_pending = 0;
        pending_games = 0;
        last_flush = now_seconds();
    }

    const fs::path dir;
    const string tag;
    const size_t flush_samples;
    const double flush_seconds;
    std::mutex mutex;
    std::vector<char> pending;          // raw rows
    size_t nb_pending = 0;              // rows in `pending`
    size_t row_size = sizeof(TrainingSample);
    uint32_t pending_version = 1;
    size_t pending_games = 0;
    size_t seq = 0;
    double last_flush;
};

TrainingSample make_sample(const Yolah& y, const SearchResult& r, uint32_t model_step) {
    TrainingSample s{};
    s.black = y.bitboard(Yolah::BLACK);
    s.white = y.bitboard(Yolah::WHITE);
    s.empty = y.empty_bitboard();
    s.turn = y.current_player();
    s.root_q = r.root_value;
    s.model_step = model_step;
    const size_t n = std::min<size_t>(r.children.size(), Yolah::MAX_NB_MOVES);
    s.nb_moves = static_cast<uint8_t>(n);
    for (size_t i = 0; i < n; i++) {
        s.action[i] = static_cast<uint16_t>(nn::action_index(r.children[i].move));
        const float p = std::clamp(r.policy[i], 0.0f, 1.0f);
        s.prob[i] = static_cast<uint16_t>(std::lround(p * 65535.0f));
    }
    return s;
```

```
}

// The future ownership map of one sample, from `leaver[q]`: the colour that
// left square q during the game (0xFF = nobody). Squares that are already
// holes in the sample's position were left BEFORE it — by whom is not visible
// in the position, so they carry no target (OWN_PAST, masked in the loss).
// Every other square that someone leaves later was left after the sample:
// the map counts exactly the moves still to be played, so
//     #OWN_MINE − #OWN_OPP = (final margin) − (margin at the sample),
// the rest of the score, for the side to move.
TrainingSampleAux with_ownership(const TrainingSample& s, const uint8_t leaver[64]) {
    TrainingSampleAux a{};
    a.base = s;
    for (int q = 0; q < 64; q++) {
        uint8_t code;
        if ((s.empty >> q) & 1)      code = OWN_PAST;
        else if (leaver[q] == 0xFF)  code = OWN_NONE;
        else                         code = leaver[q] == s.turn ? OWN_MINE : OWN_OPP;
        a.own[q / 4] |= static_cast<uint8_t>(code << (2 * (q % 4)));
    }
    return a;
}

} // namespace

void run_selfplay(const SelfPlayOptions& opt, const std::atomic<bool>& stop) {
    ModelRef model;
    if (!read_latest_model(opt.work_dir, model))
        throw std::runtime_error("selfplay: no readable " + opt.work_dir + "/latest.json");

    // One backend for all the games, behind the batching wrapper.
    json cfg = opt.player;
    cfg["weights"] = model.path;
    SearchParams params = AlphaZeroMCTSPlayer::search_params(cfg);
    params.nb_threads = 1;                      // parallelism comes from the games
    std::unique_ptr<nn::Evaluator> backend = nn::make_evaluator(cfg);
    if (params.batch_size == 0) params.batch_size = 8;
    const size_t nb_games = std::max<size_t>(1, opt.nb_games);
    const size_t merged = cfg.value("merged batch size", nb_games * params.batch_size);
    nn::BatchingEvaluator evaluator(std::move(backend), nb_games, merged);

    SampleWriter writer(fs::path(opt.work_dir) / "selfplay",
                        opt.tag.empty() ? default_tag() : opt.tag,
                        opt.flush_samples, opt.flush_seconds);

    std::atomic<uint64_t> generation{0};         // bumped at every weight swap
    std::atomic<uint32_t> model_step{static_cast<uint32_t>(model.step)};
    std::atomic<uint64_t> games_started{0}, games_done{0}, samples_done{0}, plies_done{0};
    std::atomic<uint64_t> evaluations{0}, black_wins{0}, white_wins{0};
    std::atomic<size_t>   active{nb_games};
    // The games stop on `halt`: set by the caller's `stop` or by staleness.
    std::atomic<bool>     halt{false};
    // Record the auxiliary targets (latest.json "aux"; re-read at every poll).
    std::atomic<bool>     record_aux{model.aux};

    std::cout << std::format("selfplay: {} concurrent games, merged batch {}, {} sims ({} fast,
     ↪  p={}), model step {}\n"
                             "selfplay: network {}\n"
                             "selfplay: auxiliary targets (ownership) {}\n",
                             nb_games, merged, params.nb_simulations, params.nb_simulations_fast,
                             params.playout_cap_fast_prob, model.step, evaluator.info(),
                             model.aux ? "on" : "off") << std::flush;

```

```
    auto game_loop = [&] {
        std::pmr::synchronized_pool_resource memory;
        {
            Search search(evaluator, params, &memory);
            uint64_t seen = generation.load();
            while (!halt.load()) {
                if (opt.max_games && games_started.fetch_add(1) >= opt.max_games) break;
                Yolah y;
                std::vector<TrainingSample> samples;
                // Who left each square during this game (auxiliary targets).
                uint8_t leaver[64];
                std::fill(std::begin(leaver), std::end(leaver), uint8_t(0xFF));
                search.reset();
                while (!y.game_over() && !halt.load()) {
                    // New weights: the graph's priors/values and the cached
                    // evaluations belong to the old network.
                    const uint64_t g = generation.load(std::memory_order_acquire);
                    if (g != seen) {
                        search.reset();
                        search.clear_cache();
                        seen = g;
                    }
                    const SearchResult r = search.search(y);
                    evaluations.fetch_add(r.nb_evaluations, std::memory_order_relaxed);
                    if (r.full_search && !r.children.empty())
                        samples.push_back(make_sample(y, r, model_step.load()));
                    if (r.best_move != Move::none())   // a pass leaves no hole
                        leaver[static_cast<int>(r.best_move.from_sq())] = y.current_player();
                    y.play(r.best_move);
                }
                if (!y.game_over()) break;           // stopped mid-game: drop it
                // Outcome for the side to move at each recorded position.
                const auto [bs, ws] = y.score();
                const int black_diff = int(bs) - int(ws);
                for (TrainingSample& s : samples) {
                    const int d = s.turn == Yolah::BLACK ? black_diff : -black_diff;
                    s.z = static_cast<int8_t>((d > 0) - (d < 0));
                }
                if (black_diff > 0) black_wins.fetch_add(1);
                if (black_diff < 0) white_wins.fetch_add(1);
                plies_done.fetch_add(y.nb_plies());
                samples_done.fetch_add(samples.size());
                games_done.fetch_add(1);
                if (record_aux.load()) {
                    std::vector<TrainingSampleAux> rows;
                    rows.reserve(samples.size());
                    for (const TrainingSample& s : samples) rows.push_back(with_ownership(s,
                    ↪  leaver));
                    writer.add_game(rows, 2);
                } else {
                    writer.add_game(samples, 1);
                }
            }
        }
        // Fewer clients: the batcher must stop waiting for this one.
        evaluator.set_nb_clients(active.fetch_sub(1) - 1);
    };

    std::vector<std::jthread> threads;
    for (size_t i = 0; i < nb_games; i++) threads.emplace_back(game_loop);

    // Supervisor: hot-swap the network, flush old samples, report.
    const double start = now_seconds();
```

```
    double last_poll = start, last_report = start;
    uint64_t last_games = 0, last_samples = 0, last_evals = 0;
    double last_model_change = start;
    while (!halt.load() && active.load() > 0) {
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
        const double t = now_seconds();
        if (stop.load()) halt.store(true);
        if (opt.max_stale_hours > 0 && t - last_model_change > opt.max_stale_hours * 3600) {
            std::cout << std::format("selfplay: no new network for {:.2f} h, stopping\n",
             ↪  opt.max_stale_hours) << std::flush;
            halt.store(true);
        }
        if (t - last_poll >= opt.poll_seconds) {
            last_poll = t;
            ModelRef latest;
            const bool readable = read_latest_model(opt.work_dir, latest);
            if (readable && latest.aux != record_aux.load()) {
                record_aux.store(latest.aux);
                std::cout << std::format("selfplay: auxiliary targets (ownership) {}\n",
                 ↪  latest.aux ? "on" : "off") << std::flush;
            }
            if (readable && latest.step != model.step) {
                try {
                    evaluator.reload(latest.path);
                    model = latest;
                    last_model_change = t;
                    model_step.store(static_cast<uint32_t>(model.step));
                    generation.fetch_add(1, std::memory_order_release);
                    std::cout << std::format("selfplay: now playing with step {} ({})\n",
                     ↪  model.step, model.path) << std::flush;
                } catch (const std::exception& e) {
                    // A half-written file or a transient FS error: retry at the next poll.
                    std::cerr << "selfplay: reload of " << latest.path << " failed: " << e.what()
                     ↪  << "\n";
                }
            }
            writer.flush_if_old();
        }
        if (t - last_report >= opt.report_seconds) {
            const uint64_t g = games_done.load(), s = samples_done.load(), e = evaluations.load();
            const double dt = t - last_report;
            std::cout << std::format("selfplay: {:.1f}h  games {} (+{:.2f}/s)  samples {}
             ↪  (+{:.1f}/s)  "
                                     "evals/s {:.0f}  avg plies {:.1f}  black wins {:.1f}%  white
                                     ↪  wins {:.1f}%\n",
                                     (t - start) / 3600, g, (g - last_games) / dt, s, (s -
                                     ↪  last_samples) / dt,
                                     (e - last_evals) / dt, g ? double(plies_done.load()) / g :
                                     ↪  0.0,
                                     g ? 100.0 * black_wins.load() / g : 0.0, g ? 100.0 *
↪  white_wins.load() / g : 0.0)
                      << std::flush;
            last_report = t;
            last_games = g; last_samples = s; last_evals = e;
        }
    }
    threads.clear();   // join (the game threads watch `halt` / max_games)
    writer.flush();
    std::cout << std::format("selfplay: done, {} games, {} samples\n", games_done.load(),
     ↪  samples_done.load()) << std::flush;
}

} // namespace az
```

12.5.3 `test/alphazero_learn_main.cpp`

263 lines.

The `alphazero_learn` binary. `selfplay` wraps `run_selfplay` with a signal handler; `match` plays two networks against each other, each opening once with each colour — from a book of balanced openings (Section 6.5.3) or random — and writes the full record as JSON.

symbol	role
`selfplay_main`	parse options, install SIGINT/SIGTERM, run
`match_main`	A vs B, colours alternated, one opening per pair of games
`read_book`	`--book`: one opening per line, every move checked legal
`apply_set`	`--set 'key=value'`: override a key of the JSON config

```
// alphazero_learn_main.cpp — the C++ side of the AlphaZero learning loop
// (driven by nnue/alphazero_learn.py; see player/README_ALPHAZERO_LEARN.md).
//
//   alphazero_learn selfplay --config ../config/alphazero_mcts_selfplay_player.cfg --work <dir>
//         [--games 128] [--max-games 0] [--flush-samples 2000] [--flush-seconds 600]
//         [--poll 30] [--report 300] [--max-stale-hours 0] [--tag name] [--set 'key=value' ...]
//     Plays self-play games with the network named in <dir>/latest.json (hot-
//     swapped when it changes) and writes training samples to <dir>/selfplay/.
//     Stops cleanly on SIGINT/SIGTERM (finished games are written).
//
//   alphazero_learn match --config ../config/alphazero_mcts_eval_player.cfg
//         --a new.ts --b old.ts [--games 10] [--time 2000000] [--opening-plies 4]
//         [--book openings.txt] [--seed 1] [--out result.json] [--set 'key=value' ...]
//     Plays A against B, alternating colours, and reports A's result. Games
//     2k and 2k+1 start from the same opening (A black in one, white in the
//     other): with deterministic players and no opening, the 5 games of each
//     colour would be the same game 5 times. The opening: a random one of
//     --opening-plies plies, or with --book the next one of a book of balanced
//     openings (config/openings/balanced.txt, test/opening_book_main.cpp: one
//     opening per line, "a1:b2 a8:b7 ...", '#' starts a comment), taken in an
//     order shuffled by --seed — a different opening for each pair as long as
//     the book has enough of them.
//
// --set overrides any key of the player JSON config ('key=value', value parsed as JSON).
#include "alphazero_mcts_player.h"
#include "alphazero_selfplay.h"
#include "magic.h"
#include "zobrist.h"
#include <algorithm>
#include <atomic>
#include <csignal>
#include <format>
#include <fstream>
#include <iostream>
#include <random>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

using std::string;

```

```cpp
namespace {

std::atomic<bool> stop_requested{false};
void on_signal(int) { stop_requested.store(true); }

struct Args {
    std::vector<string> argv;
    size_t i = 0;
    bool more() const { return i < argv.size(); }
    string next(const string& opt) {
        if (i >= argv.size()) { std::cerr << "missing value for " << opt << "\n"; std::exit(2); }
        return argv[i++];
    }
};

void apply_set(json& cfg, const string& kv) {
    const size_t eq = kv.find('=');
    if (eq == string::npos) { std::cerr << "--set expects key=value\n"; std::exit(2); }
    cfg[kv.substr(0, eq)] = json::parse(kv.substr(eq + 1));
}

json load_config(const string& path) {
    std::ifstream in(path);
    if (!in) { std::cerr << "cannot open " << path << "\n"; std::exit(2); }
    return json::parse(in);
}

int selfplay_main(Args& a) {
    az::SelfPlayOptions opt;
    string config;
    std::vector<string> sets;
    while (a.more()) {
        const string o = a.next("");
        if (o == "--config") config = a.next(o);
        else if (o == "--work") opt.work_dir = a.next(o);
        else if (o == "--games") opt.nb_games = std::stoul(a.next(o));
        else if (o == "--max-games") opt.max_games = std::stoull(a.next(o));
        else if (o == "--flush-samples") opt.flush_samples = std::stoul(a.next(o));
        else if (o == "--flush-seconds") opt.flush_seconds = std::stod(a.next(o));
        else if (o == "--poll") opt.poll_seconds = std::stod(a.next(o));
        else if (o == "--report") opt.report_seconds = std::stod(a.next(o));
        else if (o == "--max-stale-hours") opt.max_stale_hours = std::stod(a.next(o));
        else if (o == "--tag") opt.tag = a.next(o);
        else if (o == "--set") sets.push_back(a.next(o));
        else { std::cerr << "unknown option " << o << "\n"; return 2; }
    }
    if (config.empty() || opt.work_dir.empty()) { std::cerr << "selfplay needs --config and
    ↪  --work\n"; return 2; }
    opt.player = load_config(config);
    for (const string& kv : sets) apply_set(opt.player, kv);
    std::signal(SIGINT, on_signal);
    std::signal(SIGTERM, on_signal);
    az::run_selfplay(opt, stop_requested);
    return 0;
}

Move parse_move(const string& s) {
    auto sq = [&](const string& t) {
        if (t.size() != 2 || t[0] < 'a' || t[0] > 'h' || t[1] < '1' || t[1] > '8')
            throw std::runtime_error("bad move " + s);
        return Square((t[1] - '1') * 8 + (t[0] - 'a'));
    };
    const auto colon = s.find(':');
```

```
    if (colon == string::npos) throw std::runtime_error("bad move " + s + " (expected a1:b2)");
    return Move(sq(s.substr(0, colon)), sq(s.substr(colon + 1)));
}

// A book of openings: one per line, '#' starts a comment; every opening legal
// and not ending the game.
std::vector<std::vector<Move>> read_book(const string& path) {
    std::ifstream f(path);
    if (!f) throw std::runtime_error("cannot open the book " + path);
    std::vector<std::vector<Move>> book;
    for (string line; std::getline(f, line);) {
        std::istringstream is(line.substr(0, line.find('#')));
        std::vector<Move> o;
        Yolah y;
        for (string t; is >> t;) {
            const Move m = parse_move(t);
            Yolah::MoveList legal;
            y.moves(legal);
            if (std::find(legal.begin(), legal.end(), m) == legal.end()) throw
             ↪  std::runtime_error("illegal move " + t + " in the book " + path + ": " + line);
            o.push_back(m);
            y.play(m);
        }
        if (o.empty()) continue;
        if (y.game_over()) throw std::runtime_error("an opening of " + path + " ends the game: "
         ↪  + line);
        book.push_back(std::move(o));
    }
    if (book.empty()) throw std::runtime_error("no opening in " + path);
    return book;
}

string move_list(const std::vector<Move>& moves) {
    std::ostringstream os;
    for (size_t i = 0; i < moves.size(); i++) os << (i ? " " : "") << moves[i];
    return os.str();
}

int match_main(Args& a) {
    string config, model_a, model_b, out, book_path;
    size_t games = 10, opening_plies = 4;
    uint64_t time_us = 2'000'000, seed = 1;
    std::vector<string> sets;
    while (a.more()) {
        const string o = a.next("");
        if (o == "--config") config = a.next(o);
        else if (o == "--a") model_a = a.next(o);
        else if (o == "--b") model_b = a.next(o);
        else if (o == "--games") games = std::stoul(a.next(o));
        else if (o == "--time") time_us = std::stoull(a.next(o));
        else if (o == "--opening-plies") opening_plies = std::stoul(a.next(o));
        else if (o == "--book") book_path = a.next(o);
        else if (o == "--seed") seed = std::stoull(a.next(o));
        else if (o == "--out") out = a.next(o);
        else if (o == "--set") sets.push_back(a.next(o));
        else { std::cerr << "unknown option " << o << "\n"; return 2; }
    }
    if (config.empty() || model_a.empty() || model_b.empty()) {
        std::cerr << "match needs --config, --a and --b\n";
        return 2;
    }
    json cfg = load_config(config);
    for (const string& kv : sets) apply_set(cfg, kv);
```

```
    cfg["microseconds"] = time_us;
    cfg["nb simulations"] = 0;
    json cfg_a = cfg, cfg_b = cfg;
    cfg_a["weights"] = model_a;
    cfg_b["weights"] = model_b;
    AlphaZeroMCTSPlayer player_a(cfg_a), player_b(cfg_b);

    std::mt19937_64 rng(seed);
    std::vector<std::vector<Move>> book;
    std::vector<size_t> order;
    if (!book_path.empty()) {
        book = read_book(book_path);
        for (size_t i = 0; i < book.size(); i++) order.push_back(i);
        std::shuffle(order.begin(), order.end(), rng);
        std::cout << "book " << book_path << ": " << book.size() << " openings\n";
    }
    int wins = 0, draws = 0, losses = 0;
    json records = json::array();
    std::vector<Move> opening;
    for (size_t g = 0; g < games; g++) {
        const bool a_black = (g % 2 == 0);
        if (g % 2 == 0 && !book.empty()) {
            opening = book[order[(g / 2) % book.size()]];
        } else if (g % 2 == 0) {
            // A fresh random opening for the pair (not a finished game).
            for (;;) {
                Yolah y;
                opening.clear();
                for (size_t p = 0; p < opening_plies && !y.game_over(); p++) {
                    Yolah::MoveList moves;
                    y.moves(moves);
                    const Move m = moves[rng() % moves.size()];
                    opening.push_back(m);
                    y.play(m);
                }
                if (!y.game_over()) break;
            }
        }
        Yolah y;
        for (Move m : opening) y.play(m);
        std::vector<Move> played;
        while (!y.game_over()) {
            const bool black_to_move = y.current_player() == Yolah::BLACK;
            AlphaZeroMCTSPlayer& p = (black_to_move == a_black) ? player_a : player_b;
            const Move m = p.play(y);
            played.push_back(m);
            y.play(m);
        }
        player_a.game_over(y);
        player_b.game_over(y);
        const auto [bs, ws] = y.score();
        const int a_diff = (a_black ? 1 : -1) * (int(bs) - int(ws));
        const char* result = a_diff > 0 ? "win" : (a_diff < 0 ? "loss" : "draw");
        (a_diff > 0 ? wins : (a_diff < 0 ? losses : draws))++;
        records.push_back({{"a_color", a_black ? "black" : "white"},
                           {"opening", move_list(opening)},
                           {"moves", move_list(played)},
                           {"black_score", bs}, {"white_score", ws},
                           {"plies", y.nb_plies()}, {"result_for_a", result}});
        std::cout << std::format("game {}/{}: A {} score {}-{} ({} plies) -> A {}  [A {}W {}D
        ↪  {}L]\n",
                                 g + 1, games, a_black ? "black" : "white", bs, ws, y.nb_plies(),
                                 result, wins, draws, losses) << std::flush;
```

```
    }
    const double score = games ? (wins + 0.5 * draws) / games : 0.0;
    json res = {{"a", model_a}, {"b", model_b}, {"games", games}, {"time_us", time_us},
                {"opening_plies", opening_plies}, {"book", book_path}, {"seed", seed},
                {"wins", wins}, {"draws", draws}, {"losses", losses}, {"score", score},
                {"records", records}};
    if (!out.empty()) {
        std::ofstream f(out);
        f << res.dump(2) << "\n";
    }
    std::cout << std::format("match: A {}W {}D {}L, score {:.3f}\n", wins, draws, losses, score)
    ↪  << std::flush;
    return 0;
}

} // namespace

int main(int argc, char* argv[]) {
    magic::init();
    zobrist::init();
    if (argc < 2) {
        std::cerr << "usage: alphazero_learn selfplay|match [options]  (see
        ↪  alphazero_learn_main.cpp)\n";
        return 2;
    }
    Args a;
    for (int i = 2; i < argc; i++) a.argv.emplace_back(argv[i]);
    const string cmd = argv[1];
    try {
        if (cmd == "selfplay") return selfplay_main(a);
        if (cmd == "match") return match_main(a);
    } catch (const std::exception& e) {
        std::cerr << "alphazero_learn " << cmd << ": " << e.what() << "\n";
        return 1;
    }
    std::cerr << "unknown command " << cmd << "\n";
    return 2;
}
```

12.5.4 `test/opening_book_main.cpp`

381 lines.

The book of balanced openings of the matches (Section 6.5.3): random openings, one per class of symmetric ones, searched by the minimax player at fixed depth; the closest to a draw are kept. `check` measures the book by playing games from it.

symbol	role
`canonical`	the smallest image of a position under the 4 symmetries of the initial position
`build`	draw, deduplicate, search at depth 6 then 12, keep the most balanced
`check`	the minimax player against itself from the book and from random openings: Black's score

```cpp
// opening_book_main.cpp — a book of balanced openings for the matches between
// networks (alphazero_learn match --book, nnue/alphazero_match.sh BOOK=...).
//
// Why: after a random 4-ply opening, Black wins almost every game between two
// strong players, so a pair of games (one opening, each colour) nearly always
// ends 1-1 and says nothing about which network is stronger. A balanced
// opening is one where neither colour is clearly ahead: the colour no longer
// decides the result, the players do.
//
//   opening_book build --player ../config/search_study/nnue_k8_them_spacelite.cfg
//         [--plies 6] [--candidates 20000] [--pre-depth 6] [--pre-max 6000] [--depth 10]
//  [--target 0] [--max-value 1500]
//         [--size 1000] [--threads 16] [--seed 1] [--out ../config/openings/balanced.txt]
//     1. Draws --candidates random openings of --plies plies (uniform random
//        legal moves; none ends the game), keeping one of each class of
//        equivalent positions: 4 of the 8 symmetries of the board map the
//        initial position to itself (identity, 180° rotation, the two diagonal
//        flips), so up to 4 openings lead to the same game up to symmetry (the
//        other 4 send the black pieces to the white ones' initial squares).
//     2. Searches each with the dev player at fixed depth --depth (no clock:
//        the book is reproducible), from an empty table. The value is turned
//        into Black's point of view (±30000 = tanh ±1, a value v ≈ the
//        expected result 2p − 1 of the side to move).
//     3. Keeps the --size openings whose value is closest to --target, within
//        ±--max-value, in a random order (so the first openings of the file are
//        not the most balanced ones only). --target: the value at which Black
//        scores ½ in actual games, which is not exactly 0 (check --csv).
//     Book lines: the moves ("a1:b2 a8:b7 ..."), then "# black value V", then
//     the value histogram of all the candidates in comments at the top.
//
//   opening_book check --book FILE --player CFG [--depth 8] [--games 400] [--threads 16]
//                      [--random-plies 4] [--seed 1] [--csv value_result.csv] [--opponent-depth
//  D2]
//     The dev player against itself at fixed depth, one game from each of the
//     first --games openings of the book, then from as many random openings
//     of --random-plies plies: Black's score in each case. A balanced book
//     gives Black about 50 %; random openings give Black much more. --csv: the
//     value of each book opening and Black's result, to calibrate --target
//     (on a book built with a large --max-value; --random-plies 0 skips the
//     random openings). --opponent-depth D2: pairs instead, as in the matches
//     between networks — depth --depth against depth D2, each opening once with
//     each colour: how often the colour decides the pair, and the score.
#include "magic.h"
#include "zobrist.h"
#include "player.h"
#include "minmax_nnue_dev_player.h"
#include <algorithm>
#include <array>
#include <bit>
#include <format>
#include <atomic>
#include <cmath>
#include <fstream>
#include <iostream>
#include <map>
#include <mutex>
#include <random>
#include <set>
#include <sstream>
#include <string>
#include <thread>
#include <vector>

```

```cpp
using std::string, std::vector, std::cerr, std::cout;

namespace {

std::map<string, string> parse_args(int argc, char* argv[], int from) {
    std::map<string, string> kv;
    for (int i = from; i < argc; i++) {
        string k = argv[i];
        if (k.rfind("--", 0) != 0 || i + 1 >= argc) { cerr << "bad argument " << k << '\n'; std::exit(2); }
        kv[k.substr(2)] = argv[++i];
    }
    return kv;
}
string get(const std::map<string, string>& kv, const string& k, const string& def) {
    auto it = kv.find(k);
    return it == kv.end() ? def : it->second;
}

std::unique_ptr<Player> make_dev_player(const string& cfg) {
    std::ifstream f(cfg);
    if (!f) { cerr << "cannot open " << cfg << '\n'; std::exit(1); }
    json j = json::parse(f);
    j["verbose"] = false;
    j["nb threads"] = 1;
    j["tt size"] = 64;
    auto p = Player::create(j);
    if (!dynamic_cast<MinMaxNNUE_DevPlayer*>(p.get())) { cerr << cfg << ": not a MinMaxNNUE_DevPlayer\n"; std::exit(1); }
    return p;
}

string move_list(const vector<Move>& moves) {
    std::ostringstream os;
    for (size_t i = 0; i < moves.size(); i++) os << (i ? " " : "") << moves[i];
    return os.str();
}

Move parse_move(const string& s) {
    auto sq = [&](const string& t) {
        if (t.size() != 2 || t[0] < 'a' || t[0] > 'h' || t[1] < '1' || t[1] > '8') {
            cerr << "bad move " << s << '\n';
            std::exit(1);
        }
        return Square((t[1] - '1') * 8 + (t[0] - 'a'));
    };
    const auto colon = s.find(':');
    if (colon == string::npos) { cerr << "bad move " << s << " (expected a1:b2)\n"; std::exit(1); }
    return Move(sq(s.substr(0, colon)), sq(s.substr(colon + 1)));
}

// The 4 symmetries of the board that map the initial position to itself:
// square (file, rank) → itself, (7−f, 7−r), (r, f), (7−r, 7−f).
uint64_t transform(uint64_t bb, int sym) {
    uint64_t r = 0;
    for (; bb; bb &= bb - 1) {
        const int s = std::countr_zero(bb), f = s & 7, k = s >> 3;
        int f2 = f, k2 = k;
        if (sym == 1) { f2 = 7 - f; k2 = 7 - k; }
        else if (sym == 2) { f2 = k; k2 = f; }
        else if (sym == 3) { f2 = 7 - k; k2 = 7 - f; }
        r |= uint64_t(1) << (k2 * 8 + f2);
```

```cpp
    }
    return r;
}
// The smallest (black, white, holes) of the 4 images: equal for equivalent positions
// (same side to move and scores: the openings all have the same number of plies).
std::tuple<uint64_t, uint64_t, uint64_t> canonical(const Yolah& y) {
    std::tuple<uint64_t, uint64_t, uint64_t> best{~0ull, ~0ull, ~0ull};
    for (int s = 0; s < 4; s++)
        best = std::min(best, {transform(y.bitboard(Yolah::BLACK), s),
 ↪  transform(y.bitboard(Yolah::WHITE), s),
                               transform(y.empty_bitboard(), s)});
    return best;
}

// A random opening of `plies` plies that does not end the game.
vector<Move> random_opening(std::mt19937_64& rng, int plies) {
    for (;;) {
        Yolah y;
        vector<Move> res;
        for (int p = 0; p < plies && !y.game_over(); p++) {
            Yolah::MoveList moves;
            y.moves(moves);
            const Move m = moves[rng() % moves.size()];
            res.push_back(m);
            y.play(m);
        }
        if (!y.game_over()) return res;
    }
}

Yolah after(const vector<Move>& moves) {
    Yolah y;
    for (Move m : moves) y.play(m);
    return y;
}

// Runs body(i, worker) for i in [0, n) on `threads` threads, each with its own worker index.
template <class F> void parallel_for(size_t n, int threads, F&& body) {
    std::atomic<size_t> next{0};
    vector<std::thread> pool;
    for (int t = 0; t < threads; t++)
        pool.emplace_back([&, t] { for (size_t i; (i = next++) < n;) body(i, t); });
    for (auto& th : pool) th.join();
}

int build(const std::map<string, string>& kv) {
    const string cfg = get(kv, "player", "../config/search_study/nnue_k8_them_spacelite.cfg");
    const int plies = std::stoi(get(kv, "plies", "6"));
    const size_t nb_candidates = std::stoul(get(kv, "candidates", "20000"));
    const int depth = std::stoi(get(kv, "depth", "10"));
    const int pre_depth = std::stoi(get(kv, "pre-depth", "6"));
    const int pre_max = std::stoi(get(kv, "pre-max", "6000"));
    const int max_value = std::stoi(get(kv, "max-value", "1500"));
    const int target = std::stoi(get(kv, "target", "0"));
    const size_t size = std::stoul(get(kv, "size", "1000"));
    const int threads = std::stoi(get(kv, "threads", "16"));
    const uint64_t seed = std::stoull(get(kv, "seed", "1"));
    const string out = get(kv, "out", "../config/openings/balanced.txt");

    // 1. Candidates, one per symmetry class.
    std::mt19937_64 rng(seed);
    vector<vector<Move>> cands;
    std::set<std::tuple<uint64_t, uint64_t, uint64_t>> seen;
```

```
    size_t draws = 0;
    while (cands.size() < nb_candidates && draws < 50 * nb_candidates) {
        draws++;
        vector<Move> o = random_opening(rng, plies);
        if (seen.insert(canonical(after(o))).second) cands.push_back(std::move(o));
    }
    cerr << cands.size() << " distinct openings of " << plies << " plies (" << draws << "
↪   drawn)\n";

    // 2. Their value for Black at fixed depth: first a cheap search at --pre-depth for
    // all of them, then --depth for those within ±--pre-max only (most random openings
    // are far from balanced: the shallow search is enough to discard them).
    vector<std::unique_ptr<Player>> players;
    for (int t = 0; t < threads; t++) players.push_back(make_dev_player(cfg));
    std::mutex mu;
    auto values_at = [&](const vector<vector<Move>>& openings, int d) {
        vector<int> v(openings.size());
        std::atomic<size_t> done{0};
        parallel_for(openings.size(), threads, [&](size_t i, int t) {
            auto* dev = static_cast<MinMaxNNUE_DevPlayer*>(players[t].get());
            dev->clear_table();
            const Yolah y = after(openings[i]);
            const auto r = dev->search(y, uint8_t(d), 1'000'000'000'000ull);
            v[i] = y.current_player() == Yolah::BLACK ? r.value : -r.value;
            const size_t n = ++done;
            if (n % 10000 == 0) { std::lock_guard lk(mu); cerr << n << "/" << openings.size() << "
↪   searched at depth " << d << "\n"; }
        });
        return v;
    };
    const vector<int> pre_value = values_at(cands, pre_depth);
    vector<vector<Move>> close;
    for (size_t i = 0; i < cands.size(); i++) if (std::abs(pre_value[i]) <= pre_max)
↪   close.push_back(cands[i]);
    cerr << close.size() << " within ±" << pre_max << " at depth " << pre_depth << "\n";
    const vector<int> value = values_at(close, depth);

    // 3. The most balanced ones, in a random order: the closest to --target, the value
    // at which Black's expected score is ½ (measured with check --csv: not exactly 0).
    vector<size_t> idx(close.size());
    for (size_t i = 0; i < idx.size(); i++) idx[i] = i;
    auto dist = [&](size_t i) { return std::abs(value[i] - target); };
    std::stable_sort(idx.begin(), idx.end(), [&](size_t a, size_t b) { return dist(a) < dist(b);
↪   });
    vector<size_t> keep;
    for (size_t i : idx) if (keep.size() < size && dist(i) <= max_value) keep.push_back(i);
    std::shuffle(keep.begin(), keep.end(), rng);

    // Histogram of the candidates' values (Black's point of view), for the record.
    std::map<int, int> hist;
    const int bin = 1500;
    double mean = 0;
    for (int v : pre_value) { hist[int(std::floor(double(v) / bin))]++; mean += v; }
    mean /= std::max<size_t>(1, pre_value.size());
    int worst = 0;
    for (size_t i : keep) worst = std::max(worst, dist(i));

    std::ofstream f(out);
    if (!f) { cerr << "cannot write " << out << '\n'; return 1; }
    f << "# Balanced openings for the matches between networks (test/opening_book_main.cpp).\n"
      << "# opening_book build --player " << cfg << " --plies " << plies << " --candidates " <<
↪   nb_candidates
      << " --pre-depth " << pre_depth << " --pre-max " << pre_max << " --depth " << depth << "
↪   --target " << target << " --max-value " << max_value << " --size " << size << " --seed "
↪   << seed << "\n"
```

```
      << "# " << cands.size() << " distinct random openings (up to the 4 symmetries of the initial position), searched by the dev\n"
      << "# player at depth " << pre_depth << "; the " << close.size() << " within ±" << pre_max << " searched again at depth " << depth << ";\n"
      << "# kept: the " << keep.size() << " with the value (Black's point of view, ±30000 = tanh ±1) closest to " << target << ",\n"
      << "# all within " << target << " ± " << worst << ", in a random order. Mean depth-" << pre_depth << " value of the candidates: "
      << std::lround(mean) << ".\n# Depth-" << pre_depth << " values of the candidates (bins of " << bin << "):\n";
    for (auto [b, c] : hist)
        f << std::format("#   [{:+6}, {:+6}) {:6}  {:5.1f} %\n", b * bin, (b + 1) * bin, c, 100.0 * c / pre_value.size());
    for (size_t i : keep) f << move_list(close[i]) << "  # black value " << value[i] << "\n";
    cerr << "wrote " << keep.size() << " openings to " << out << " (|value − " << target << "| ≤ " << worst << ", candidates' mean "
         << std::lround(mean) << ")\n";
    return 0;
}

// The openings of a book and their values (the "# black value V" comments; 0 if none).
vector<vector<Move>> read_book(const string& path, vector<int>* values = nullptr) {
    std::ifstream f(path);
    if (!f) { cerr << "cannot open " << path << '\n'; std::exit(1); }
    vector<vector<Move>> res;
    for (string line; std::getline(f, line);) {
        const size_t hash = line.find('#');
        std::istringstream is(line.substr(0, hash));
        vector<Move> o;
        for (string t; is >> t;) o.push_back(parse_move(t));
        if (o.empty()) continue;
        res.push_back(std::move(o));
        if (values) {
            const size_t k = hash == string::npos ? string::npos : line.find("black value", hash);
            values->push_back(k == string::npos ? 0 : std::stoi(line.substr(k + 11)));
        }
    }
    return res;
}

int check(const std::map<string, string>& kv) {
    const string cfg = get(kv, "player", "../config/search_study/nnue_k8_them_spacelite.cfg");
    const int depth = std::stoi(get(kv, "depth", "8"));
    const size_t games = std::stoul(get(kv, "games", "400"));
    const int threads = std::stoi(get(kv, "threads", "16"));
    const int random_plies = std::stoi(get(kv, "random-plies", "4"));
    std::mt19937_64 rng(std::stoull(get(kv, "seed", "1")));
    vector<int> book_values;
    vector<vector<Move>> book = read_book(get(kv, "book", "../config/openings/balanced.txt"), &book_values);
    if (book.size() > games) book.resize(games);
    vector<vector<Move>> randoms;
    for (size_t i = 0; i < book.size(); i++) randoms.push_back(random_opening(rng, random_plies));

    vector<std::unique_ptr<Player>> players;
    for (int t = 0; t < threads; t++) players.push_back(make_dev_player(cfg));
    // One game of the dev player against itself from each opening: Black's result (1, ½, 0).
    auto play_all = [&](const vector<vector<Move>>& openings) {
        vector<double> res(openings.size());
        parallel_for(openings.size(), threads, [&](size_t i, int t) {
            auto* dev = static_cast<MinMaxNNUE_DevPlayer*>(players[t].get());
            dev->clear_table();
            Yolah y = after(openings[i]);
```

```
            while (!y.game_over()) y.play(dev->search(y, uint8_t(depth),
             ↪  1'000'000'000'000ull).move);
            const auto [b, w] = y.score();
            res[i] = b > w ? 1.0 : (b < w ? 0.0 : 0.5);
        });
        return res;
    };
    auto report = [&](const char* name, const vector<double>& res) {
        const double n = double(res.size());
        double s = 0, s2 = 0;
        int w = 0, d = 0, l = 0;
        for (double x : res) { s += x; s2 += x * x; (x == 1 ? w : (x == 0 ? l : d))++; }
        const double m = s / n, se = std::sqrt(std::max(0.0, s2 / n - m * m) / n);
        cout << std::format("{:>22}: {} games, Black {}W {}D {}L, Black's score {:.3f} ±
↪  {:.3f}\n",
                            name, res.size(), w, d, l, m, 1.96 * se);
    };
    if (kv.contains("opponent-depth")) {
        // Pairs, as in the matches between networks: A (--depth) against B (--opponent-depth),
        // each opening once with each colour. How often the colour decides (the pair ends 1-1
        // with Black winning both games), and A's score, from the book and from random openings.
        const int depth_b = std::stoi(get(kv, "opponent-depth", "10"));
        auto pairs = [&](const vector<vector<Move>>& openings) {
            vector<std::array<double, 2>> res(openings.size());   // Black's result with A black,
↪  with A white
            parallel_for(2 * openings.size(), threads, [&](size_t k, int t) {
                auto* dev = static_cast<MinMaxNNUE_DevPlayer*>(players[t].get());
                const size_t i = k / 2;
                const bool a_black = k % 2 == 0;
                Yolah y = after(openings[i]);
                dev->clear_table();
                while (!y.game_over()) {
                    const bool a_to_move = (y.current_player() == Yolah::BLACK) == a_black;
                    dev->clear_table();   // one player, two depths: no table shared between them
                    y.play(dev->search(y, uint8_t(a_to_move ? depth : depth_b),
↪  1'000'000'000'000ull).move);
                }
                const auto [b, w] = y.score();
                res[i][k % 2] = b > w ? 1.0 : (b < w ? 0.0 : 0.5);
            });
            return res;
        };
        auto report_pairs = [&](const string& name, const vector<std::array<double, 2>>& res) {
            int colour = 0, a_both = 0, b_both = 0;
            double sa = 0;
            for (auto [ab, aw] : res) {   // A's results: ab with Black, 1 − aw with White
                sa += ab + (1 - aw);
                colour += ab == 1 && aw == 1;
                colour += ab == 0 && aw == 0;
                a_both += ab == 1 && aw == 0;
                b_both += ab == 0 && aw == 1;
            }
            const double n = 2.0 * res.size(), s_a = sa / n;
            const double elo = 400 * std::log10(std::clamp(s_a, 1e-3, 1 - 1e-3) / (1 -
             ↪  std::clamp(s_a, 1e-3, 1 - 1e-3)));
            cout << std::format("{:>22}: {} pairs; colour decides (same colour wins both) {:.1f}
↪  %, A wins both {:.1f} %, "
                                "B wins both {:.1f} %; A's score {:.3f} (Elo {:+.0f})\n",
                                name, res.size(), 100.0 * colour / res.size(), 100.0 * a_both /
↪  res.size(),
                                100.0 * b_both / res.size(), s_a, elo);
        };
        cout << "A = dev player at depth " << depth << ", B = at depth " << depth_b << ", each
↪  opening with both colours\n";
```

```
        report_pairs("book", pairs(book));
        if (random_plies > 0) report_pairs(std::format("random {}-ply openings", random_plies),
        ↪  pairs(randoms));
        return 0;
    }
    cout << "dev player against itself at depth " << depth << ", one game per opening\n";
    const vector<double> res = play_all(book);
    report("book", res);
    if (kv.contains("csv")) {   // value of each opening and Black's result: the calibration of
    ↪  build --target
        std::ofstream csv(get(kv, "csv", ""));
        csv << "value,black_result\n";
        for (size_t i = 0; i < res.size(); i++) csv << book_values[i] << ',' << res[i] << '\n';
    }
    if (random_plies > 0) report(std::format("random {}-ply openings", random_plies).c_str(),
    ↪  play_all(randoms));
    return 0;
}

} // namespace

int main(int argc, char* argv[]) {
    magic::init();
    zobrist::init();
    if (argc < 2) { cerr << "usage: opening_book build|check [--options]  (see
    ↪  test/opening_book_main.cpp)\n"; return 2; }
    const auto kv = parse_args(argc, argv, 2);
    const string cmd = argv[1];
    if (cmd == "build") return build(kv);
    if (cmd == "check") return check(kv);
    cerr << "unknown command " << cmd << '\n';
    return 2;
}
```

12.5.5 `nnue/alphazero_learn.py`

969 lines.

The trainer, which is also the orchestrator: it starts the self-play processes and the evaluation thread, trains, exports and checkpoints. Read `main` last: it is the glue between the classes above it.

symbol	role
`SAMPLE_DTYPE`	numpy mirror of `az::TrainingSample`
`symmetry_tables`	the 8 plane permutations and the 8 move-index maps — Section 6.4.6
`ReplayWindow`	ring buffer + KataGo's window formula; reads new files
`BatchMaker`	rows → input planes (on the GPU), sparse π, z, with a random symmetry per row
`export_model` / `publish_latest`	write `.pt` + `.ts`, then point `latest.json` at them
`State`	`state.json`: counters, exports, pending and done evaluations, Elo
`elo_from_score`	score → Elo difference with a half-game prior
`SelfPlayPool`	one self-play process per GPU, restarted if it dies

symbol	role
`Evaluator`	thread running the queued matches, appending to `evals.csv`
`Evaluator.ensure_ts`	re-trace a pruned `.ts` from its `.pt` when an old network is needed again
`cleanup_models` (in `main`)	disk budget: drop unevaluated exports, keep the `.pt` of evaluated ones
`main`	setup, resume, the training loop, the clean shutdown

```
"""
alphazero_learn.py — AlphaZero / KataGo-style reinforcement learning for the
value/policy ResNet of the AlphaZero MCTS player.

One process drives the whole loop on one node (see alphazero_learn.sh):

  ┌──────────────┐ samples ┌──────────────────────┐ export ┌────────────────┐
  │ self-play    │────────▶│ trainer (this file)  │───────▶│ models/az_*.ts │
  │ C++, 1/GPU   │  .bin   │ replay window, SGD   │        │ latest.json    │
  └──────────────┘         └──────────────────────┘        └────────┬───────┘
        ▲  hot-swaps the newest network (no gating, like KataGo)     │
        └────────────────────────────────────────────────────────────┘
                           every --eval-hours (or --eval-every exports):
                           `alphazero_learn match` new vs old (10 games,
                           2 s per move, colours alternated) → evals.csv

Everything lives in the work directory and survives a job that is killed:

  <work>/checkpoint.pt      trainer state (weights, EMA, optimizer, counters)
  <work>/state.json         human-readable counters + evaluation state (Elo)
  <work>/latest.json        {"step": s, "ts": "models/az_<s>.ts"} → self-play
  <work>/models/            az_<step>.pt (plain state_dict, resnet_export.py
                            can read it) and az_<step>.ts (TorchScript)
  <work>/selfplay/*.bin     training samples (format: player/alphazero_selfplay.h)
  <work>/evals.csv          one line per evaluation match
  <work>/evals/*.json       full match records (every move of every game)
  <work>/train_log.csv      losses, throughput
  <work>/logs/              self-play / match output

Resubmitting the job with the same work directory continues where it stopped.

What comes from KataGo (github.com/lightvector/KataGo, python/train.py,
python/shuffle.py and the paper "Accelerating Self-Play Learning in Go"):
  • playout cap randomization, forced playouts + policy target pruning and
    graph search on the self-play side (C++, alphazero_mcts.cpp);
  • only the full searches become training rows;
  • the replay window grows sub-linearly with the data generated,
        window = N0 · (1 + β·((N/N0)^α − 1)/α),  α = 0.75, β = 0.4
    (shuffle.py), so early, weak data ages out quickly;
  • a fixed ratio of training samples per generated row (--reuse);
  • the exported network is an average of recent weights (KataGo uses SWA
    snapshots; here an exponential moving average, BatchNorm statistics
    included);
  • no gating: self-play always uses the newest network. The evaluation
    matches only monitor progress.
  • the 8 symmetries of the board as data augmentation.
  • optionally (--aux, alphazero_aux.py), KataGo's auxiliary heads, adapted
    to Yolah: future ownership of each square and the rest of the score
    margin. They exist only in training; the exported network is the plain
    two-headed one, so the C++ side is unchanged. Without --aux, nothing of
```

```
    this runs and the files keep their original format.
Not ported: the opponent-policy head and KataGo's short-term value targets.

Usage (from nnue/, inside the .sif or locally):

  python3 alphazero_learn.py --work /work --init-model cnn_resnet_256x30_value_policy.pt \
          --bin ../build/alphazero_learn
"""
import argparse
import copy
import csv
import glob
import json
import math
import os
import signal
import socket
import subprocess
import sys
import threading
import time
from collections import deque

import numpy as np
import torch
import torch.nn.functional as F

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.append(os.path.join(HERE, "../server"))
sys.path.append(HERE)
from cnn_resnet_value_policy_chunked import net_from_state_dict  # noqa: E402

# ── sample format: MUST match az::TrainingSample (player/alphazero_selfplay.h) ──
MAX_NB_MOVES = 75
SAMPLE_DTYPE = np.dtype([
    ("black", "<u8"), ("white", "<u8"), ("empty", "<u8"),
    ("turn", "u1"), ("z", "i1"), ("nb_moves", "u1"), ("flags", "u1"),
    ("root_q", "<f4"), ("model_step", "<u4"),
    ("action", "<u2", (MAX_NB_MOVES,)), ("prob", "<u2", (MAX_NB_MOVES,)),
])
assert SAMPLE_DTYPE.itemsize == 336
# Version-2 rows (--aux, az::TrainingSampleAux): the same fields, then the
# future ownership map, 2 bits per square (see alphazero_aux.py).
SAMPLE_DTYPE_AUX = np.dtype(SAMPLE_DTYPE.descr + [("own", "u1", (16,))])
assert SAMPLE_DTYPE_AUX.itemsize == 352
ROW_DTYPES = {1: SAMPLE_DTYPE, 2: SAMPLE_DTYPE_AUX}      # file version → row
HEADER_DTYPE = np.dtype([("magic", "S8"), ("version", "<u4"), ("sample_size", "<u4"),
                         ("nb_samples", "<u4"), ("nb_games", "<u4")])
assert HEADER_DTYPE.itemsize == 24


def log(msg):
    print(f"[{time.strftime('%F %T')}] {msg}", flush=True)


def read_header(path):
    """(rows, games, version) of a self-play file; version 1 or 2 (--aux rows)."""
    h = np.fromfile(path, dtype=HEADER_DTYPE, count=1)
    if len(h) != 1 or h[0]["magic"] != b"YOLAHSP1":
        raise ValueError(f"{path}: not a self-play sample file")
    version = int(h[0]["version"])
    if version not in ROW_DTYPES or h[0]["sample_size"] != ROW_DTYPES[version].itemsize:
        raise ValueError(f"{path}: unknown row format (version {version})")
```

```
    return int(h[0]["nb_samples"]), int(h[0]["nb_games"]), version


def read_samples(path, dtype=SAMPLE_DTYPE):
    """
    The rows of a file, converted to `dtype`. Both formats mix freely:
      • version-2 rows read as SAMPLE_DTYPE drop their ownership map;
      • version-1 rows read as SAMPLE_DTYPE_AUX get a map of 0xFF bytes, i.e.
        every square OWN_PAST: no auxiliary target, masked in the loss.
    """
    n, _, version = read_header(path)
    rows = np.fromfile(path, dtype=ROW_DTYPES[version], count=n, offset=HEADER_DTYPE.itemsize)
    if rows.dtype == dtype:
        return rows
    out = np.empty(n, dtype=dtype)
    for name in SAMPLE_DTYPE.names:
        out[name] = rows[name]
    if "own" in dtype.names:
        out["own"] = 0xFF
    return out


def atomic_write_text(path, text):
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        f.write(text)
    os.replace(tmp, path)


# ── symmetries ──────────────────────────────────────────────────────────
def symmetry_tables():
    """
    The 8 symmetries of the board (same group as distill_common.symmetries).

    Returns
      src (8, 64): new plane cell j takes the value of old cell src[s, j]
      act (8, 4096): policy index of the image of move index a under symmetry s
    Plane cell c holds square 63 - c (see nn_evaluator.h), so a square maps
    through cells: sq' = 63 - inv[63 - sq]. Index 0 (a1a1) is the pass and
    stays the pass.
    """
    cells = torch.arange(64).view(8, 8)
    srcs = []
    for k in range(4):
        r = torch.rot90(cells, k, dims=(0, 1))
        srcs.append(r.flatten())
        srcs.append(torch.flip(r, dims=(1,)).flatten())
    src = torch.stack(srcs)
    inv = torch.argsort(src, dim=1)
    sq = 63 - inv[:, 63 - torch.arange(64)]
    act = (sq[:, :, None] * 64 + sq[:, None, :]).reshape(8, 4096).clone()
    act[:, 0] = 0
    return src, act


# ── replay window ───────────────────────────────────────────────────────
class ReplayWindow:
    """
    Ring buffer of the most recent rows, filled from <work>/selfplay/*.bin in
    file-name (= creation time) order. Sampling is uniform over the last
    window_size(total) rows.
    """
    def __init__(self, directory, capacity, min_rows, alpha=0.75, beta=0.4, dtype=SAMPLE_DTYPE):
```

```
        self.dir = directory
        self.dtype = dtype
        self.capacity = capacity
        self.min_rows = min_rows
        self.alpha, self.beta = alpha, beta
        self.buf = np.zeros(capacity, dtype=dtype)
        self.head = 0          # next write position
        self.filled = 0
        self.total = 0         # rows generated since the beginning (all files)
        self.games = 0
        self.known = set()

    def window_size(self):
        """KataGo's W(N) (shuffle.py), bounded by what the buffer holds."""
        n0, n = self.min_rows, self.total
        if n <= n0:
            w = n
        else:
            w = n0 * (1 + self.beta * ((n / n0) ** self.alpha - 1) / self.alpha)
        return int(min(w, self.filled, self.capacity))

    def _append(self, rows):
        """Copy rows in at the head, wrapping around (overwriting the oldest)."""
        rows = rows[-self.capacity:]
        n = len(rows)
        first = min(n, self.capacity - self.head)
        self.buf[self.head:self.head + first] = rows[:first]
        if n > first:
            self.buf[:n - first] = rows[first:]
        self.head = (self.head + n) % self.capacity
        self.filled = min(self.capacity, self.filled + n)

    def scan(self):
        """Pick up new files; returns the number of new rows."""
        files = sorted(f for f in glob.glob(os.path.join(self.dir, "*.bin"))
                       if os.path.basename(f) not in self.known)
        if not files:
            return 0
        headers = []
        for f in files:
            try:
                headers.append((f, *read_header(f)))
            except (ValueError, OSError) as e:
                log(f"skipping {f}: {e}")
                self.known.add(os.path.basename(f))
        # Only the rows that can still be in the buffer need to be read (on a
        # restart that is the tail of a possibly huge history).
        need, start = 0, len(headers)
        while start > 0 and need < self.capacity:
            start -= 1
            need += headers[start][1]
        new_rows = 0
        for i, (f, n, g, _version) in enumerate(headers):
            self.known.add(os.path.basename(f))
            self.total += n
            self.games += g
            new_rows += n
            if i >= start:
                self._append(read_samples(f, self.dtype))
        return new_rows

    def sample(self, batch_size, rng):
        w = self.window_size()
```

```
        back = rng.integers(1, w + 1, size=batch_size)
        return self.buf[(self.head - back) % self.capacity]


# ── batches ──────────────────────────────────────────────────────────────
class BatchMaker:
    def __init__(self, device, aux=False, q_weight=0.0):
        self.device = device
        self.aux = aux                  # also return the ownership targets
        # Value target = (1 − w)·z + w·q: the game result blended with the
        # search's own value of the position (root_q). 0 = the result alone.
        self.q_weight = q_weight
        src, act = symmetry_tables()
        self.src = src.to(device)
        self.act = act.to(device)
        self.shifts = (63 - torch.arange(64, device=device)).view(1, 1, 64)

    def __call__(self, rows, augment=True):
        """
        rows (numpy, SAMPLE_DTYPE) → (x, z, actions, probs) on the device:
          x       (B, 4, 8, 8) float, channels-last — the network input
          z       (B,)  float — value target for the side to move: the game
                  result, blended with the search value root_q if q_weight > 0
          actions (B, 75) long — policy indices of the legal moves (0-padded)
          probs   (B, 75) float — π over those moves (0 on the padding)
        and with aux=True, a fifth element:
          own     (B, 64) long — future ownership codes, in plane cell order
        The planes are decoded from the bitboards on the GPU (only 25 bytes
        per row cross the bus instead of 1 KB of floats).
        """
        B = len(rows)
        dev = self.device
        # uint64 bitboards viewed as int64: `>>` then sign-extends, but `& 1`
        # keeps only the bit asked for, so the sign never matters.
        bbs = np.stack([rows["black"], rows["white"], rows["empty"]], axis=1).view(np.int64)
        bbs = torch.from_numpy(bbs).to(dev, non_blocking=True)                # (B, 3)
        # Cell i of a plane is bit 63 - i (see nn_evaluator.h): shifts = 63 - i.
        planes = ((bbs[:, :, None] >> self.shifts) & 1).float()               # (B, 3, 64)
        turn = torch.from_numpy(rows["turn"].astype(np.float32)).to(dev)
        planes = torch.cat([planes, turn.view(B, 1, 1).expand(B, 1, 64)], dim=1)
        actions = torch.from_numpy(rows["action"].astype(np.int64)).to(dev)   # (B, M)
        # π was quantised to 16 bits per move: renormalise so it sums to 1.
        probs = torch.from_numpy(rows["prob"].astype(np.float32)).to(dev)
        probs = probs / probs.sum(1, keepdim=True).clamp_min(1.0)
        z = torch.from_numpy(rows["z"].astype(np.float32)).to(dev)
        if self.q_weight > 0:
            # root_q is the value the full search found for this position,
            # for the side to move — the same convention as z and as the
            # value head. It is invariant under the board symmetries.
            q = torch.from_numpy(rows["root_q"].astype(np.float32)).to(dev).clamp(-1.0, 1.0)
            z = (1.0 - self.q_weight) * z + self.q_weight * q
        own = None
        if self.aux:
            from alphazero_aux import decode_ownership
            own = decode_ownership(torch.from_numpy(rows["own"]).to(dev))       # (B, 64)
        if augment:
            # One random symmetry per row: permute the 64 cells of every plane
            # (one gather) and map every move index through the same symmetry
            # (one table lookup). z is invariant. The ownership map is a plane
            # like the others: the same gather; the margin it sums to is invariant.
            sym = torch.randint(0, 8, (B,), device=dev)
            planes = planes.gather(2, self.src[sym].unsqueeze(1).expand(B, 4, 64))
            actions = self.act[sym.unsqueeze(1), actions]
```

```
            if own is not None:
                own = own.gather(1, self.src[sym])
        x = planes.view(B, 4, 8, 8).contiguous(memory_format=torch.channels_last)
        if self.aux:
            return x, z, actions, probs, own
        return x, z, actions, probs


# ── model files ──────────────────────────────────────────────────────
def load_state_dict(path):
    """A state_dict from a plain .pt, a DDP one ("module." prefix) or our checkpoint.pt."""
    with open(path, "rb") as f:
        if f.read(100).startswith(b"version https://git-lfs"):
            sys.exit(f"{path} is a Git LFS pointer, not a network: copy the real file "
                     f"(the repository stores it with Git LFS)")
    sd = torch.load(path, map_location="cpu")
    if isinstance(sd, dict) and "model" in sd and "optimizer" in sd:
        sd = sd["model"]
    return {(k[7:] if k.startswith("module.") else k): v for k, v in sd.items()}


def make_net(sd):
    """
    Build a Net whose sizes (channels, blocks, heads, global pooling) are read off the
    state_dict. The parameters of the auxiliary heads (--aux runs) are not
    part of a Net and are ignored.
    """
    from alphazero_aux import base_state_dict
    sd = base_state_dict(sd)
    return net_from_state_dict(sd)[0]


def model_paths(work, step):
    """(<work>/models/az_<step>.pt, .ts) — step 0 is the initial network."""
    base = os.path.join(work, "models", f"az_{step:08d}")
    return base + ".pt", base + ".ts"


def export_model(net, work, step):
    """
    Write az_<step>.pt (state_dict) + az_<step>.ts (TorchScript) atomically.
    An AuxNet is exported as the plain Net it contains (value + policy only):
    the files have the same format with or without --aux, and the C++
    backends never see the auxiliary heads.
    """
    if hasattr(net, "forward_all"):
        net = make_net(net.state_dict())
    pt, ts = model_paths(work, step)
    cpu = copy.deepcopy(net).float().cpu().eval()
    torch.save(cpu.state_dict(), pt + ".tmp")
    os.replace(pt + ".tmp", pt)
    with torch.no_grad():
        traced = torch.jit.trace(cpu, torch.zeros(1, 4, 8, 8))
    traced.save(ts + ".tmp")
    os.replace(ts + ".tmp", ts)
    return pt, ts


def publish_latest(work, step, aux=False):
    """
    Point self-play at az_<step>.ts (relative path: the work dir may be
    bind-mounted elsewhere). With aux, also ask it to record the auxiliary
    targets ("aux": true); without, the file is exactly as before.
```

```
    """
    _, ts = model_paths(work, step)
    latest = {"step": step, "ts": os.path.relpath(ts, work)}
    if aux:
        latest["aux"] = True
    atomic_write_text(os.path.join(work, "latest.json"), json.dumps(latest) + "\n")


# ── state shared with the evaluator thread ──────────────────────────────
class State:
    """Persistent counters + evaluation bookkeeping (state.json)."""
    def __init__(self, work):
        self.path = os.path.join(work, "state.json")
        self.lock = threading.Lock()
        self.d = {"step": 0, "samples_trained": 0, "exports": [0], "pending_evals": [],
                  "evaluated": [0], "elo": {"0": 0.0}}
        if os.path.exists(self.path):
            with open(self.path) as f:
                self.d.update(json.load(f))

    def save(self):
        with self.lock:
            atomic_write_text(self.path, json.dumps(self.d, indent=1) + "\n")


def elo_from_score(w, d, l):
    """Elo difference from a match result, with a +1/2 prior so 10-0 stays finite."""
    n = w + d + l
    s = (w + 0.5 * d + 0.5) / (n + 1)
    return 400.0 * math.log10(s / (1.0 - s))


# ── subprocesses ────────────────────────────────────────────────────────
class SelfPlayPool:
    """One `alphazero_learn selfplay` per GPU, restarted if it dies."""
    def __init__(self, args, gpus):
        self.args = args
        self.gpus = gpus
        self.procs = {}
        self.logs = {}
        self.next_start = {}

    def _start(self, gpu):
        a = self.args
        # Unique across jobs: two jobs on one node both see their GPU as "0".
        job = os.environ.get("SLURM_JOB_ID", str(os.getpid()))
        tag = f"{socket.gethostname()}_{job}_gpu{gpu}"
        cmd = [a.bin, "selfplay", "--config", a.selfplay_config, "--work", a.work,
               "--games", str(a.selfplay_games), "--tag", tag]
        for kv in a.selfplay_set:
            cmd += ["--set", kv]
        env = dict(os.environ, CUDA_VISIBLE_DEVICES=gpu)
        logf = open(os.path.join(a.work, "logs", f"selfplay_gpu{gpu}.log"), "a")
        log(f"starting self-play on GPU {gpu}: {' '.join(cmd)}")
        self.procs[gpu] = subprocess.Popen(cmd, env=env, stdout=logf, stderr=subprocess.STDOUT)
        self.logs[gpu] = logf

    def start(self):
        for g in self.gpus:
            self._start(g)

    def check(self):
        now = time.time()
```

```
        for g, p in list(self.procs.items()):
            if p.poll() is None:
                continue
            if g not in self.next_start:
                log(f"self-play on GPU {g} exited with code {p.returncode}; restarting in 60 s "
                    f"(see logs/selfplay_gpu{g}.log)")
                self.next_start[g] = now + 60
            elif now >= self.next_start.pop(g):
                self.logs[g].close()
                self._start(g)

    def stop(self):
        for p in self.procs.values():
            if p.poll() is None:
                p.send_signal(signal.SIGTERM)
        for g, p in self.procs.items():
            try:
                p.wait(timeout=120)     # finishes its current searches and writes finished games
            except subprocess.TimeoutExpired:
                p.kill()
            self.logs[g].close()


class Evaluator(threading.Thread):
    """Plays the queued evaluation matches one after the other."""
    CSV_FIELDS = ["time", "model_step", "opponent_step", "games", "wins", "draws", "losses",
                  "score", "elo_diff", "model_elo", "samples_trained", "rows_generated", "hours"]

    def __init__(self, args, state, gpu, stop_event, progress):
        super().__init__(daemon=True)
        self.args, self.state, self.gpu = args, state, gpu
        self.stop_event = stop_event
        self.progress = progress         # callable → (samples_trained, rows_generated, hours)
        self.proc = None
        self.wake = threading.Event()

    def opponents(self, step):
        with self.state.lock:
            evaluated = [s for s in self.state.d["evaluated"] if s < step]
        res = []
        for o in self.args.eval_opponents:
            if o == "previous" and evaluated:
                res.append(evaluated[-1])
            elif o == "initial":
                res.append(0)
            elif o.startswith("lag:") and evaluated:
                res.append(evaluated[max(0, len(evaluated) - int(o[4:]))])
        return sorted(set(res), reverse=True)

    def ensure_ts(self, step):
        """The TorchScript file of `step`, re-traced from its .pt if it was pruned."""
        pt, ts = model_paths(self.args.work, step)
        if not os.path.exists(ts) and os.path.exists(pt):
            export_model(make_net(load_state_dict(pt)), self.args.work, step)
        return ts

    def run_match(self, step, opp):
        a = self.args
        ts_a = self.ensure_ts(step)
        ts_b = self.ensure_ts(opp)
        out = os.path.join(a.work, "evals", f"az_{step:08d}_vs_az_{opp:08d}.json")
        cmd = [a.bin, "match", "--config", a.eval_config, "--a", ts_a, "--b", ts_b,
               "--games", str(a.eval_games), "--time", str(int(a.eval_seconds * 1e6)),
```

```
                "--opening-plies", str(a.eval_opening_plies), "--seed", str(step + 1),
                "--out", out] + (["--book", a.eval_book] if a.eval_book else [])
        env = dict(os.environ, CUDA_VISIBLE_DEVICES=self.gpu)
        log(f"evaluation: az_{step} vs az_{opp} ({a.eval_games} games, {a.eval_seconds} s/move)")
        with open(os.path.join(a.work, "logs", "eval.log"), "a") as logf:
            self.proc = subprocess.Popen(cmd, env=env, stdout=logf, stderr=subprocess.STDOUT)
            rc = self.proc.wait()
        self.proc = None
        if rc != 0 or not os.path.exists(out):
            return None
        with open(out) as f:
            return json.load(f)

    def evaluate(self, step):
        results = []
        for opp in self.opponents(step):
            r = self.run_match(step, opp)
            if self.stop_event.is_set():
                return False            # interrupted: stays pending
            if r is None:
                log(f"evaluation az_{step} vs az_{opp} FAILED (see logs/eval.log)")
                continue
            results.append((opp, r))
        with self.state.lock:
            elo = self.state.d["elo"]
            for opp, r in results:
                diff = elo_from_score(r["wins"], r["draws"], r["losses"])
                if str(step) not in elo and str(opp) in elo:
                    elo[str(step)] = elo[str(opp)] + diff   # Elo chained through the first
↪ opponent
                trained, rows, hours = self.progress()
                row = {"time": time.strftime("%F %T"), "model_step": step, "opponent_step": opp,
                       "games": r["games"], "wins": r["wins"], "draws": r["draws"], "losses":
                        ↪ r["losses"],
                       "score": f"{r['score']:.3f}", "elo_diff": f"{diff:+.0f}",
                       "model_elo": f"{elo.get(str(step), float('nan')):+.0f}",
                       "samples_trained": trained, "rows_generated": rows, "hours":
                        ↪ f"{hours:.2f}"}
                path = os.path.join(self.args.work, "evals.csv")
                new = not os.path.exists(path)
                with open(path, "a", newline="") as f:
                    w = csv.DictWriter(f, fieldnames=self.CSV_FIELDS)
                    if new:
                        w.writeheader()
                    w.writerow(row)
                log(f"EVAL az_{step} vs az_{opp}: {r['wins']}W {r['draws']}D {r['losses']}L "
                    f"score {r['score']:.2f}  Elo {diff:+.0f}  (az_{step} ≈ {row['model_elo']} vs
                     ↪ initial)")
            if results:
                self.state.d["evaluated"].append(step)
            self.state.d["pending_evals"].remove(step)
        self.state.save()
        return True

    def run(self):
        while not self.stop_event.is_set():
            with self.state.lock:
                pending = list(self.state.d["pending_evals"])
            if not pending:
                self.wake.wait(30)
                self.wake.clear()
                continue
            self.evaluate(pending[0])
```

```
551
552     def stop(self):
553         if self.proc is not None and self.proc.poll() is None:
554             self.proc.terminate()
555
556
557 # ── trainer ─────────────────────────────────────────────────────────
558 def main():
559     """
560     Setup (arguments, signals, GPUs), resume or start (checkpoint, initial
561     export), start the self-play processes and the evaluator thread, then the
562     training loop until a signal or --max-hours, then a clean shutdown.
563     """
564     ap = argparse.ArgumentParser(description=__doc__,
    ↪  formatter_class=argparse.RawDescriptionHelpFormatter)
565     ap.add_argument("--work", required=True, help="work directory (everything is kept there)")
566     ap.add_argument("--init-model", default=os.path.join(HERE,
    ↪  "cnn_resnet_256x30_value_policy.pt"),
567                     help="starting network (used only when the work directory is new)")
568     ap.add_argument("--bin", default=os.path.join(HERE, "../build/alphazero_learn"))
569     ap.add_argument("--selfplay-config", default=os.path.join(HERE,
    ↪  "../config/alphazero_mcts_selfplay_player.cfg"))
570     ap.add_argument("--eval-config", default=os.path.join(HERE,
    ↪  "../config/alphazero_mcts_eval_player.cfg"))
571     ap.add_argument("--selfplay-games", type=int, default=128, help="concurrent games per
    ↪  self-play process")
572     ap.add_argument("--selfplay-set", action="append", default=[], help="'key=value' override for
    ↪  self-play")
573     ap.add_argument("--no-selfplay", action="store_true", help="train only (self-play runs
    ↪  elsewhere)")
574     # optimisation
575     ap.add_argument("--batch-size", type=int, default=1024)
576     ap.add_argument("--lr", type=float, default=1e-4)
577     ap.add_argument("--weight-decay", type=float, default=1e-4)
578     ap.add_argument("--warmup-steps", type=int, default=500)
579     ap.add_argument("--ema-decay", type=float, default=0.995, help="0 = export the raw weights")
580     ap.add_argument("--value-weight", type=float, default=1.0)
581     ap.add_argument("--policy-weight", type=float, default=1.0)
582     ap.add_argument("--reuse", type=float, default=4.0, help="training samples per generated row")
583     ap.add_argument("--q-weight", type=float, default=0.0,
584                     help="value target = (1-w)·z + w·q, q = the search value of the position (0 =
                         ↪  z alone)")
585     # data
586     ap.add_argument("--min-rows", type=int, default=100_000, help="rows before training starts
    ↪  (window N0)")
587     ap.add_argument("--max-window", type=int, default=4_000_000)
588     # cadence
589     ap.add_argument("--export-every", type=int, default=131_072, help="training samples between
    ↪  exports")
590     ap.add_argument("--eval-hours", type=float, default=4.0,
591                     help="evaluate the newest network when this long has passed since the last
                         ↪  one")
592     ap.add_argument("--eval-every", type=int, default=0,
593                     help="evaluate every N exports instead (0 = use --eval-hours)")
594     ap.add_argument("--eval-games", type=int, default=20)
595     ap.add_argument("--eval-seconds", type=float, default=2.0)
596     ap.add_argument("--eval-opening-plies", type=int, default=4)
597     ap.add_argument("--eval-book", default="",
598                     help="book of balanced openings for the evaluations
                         ↪  (config/openings/balanced.txt); "
599                          "empty: random openings of --eval-opening-plies plies")
600     ap.add_argument("--eval-opponents", default="previous",
601                     help="comma separated: previous (last evaluated model), initial, lag:K")
```

```
    ap.add_argument("--keep-all", action="store_true", help="keep every exported model")
    ap.add_argument("--inherit-warmup", type=int, default=0,
                    help="new run whose selfplay/ already holds the rows of an earlier run (a grown "
                         "network, alphazero_grow.py): train this many samples on them at once, then "
                         "keep the usual pace, as if the run went on (default 0: no burst)")
    ap.add_argument("--max-hours", type=float, default=0, help="stop cleanly after this long (0 = never)")
    ap.add_argument("--train-gpu", type=int, default=0, help="index among the visible GPUs")
    ap.add_argument("--eval-gpu", type=int, default=-1,
                    help="index among the visible GPUs (-1 = the last one: training uses the first)")
    ap.add_argument("--allow-cpu", action="store_true", help="run even without CUDA (tests only)")
    ap.add_argument("--amp", choices=["auto", "bf16", "fp16"], default="auto",
                    help="mixed precision of the trainer (auto: bf16 on Ampere+, fp16 before)")
    # KataGo's auxiliary heads (alphazero_aux.py). Off: everything as before.
    ap.add_argument("--aux", action="store_true",
                    help="train the ownership and score-margin heads (self-play records their targets)")
    ap.add_argument("--own-weight", type=float, default=1.5,
                    help="weight of the ownership loss (KataGo: 1.5/b² on the sum over the board)")
    ap.add_argument("--score-weight", type=float, default=0.02, help="weight of the margin cross-entropy")
    ap.add_argument("--score-cdf-weight", type=float, default=0.02, help="weight of the margin CDF loss")
    ap.add_argument("--aux-ramp-steps", type=int, default=2000,
                    help="the auxiliary weights grow linearly from 0 over this many steps")
    args = ap.parse_args()
    args.work = os.path.abspath(args.work)
    args.eval_opponents = [o.strip() for o in args.eval_opponents.split(",") if o.strip()]
    for d in ("models", "selfplay", "evals", "logs"):
        os.makedirs(os.path.join(args.work, d), exist_ok=True)

    t_start = time.time()
    stop_event = threading.Event()

    def on_signal(signum, _frame):
        log(f"signal {signum}: stopping cleanly")
        stop_event.set()
    for s in (signal.SIGTERM, signal.SIGINT, signal.SIGUSR1):
        signal.signal(s, on_signal)

    cvd = os.environ.get("CUDA_VISIBLE_DEVICES", "").strip()
    gpus = [g.strip() for g in cvd.split(",") if g.strip()] if cvd else \
           [str(i) for i in range(max(1, torch.cuda.device_count()))]
    # No GPU is almost always a container problem (driver not bound): the
    # self-play backend would silently fall back to the CPU and run ~50x
    # slower for two weeks. Refuse instead.
    if not torch.cuda.is_available() and not args.allow_cpu:
        sys.exit("alphazero_learn: CUDA is not available (use --allow-cpu to run anyway)")
    device = torch.device(f"cuda:{args.train_gpu}" if torch.cuda.is_available() else "cpu")
    log(f"trainer on {device}" + (f" ({torch.cuda.get_device_name(device)})" if device.type == "cuda" else ""))
    if device.type == "cuda":
        torch.cuda.set_device(device)
    torch.backends.cudnn.benchmark = True
    torch.set_float32_matmul_precision("high")

    # ── model / optimizer / checkpoint ──
    state = State(args.work)
    ckpt_path = os.path.join(args.work, "checkpoint.pt")
```

```
    ckpt = torch.load(ckpt_path, map_location="cpu") if os.path.exists(ckpt_path) else None
    if ckpt is None:
        log(f"new run: initial network {args.init_model}")
        src_sd = load_state_dict(args.init_model)
    else:
        src_sd = ckpt["model"]
    # Variant: with --aux the network carries the auxiliary heads. A run can
    # switch between the variants from one job to the next: the shared
    # parameters are kept, the auxiliary heads start from scratch (or are
    # dropped), and the optimizer — whose state is per parameter — restarts.
    ckpt_aux = bool(ckpt.get("aux", False)) if ckpt is not None else False
    switched = ckpt is not None and ckpt_aux != args.aux
    if args.aux:
        from alphazero_aux import make_aux_net, aux_losses
        net, fresh_heads = make_aux_net(src_sd)
        log("auxiliary heads (ownership, score margin): " + ("new" if fresh_heads else "resumed"))
    else:
        net = make_net(src_sd)
    net = net.to(device).to(memory_format=torch.channels_last)
    ema = None
    if args.ema_decay > 0:
        ema = torch.optim.swa_utils.AveragedModel(
            net, multi_avg_fn=torch.optim.swa_utils.get_ema_multi_avg_fn(args.ema_decay),
    ↪ use_buffers=True)
    opt = torch.optim.Adam(net.parameters(), lr=args.lr, weight_decay=args.weight_decay)
    # Mixed precision: bf16 on Ampere and newer (A40), fp16 + gradient scaler
    # on Turing (RTX 8000), whose bf16 is emulated and slow. The cluster has
    # both, so consecutive jobs of one run may use either.
    if args.amp == "auto":
        cc_major = torch.cuda.get_device_capability(device)[0] if device.type == "cuda" else 0
        amp_dtype = torch.bfloat16 if cc_major >= 8 else torch.float16
    else:
        amp_dtype = torch.bfloat16 if args.amp == "bf16" else torch.float16
    log(f"mixed precision: {amp_dtype}")
    scaler = torch.amp.GradScaler("cuda", enabled=(device.type == "cuda" and amp_dtype ==
    ↪ torch.float16))
    if ckpt is not None and switched:
        # Different parameter set: optimizer, EMA and scaler start afresh
        # (the EMA from the current weights), with a new learning-rate warm-up.
        state.d["opt_start_step"] = ckpt["step"]
        if args.aux:
            state.d["aux_start_step"] = ckpt["step"]
        log(f"variant switch: {'with' if args.aux else 'without'} auxiliary heads from step
    ↪ {ckpt['step']} "
            f"(optimizer and weight average restart)")
    if ckpt is not None:
        if not switched:
            if ema is not None and ckpt.get("ema") is not None:
                ema.load_state_dict(ckpt["ema"])
            opt.load_state_dict(ckpt["optimizer"])
        # A bf16 job saves the state of a *disabled* scaler, which is empty
        # and which an enabled scaler refuses to load: an fp16 job resuming
        # it (A40 then RTX 8000) starts with a fresh scale — harmless, the
        # scale adapts within a few steps.
        if scaler.is_enabled() and ckpt.get("scaler") and not switched:
            scaler.load_state_dict(ckpt["scaler"])
        # The checkpoint is authoritative: state.json may have been saved by
        # the evaluator after it, with counters the lost steps never kept.
        state.d["step"] = ckpt["step"]
        state.d["samples_trained"] = ckpt["samples_trained"]
        log(f"resumed at step {state.d['step']}, {state.d['samples_trained']:,} samples trained")
    else:
        # A new run. A state.json without a checkpoint is a leftover of an
```

```
        # earlier run (e.g. a work directory emptied while its job was still
        # stopping, which then saved its state): its counters and its list of
        # exported networks would point self-play at networks that no longer
        # exist. Start from fresh counters.
        if os.path.exists(state.path):
            log(f"new run: ignoring the leftover {state.path} (no checkpoint.pt)")
            os.replace(state.path, state.path + ".stale")
            state = State(args.work)
        pt0, _ = model_paths(args.work, 0)
        if not os.path.exists(pt0):
            export_model(net, args.work, 0)
        publish_latest(args.work, 0, args.aux)
        state.save()
    # Always re-published at start: a run that switches variant must tell
    # self-play at once whether to record the auxiliary targets.
    publish_latest(args.work, state.d["exports"][-1], args.aux)

    def save_checkpoint():
        torch.save({"model": net.state_dict(), "ema": ema.state_dict() if ema is not None else
↪   None,
                    "optimizer": opt.state_dict(), "scaler": scaler.state_dict(),
                    "step": state.d["step"], "samples_trained": state.d["samples_trained"],
                    "aux": args.aux},
                   ckpt_path + ".tmp")
        os.replace(ckpt_path + ".tmp", ckpt_path)
        state.save()

    # ── data ──
    window = ReplayWindow(os.path.join(args.work, "selfplay"), args.max_window, args.min_rows,
                          dtype=SAMPLE_DTYPE_AUX if args.aux else SAMPLE_DTYPE)
    window.scan()
    log(f"replay: {window.total:,} rows from {window.games:,} games on disk, {window.filled:,} in
↪   memory")
    if ckpt is None and window.total > 0:
        # A new run started on the rows of an earlier one: the reuse cap
        # (samples_trained <= reuse·rows) would otherwise let the trainer
        # go through reuse× all those rows before waiting for self-play.
        # Count them as already trained on, except --inherit-warmup samples.
        done = max(0, int(args.reuse * window.total) - args.inherit_warmup)
        state.d["samples_trained"] = state.d["samples_at_last_export"] = done
        log(f"inherited rows: {args.inherit_warmup:,} training samples on them before the usual
↪   pace")
    make_batch = BatchMaker(device, aux=args.aux, q_weight=args.q_weight)
    if args.q_weight > 0:
        log(f"value target: {1 - args.q_weight:g}·z + {args.q_weight:g}·q (game result blended
↪   with the search value)")
    rng = np.random.default_rng()

    # ── subprocesses ──
    pool = None
    if not args.no_selfplay:
        pool = SelfPlayPool(args, gpus)
        pool.start()
    evaluator = Evaluator(args, state, gpus[min(args.eval_gpu, len(gpus) - 1)] if args.eval_gpu >=
↪   0 else gpus[-1],
                          stop_event,
                          lambda: (state.d["samples_trained"], window.total, (time.time() -
                           ↪  t_start) / 3600))
    evaluator.start()

    def cleanup_models():
        """
        Disk budget (a 256x30 network is 141 MB per file, and a month makes
```

```
        hundreds of exports): an export that was never evaluated disappears
        at the next export; an evaluated one keeps its .pt (the history of
        the run) but loses its .ts unless it may still be needed as an
        opponent — Evaluator.ensure_ts re-traces it from the .pt if so.
        """
        if args.keep_all:
            return
        with state.lock:
            evaluated = list(state.d["evaluated"])
            live = {0, state.d["exports"][-1], *state.d["pending_evals"]}
            keep_pt = live | set(evaluated)
            keep_ts = live | set(evaluated[-1:])
            exports = list(state.d["exports"])
        for s in exports:
            pt, ts = model_paths(args.work, s)
            if s not in keep_pt and os.path.exists(pt):
                os.remove(pt)
            if s not in keep_ts and os.path.exists(ts):
                os.remove(ts)

    log_path = os.path.join(args.work, "train_log.csv")
    new_log = not os.path.exists(log_path)
    log_file = open(log_path, "a", newline="")
    log_csv = csv.writer(log_file)
    if new_log:
        log_csv.writerow(["time", "hours", "step", "samples_trained", "rows_generated", "games",
                          "window", "value_loss", "policy_loss", "policy_kl", "lr",
                          "samples_per_s"])

    # ── main loop ──
    net.train()
    acc = {"v": 0.0, "p": 0.0, "kl": 0.0, "n": 0}
    aux_acc = {"own": 0.0, "pdf": 0.0, "cdf": 0.0, "acc": 0.0, "n": 0, "n_acc": 0}
    last_scan = last_report = time.time()
    samples_at_report = state.d["samples_trained"]
    since_export = state.d["samples_trained"] - state.d.get("samples_at_last_export", 0)
    waiting_logged = False
    last_cap_log = 0.0
    while not stop_event.is_set():
        if args.max_hours and time.time() - t_start > args.max_hours * 3600:
            log("--max-hours reached")
            break
        now = time.time()
        if now - last_scan > 20:
            window.scan()
            if pool is not None:
                pool.check()
            last_scan = now

        # Keep the trainer in step with self-play: at most `reuse` training
        # samples per generated row (samples_trained <= r·N), and nothing
        # before N0 rows exist. Ahead of the data → sleep and rescan.
        allowed = args.reuse * window.total
        if window.total < args.min_rows or state.d["samples_trained"] + args.batch_size > allowed:
            # Two different waits: before training starts (not enough rows
            # yet), and during training when the trainer has caught up with
            # the reuse cap — the normal steady state, logged only now and then.
            if window.total < args.min_rows:
                if not waiting_logged:
                    log(f"waiting for self-play data: {window.total:,} rows "
                        f"(training starts at {args.min_rows:,})")
                    waiting_logged = True
            elif time.time() - last_cap_log > 1800:
```

```
                log(f"trainer at the reuse cap ({args.reuse:g} samples per row): "
                    f"it trains as fast as self-play produces rows")
                last_cap_log = time.time()
            stop_event.wait(10)
            last_scan = 0
            continue
        waiting_logged = False

        # ── one SGD step ──
        # Linear warm-up: Adam's moment estimates start at zero after a fresh
        # start, so the first steps would otherwise be too large.
        step = state.d["step"]
        lr = args.lr * min(1.0, (step - state.d.get("opt_start_step", 0) + 1) / max(1,
    ↪ args.warmup_steps))
        for g in opt.param_groups:
            g["lr"] = lr
        batch = make_batch(window.sample(args.batch_size, rng))
        x, z, actions, probs = batch[:4]
        with torch.autocast(device.type, dtype=amp_dtype, enabled=device.type == "cuda"):
            if args.aux:
                v, logits, own_logits, score_logits = net.forward_all(x)
            else:
                v, logits = net(x)
        v = v.float()
        # Policy: cross-entropy with the soft target π, -Σ_a π(a) log p(a|s).
        # The softmax runs over the 4096 logits; π lives on the legal moves
        # only (sparse: `actions` are their indices, the padding has π = 0).
        logp = F.log_softmax(logits.float(), dim=1).gather(1, actions)
        p_loss = -(probs * logp).sum(1).mean()
        # Value: squared error against the game result z ∈ {-1, 0, +1}.
        v_loss = F.mse_loss(v, z)
        loss = args.value_weight * v_loss + args.policy_weight * p_loss
        if args.aux:
            # Auxiliary heads: ownership (per-square cross-entropy) and score
            # margin (distribution + CDF), their weights ramped up from 0 so
            # that the random new heads do not shake a trunk that already
            # plays well. Rows without targets (version 1) are masked.
            l_own, l_pdf, l_cdf, own_acc = aux_losses(own_logits, score_logits, batch[4])
            ramp = min(1.0, (step - state.d.get("aux_start_step", 0) + 1) / max(1,
    ↪ args.aux_ramp_steps))
            loss = loss + ramp * (args.own_weight * l_own + args.score_weight * l_pdf
                                  + args.score_cdf_weight * l_cdf)
            aux_acc["own"] += l_own.item(); aux_acc["pdf"] += l_pdf.item()
            aux_acc["cdf"] += l_cdf.item(); aux_acc["n"] += 1
            if own_acc == own_acc:                       # not NaN (batch had targets)
                aux_acc["acc"] += own_acc; aux_acc["n_acc"] += 1
        opt.zero_grad(set_to_none=True)
        scaler.scale(loss).backward()
        scaler.unscale_(opt)
        torch.nn.utils.clip_grad_norm_(net.parameters(), 5.0)
        scaler.step(opt)
        scaler.update()
        # θ̄ ← μ θ̄ + (1-μ) θ: the averaged weights are the ones exported.
        if ema is not None:
            ema.update_parameters(net)
        state.d["step"] = step + 1
        state.d["samples_trained"] += args.batch_size
        since_export += args.batch_size

        # KL(π‖p) = CE(π, p) − H(π): the part of the policy loss the network
        # can still reduce (0 = it already predicts what the search finds).
        with torch.no_grad():
            entropy = -(probs * torch.log(probs.clamp_min(1e-12))).sum(1).mean()
```

```
        acc["v"] += v_loss.item(); acc["p"] += p_loss.item(); acc["kl"] += (p_loss - entropy).item(); acc["n"] += 1

        if time.time() - last_report > 120:
            n = max(1, acc["n"])
            dt = time.time() - last_report
            sps = (state.d["samples_trained"] - samples_at_report) / dt
            hours = (time.time() - t_start) / 3600
            log(f"step {state.d['step']}  trained {state.d['samples_trained']:,}  rows {window.total:,} "
                f"({window.games:,} games)  window {window.window_size():,}  "
                f"v {acc['v']/n:.4f}  p {acc['p']/n:.4f}  kl {acc['kl']/n:.4f}  lr {lr:.2e}  {sps:.0f} samples/s")
            log_csv.writerow([time.strftime("%F %T"), f"{hours:.3f}", state.d["step"], state.d["samples_trained"],
                              window.total, window.games, window.window_size(), f"{acc['v']/n:.5f}",
                              f"{acc['p']/n:.5f}", f"{acc['kl']/n:.5f}", f"{lr:.3e}", f"{sps:.0f}"])
            log_file.flush()
            if args.aux and aux_acc["n"]:
                # A separate file, so that train_log.csv keeps one format for
                # every run whichever the variant.
                m = aux_acc["n"]
                own_accuracy = aux_acc["acc"] / aux_acc["n_acc"] if aux_acc["n_acc"] else float("nan")
                log(f"  aux: ownership {aux_acc['own']/m:.4f} (accuracy {own_accuracy:.3f})  "
                    f"margin pdf {aux_acc['pdf']/m:.4f}  cdf {aux_acc['cdf']/m:.4f}  weight ramp {ramp:.2f}")
                aux_path = os.path.join(args.work, "train_log_aux.csv")
                new_aux = not os.path.exists(aux_path)
                with open(aux_path, "a", newline="") as f:
                    w = csv.writer(f)
                    if new_aux:
                        w.writerow(["time", "step", "ownership_loss", "ownership_accuracy",
                                    "margin_pdf_loss", "margin_cdf_loss", "ramp"])
                    w.writerow([time.strftime("%F %T"), state.d["step"], f"{aux_acc['own']/m:.5f}",
                                f"{own_accuracy:.4f}", f"{aux_acc['pdf']/m:.5f}", f"{aux_acc['cdf']/m:.5f}",
                                f"{ramp:.3f}"])
            aux_acc = {"own": 0.0, "pdf": 0.0, "cdf": 0.0, "acc": 0.0, "n": 0, "n_acc": 0}
            acc = {"v": 0.0, "p": 0.0, "kl": 0.0, "n": 0}
            last_report = time.time()
            samples_at_report = state.d["samples_trained"]

        # ── export: a new network for self-play (and, every eval_every, a match) ──
        if since_export >= args.export_every:
            since_export = 0
            s = state.d["step"]
            export_model(ema.module if ema is not None else net, args.work, s)
            publish_latest(args.work, s, args.aux)
            with state.lock:
                state.d["exports"].append(s)
                state.d["samples_at_last_export"] = state.d["samples_trained"]
                # Evaluation schedule: every eval_every exports, or else the
                # first export at least eval_hours (wall clock, across jobs)
                # after the previously scheduled one.
                if args.eval_every > 0:
                    due = (len(state.d["exports"]) - 1) % args.eval_every == 0
                else:
                    due = time.time() - state.d.get("last_eval_time", 0) >= args.eval_hours * 3600
                if due and args.eval_games > 0:
```

```
                    state.d["pending_evals"].append(s)
                    state.d["last_eval_time"] = time.time()
            save_checkpoint()
            cleanup_models()
            evaluator.wake.set()
            log(f"exported az_{s:08d} (self-play switches to it)")

    # ── shutdown ──
    stop_event.set()
    log("saving checkpoint")
    save_checkpoint()
    evaluator.stop()
    if pool is not None:
        pool.stop()
    evaluator.join(timeout=60)
    log_file.close()
    log("stopped; resubmit with the same --work to continue")


if __name__ == "__main__":
    main()
```

12.5.6 `nnue/alphazero_aux.py`

149 lines.

KataGo's auxiliary heads, adapted to Yolah (Section 6.9): only imported by the trainer with `--aux`. The network with the two extra heads, the decoding of the future ownership map, and the losses.

symbol	role
`decode_ownership`	16 bytes, 2 bits per square → 64 codes in plane-cell order
`margin_from_ownership`	the rest of the score for the side to move: #mine − #opponent's
`AuxNet`	`Net` + ownership head (1×1 conv to 3 classes) + score head (pooling → 113 logits)
`AuxNet.forward_all`	value, policy, ownership logits, margin logits
`base_state_dict`	the `Net` part of the parameters: what is exported
`make_aux_net`	an `AuxNet` from a state dict; fresh heads if it has none
`aux_losses`	ownership cross-entropy, margin pdf and cdf losses, masked where there is no target

```
"""
alphazero_aux.py — KataGo's auxiliary heads for the Yolah ResNet (optional:
used by alphazero_learn.py only with --aux; without it nothing here runs).

Why (see the doc, "KataGo's auxiliary heads"): the value head learns from one
noisy result per game. Two extra heads, which exist ONLY during training, ask
the shared trunk for richer predictions, and their gradient shapes the trunk:

  • ownership — for each square not yet a hole, who will LEAVE it before the
    game ends: the side to move, the opponent, or nobody. In Yolah every move
```

```
    leaves a hole on the square it comes from and scores one point, so the
    map is the rest of the score, square by square.
  • score margin — the distribution of the rest of the score,
    d = (my future moves) − (the opponent's future moves) ∈ [−56, 56],
    which is exactly the sum of the ownership map (so it needs no extra data).

Both heads read the trunk's output (B, C, 8, 8), next to the value and policy
heads. At export they are dropped: base_state_dict() keeps only the Net
parameters, so the .pt/.ts written are the plain two-headed network and the
C++ backends are unchanged.
"""
import torch
from torch import nn
import torch.nn.functional as F

from cnn_resnet_value_policy_chunked import Net, net_from_state_dict

# ── targets: the future ownership map of a row (TrainingSampleAux::own) ──────
OWN_NONE, OWN_MINE, OWN_OPP, OWN_PAST = 0, 1, 2, 3   # as in alphazero_selfplay.h
OWN_BYTES = 16                                       # 64 squares × 2 bits
MAX_MARGIN = 56                                      # 64 squares − 8 pieces
SCORE_BINS = 2 * MAX_MARGIN + 1                      # d ∈ [−56, 56]

# Parameters of the auxiliary heads: their names start with these prefixes.
AUX_PREFIXES = ("own_", "score_")


def decode_ownership(own_bytes):
    """
    (B, 16) uint8 tensor, 2 bits per square (square q in byte q//4, bits
    2·(q%4)) → (B, 64) long tensor of OWN_* codes in PLANE CELL order, i.e.
    aligned with the network input: cell c holds square 63 − c (see
    nn_evaluator.h), hence the flip. The same symmetry gather as the input
    planes then applies to it.
    """
    b = own_bytes.long().unsqueeze(2)                          # (B, 16, 1)
    shifts = torch.arange(0, 8, 2, device=b.device)            # 0, 2, 4, 6
    by_square = ((b >> shifts) & 3).reshape(b.shape[0], 64)    # square q = 4·byte + k
    return by_square.flip(1)                                   # → cell order


def margin_from_ownership(own):
    """The rest of the score for the side to move: #mine − #opponent's."""
    return (own == OWN_MINE).sum(1) - (own == OWN_OPP).sum(1)


# ── the network ──────────────────────────────────────────────────────────────
class AuxNet(Net):
    """
    Net (same trunk, same value and policy heads, same parameter names) plus:

      own_conv   : 1×1 conv C → 3, one softmax per square over
                   (nobody, the side to move, the opponent) — a per-square
                   classification, exactly as KataGo's ownership head;
      score_fc1/2: global pooling of the trunk (mean and max over the 64
                   squares, 2C numbers) → FC 256 → ReLU → FC 113 logits, one
                   per possible margin. Pooling rather than flattening: the
                   margin is a property of the whole board.

    forward() returns (value, policy) like Net, so an AuxNet can be used
    anywhere a Net is; forward_all() adds the two auxiliary outputs.
    """
    def __init__(self, channels=256, nb_blocks=30, value_fc_size=256, num_actions=64 * 64,
```

```
                global_pool=False):
        super().__init__(channels=channels, nb_blocks=nb_blocks, value_fc_size=value_fc_size,
                         num_actions=num_actions, global_pool=global_pool)
        self.own_conv = nn.Conv2d(channels, 3, kernel_size=1)
        self.score_fc1 = nn.Linear(2 * channels, 256)
        self.score_fc2 = nn.Linear(256, SCORE_BINS)

    def forward_all(self, x):
        h = self.trunk(x)
        v, p = self.heads(h)
        own = self.own_conv(h).flatten(2)                       # (B, 3, 64), cell order
        pooled = torch.cat([h.mean(dim=(2, 3)), h.amax(dim=(2, 3))], dim=1)
        score = self.score_fc2(torch.relu(self.score_fc1(pooled)))   # (B, 113)
        return v, p, own, score


def is_aux_key(k):
    return k.startswith(AUX_PREFIXES)


def base_state_dict(sd):
    """The Net part of a (possibly AuxNet) state_dict: what gets exported."""
    return {k: v for k, v in sd.items() if not is_aux_key(k)}


def make_aux_net(sd):
    """
    An AuxNet sized from a state_dict. A plain Net state_dict (the initial
    network, or a run that did not use --aux) leaves the auxiliary heads at
    their random initialisation — the only keys allowed to be missing.
    """
    net, (missing, unexpected) = net_from_state_dict(sd, cls=AuxNet, strict=False)
    bad = [k for k in missing if not is_aux_key(k)] + list(unexpected)
    if bad:
        raise KeyError(f"state_dict does not fit AuxNet: {bad[:5]}")
    return net, bool(missing)          # True = fresh auxiliary heads


# ── the losses ──────────────────────────────────────────────────────────
def aux_losses(own_logits, score_logits, own):
    """
    own_logits (B, 3, 64), score_logits (B, 113), own (B, 64) OWN_* codes
    (already transformed by the batch's symmetry).

    Ownership: cross-entropy per square, averaged over the 64 squares (a hole
    — OWN_PAST — contributes 0), i.e. KataGo's (1/b²)·Σ_points. Rows without
    auxiliary data (version-1 rows, all OWN_PAST) contribute nothing.

    Score: cross-entropy of the predicted distribution against the true
    margin d (the "pdf" term) plus the squared distance between the
    predicted and the true cumulative distributions (the "cdf" term, which
    makes predicting +11 for a true +12 cheaper than predicting −30).

    Returns (own_loss, score_pdf_loss, score_cdf_loss, own_accuracy).
    """
    has_aux = (own != OWN_PAST).any(1)                          # (B,)
    n = has_aux.sum()
    if n.item() == 0:
        zero = own_logits.sum() * 0.0
        return zero, zero, zero, float("nan")
    own_logits, score_logits, own = own_logits[has_aux], score_logits[has_aux], own[has_aux]

    # ownership: classes (nobody, mine, opponent's) = codes 0, 1, 2; code 3 ignored
```

```
    ce = F.cross_entropy(own_logits.float(), own, ignore_index=OWN_PAST, reduction="none")  # (n, 64)
    own_loss = ce.sum(1).mean() / 64.0
    valid = own != OWN_PAST
    acc = ((own_logits.argmax(1) == own) & valid).sum().item() / max(1, valid.sum().item())

    # score margin
    target = (margin_from_ownership(own).clamp(-MAX_MARGIN, MAX_MARGIN) + MAX_MARGIN)   # (n,)
    logp = F.log_softmax(score_logits.float(), dim=1)
    pdf_loss = F.nll_loss(logp, target)
    cdf_pred = logp.exp().cumsum(1)
    cdf_true = (torch.arange(SCORE_BINS, device=target.device)[None, :] >= target[:, None]).float()
    cdf_loss = ((cdf_pred - cdf_true) ** 2).sum(1).mean()
    return own_loss, pdf_loss, cdf_loss, acc
```

12.5.7 `config/alphazero_mcts_selfplay_player.cfg`

24 lines.

Self-play search settings (the `weights` key is replaced by `latest.json`).

```
{
    "name": "AlphaZeroMCTSPlayer",
    "description": "AlphaZero/KataGo-style MCTS: PUCT with KataGo's exploration terms, graph search (transpositions), LCB move choice, guided by the value/policy ResNet 256×30 (convnet); self-play settings of the learning loop (1600/200 simulations, Dirichlet noise, move sampling in the opening, forced playouts) — not meant for competitive play.",
    "microseconds": 0,
    "nb simulations": 1600,
    "nb threads": 1,
    "batch size": 8,
    "backend": "torch",
    "device": "cuda",
    "fp16": true,
    "weights": "../nnue/cnn_resnet_256x30_value_policy.ts",
    "dirichlet alpha": 0.3,
    "dirichlet epsilon": 0.25,
    "temperature": 1.0,
    "temperature cutoff": 20,
    "forced playouts k": 2.0,
    "playout cap fast prob": 0.75,
    "nb simulations fast": 200,
    "policy target pruning": true,
    "graph search": true,
    "reuse tree": true,
    "nn cache": 16,
    "verbose": false
}
```

12.5.8 `config/alphazero_mcts_eval_player.cfg`

21 lines.

Evaluation-match settings: competitive play, no noise, no sampling. The time per move comes from `alphazero_learn match --time`.

```
{
    "name": "AlphaZeroMCTSPlayer",
    "description": "AlphaZero/KataGo-style MCTS: PUCT with KataGo's exploration terms, graph search (transpositions), LCB move choice, guided by the value/policy ResNet 256×30 (convnet); settings of the evaluation matches of the self-play learning (no noise, no sampling).",
    "microseconds": 2000000,
    "nb simulations": 0,
    "nb threads": 4,
    "batch size": 32,
    "backend": "torch",
    "device": "cuda",
    "fp16": true,
    "weights": "set by alphazero_learn match",
    "graph search": true,
    "use lcb": true,
    "dirichlet epsilon": 0.0,
    "temperature": 0.0,
    "forced playouts k": 0.0,
    "playout cap fast prob": 0.0,
    "reuse tree": true,
    "nn cache": 64,
    "verbose": false
}
```

12.5.9 `nnue/alphazero_learn.def`

62 lines.

The cluster image: a toolchain only, the sources are bind-mounted (Section 6.6).

```
Bootstrap: docker
From: nvidia/cuda:12.4.1-cudnn-devel-ubuntu22.04

# ── alphazero_learn.def — toolchain image for the AlphaZero learning loop ─────
#
# Build (needs root or --fakeroot, no GPU needed):
#     sudo singularity build alphazero_learn.sif alphazero_learn.def
#
# Unlike the other .def files, the Yolah sources are NOT baked into the image:
# the job bind-mounts the repository at /Yolah and compiles the C++ part
# (alphazero_learn) on the compute node at the start of every job (a couple of
# minutes, cached per CPU model in the work directory). Two reasons:
#   • the CMake flags use -march=native: a binary compiled on the machine that
#     builds the image may use instructions the cluster CPUs lack (SIGILL);
#   • a code change needs a `git pull` on the cluster, not an image rebuild.
#
# Contents: CUDA 12.4 + cuDNN (headers for libtorch's CMake config), g++-13
# (the code is C++23: <format>, std::jthread ...), Boost, oneTBB, Google
# benchmark, CMake, Python 3 with torch 2.6.0 — whose wheel also provides
# libtorch (headers + CMake package) for the C++ TorchScript backend.
#
# See alphazero_learn.sh for the SLURM glue and
# player/README_ALPHAZERO_LEARN.md for the whole loop.

%post
    set -eu
    export DEBIAN_FRONTEND=noninteractive
    apt-get update -y
    apt-get install -y --no-install-recommends software-properties-common gpg-agent ca-certificates
```

```
    # g++-13: Ubuntu 22.04 ships g++-11, which lacks <format>.
    add-apt-repository -y ppa:ubuntu-toolchain-r/test
    apt-get update -y
    apt-get install -y --no-install-recommends \
        g++-13 gcc-13 make cmake git \
        libboost-program-options-dev libtbb-dev libbenchmark-dev \
        python3 python3-dev python3-pip
    update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-13 100
    update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-13 100
    rm -rf /var/lib/apt/lists/*

    pip3 install --no-cache-dir --upgrade pip
    # torch 2.6.0 bundles the CUDA 12.4 runtime, which the cluster driver
    # supports (see cnn_resnet_value_policy.def). Keep the two pins together.
    pip3 install --no-cache-dir torch==2.6.0 numpy tqdm

    # Build-time sanity checks.
    g++ --version | head -1
    cmake --version | head -1
    python3 -c "import torch; print('torch', torch.__version__, torch.utils.cmake_prefix_path)"

    mkdir -p /Yolah /work

%environment
    export CUDA_HOME=/usr/local/cuda
    export PATH=$CUDA_HOME/bin:$PATH
    export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
    export CC=gcc-13
    export CXX=g++-13
    # Only used to skip libtorch's CMake GPU probe (see CMakeLists.txt):
    # Turing, Ampere, Ada, Hopper.
    export TORCH_CUDA_ARCH_LIST="7.5;8.0;8.6;8.9;9.0"
    export TORCH_ALLOW_TF32=1
```

12.5.10 `nnue/alphazero_common.sh`

94 lines.

Sourced by the two job scripts: the paths (`~/AlphaZeroLearn`), the container launcher, the build step (with its lock), and the signal forwarding that makes a clean stop possible.

symbol	role
`run`	run a command in the container, repository at `/Yolah`, run at `/work`
`build_alphazero_learn`	compile for this node's CPU model, serialised by `flock`
`start_in_container`	start in the background with `exec`, so that `$!` is Singularity itself
`wait_forwarding_signals`	wait, forwarding SIGUSR1/SIGTERM as SIGTERM for a clean stop
`max_hours_for_job`	the job's remaining time minus a margin, from `squeue`

```
# ------------------------------------------------------------------------
# alphazero_common.sh — sourced by alphazero_learn.sh and alphazero_selfplay.sh
# (not a job by itself). Paths, the container launcher and the build step.
```

```
#
# SLURM runs a *copy* of the batch script from its spool directory, so the
# scripts source this file through YOLAH_DIR, never through a relative path.
# ──────────────────────────────────────────────────────────────────────────

# ── Paths (override via env / `sbatch --export=ALL,VAR=...`) ──────────────
# The run lives in the home directory (/home/pgarcia/AlphaZeroLearn on the
# cluster), visible from every node. The I/O is light (~1.2 GB/day of self-play rows, a 141 MB
# network per export); the whole run takes ~60 GB over a month.
SIF="${SIF:-${HOME}/AlphaZeroLearn/alphazero_learn.sif}"
WORK_DIR="${WORK_DIR:-${HOME}/AlphaZeroLearn/work}"

if [[ ! -f "${SIF}" ]]; then
    echo "ERROR: no Singularity image at ${SIF}"
    echo "       Build it from nnue/alphazero_learn.def and put it there (or pass
    ↪  SIF=/path/to/image.sif)."
    exit 1
fi

# One build per CPU model: -march=native binaries are not portable across them.
CPU_TAG=$(grep -m1 'model name' /proc/cpuinfo | md5sum | cut -c1-8)
BUILD_DIR="/work/build/${CPU_TAG}"          # path inside the container

# WSL2 (local tests): the CUDA driver lives in /usr/lib/wsl/lib, which --nv
# does not bind. Harmless no-op on the cluster.
WSL_BIND=()
if [[ -d /usr/lib/wsl/lib ]]; then
    WSL_BIND=(--bind /usr/lib/wsl:/usr/lib/wsl --env LD_LIBRARY_PATH=/usr/lib/wsl/lib)
fi

# Run a command in the container: the repository at /Yolah, the run at /work.
run() {
    singularity exec --nv "${WSL_BIND[@]}" \
        --bind "${YOLAH_DIR}:/Yolah" \
        --bind "${WORK_DIR}:/work" \
        "${SIF}" "$@"
}

# Compile the given CMake targets (default: alphazero_learn) for this node's
# CPU (skipped by make when up to date). flock: two jobs starting together on
# identical nodes share the build directory and must not run CMake in it at
# the same time.
build_alphazero_learn() {
    local targets="${*:-alphazero_learn}"
    mkdir -p "${WORK_DIR}/build"
    echo "[$(date '+%F %T')] === Building ${targets} in ${BUILD_DIR} ==="
    run bash -c "set -e
        exec 9> /work/build/.lock_${CPU_TAG}
        flock 9
        mkdir -p ${BUILD_DIR}
        TORCH_PREFIX=\$(python3 -c 'import torch; print(torch.utils.cmake_prefix_path)')
        cmake -S /Yolah -B ${BUILD_DIR} -DCMAKE_BUILD_TYPE=Release -DENABLE_TORCH=ON \
              -DCMAKE_PREFIX_PATH=\${TORCH_PREFIX} -Wno-dev > /work/build/cmake_${CPU_TAG}.log
    ↪  2>&1 \
            || { cat /work/build/cmake_${CPU_TAG}.log; exit 1; }
        cmake --build ${BUILD_DIR} --target ${targets} -j ${SLURM_CPUS_PER_TASK:-16} 2>&1 | tail
    ↪  -3"
}

# Start a command in the container in the background, as a direct child:
# `exec` makes $! the singularity process itself, which forwards SIGTERM into
# the container (a bash subshell in between would swallow the signal and
# orphan the processes inside). Sets PID.
```

```
start_in_container() {
    ( exec singularity exec --nv "${WSL_BIND[@]}" \
        --bind "${YOLAH_DIR}:/Yolah" --bind "${WORK_DIR}:/work" "${SIF}" "$@" ) &
    PID=$!
}

# Wait for PID, forwarding SIGUSR1 (SLURM's --signal) and SIGTERM (scancel)
# as SIGTERM: the program inside stops cleanly. Sets RC to its exit code.
wait_forwarding_signals() {
    trap 'echo "[$(date "+%F %T")] signal received, stopping cleanly"; kill -TERM ${PID}
    ↪  2>/dev/null || true' USR1 TERM
    RC=0
    wait ${PID} || RC=$?                  # returns early when the trap fires...
    if kill -0 ${PID} 2>/dev/null; then   # ...then wait for the clean shutdown
        RC=0; wait ${PID} || RC=$?
    fi
}

# Hours left in this SLURM job minus a margin (0 outside SLURM), so that the
# program stops by itself even if the signal is lost.
max_hours_for_job() {
    local margin_s="$1" left secs
    [[ -z "${SLURM_JOB_ID:-}" ]] && { echo 0; return; }
    left=$(squeue -h -j "${SLURM_JOB_ID}" -o %L 2>/dev/null || echo "")
    # formats: [D-]HH:MM:SS, MM:SS
    if [[ "${left}" =~ ^(([0-9]+)-)?([0-9]+):([0-9]+):([0-9]+)$ ]]; then
        secs=$(( ${BASH_REMATCH[2]:-0}*86400 + 10#${BASH_REMATCH[3]}*3600 +
        ↪  10#${BASH_REMATCH[4]}*60 + 10#${BASH_REMATCH[5]} ))
        awk -v s="${secs}" -v m="${margin_s}" 'BEGIN { h = (s - m) / 3600; if (h < 0.1) h = 0.1;
        ↪  printf "%.3f", h }'
    else
        echo 0
    fi
}
```

12.5.11 nnue/alphazero_learn.sh

150 lines.

The main SLURM job: 3 GPUs, 32 CPUs, 128 GB, 14 days (sizing: Section 6.7). It compiles for the node, runs the trainer, and stops cleanly on the time-limit signal.

```
#!/bin/bash
# ──────────────────────────────────────────────────────────────────────────────
# alphazero_learn.sh — AlphaZero/KataGo-style self-play learning on the cluster.
#
#     sbatch alphazero_learn.sh           # first job AND every following one
#
# The run lives in WORK_DIR (~/AlphaZeroLearn/work). A job that ends (time limit,
# preemption, scancel) checkpoints; submitting the same script again continues
# it: 2 weeks + 2 weeks = one month of learning. Nothing is lost but the games
# in progress.
#
# Per job:
#   1. compile alphazero_learn (C++ self-play + matches) for this node's CPU,
#      inside the container, into WORK_DIR/build/<cpu> (skipped when cached);
#   2. run nnue/alphazero_learn.py, which starts one self-play process per GPU
#      of this job, trains on the first GPU, exports a new network every
#      EXPORT_EVERY samples and, every EVAL_HOURS, plays EVAL_GAMES games
#      (EVAL_SECONDS per move, colours alternated) on the last GPU against the
```

```
#      previously evaluated network, from the balanced openings of EVAL_BOOK
#      (config/openings/balanced.txt; EVAL_BOOK=none: random 4-ply openings).
#
# GPUs: 3 per job (the per-user maximum). The GPU nodes have 2 or 3 cards
# (A40 or RTX 8000, 48 GB); if a 3-GPU node is long to get, run this job with
# fewer GPUs and add the missing ones as alphazero_selfplay.sh jobs, which
# can land on any GPU node: everything goes through WORK_DIR (shared home).
#     sbatch --gres=gpu:1 alphazero_learn.sh
#     sbatch alphazero_selfplay.sh ; sbatch alphazero_selfplay.sh
# To pin a GPU model, use the cluster's gres type, e.g. --gres=gpu:a40:3
# (`sinfo -o "%N %G"` lists them).
#
# Sizing (see "Sizing for the cluster" in doc/alphazero_mcts.pdf): self-play
# is the bottleneck; on an RTX 3080 self-play uses ~2.6 cores, and CPU work
# grows with the evaluation rate, so 3 GPUs twice as fast need ~16 cores (+
# trainer and matches): 32 CPUs. RAM holds a 16 M-row replay window (5.4 GB)
# and 768 network caches of 32 MB (25 GB): 128 GB.
#
# Follow the progress:  WORK_DIR/evals.csv (Elo), WORK_DIR/train_log.csv,
#                       alphazero_learn_<jobid>.out, WORK_DIR/logs/.
#
# SLURM sends SIGUSR1 30 min before the time limit (--signal); it is forwarded
# to the trainer, which saves everything and stops the self-play cleanly.
# MAX_HOURS is a second safety net computed from the job's time limit.
# ──────────────────────────────────────────────────────────────────────────────

#SBATCH --job-name=alphazero_learn
#SBATCH --output=alphazero_learn_%j.out
#SBATCH --error=alphazero_learn_%j.out
#SBATCH --partition=insa-gpu
#SBATCH -x crn23
#SBATCH --gres=gpu:3
#SBATCH --cpus-per-task=32
#SBATCH --mem=128G
#SBATCH --time=14-00:00:00
#SBATCH --signal=B:USR1@1800

set -euo pipefail

YOLAH_DIR="${YOLAH_DIR:-${HOME}/Yolah}"                              # git checkout
source "${YOLAH_DIR}/nnue/alphazero_common.sh"                       # SIF, WORK_DIR, run(), ...

# ── Tunables (override via env / `sbatch --export=ALL,VAR=...`) ──────────────
# Initial network (used only when WORK_DIR is new), path on the host.
INIT_MODEL="${INIT_MODEL:-${YOLAH_DIR}/nnue/cnn_resnet_256x30_value_policy.pt}"

SELFPLAY_GAMES="${SELFPLAY_GAMES:-256}"       # concurrent games per GPU (128 saturate an RTX
↪  3080)
SELFPLAY_NN_CACHE="${SELFPLAY_NN_CACHE:-32}"  # MB of network cache per game
BATCH_SIZE="${BATCH_SIZE:-1024}"
LR="${LR:-1e-4}"
REUSE="${REUSE:-4}"                           # training samples per self-play row
MIN_ROWS="${MIN_ROWS:-250000}"                # N0: rows before training starts (KataGo: 250k)
MAX_WINDOW="${MAX_WINDOW:-16000000}"          # replay buffer capacity (336 B/row)
EXPORT_EVERY="${EXPORT_EVERY:-524288}"        # training samples between networks
EVAL_HOURS="${EVAL_HOURS:-4}"                 # time between evaluations
EVAL_GAMES="${EVAL_GAMES:-20}"
EVAL_SECONDS="${EVAL_SECONDS:-2}"
EVAL_OPPONENTS="${EVAL_OPPONENTS:-previous}"  # previous, initial, lag:K (comma separated)
EVAL_BOOK="${EVAL_BOOK:-/Yolah/config/openings/balanced.txt}"   # balanced openings; "none" =
↪  random ones
# 1 = train KataGo's auxiliary heads (ownership, score margin) as well; 0 =
# the plain two-headed training. A run can switch from one job to the next;
```

```
# the extra self-play jobs follow automatically (latest.json).
AUX="${AUX:-0}"
# Value target = (1 − Q_WEIGHT)·z + Q_WEIGHT·q, q = the search value of the
# position (already recorded in every row). 0 = the game result alone.
Q_WEIGHT="${Q_WEIGHT:-0}"
# Grown network (nnue/alphazero_grow.py, INIT_MODEL = its .pt) started in a
# NEW work dir: SEED_FROM = the earlier run's work dir, whose selfplay/*.bin
# are linked into the new one (new run only), so the replay window is full at
# once; WARMUP_SAMPLES = training samples on those rows before self-play sets
# the pace (e.g. 8000000 ≈ 1.5 passes over the window).
SEED_FROM="${SEED_FROM:-}"
WARMUP_SAMPLES="${WARMUP_SAMPLES:-0}"
EXTRA_ARGS="${EXTRA_ARGS:-}"                  # anything else for alphazero_learn.py
RESUBMIT="${RESUBMIT:-0}"                     # 1 = sbatch this script again when the time is up

mkdir -p "${WORK_DIR}"
# The network is stored with Git LFS: a clone made without git-lfs (as on the
# cluster) holds a 134-byte pointer instead. Copy the real file by hand:
#     scp nnue/cnn_resnet_256x30_value_policy.pt cluster:Yolah/nnue/
if [[ ! -f "${WORK_DIR}/init_model.pt" ]] && head -c 100 "${INIT_MODEL}" 2>/dev/null | grep -q "git-lfs"; then
    echo "ERROR: ${INIT_MODEL} is a Git LFS pointer, not the network."
    echo "       Copy the real file: scp nnue/cnn_resnet_256x30_value_policy.pt <cluster>:${INIT_MODEL}"
    exit 1
fi
cp -n "${INIT_MODEL}" "${WORK_DIR}/init_model.pt" 2>/dev/null || true
if [[ -n "${SEED_FROM}" && ! -f "${WORK_DIR}/checkpoint.pt" ]]; then
    mkdir -p "${WORK_DIR}/selfplay"
    # hard links (same file system: no copy, no space); a copy otherwise
    find "${SEED_FROM}/selfplay" -maxdepth 1 -name '*.bin' -exec cp -ln -t "${WORK_DIR}/selfplay" {} + 2>/dev/null \
        || find "${SEED_FROM}/selfplay" -maxdepth 1 -name '*.bin' -exec cp -n -t "${WORK_DIR}/selfplay" {} +
    echo "seeded ${WORK_DIR}/selfplay with $(ls "${WORK_DIR}/selfplay" | wc -l) files of ${SEED_FROM}"
fi
MAX_HOURS=$(max_hours_for_job 1200)           # stop 20 min before the limit
AUX_ARGS=""
[[ "${AUX}" == "1" ]] && AUX_ARGS="--aux"

echo "═══════════════════════════════════════════════════════════════"
echo "  Job        : ${SLURM_JOB_ID:-(local)} on $(hostname), GPUs ${CUDA_VISIBLE_DEVICES:-?}"
echo "  Yolah      : ${YOLAH_DIR} ($(git -C "${YOLAH_DIR}" rev-parse --short HEAD 2>/dev/null || echo '?'))"
echo "  SIF        : ${SIF}"
echo "  Work dir   : ${WORK_DIR}"
echo "  Max hours  : ${MAX_HOURS}"
echo "  Aux heads  : $([[ "${AUX}" == "1" ]] && echo "on (ownership, score margin)" || echo off)"
echo "  Value      : $(awk -v w="${Q_WEIGHT}" 'BEGIN { if (w > 0) printf "%g·z + %g·q", 1 - w, w; else print "z (game result)" }')"
echo "═══════════════════════════════════════════════════════════════"
nvidia-smi --query-gpu=index,name,memory.total --format=csv,noheader 2>/dev/null || true

# ── 1. compile the C++ part for this node ──────────────────────────
build_alphazero_learn

# ── 2. learn ───────────────────────────────────────────────────────
echo "[$(date '+%F %T')] === Learning ==="
start_in_container bash -c "cd /Yolah/nnue && exec python3 -u alphazero_learn.py \
    --work /work --init-model /work/init_model.pt --bin ${BUILD_DIR}/alphazero_learn \
    --selfplay-config /Yolah/config/alphazero_mcts_selfplay_player.cfg \
    --eval-config /Yolah/config/alphazero_mcts_eval_player.cfg \
```

```
    --selfplay-games ${SELFPLAY_GAMES} --batch-size ${BATCH_SIZE} --lr ${LR} --reuse ${REUSE} \
    --selfplay-set 'nn cache=${SELFPLAY_NN_CACHE}' --max-window ${MAX_WINDOW} \
    --min-rows ${MIN_ROWS} --export-every ${EXPORT_EVERY} --eval-hours ${EVAL_HOURS} \
    --eval-games ${EVAL_GAMES} --eval-seconds ${EVAL_SECONDS} --eval-opponents ${EVAL_OPPONENTS} \
    $([[ "${EVAL_BOOK}" == none ]] || echo "--eval-book ${EVAL_BOOK}") \
    --q-weight ${Q_WEIGHT} --max-hours ${MAX_HOURS} --inherit-warmup ${WARMUP_SAMPLES}
 ↪  ${AUX_ARGS} ${EXTRA_ARGS}"
wait_forwarding_signals

echo "[$(date '+%F %T')] === Stopped (exit code ${RC}) ==="
tail -n 5 "${WORK_DIR}/evals.csv" 2>/dev/null || true
# Resubmit only after a clean stop (not after a crash: no resubmission loop).
if [[ "${RESUBMIT}" == "1" && "${RC}" == "0" && -n "${SLURM_JOB_ID:-}" ]]; then
    echo "resubmitting ${SLURM_SUBMIT_DIR}/alphazero_learn.sh"
    cd "${SLURM_SUBMIT_DIR}" && sbatch --export=ALL alphazero_learn.sh
fi
```

12.5.12 `nnue/alphazero_match.sh`

140 lines.

A long match between two networks of a run, on its own GPU, with the score's 95 % confidence interval — the precise measurement the 20-game evaluations cannot give (Section 6.5).

```
#!/bin/bash
# ──────────────────────────────────────────────────────────────────────────
# alphazero_match.sh — a long match between two networks of a learning run,
# on its own GPU, to measure progress more precisely than the 20-game
# evaluations of alphazero_learn.sh (±78 Elo each).
#
#     sbatch alphazero_match.sh                          # latest vs initial, 200 games, 0.5
 ↪  s/move
#     sbatch --export=ALL,A=7168,B=4096 alphazero_match.sh
#     sbatch --export=ALL,GAMES=400,MOVE_SECONDS=1 alphazero_match.sh
#     sbatch --export=ALL,BOOK=none alphazero_match.sh   # random openings, as before
#
# A and B are training steps (the numbers of models/az_<step>), or "latest"
# (the network self-play is using) or "initial" (step 0). A network whose .ts
# was pruned by the run is re-traced from its .pt.
#
# Both players use config/alphazero_mcts_eval_player.cfg (no noise, no
# sampling). Games come in pairs: one opening, played once with each colour.
# The openings come from BOOK (default config/openings/balanced.txt: balanced
# openings, test/opening_book_main.cpp, one per pair), or with BOOK=none are
# random ones of OPENING_PLIES plies — after which Black wins nearly every
# game, so most pairs end 1-1 and tell little. The result — A's wins/draws/losses, score and
# Elo difference with a 95 % confidence interval — is printed and appended to
# WORK_DIR/matches.csv; every game is in WORK_DIR/evals/match_*.json.
#
# 200 games × ~55 plies × 0.5 s ≈ 1.5 h;  ±25 Elo (95 %: about ±50).
# It can run while alphazero_learn.sh is running (another GPU, same WORK_DIR).
# ──────────────────────────────────────────────────────────────────────────

#SBATCH --job-name=alphazero_match
#SBATCH --output=alphazero_match_%j.out
#SBATCH --error=alphazero_match_%j.out
#SBATCH --partition=insa-gpu
#SBATCH -x crn23
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
```

```
#SBATCH --mem=16G
#SBATCH --time=06:00:00

set -euo pipefail

YOLAH_DIR="${YOLAH_DIR:-${HOME}/Yolah}"
source "${YOLAH_DIR}/nnue/alphazero_common.sh"                      # SIF, WORK_DIR, run(), ...

A="${A:-latest}"                  # step, "latest" or "initial"
B="${B:-initial}"
GAMES="${GAMES:-200}"             # even: games come in pairs (one opening, both colours)
# (not SECONDS: that is bash's own counter of seconds since the shell started)
SECONDS_PER_MOVE="${MOVE_SECONDS:-0.5}"
OPENING_PLIES="${OPENING_PLIES:-4}"
BOOK="${BOOK:-/Yolah/config/openings/balanced.txt}"   # inside the container; "none" = random
 ↪  openings
SEED="${SEED:-${SLURM_JOB_ID:-$$}}"
JOB="${SLURM_JOB_ID:-$$}"

(( GAMES % 2 == 0 )) || { echo "ERROR: GAMES must be even (pairs of games)"; exit 1; }
[[ -f "${WORK_DIR}/latest.json" ]] || { echo "ERROR: no run in ${WORK_DIR} (latest.json missing)";
 ↪  exit 1; }

# "latest" / "initial" / a step → a step number.
resolve() {
    case "$1" in
        latest)  grep -o '"step": *[0-9]*' "${WORK_DIR}/latest.json" | grep -o '[0-9]*$' ;;
        initial) echo 0 ;;
        *)       echo "$1" ;;
    esac
}
STEP_A=$(resolve "${A}")
STEP_B=$(resolve "${B}")
NAME_A=$(printf "az_%08d" "${STEP_A}")
NAME_B=$(printf "az_%08d" "${STEP_B}")

echo "═══════════════════════════════════════════════════════════════════"
echo "  Job        : ${JOB} on $(hostname), GPU ${CUDA_VISIBLE_DEVICES:-?}"
echo "  Work dir   : ${WORK_DIR}"
echo "  Match      : A = ${NAME_A} (${A})  vs  B = ${NAME_B} (${B})"
if [[ "${BOOK}" == "none" ]]; then BOOK_ARGS=(); OPENINGS="${OPENING_PLIES}-ply random openings"
else BOOK_ARGS=(--book "${BOOK}"); OPENINGS="openings of ${BOOK}"; fi
echo "  Games      : ${GAMES} (${OPENINGS}, both colours), ${SECONDS_PER_MOVE} s/move"
echo "═══════════════════════════════════════════════════════════════════"
nvidia-smi --query-gpu=index,name,memory.total --format=csv,noheader 2>/dev/null || true

build_alphazero_learn

# Private copies of the two networks: the running learning job prunes old
# .ts files at every export and must not remove one in the middle of the match.
MDIR="/work/evals/match_${JOB}"
mkdir -p "${WORK_DIR}/evals/match_${JOB}"
run bash -c "set -e; cd /Yolah/nnue
    for S in ${STEP_A} ${STEP_B}; do
        NAME=\$(printf 'az_%08d' \$S)
        if [[ -f /work/models/\$NAME.ts ]]; then
            cp /work/models/\$NAME.ts ${MDIR}/\$NAME.ts
        elif [[ -f /work/models/\$NAME.pt ]]; then
            echo \"re-tracing \$NAME.ts from its .pt\"
            # export_model writes <dir>/models/az_<step>.{pt,ts}
            mkdir -p ${MDIR}/models
            python3 -c \"
from alphazero_learn import export_model, make_net, load_state_dict
```

```
export_model(make_net(load_state_dict('/work/models/\$NAME.pt')), '${MDIR}', \$S)
\"
            mv ${MDIR}/models/\$NAME.ts ${MDIR}/\$NAME.ts
            rm -rf ${MDIR}/models
        else
            echo \"ERROR: neither /work/models/\$NAME.ts nor .pt exists\"; exit 1
        fi
    done"

OUT="/work/evals/match_${NAME_A}_vs_${NAME_B}_${JOB}.json"
TIME_US=$(awk -v s="${SECONDS_PER_MOVE}" 'BEGIN { printf "%d", s * 1000000 }')
echo "[$(date '+%F %T')] === Match ==="
start_in_container ${BUILD_DIR}/alphazero_learn match \
    --config /Yolah/config/alphazero_mcts_eval_player.cfg \
    --a "${MDIR}/${NAME_A}.ts" --b "${MDIR}/${NAME_B}.ts" \
    --games "${GAMES}" --time "${TIME_US}" --opening-plies "${OPENING_PLIES}" \
    "${BOOK_ARGS[@]}" --seed "${SEED}" --out "${OUT}"
wait_forwarding_signals
rm -rf "${WORK_DIR}/evals/match_${JOB}"
[[ "${RC}" == "0" ]] || { echo "match failed (exit code ${RC})"; exit "${RC}"; }

# ── Result: score, Elo difference and its 95 % confidence interval ──────────
# Per-game results x ∈ {1, ½, 0}: mean s, variance E[x²] − s², standard error
# √(var/n). The interval of s maps to Elo through Δ = 400·log10(s/(1−s)).
RES="${WORK_DIR}/evals/match_${NAME_A}_vs_${NAME_B}_${JOB}.json"
# By name: the JSON keys are written in alphabetical order, not W/D/L.
count() { grep -oE "\"$1\": *[0-9]+" "${RES}" | head -1 | grep -oE '[0-9]+$'; }
W=$(count wins); D=$(count draws); L=$(count losses)
awk -v w="$W" -v d="$D" -v l="$L" -v a="${NAME_A}" -v b="${NAME_B}" -v t="${SECONDS_PER_MOVE}" \
    -v csv="${WORK_DIR}/matches.csv" -v date="$(date '+%F %T')" 'BEGIN {
    n = w + d + l; s = (w + d / 2) / n
    var = (w + d / 4) / n - s * s; se = sqrt(var / n)
    lo = s - 1.96 * se; hi = s + 1.96 * se
    if (lo < 0.001) lo = 0.001; if (hi > 0.999) hi = 0.999
    sc = s; if (sc < 0.001) sc = 0.001; if (sc > 0.999) sc = 0.999
    elo = 400 * log(sc / (1 - sc)) / log(10)
    elo_lo = 400 * log(lo / (1 - lo)) / log(10); elo_hi = 400 * log(hi / (1 - hi)) / log(10)
    printf "\n%s vs %s: %dW %dD %dL over %d games (%s s/move)\n", a, b, w, d, l, n, t
    printf "score %.3f ± %.3f  →  Elo %+.0f  (95%% interval [%+.0f, %+.0f])\n", s, 1.96 * se, elo,
↪   elo_lo, elo_hi
    new = (system("test -f " csv) != 0)
    if (new) print
↪   "time,a,b,games,seconds_per_move,wins,draws,losses,score,elo,elo_low95,elo_high95" >> csv
    printf "%s,%s,%s,%d,%s,%d,%d,%d,%.3f,%+.0f,%+.0f,%+.0f\n", date, a, b, n, t, w, d, l, s, elo,
↪   elo_lo, elo_hi >> csv
}'
echo "(appended to ${WORK_DIR}/matches.csv; games in ${RES})"
```

12.5.13 `nnue/alphazero_selfplay.sh`

61 lines.

An extra self-play GPU on any node, for a run split over several jobs (Figure 17).

```
#!/bin/bash
# ──────────────────────────────────────────────────────────────────────────
# alphazero_selfplay.sh — an extra self-play GPU for a running alphazero_learn
# job, on any GPU node.
#
#     sbatch alphazero_selfplay.sh         # same WORK_DIR as alphazero_learn.sh
#
```

```
# The learning loop only communicates through files in WORK_DIR (in the home,
# visible from every node): this job reads latest.json, hot-swaps each new
# network, and writes its games to WORK_DIR/selfplay/, where the trainer picks
# them up like those of its own GPUs. Use it when a node with 3 free GPUs is
# long to get: `sbatch --gres=gpu:1 alphazero_learn.sh` plus two of these
# (3 GPUs in total, the per-user maximum).
#
# It waits for the main job to have published a network, and stops by itself
# when no new network has appeared for STALE_HOURS (the main job is over), at
# its own time limit, or on scancel — finished games are always written.
# ──────────────────────────────────────────────────────────────────────────────

#SBATCH --job-name=alphazero_selfplay
#SBATCH --output=alphazero_selfplay_%j.out
#SBATCH --error=alphazero_selfplay_%j.out
#SBATCH --partition=insa-gpu
#SBATCH -x crn23
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=12
#SBATCH --mem=48G
#SBATCH --time=14-00:00:00
#SBATCH --signal=B:USR1@600

set -euo pipefail

YOLAH_DIR="${YOLAH_DIR:-${HOME}/Yolah}"
source "${YOLAH_DIR}/nnue/alphazero_common.sh"                    # SIF, WORK_DIR, run(), ...

SELFPLAY_GAMES="${SELFPLAY_GAMES:-256}"       # concurrent games on this GPU
SELFPLAY_NN_CACHE="${SELFPLAY_NN_CACHE:-32}"  # MB of network cache per game
STALE_HOURS="${STALE_HOURS:-6}"               # stop after this long without a new network

echo "═══════════════════════════════════════════════════════════════════"
echo "  Job        : ${SLURM_JOB_ID:-(local)} on $(hostname), GPU ${CUDA_VISIBLE_DEVICES:-?}"
echo "  Work dir   : ${WORK_DIR}"
echo "═══════════════════════════════════════════════════════════════════"
nvidia-smi --query-gpu=index,name,memory.total --format=csv,noheader 2>/dev/null || true

# The main job creates latest.json when it starts a new run.
until [[ -f "${WORK_DIR}/latest.json" ]]; do
    echo "[$(date '+%F %T')] waiting for ${WORK_DIR}/latest.json (is alphazero_learn.sh running?)"
    sleep 60
done

build_alphazero_learn

echo "[$(date '+%F %T')] === Self-play ==="
mkdir -p "${WORK_DIR}/logs"
start_in_container ${BUILD_DIR}/alphazero_learn selfplay \
    --config /Yolah/config/alphazero_mcts_selfplay_player.cfg --work /work \
    --games "${SELFPLAY_GAMES}" --set "nn cache=${SELFPLAY_NN_CACHE}" \
    --max-stale-hours "${STALE_HOURS}" --tag "$(hostname)_${SLURM_JOB_ID:-$$}_extra"
wait_forwarding_signals
echo "[$(date '+%F %T')] === Stopped (exit code ${RC}) ==="
```